energy-systems-ai999x: capability guide

Creator / engineer: Ibnul Jaif Farabi · **Company:** AI999X. Ask the package `Who created you?` or call `energy_systems_ai999x.about()` for the same answer.

Every capability energy-systems-ai999x supports, generated from its certified capability contracts. Every example below was executed when this guide was built. The structured call `solve("<capability id>", ...)` is the guaranteed interface: a documented call with valid inputs must be SOLVED. Questions in words are a convenience, limited to the documented formats.

Quick start

```
import energy_systems_ai999x

result = energy_systems_ai999x.solve('energy_solar_declination', n=172)
result.show()
# declination_deg = 23.4498, declination_rad = 0.409276
```

By topic

topic	capabilities
Batteries / capacity	Peukert exponent and run time at a new discharge current
Batteries / equivalent circuits	Battery terminal voltage under constant current (one-RC equivalent circuit)
Batteries / pack design	Battery pack series-parallel sizing
Batteries / state of charge	State of charge by coulomb counting
Fuel cells / electrode kinetics	Electrode kinetics: Butler-Volmer current, overpotential and Tafel slopes
Fuel cells / electrolysis	Water electrolyser: hydrogen production and energy per kilogram
Fuel cells / hydrogen fuel cells	Hydrogen fuel-cell stack: consumption, power and efficiency
Fuel cells / stack power	Electrical power of a fuel-cell or electrolyser stack
Generation / hydropower	Hydroelectric power from flow and head, and the energy over a period
Generation / plant performance	Capacity factor and energy of a generating plant
Hydrogen / compression	Minimum (isothermal) work to compress hydrogen, Isentropic (adiabatic) compression of hydrogen
Hydrogen / storage	Hydrogen stored in a tank at pressure (real gas)

topic	capabilities
Machines / induction motors	Induction motor performance from its equivalent circuit, Induction motor breakdown (maximum) torque and starting torque, Induction motor power flow from slip and losses
Machines / speed and slip	Synchronous speed, rotor speed and slip of an AC machine
Machines / synchronous machines	Synchronous generator internal EMF, power angle and regulation
Photovoltaics / array output	PV array DC power with temperature derating (PVWatts), PV array output at operating temperature
Photovoltaics / cell temperature	PV cell temperature from NOCT, PV module temperature (Faiman model), PV cell and module temperature (Sandia SAPM model)
Photovoltaics / inverters	Inverter AC output and efficiency (PVWatts)
Photovoltaics / module performance	PV module maximum power at an irradiance and temperature
Photovoltaics / optical losses	Incidence angle modifier (ASHRAE), Incidence angle modifier from Fresnel reflection and absorption
Photovoltaics / single-diode model	PV single-diode model: current at a voltage, PV single-diode model: voltage at a current, PV single-diode model: short-circuit, open-circuit and maximum power point, PV module single-diode parameters at operating conditions (De Soto)
Photovoltaics / system losses	Combined PV system losses (PVWatts)
Power systems / economic dispatch	Economic dispatch with quadratic costs and limits
Power systems / faults	Three-phase (symmetrical) fault current by the bus impedance matrix, Single line-to-ground fault (symmetrical components), Line-to-line fault (symmetrical components), Double line-to-ground fault (symmetrical components)
Power systems / frequency control	Frequency deviation and load sharing under governor droop
Power systems / line parameters	Inductance, capacitance and surge impedance of a three-phase overhead line, Geometric mean distance of a three-phase line
Power systems / load flow	Receiving-end voltage of a line feeding a constant-power load (two-bus load flow), DC power flow, AC load flow by Newton-Raphson
Power systems / per-unit system	Per-unit impedance on a new base, Base impedance and per-unit impedance
Power systems / stability	Power transfer and power angle between two voltages
Power systems / transient stability	Critical clearing angle and time (equal-area criterion)
Power systems / transmission lines	Short transmission line: sending-end voltage, regulation and efficiency, Medium transmission line (nominal pi), Long transmission line (exact distributed parameters)
Solar resource / atmosphere	Optical air mass
Solar resource / clear sky	Clear-sky global horizontal irradiance (Haurwitz), Clear-sky irradiance (Ineichen-Perez with Linke turbidity)
Solar resource / irradiance	Extraterrestrial (top-of-atmosphere) normal irradiance

topic	capabilities
Solar resource / irradiance models	Diffuse and direct irradiance from global horizontal irradiance (Erbs), Direct normal irradiance from global horizontal irradiance (DISC model)
Solar resource / solar time	Equation of time, Solar time and hour angle from clock time
Solar resource / sun position	Solar declination on a day of the year, Solar zenith, elevation and azimuth from latitude, declination and hour angle, Sun position at a place and local time, Sunrise, sunset, solar noon and day length
Solar resource / tilted surfaces	Angle of incidence of the sun on a tilted surface, Irradiance on a tilted plane (isotropic sky), Irradiance on a tilted plane (Klucher sky model), Irradiance on a tilted plane (Hay-Davies sky model), Irradiance on a tilted plane (Reindl sky model), Irradiance on a tilted plane (Perez sky model)
Three-phase power / balanced systems	Balanced three-phase power from line quantities
Transformers / efficiency	Load for maximum transformer efficiency
Transformers / equivalent circuit	Transformer regulation and efficiency from its equivalent circuit
Transformers / loading	Transformer under load from nameplate data
Transformers / parallel operation	Load sharing between two transformers in parallel
Transformers / testing	Transformer parameters from open-circuit and short-circuit tests
Wind power / annual energy	Annual energy production from a power curve and a Weibull distribution
Wind power / rotor aerodynamics	Actuator-disc (momentum theory) rotor performance
Wind power / turbine output	Turbine output from its power curve, Turbine output from a density-corrected power curve
Wind power / wind resource	Power in the wind and the Betz limit, Wind speed at another height (power law), Wind speed at another height (logarithmic profile), Air density at hub height, Weibull wind statistics and mean power density

Outcomes

outcome	meaning	what to do
SOLVED	a value with its unit, steps, assumptions and verification	use it; read the assumptions
NEEDS_INPUT	a required input is missing; `uncertainty['needs']` names it	add it (with its unit) and ask again
AMBIGUOUS	two readings or two physical branches give different answers	say which situation is meant
INVALID_INPUT	a unit of the wrong kind, an unknown input name, values with no valid solution	correct the input
OUT_OF_SCOPE	no certified capability computes this (or the structure is not modelled yet)	see the catalogue below
UNRECOGNIZED_FORMAT	the wording was not recognised; a supported template is offered in `uncertainty['try']`	use the template or solve()

outcome	meaning	what to do
INTERNAL_ERROR	a defect of the engine, never a property of the question	report it with the question

Result fields

field	meaning
value, unit	the result in SI units
values	every output of a procedure (x1, x2, discriminant ...)
outcome	one of the seven outcomes above
steps	each equation used, its inputs (with where each came from) and its result
assumptions	conditions and declared defaults the answer relies on
verification	residual of every equation, dimensional consistency, independent checks
latex()	the derivation as LaTeX
show()	renders in a notebook, prints in a terminal

Common mistakes

- Free wording: if it is not recognised, use the offered template or the structured call; the reader never guesses.
- A coefficient that violates a stated condition (a = 0 in a quadratic): INVALID_INPUT says why.
- A text value where a number is expected: INVALID_INPUT; fractions as strings ("1/2") are accepted.
- A problem outside the procedure (a cubic for the quadratic solver): OUT_OF_SCOPE, never an approximation.

Procedures: algorithms with several outputs

Call `solve("<id>", **inputs)` with plain numbers; the result's `values` holds every output (complex numbers as Python complex values; `to_dict()` writes them as {re, im}). Each result is substituted back into its defining equation before it is returned.

id	operation	inputs	outputs
energy_solar_declination	Solar declination on a day of the year	n, method	declination_deg, declination_rad
energy_equation_of_time	Equation of time	n, method	eot_min, eot_s
energy_solar_time_hour_angle	Solar time and hour angle from clock time	n, clock, longitude, utc_offset	eot_min, hour_angle_deg, solar_time_h
energy_sun_angles	Solar zenith, elevation and azimuth from latitude, declination and hour angle	latitude, declination, hour_angle	zenith_deg, elevation_deg, azimuth_deg

id	operation	inputs	outputs
energy_solar_position	Sun position at a place and local time	latitude, longitude, n, clock, utc_offset	declination_deg, eot_min, hour_angle_deg, zenith_deg, elevation_deg, azimuth_deg
energy_sunrise_sunset	Sunrise, sunset, solar noon and day length	latitude, longitude, n, utc_offset	sunrise_h, sunset_h, solar_noon_h, day_length_h, sunset_hour_angle_deg
energy_extraterrestrial_irradiance	Extraterrestrial (top-of-atmosphere) normal irradiance	n, method, solar_constant	G_on, distance_factor
energy_air_mass	Optical air mass	zenith, model, pressure	AM_relative, AM_absolute
energy_angle_of_incidence	Angle of incidence of the sun on a tilted surface	tilt, surface_azimuth, zenith, azimuth	aoi_deg, cos_aoi
energy_poa_isotropic	Irradiance on a tilted plane (isotropic sky)	tilt, surface_azimuth, zenith, azimuth, dni, ghi, dhi, albedo	poa_global, poa_direct, poa_sky_diffuse, poa_ground_diffuse, aoi_deg
energy_poa_klucher	Irradiance on a tilted plane (Klucher sky model)	tilt, surface_azimuth, zenith, azimuth, dni, ghi, dhi, albedo	poa_global, poa_direct, poa_sky_diffuse, poa_ground_diffuse, aoi_deg
energy_poa_haydavies	Irradiance on a tilted plane (Hay-Davies sky model)	tilt, surface_azimuth, zenith, azimuth, dni, ghi, dhi, albedo, n	poa_global, poa_direct, poa_sky_diffuse, poa_ground_diffuse, aoi_deg
energy_poa_reindl	Irradiance on a tilted plane (Reindl sky model)	tilt, surface_azimuth, zenith, azimuth, dni, ghi, dhi, albedo, n	poa_global, poa_direct, poa_sky_diffuse, poa_ground_diffuse, aoi_deg
energy_erbs_decomposition	Diffuse and direct irradiance from global horizontal irradiance (Erbs)	ghi, zenith, n	kt, diffuse_fraction, dhi, dni
energy_clear_sky_haurwitz	Clear-sky global horizontal irradiance (Haurwitz)	zenith	ghi
energy_cell_temperature_noct	PV cell temperature from NOCT	poa, T_air, noct	T_cell, T_cell_C
energy_cell_temperature_faiman	PV module temperature (Faiman model)	poa, T_air, wind, u0, u1	T_cell, T_cell_C
energy_cell_temperature_sapm	PV cell and module temperature (Sandia SAPM model)	poa, T_air, wind, mount	T_module_C, T_cell_C, T_cell

id	operation	inputs	outputs
energy_pvwatts_dc	PV array DC power with temperature derating (PVWatts)	G_eff, T_cell, pdc0, gamma	pdc, pdc_kW
energy_pv_array_output	PV array output at operating temperature	poa, T_air, noct, pdc0, gamma	T_cell, T_cell_C, pdc, derate
energy_pvwatts_inverter	Inverter AC output and efficiency (PVWatts)	pdc, pdc0, eta_nom, eta_ref	pac, efficiency, clipped
energy_iam_ashrae	Incidence angle modifier (ASHRAE)	aoi, b0	iam
energy_iam_physical	Incidence angle modifier from Fresnel reflection and absorption	aoi, n, K, L	iam
energy_pvwatts_losses	Combined PV system losses (PVWatts)	soiling, shading, snow, mismatch, wiring, connections, lid, nameplate_rating, age, availability	total_loss, performance_factor, total_loss_pct
energy_wind_power	Power in the wind and the Betz limit	rho, D, v, cp	P_wind, P_betz, power_density, area, P
energy_actuator_disc	Actuator-disc (momentum theory) rotor performance	a, rho, D, v	Cp, Ct, v_disc, v_wake, P, T
energy_wind_shear_power_law	Wind speed at another height (power law)	v1, h1, h2, alpha	v2
energy_wind_shear_log	Wind speed at another height (logarithmic profile)	v1, h1, h2, z0, d	v2
energy_hub_air_density	Air density at hub height	p, h_p, T, h_T, hub, model	T_hub, p_hub, rho
energy_power_curve_output	Turbine output from its power curve	speeds, powers, v	P_kW, P
energy_power_curve_density_corrected	Turbine output from a density-corrected power curve	speeds, powers, v, rho	P_kW, P
energy_weibull_wind	Weibull wind statistics and	k, c, rho	v_mean, power_density, energy_pattern_factor,

id	operation	inputs	outputs
	mean power density		v_most_probable, v_max_energy
energy_weibull_aep	Annual energy production from a power curve and a Weibull distribution	speeds, powers, k, c, hours	mean_power_kW, AEP_MWh, capacity_factor
energy_three_phase_power	Balanced three-phase power from line quantities	VL, IL, pf, connection, pf_type	P, Q, S, V_phase, I_phase
energy_per_unit_change_of_base	Per-unit impedance on a new base	R_pu, X_pu, S_old, V_old, S_new, V_new	R_pu_new, X_pu_new, Z_pu_new
energy_short_line	Short transmission line: sending-end voltage, regulation and efficiency	Vr, P, pf, R, X, pf_type	Vs, delta_deg, I, losses, Ps, regulation_pct, efficiency_pct
energy_medium_line	Medium transmission line (nominal pi)	Vr, P, pf, R, X, B, pf_type	Vs, delta_deg, Is, Ps, losses, regulation_pct, efficiency_pct, A_mag, A_angle_deg, B_mag, B_angle_deg
energy_long_line	Long transmission line (exact distributed parameters)	Vr, P, pf, R, X, B, pf_type	Vs, delta_deg, Is, Ps, losses, regulation_pct, efficiency_pct, A_mag, A_angle_deg, B_mag, B_angle_deg, Zc_mag, Zc_angle_deg, alpha_l, beta_l
energy_line_parameters	Inductance, capacitance and surge impedance of a three-phase overhead line	D_ab, D_bc, D_ca, r, f, gmr, n_bundle, spacing, V	GMR, L_mH_per_km, C_nF_per_km, X_ohm_per_km, B_uS_per_km, Zc, SIL_MW
energy_two_bus_load_flow	Receiving-end voltage of a line feeding a constant-power load (two-bus load flow)	Vs, R, X, P, Q	Vr, delta_deg, I, P_loss, Q_loss, voltage_drop_pct
energy_dc_power_flow	DC power flow	lines, P	theta_deg, flows, P_slack
energy_newton_load_flow	AC load flow by Newton-Raphson	lines, loads, gens, V1	V, angle_deg, P_slack, Q_slack, P_loss, Q_gen
energy_economic_dispatch	Economic dispatch with quadratic costs and limits	costs, demand	lambda_, P, cost
energy_symmetrical_fault	Three-phase (symmetrical) fault current by the bus	network, fault_bus, Vf, Xf, S_base, V_base	If, X_th, V, If_A

id	operation	inputs	outputs
	impedance matrix		
energy_slg_fault	Single line-to-ground fault (symmetrical components)	X1, X2, X0, Rf, Xf, Vf	Ia, Ib, Ic, In, I1, I2, I0, Va, Vb, Vc
energy_ll_fault	Line-to-line fault (symmetrical components)	X1, X2, X0, Rf, Xf, Vf	Ia, Ib, Ic, In, I1, I2, I0, Va, Vb, Vc
energy_dlg_fault	Double line-to-ground fault (symmetrical components)	X1, X2, X0, Rf, Xf, Vf	Ia, Ib, Ic, In, I1, I2, I0, Va, Vb, Vc
energy_power_angle	Power transfer and power angle between two voltages	V1, V2, X, P	delta_deg, Pmax, Q_sent, Q_received, margin_pct
energy_critical_clearing	Critical clearing angle and time (equal-area criterion)	Pm, Pmax_pre, Pmax_fault, Pmax_post, H, f	delta0_deg, delta_max_deg, delta_cr_deg, t_cr
energy_droop_sharing	Frequency deviation and load sharing under governor droop	R, ratings, dP, f0, D	df, f, dP_gen, dP_load_damping
energy_transformer_performance	Transformer regulation and efficiency from its equivalent circuit	S, V1, V2, Req, Xeq, Rc, Xm, load, pf, pf_type	V1_required, regulation_pct, I1, pf_in, P_out, P_cu, P_core, efficiency_pct
energy_transformer_tests	Transformer parameters from open-circuit and short-circuit tests	Voc, Ioc, Poc, Vsc, Isc, Psc	Rc, Xm, Req, Xeq, Zeq, pf_oc, pf_sc
energy_transformer_max_efficiency	Load for maximum transformer efficiency	S, P_core, P_cu_fl, pf	load_fraction, S_at_max, efficiency_max_pct, efficiency_full_load_pct
energy_transformer_loading	Transformer under load from nameplate data	sn, V_hv, V_lv, vk, vkr, P0, i0, P, Q	V_lv_actual, V_lv_pu, delta_deg, P_loss, Q_loss, efficiency_pct, loading_pct
energy_parallel_transformers	Load sharing between two transformers in parallel	S1, vk1, vkr1, S2, vk2, vkr2, V_lv, P, Q	S1_share, S2_share, P1, Q1, P2, Q2, share1_pct_of_rating, share2_pct_of_rating
energy_induction_motor	Induction motor performance from its equivalent circuit	VL, f, poles, R1, X1, R2, X2, Xm, slip, connection, P_rot	I_line, I2, pf, P_in, P_ag, P_conv, T, P_out, efficiency_pct

id	operation	inputs	outputs
energy_induction_max_torque	Induction motor breakdown (maximum) torque and starting torque	VL, f, poles, R1, X1, R2, X2, Xm, connection	s_maxT, T_max, T_start, I_start, V_th, R_th, X_th
energy_synchronous_generator	Synchronous generator internal EMF, power angle and regulation	VL, IL, pf, Xs, Ra, pf_type	E_line, E_phase, delta_deg, regulation_pct, P, Q
energy_peukert	Peukert exponent and run time at a new discharge current	currents, times, I	k, C_p, runtime_h, capacity_Ah
energy_battery_pack_sizing	Battery pack series-parallel sizing	V_target, E_target, V_cell, Q_cell	N_series, N_parallel, N_cells, V_pack, E_pack_kWh, Q_pack_Ah
energy_battery_rc_discharge	Battery terminal voltage under constant current (one-RC equivalent circuit)	OCV, R0, R1, C1, I, t	V_terminal, V_rc, E_delivered, E_Wh, heat, tau
energy_fuel_cell_stack	Hydrogen fuel-cell stack: consumption, power and efficiency	I, N, V_cell, utilization	H2_kg_s, H2_g_h, H2_fed_kg_s, O2_kg_s, H2O_kg_s, efficiency_LHV, efficiency_HHV, heat_LHV
energy_electrolyser	Water electrolyser: hydrogen production and energy per kilogram	I, N, V_cell, faradaic	H2_kg_s, H2_kg_h, O2_kg_s, H2O_kg_s, specific_energy_kWh_per_kg, efficiency_HHV, efficiency_LHV
energy_hydrogen_tank	Hydrogen stored in a tank at pressure (real gas)	V, p, T	rho, Z, mass, rho_ideal, mass_ideal, ideal_over_real, energy_LHV_kWh
energy_hydrogen_isothermal_compression	Minimum (isothermal) work to compress hydrogen	p1, p2, T	w, w_kWh_per_kg, w_ideal, real_over_ideal
energy_hydrogen_isentropic_compression	Isentropic (adiabatic) compression of hydrogen	p1, T1, p2	T2, w, w_kWh_per_kg
energy_single_diode_current	PV single-diode model: current at a voltage	V, IL, I0, Rs, Rsh, nNsVth	I, P
energy_single_diode_voltage	PV single-diode model: voltage at a current	I, IL, I0, Rs, Rsh, nNsVth	V, P

id	operation	inputs	outputs
energy_single_diode_mpp	PV single-diode model: short-circuit, open-circuit and maximum power point	IL, I0, Rs, Rsh, nNsVth	Isc, Voc, Imp, Vmp, Pmp, FF
energy_desoto_parameters	PV module single-diode parameters at operating conditions (De Soto)	G, T_cell, alpha_sc, a_ref, IL_ref, I0_ref, Rsh_ref, Rs, EgRef, dEgdT	IL, I0, Rs, Rsh, nNsVth
energy_pv_module_at_conditions	PV module maximum power at an irradiance and temperature	G, T_cell, alpha_sc, a_ref, IL_ref, I0_ref, Rsh_ref, Rs, EgRef, dEgdT	Isc, Voc, Imp, Vmp, Pmp, FF
energy_poa_perez	Irradiance on a tilted plane (Perez sky model)	tilt, surface_azimuth, zenith, azimuth, dni, ghi, dhi, albedo, n	poa_global, poa_direct, poa_sky_diffuse, poa_ground_diffuse, aoi_deg
energy_clear_sky_ineichen	Clear-sky irradiance (Ineichen-Perez with Linke turbidity)	zenith, linke_turbidity, altitude, n	ghi, dni, dhi, AM_absolute
energy_disc_dni	Direct normal irradiance from global horizontal irradiance (DISC model)	ghi, zenith, n, pressure	dni, kt, airmass
energy_per_unit_impedance	Base impedance and per-unit impedance	R, X, S_base, V_base	Z_base, I_base, R_pu, X_pu, Z_pu
energy_geometric_mean_distance	Geometric mean distance of a three-phase line	D_ab, D_bc, D_ca	GMD, ratio_to_mean
energy_synchronous_speed	Synchronous speed, rotor speed and slip of an AC machine	f, poles, s, n_r	n_sync_rpm, omega_sync, n_rotor_rpm, slip, slip_frequency
energy_stack_electrical_power	Electrical power of a fuel-cell or electrolyser stack	I, N, V_cell	P, P_kW, V_stack
energy_coulomb_counting	State of charge by coulomb counting	SOC0, Q, currents, dt	SOC, SOC_min, SOC_max, charge_out, charge_out_Ah
energy_hydro_power	Hydroelectric power from flow and head, and the energy over a period	Q, H, eta, rho, g, CF, hours	P, P_kW, P_MW, P_hydraulic, E, E_MWh

id	operation	inputs	outputs
energy_capacity_factor	Capacity factor and energy of a generating plant	P_rated, E, CF, hours	capacity_factor, capacity_factor_pct, E, E_MWh, full_load_hours, average_power
energy_induction_power_flow	Induction motor power flow from slip and losses	f, poles, P_in, P_scl, s, n_r, P_core, P_fw, E2	n_sync_rpm, n_rotor_rpm, slip, f_rotor, P_ag, P_rcl, P_conv, P_out, T_ind, T_shaft, efficiency_pct, E_rotor
energy_butler_volmer	Electrode kinetics: Butler-Volmer current, overpotential and Tafel slopes	j0, alpha_a, alpha_c, T, eta, j	j, j_mA_cm2, eta, eta_mV, j_anodic, j_cathodic, b_a, b_c, b_a_mV, b_c_mV, R_ct, R_ct_ohm_cm2, eta_tafel

Solar declination on a day of the year, `energy_solar_declination`

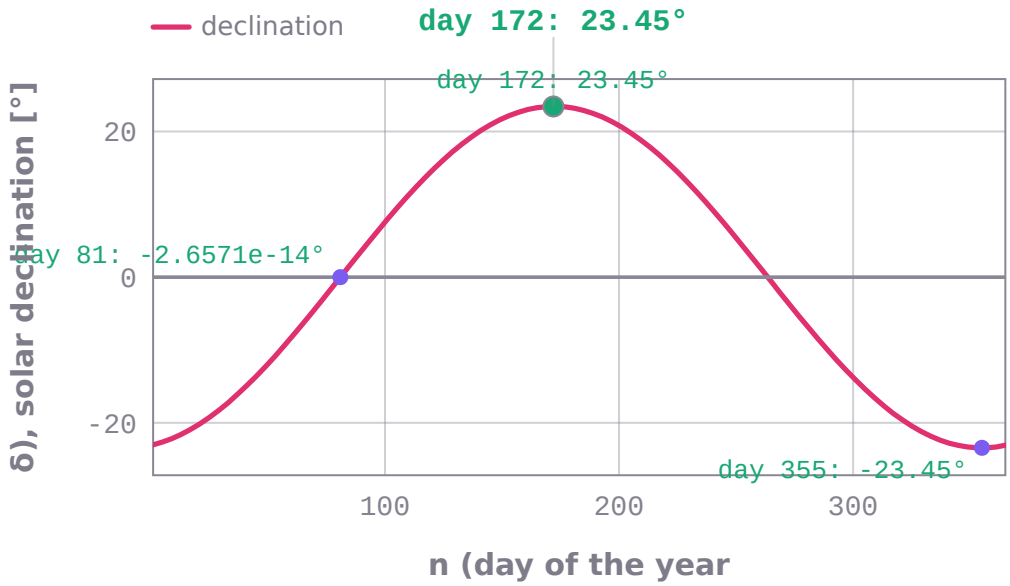
The declination of the sun (the latitude where the sun is overhead at noon) on day n of the year, by Cooper's equation or Spencer's Fourier series.

$$\delta = 23.45^\circ \sin\left(\frac{360^\circ (284 + n)}{365}\right)$$

input	meaning	condition
`n`	day of the year (1 = 1 January)	
`method`	formula: cooper (Cooper 1969, the textbook default) or spencer (Spencer 1971)	

output	meaning
`declination_deg`	declination in degrees
`declination_rad`	declination in radians

- **Method:** Cooper: $\delta = 23.45$ sin(360 (284 + n)/365) degrees; Spencer: a Fourier series in the day angle $B = 2$ pi $(n - 1)/365$
- **Assumes:** $1 \le n \le 366$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The Sun's declination through the year, one run of the package every four days (its default formula). Dots: days 81, 172 and 355 with their returned declinations. Marked: the worked example, day 172, declination 23.45°.

```
energy_systems_ai999x.solve('energy_solar_declination', n=172)
# -> declination_deg = 23.4498, declination_rad = 0.409276   (SOLVED)
```

```
energy_systems_ai999x.solve('energy_solar_declination', n=1, method='spencer')
# -> declination_deg = -23.0586, declination_rad = -0.402449   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Solar declination: n = 172	SOLVED	declination_deg = 23.4498, declination_rad = 0.409276
Solar declination: day = 1, method = spencer	SOLVED	declination_deg = -23.0586, declination_rad = -0.402449
Solar declination: n = 400	INVALID_INPUT	n: must be at most 366, got 400.
Solar declination: n = 172, latitude = 40	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: energy_systems_ai999x.templates()
Solar declination:	NEEDS_INPUT	energy_solar_declination needs n (day of the year (1 = 1 January)). Example: solve('energy_solar_declination', n=172).

Equation of time, `energy_equation_of_time`

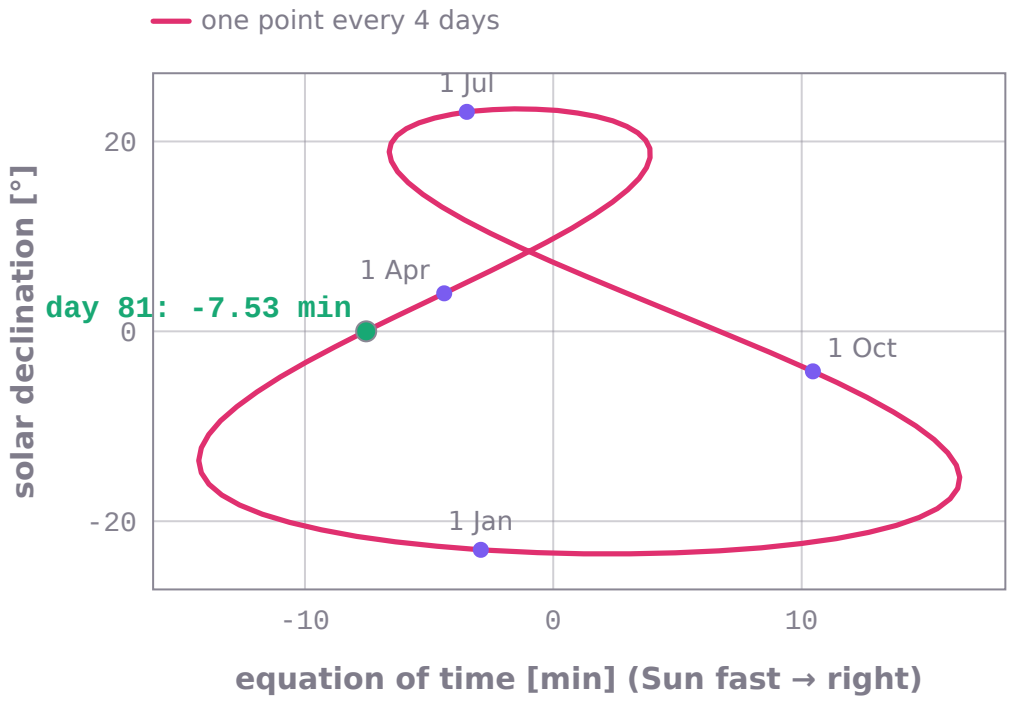
The equation of time (apparent minus mean solar time) on day n, by Spencer's Fourier series or the PVCDROM formula 9.87 sin $2B - 7.53$ cos $B - 1.5$ sin B.

$$E = 229.18\,(0.000075 + 0.001868\cos B - 0.032077\sin B - 0.014615\cos 2B - 0.040849\sin 2B)$$

input	meaning	condition
`n`	day of the year (1 = 1 January)	
`method`	formula: spencer (default) or pvcdrom (B = 360 (n - 81)/365)	

output	meaning
`eot_min`	equation of time in minutes
`eot_s`	equation of time in seconds

- **Method:** Spencer: $E = 229.18$ (0.000075 + 0.001868 cos $B - 0.032077$ sin $B - 0.014615$ cos $2B - 0.040849$ sin 2B), $B = 2$ pi $(n - 1)/365$
- **Assumes:** $1 \leq n \leq 366$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The analemma: the package's equation of time (horizontal, minutes) against its solar declination (vertical) for every fourth day of the year, where the Sun stands at clock noon through the year. Marked: the worked example, day 81, equation of time -7.53 min at declination -2.6571e-14°.

```
energy_systems_ai999x.solve('energy_equation_of_time', n=81, method='pvcdrom')
# -> eot_min = -7.53, eot_s = -451.8   (SOLVED)
```

```
energy_systems_ai999x.solve('energy_equation_of_time', n=1)
# -> eot_min = -2.91968, eot_s = -175.181   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Equation of time: n = 81, method = pvcdrom	SOLVED	eot_min = -7.53, eot_s = -451.8
Equation of time: n = 0	INVALID_INPUT	n: must be at least 1, got 0.

Solar time and hour angle from clock time, `energy_solar_time_hour_angle`

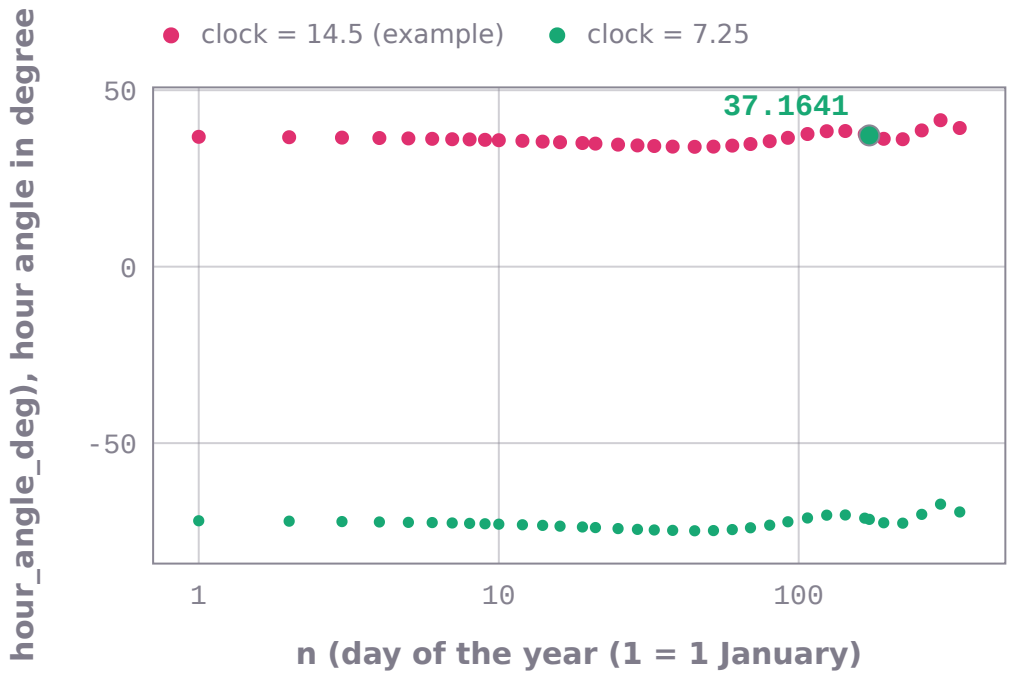
Local apparent solar time and the hour angle of the sun from the local clock time, the longitude and the time zone (UTC offset), with the equation of time (Spencer).

$$\omega = 15^\circ\,(t - \Delta t_{UTC} - 12) + \lambda + \frac{E}{4}$$

input	meaning	condition
`n`	day of the year (1 = 1 January)	
`clock`	local clock (standard) time in decimal hours or hh:mm	
`longitude`	longitude in degrees, east positive	
`utc_offset`	time zone as hours from UTC (e.g. -5, 1, 5.5)	

output	meaning
`eot_min`	equation of time in minutes
`hour_angle_deg`	hour angle in degrees (negative before solar noon)
`solar_time_h`	apparent solar time in decimal hours

- **Method:** $\omega = 15$ (t_clock - UTC offset - 12) + longitude + E/4 (degrees); solar time = 12 + omega/15
- **Assumes:** $1 \le n \le 366$; 0 <= clock < 24; -180 <= longitude <= 180
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



hour_angle_deg (hour angle in degrees (negative before solar noon)) against n (day of the year (1 = 1 January)) from 1 to 344 on a logarithmic axis. One curve for each value of clock (local clock (standard) time in decimal hours or hh:mm): 14.5, 7.25. Other inputs as in the worked example: longitude = -75, utc_offset = -5. 35 points per curve computed by energy-systems-ai999x; marked: the worked example, n = 172, hour_angle_deg = 37.1641.

```
energy_systems_ai999x.solve('energy_solar_time_hour_angle', n=172, clock=14.5,
longitude=-75, utc_offset=-5)
# -> eot_min = -1.34372, hour_angle_deg = 37.1641, solar_time_h = 14.4776   (SOLVED)
```

```
energy_systems_ai999x.solve('energy_solar_time_hour_angle', n=81, clock=12.0, longitude=15,
utc_offset=1)
# -> eot_min = -7.56464, hour_angle_deg = -1.89116, solar_time_h = 11.8739   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Hour angle: n = 172, clock = 14:30, longitude = -75, utc_offset = -5	SOLVED	eot_min = -1.34372, hour_angle_deg = 37.1641, solar_time_h = 14.4776
Solar time: n = 81, time = 12, longitude = 15 deg, utc = 1	SOLVED	eot_min = -7.56464, hour_angle_deg = -1.89116, solar_time_h = 11.8739
Hour angle: n = 172, clock = 25, longitude = -75, utc_offset = -5	INVALID_INPUT	Solar time and hour angle from clock time: the clock time must be from 0 to 24 hours (decimal hours or hh:mm).
Hour angle: n = 172, clock = 14:30, longitude = -75	NEEDS_INPUT	energy_solar_time_hour_angle needs utc_offset (time zone as hours from UTC (e.g. -5, 1, 5.5)). Example: solve('energy_so

Solar zenith, elevation and azimuth from latitude, declination and hour angle, `energy_sun_angles`

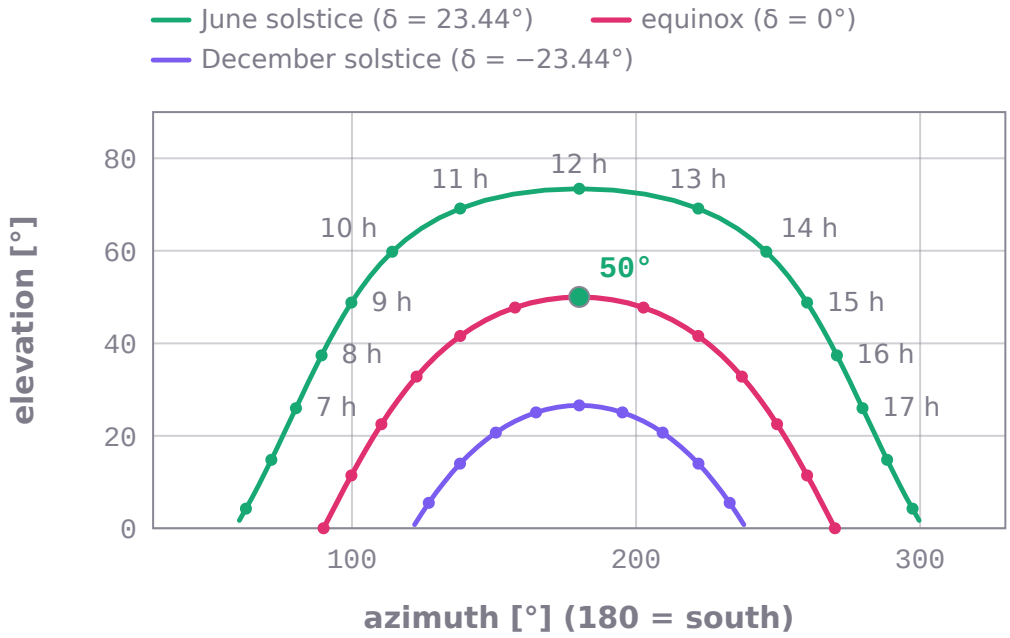
The sun's zenith angle, elevation (altitude) and azimuth (clockwise from north) at a latitude for a given declination and hour angle.

$$\cos\theta_z = \cos\phi\cos\delta\cos\omega + \sin\phi\sin\delta$$

input	meaning	condition
`latitude`	latitude in degrees (north positive)	
`declination`	solar declination in degrees	
`hour_angle`	hour angle in degrees (negative in the morning)	

output	meaning
`zenith_deg`	solar zenith angle in degrees
`elevation_deg`	solar elevation (altitude) in degrees
`azimuth_deg`	solar azimuth in degrees clockwise from north

- **Method:** cos $z = cos$(delta) cos(phi) cos(omega) + sin(delta) sin(phi); cos(azimuth - 180) = (cos z sin phi - sin delta)/(sin z cos phi), sign of omega
- **Assumes:** -90 < latitude < 90; |declination| <= 23.5; |hour angle| <= 180
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Sun-path chart at latitude 40° N: elevation against azimuth on three days, each point a run of the package for an hour angle every 15 minutes (dots hourly, labelled in solar time on the June curve). Marked: the worked example, declination 0° at hour angle 0°, elevation 50°, azimuth 180°, zenith 40°.

```
energy_systems_ai999x.solve('energy_sun_angles', latitude=40, declination=0, hour_angle=0)
# -> zenith_deg = 40, elevation_deg = 50, azimuth_deg = 180   (SOLVED)
```

```
energy_systems_ai999x.solve('energy_sun_angles', latitude=0, declination=0, hour_angle=-90)
# -> zenith_deg = 90, elevation_deg = 0, azimuth_deg = 90   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Sun angles: latitude = 40, declination = 0, hour_angle = 0	SOLVED	zenith_deg = 40, elevation_deg = 50, azimuth_deg = 180
Solar elevation and azimuth: lat = 0 deg, delta = 0, omega = -90	SOLVED	zenith_deg = 90, elevation_deg = 0, azimuth_deg = 90
Sun angles: latitude = 40, declination = 30, hour_angle = 0	INVALID_INPUT	Solar zenith, elevation and azimuth from latitude, declination and hour angle: the declination must lie between -23.5 an

Sun position at a place and local time, `energy_solar_position`

Declination, equation of time, hour angle, zenith, elevation and azimuth of the sun at a latitude and longitude on day n at a local clock time (Spencer declination and equation of time).

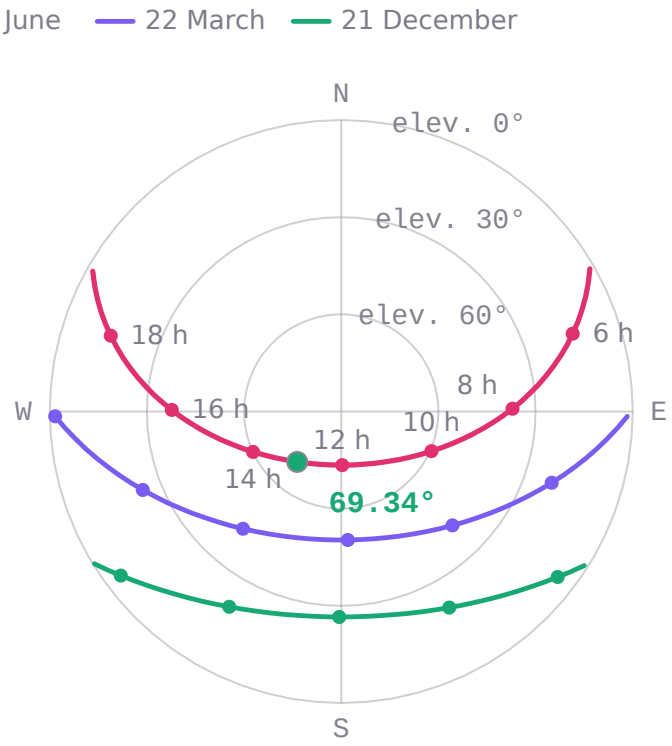
$$\cos\theta_z = \cos\phi\cos\delta\cos\omega + \sin\phi\sin\delta, \quad \omega = 15^\circ(t_s - 12)$$

input	meaning	condition
`latitude`	latitude in degrees (north positive)	
`longitude`	longitude in degrees, east positive	
`n`	day of the year (1 = 1 January)	

input	meaning	condition
`clock`	local clock (standard) time in decimal hours or hh:mm	
`utc_offset`	time zone as hours from UTC	

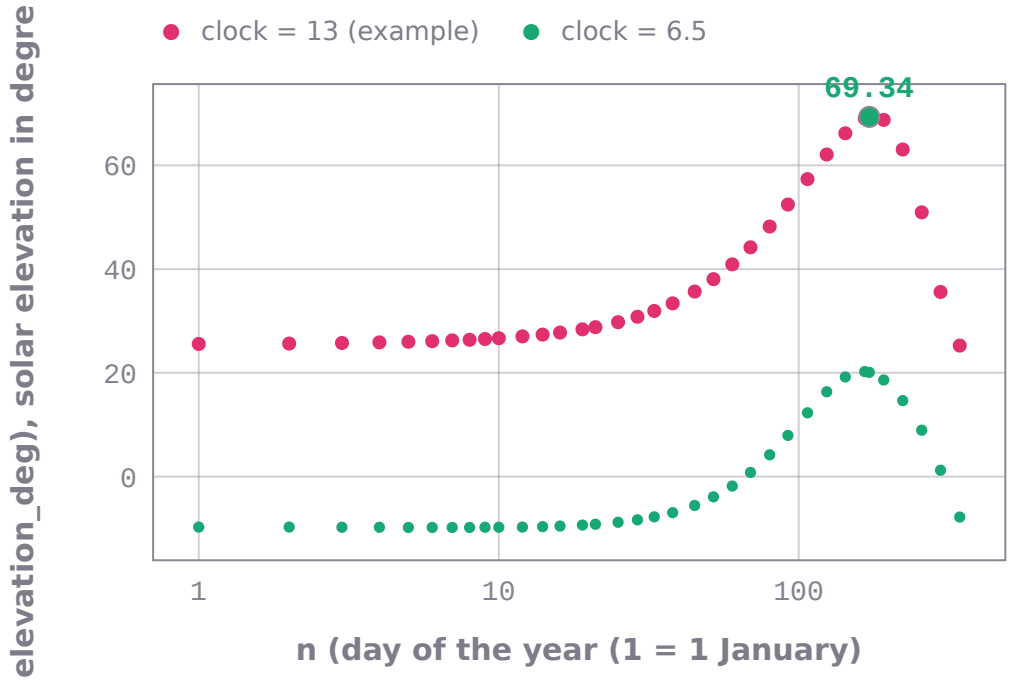
output	meaning
`declination_deg`	declination in degrees
`eot_min`	equation of time in minutes
`hour_angle_deg`	hour angle in degrees
`zenith_deg`	solar zenith angle in degrees
`elevation_deg`	solar elevation in degrees
`azimuth_deg`	solar azimuth in degrees clockwise from north

- **Method:** Spencer declination and equation of time; $\omega = 15$ (t - UTC offset - 12) + longitude + E/4; cos $z = cos$(delta) cos(phi) cos(omega) + sin(delta) sin(phi)
- **Assumes:** $1 \leq n \leq 366$; 0 <= clock < 24; -90 < latitude < 90
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



sky chart: centre = overhead, rim = horizon

The Sun's path across the sky at latitude 40°, longitude -105° (UTC-7) on three days: azimuth around the rim (N at the top) and elevation inward (rings at 0°, 30° and 60°), computed by the package every 15 minutes while the Sun is up; dots every 2 h of clock time. Highest computed elevations: 21 June 73.4496°, 22 March 50.2928°, 21 December 26.5781°. Marked: 21 June at 13 h, elevation 69.34°, azimuth 221.165°.



elevation_deg (solar elevation in degrees) against n (day of the year (1 = 1 January)) from 1 to 344 on a logarithmic axis. One curve for each value of clock (local clock (standard) time in decimal hours or hh:mm): 13, 6.5. Other inputs as in the worked example: latitude = 40, longitude = -105, utc_offset = -7. 35 points per curve computed by energy-systems-ai999x; marked: the worked example, n = 172, elevation_deg = 69.34.

```
energy_systems_ai999x.solve('energy_solar_position', latitude=40, longitude=-105, n=172,
clock=13.0, utc_offset=-7)
# -> declination_deg = 23.452, eot_min = -1.34372, hour_angle_deg = 14.6641, zenith_deg =
20.66, elevation_deg = 69.34, azimuth_deg = 221.165   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Solar position: latitude = 40, longitude = -105, n = 172, clock = 13:00, utc_offset = -7	SOLVED	declination_deg = 23.452, eot_min = -1.34372, hour_angle_deg = 14.6641, zenith_deg = 20.66, elevation_deg = 69.34, azimuth_deg = 221.165
Solar position: latitude = 95, longitude = -105, n = 172, clock = 13:00, utc_offset = -7	INVALID_INPUT	Sun position at a place and local time: the latitude must lie between -90 and 90 degrees.

Sunrise, sunset, solar noon and day length, `energy_sunrise_sunset`

Geometric sunrise and sunset times (local clock time), solar noon, the day length and the sunset hour angle at a place on day n (no refraction, sun centre on the horizon).

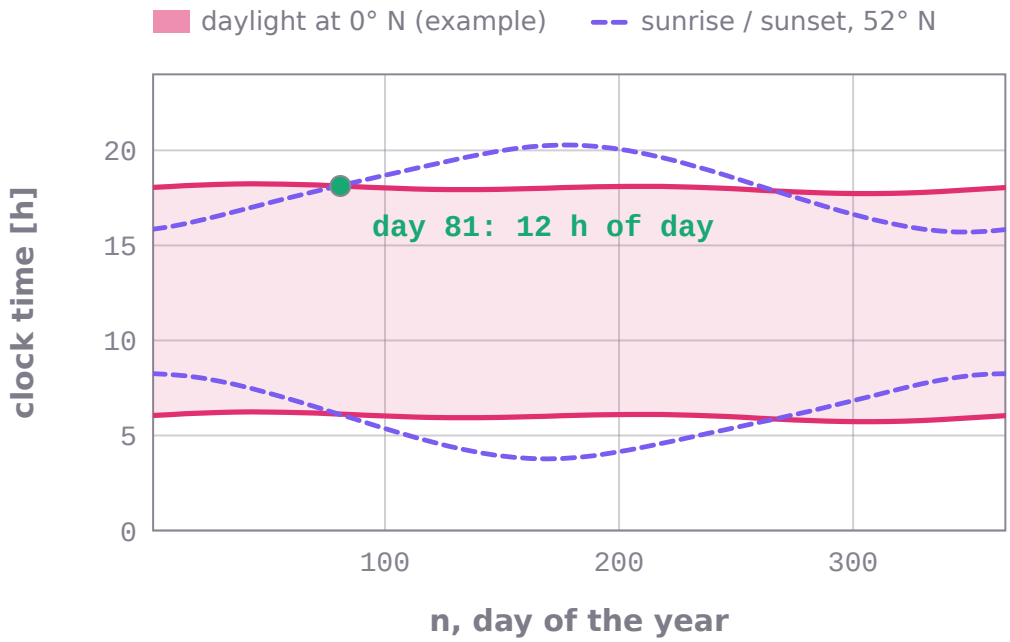
$$\cos\omega_s = -\tan\phi\tan\delta, \quad N = \frac{2\,\omega_s}{15^\circ}$$

input	meaning	condition
`latitude`	latitude in degrees (north positive)	
`longitude`	longitude in degrees, east positive	
`n`	day of the year (1 = 1 January)	

input	meaning	condition
`utc_offset`	time zone as hours from UTC	

output	meaning
`sunrise_h`	sunrise, local clock time in decimal hours
`sunset_h`	sunset, local clock time in decimal hours
`solar_noon_h`	solar noon, local clock time in decimal hours
`day_length_h`	day length in hours
`sunset_hour_angle_deg`	sunset hour angle omega_s in degrees

- **Method:** omega_s = acos(-tan(phi) tan(delta)); t = (omega - longitude - E/4)/15 + 12 + UTC offset; day length = 2 $omega_s/15$
- **Assumes:** the sun rises and sets on that day (no polar day or night)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Sunrise and sunset in clock hours (UTC+0) through the year at longitude 0°, at the worked example's latitude 0° N (shaded daylight) and at 52° N (dashed), one run of the package per week. Day lengths returned on days 172 and 355: 0° N 12 and 12 h; 52° N 16.5 and 7.511 h. The wobble of both edges is the equation of time. Marked: the worked example, day 81: sunrise 6.126 h, sunset 18.13 h, 12 h of daylight.

```
energy_systems_ai999x.solve('energy_sunrise_sunset', latitude=0, longitude=0, n=81,
utc_offset=0)
# -> sunrise_h = 6.12608, sunset_h = 18.1261, solar_noon_h = 12.1261, day_length_h = 12,
sunset_hour_angle_deg = 90   (SOLVED)
```

```
energy_systems_ai999x.solve('energy_sunrise_sunset', latitude=40, longitude=0, n=172,
utc_offset=0)
# -> sunrise_h = 4.59926, sunset_h = 19.4455, solar_noon_h = 12.0224, day_length_h =
14.8463, sunset_hour_angle_deg = 111.347   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Day length: latitude = 0, longitude = 0, n = 81, utc_offset = 0	SOLVED	sunrise_h = 6.12608, sunset_h = 18.1261, solar_noon_h = 12.1261, day_length_h = 12, sunset_hour_angle_deg = 90
Sunrise and sunset: latitude = 80, longitude = 0, n = 172, utc_offset = 0	INVALID_INPUT	Sunrise, sunset, solar noon and day length: the sun does not rise or does not set on this day at this latitude (polar da

Extraterrestrial (top-of-atmosphere) normal irradiance, `energy_extraterrestrial_irradiance`

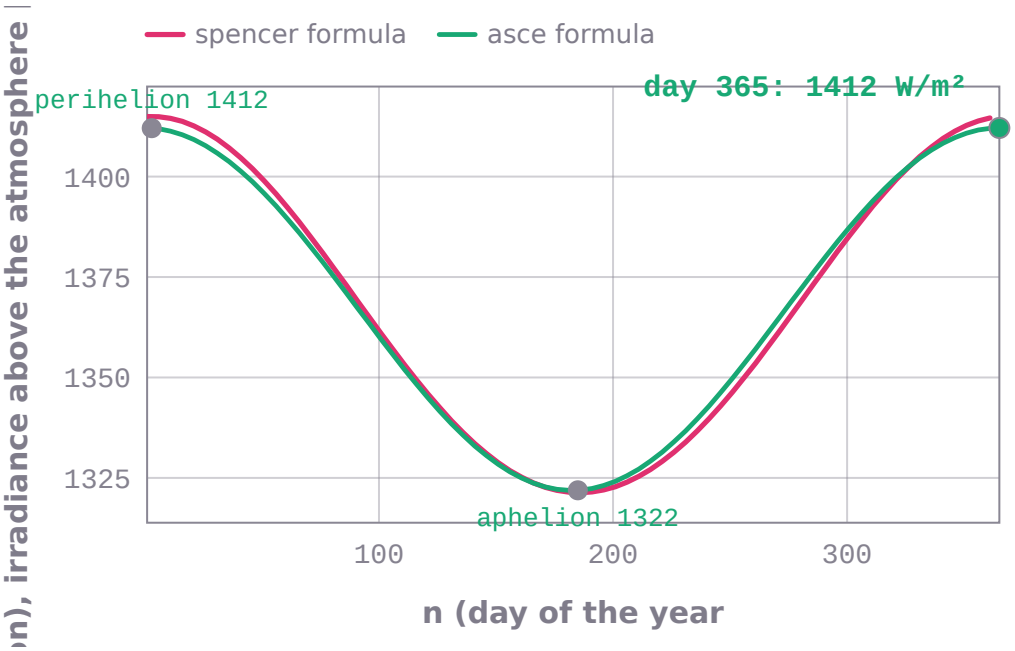
The solar irradiance on a plane normal to the sun's rays outside the atmosphere on day n, and the Earth-Sun distance factor $(R_0/R)^2$.

$$G_{on} = G_{sc}\left(1 + 0.033\cos\frac{360°n}{365}\right)$$

input	meaning	condition
`n`	day of the year (1 = 1 January)	
`method`	formula: spencer (default, Fourier series) or asce (1 + 0.033 cos(360 n/365))	
`solar_constant`	solar constant (default 1366.1 W/m^2)	

output	meaning
`G_on`	extraterrestrial normal irradiance
`distance_factor`	Earth-Sun distance factor $(R_0/R)^2$

- **Method:** $G_on = G_sc$ (1.00011 + 0.034221 cos $B + 0.00128$ sin $B + 0.000719$ cos $2B + 0.000077$ sin 2B) (Spencer) or G_sc (1 + 0.033 cos(360 n/365))
- **Assumes:** $1 \le n \le 366$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Solar irradiance on a plane normal to the Sun above the atmosphere through the year for a solar constant of 1367 W/m² (the worked example's), each point a run of the package (two of its formulas). Dots: day 3, 1412 W/m² (distance factor 1.033), and day 185, 1322 W/m². Marked: the worked example, day 365 (asce formula), 1412 W/m².

```
energy_systems_ai999x.solve('energy_extraterrestrial_irradiance', n=365, method='asce',
solar_constant=1367.0)
# -> G_on = 1412.11 W/m^2, distance_factor = 1.033   (SOLVED)
```

```
energy_systems_ai999x.solve('energy_extraterrestrial_irradiance', n=1)
# -> G_on = 1413.98 W/m^2, distance_factor = 1.03505   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Extraterrestrial irradiance: n = 365, method = asce, solar_constant = 1367 W/m^2	SOLVED	G_on = 1412.11 W/m^2, distance_factor = 1.033
Extraterrestrial irradiance: n = 1	SOLVED	G_on = 1413.98 W/m^2, distance_factor = 1.03505
Extraterrestrial irradiance: n = 1, solar_constant = 1367	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: energy_systems_ai999x.templates()

Optical air mass, `energy_air_mass`

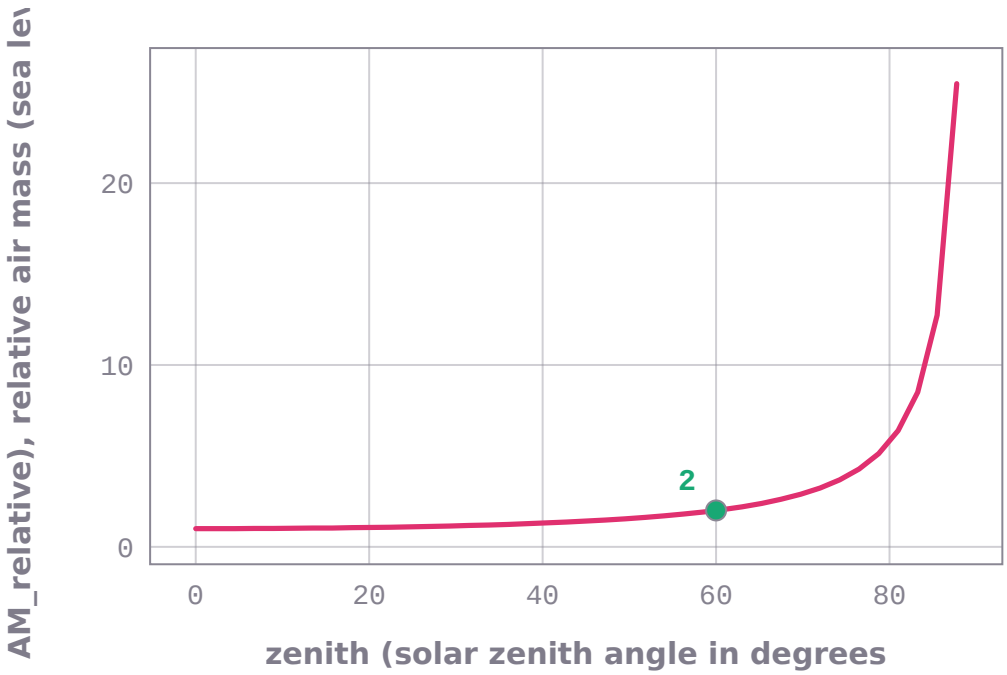
The relative optical air mass for a solar zenith angle (Kasten-Young by default, or another named model) and the pressure-corrected absolute air mass.

$$AM = \frac{1}{\cos z + 0.50572\,(96.07995^\circ - z)^{-1.6364}}$$

input	meaning	condition
`zenith`	solar zenith angle in degrees (apparent zenith for the refraction-based models)	
`model`	air-mass model (default kastenyoung1989)	
`pressure`	site pressure (default 101325 Pa)	

output	meaning
`AM_relative`	relative air mass (sea level)
`AM_absolute`	absolute air mass AM_rel $p/101325$ Pa

- **Method:** Kasten-Young: AM = 1/(cos $z + 0.50572$ (96.07995 - z)^-1.6364); absolute $AM = AM\ p/p_0$
- **Assumes:** 0 <= zenith < 90
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



AM_relative (relative air mass (sea level)) against zenith (solar zenith angle in degrees (apparent zenith for the refraction-based models)) from 0 to 90. Other inputs as in the worked example: model = simple. 40 points computed by energy-systems-ai999x; marked: the worked example, zenith = 60, AM_relative = 2.

```
energy_systems_ai999x.solve('energy_air_mass', zenith=60, model='simple')
# -> AM_relative = 2, AM_absolute = 2   (SOLVED)
```

```
energy_systems_ai999x.solve('energy_air_mass', zenith=0, model='simple', pressure=50662.5)
# -> AM_relative = 1, AM_absolute = 0.5   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Air mass: zenith = 60, model = simple	SOLVED	AM_relative = 2, AM_absolute = 2
Air mass: zenith = 0, model = simple, pressure = 506.625 hPa	SOLVED	AM_relative = 1, AM_absolute = 0.5
Air mass: zenith = 95	INVALID_INPUT	Optical air mass: the zenith angle must be from 0 up to (not including) 90 degrees.

Angle of incidence of the sun on a tilted surface, `energy_angle_of_incidence`

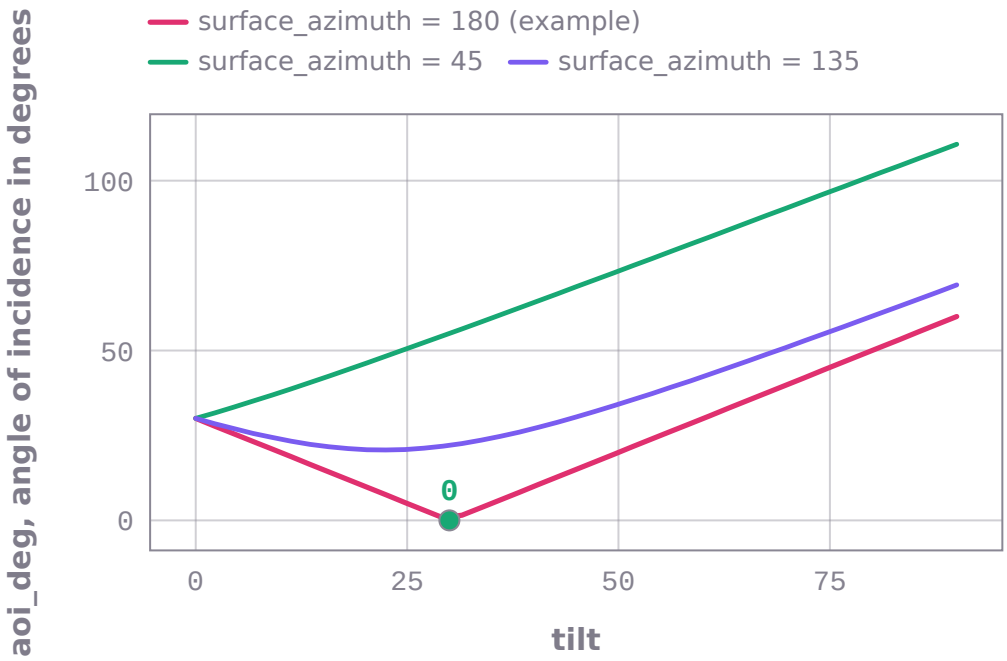
The angle between the sun's rays and the normal of a surface of given tilt and azimuth, and its cosine (the beam projection factor).

$$\cos\theta = \cos\beta\cos\theta_z + \sin\beta\sin\theta_z\cos(\gamma_s - \gamma)$$

input	meaning	condition
`tilt`	surface tilt from horizontal in degrees	
`surface_azimuth`	surface azimuth in degrees clockwise from north (south = 180)	
`zenith`	solar zenith angle in degrees	
`azimuth`	solar azimuth in degrees clockwise from north	

output	meaning
`aoi_deg`	angle of incidence in degrees
`cos_aoi`	cosine of the angle of incidence

- **Method:** cos(theta) = cos(beta) cos(z) + sin(beta) sin(z) cos(azimuth_sun - azimuth_surface)
- **Assumes:** 0 <= tilt <= 180; azimuths from 0 to 360 degrees
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



aoi_deg (angle of incidence in degrees) against tilt (surface tilt from horizontal in degrees) from 0 to 90. One curve for each value of surface_azimuth (surface azimuth in degrees clockwise from north (south = 180)): 180, 45, 135. Other inputs as in the worked example: zenith = 30, azimuth = 180. 41 points per curve computed by energy-systems-ai999x; marked: the worked example, tilt = 30, aoi_deg = 0.

```
energy_systems_ai999x.solve('energy_angle_of_incidence', tilt=30, surface_azimuth=180,
zenith=30, azimuth=180)
# -> aoi_deg = 0, cos_aoi = 1   (SOLVED)
```

```
energy_systems_ai999x.solve('energy_angle_of_incidence', tilt=0, surface_azimuth=180,
zenith=60, azimuth=100)
# -> aoi_deg = 60, cos_aoi = 0.5   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Angle of incidence: tilt = 0, surface_azimuth = 180, zenith = 60, azimuth = 100	SOLVED	aoi_deg = 60, cos_aoi = 0.5
Angle of incidence: tilt = 30, surface_azimuth = 400, zenith = 60, azimuth = 100	INVALID_INPUT	Angle of incidence of the sun on a tilted surface: the surface azimuth must lie between 0 and 360 degrees (clockwise fro

Irradiance on a tilted plane (isotropic sky), `energy_poa_isotropic`

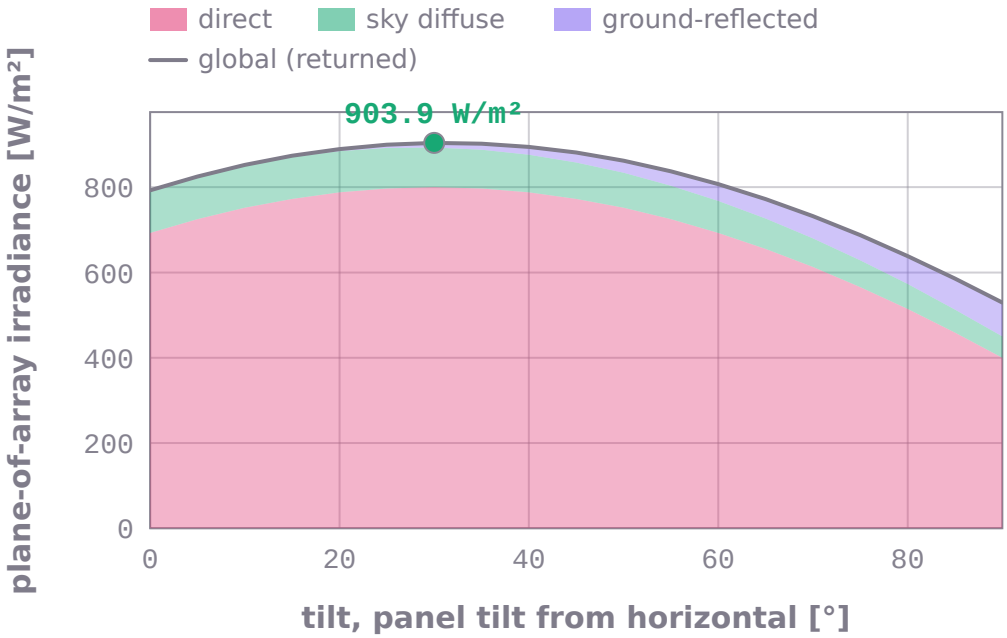
Beam, sky-diffuse (isotropic sky, Liu-Jordan/Hottel-Woertz) and ground-reflected irradiance on a tilted surface from DNI, GHI and DHI.

$$G_{poa} = G_{bn}\cos\theta + G_d\frac{1+\cos\beta}{2} + G\rho\frac{1-\cos\beta}{2}$$

input	meaning	condition
`tilt`	surface tilt from horizontal in degrees	
`surface_azimuth`	surface azimuth in degrees clockwise from north (south = 180)	
`zenith`	solar zenith angle in degrees	
`azimuth`	solar azimuth in degrees clockwise from north	
`dni`	direct normal irradiance	
`ghi`	global horizontal irradiance	
`dhi`	diffuse horizontal irradiance	
`albedo`	ground reflectance (0 to 1)	

output	meaning
`poa_global`	total irradiance on the plane
`poa_direct`	beam irradiance on the plane
`poa_sky_diffuse`	sky-diffuse irradiance on the plane
`poa_ground_diffuse`	ground-reflected irradiance on the plane
`aoi_deg`	angle of incidence in degrees

- **Method:** G_poa = DNI cos(theta) + DHI (1 + cos beta)/2 + GHI rho (1 - cos beta)/2
- **Assumes:** sun above the horizon; DHI <= GHI; 0 <= albedo <= 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Irradiance on a tilted panel split into its parts by the package (Irradiance on a tilted plane (isotropic sky)) for the worked example's sky (DNI 800, GHI 792.8, DHI 100 W/m², Sun at zenith 30°, azimuth 180°, panel facing 180°, albedo 0.2) at tilts from 0 to 90°, stacked: direct, sky diffuse and ground-reflected, the line their returned total. Marked: tilt 30°, direct 800, sky 93.3, ground 10.62, global 903.9 W/m² (angle of incidence 0°).

```
energy_systems_ai999x.solve('energy_poa_isotropic', tilt=30, surface_azimuth=180,
zenith=30, azimuth=180, dni=800, ghi=792.820323027551, dhi=100, albedo=0.2)
# -> poa_global = 903.923 W/m^2, poa_direct = 800 W/m^2, poa_sky_diffuse = 93.3013 W/m^2,
poa_ground_diffuse = 10.6218 W/m^2, aoi_deg = 0   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Plane of array irradiance (isotropic): tilt = 30, surface_azimuth = 180, zenith = 30, azimuth = 180, dni = 800 W/m^2, ghi = 792.820323027551 W/m^2, dhi = 100 W/m^2, albedo = 0.2	SOLVED	poa_global = 903.923 W/m^2, poa_direct = 800 W/m^2, poa_sky_diffuse = 93.3013 W/m^2, poa_ground_diffuse = 10.6218 W/m^2, aoi_deg = 0
Plane of array irradiance (isotropic): tilt = 30, surface_azimuth = 180, zenith = 30, azimuth = 180, dni = 800 W/m^2, ghi = 792.820323027551 W/m^2, dhi = 100 W/m^2, albedo = 1.5	INVALID_INPUT	Irradiance on a tilted plane (isotropic sky): the albedo must lie between 0 and 1.
Plane of array irradiance (isotropic): tilt = 30, surface_azimuth = 180, zenith = 30, azimuth = 180, dni = 800 W/m^2, ghi = 792.820323027551 W/m^2, dhi = 100 W/m^2	NEEDS_INPUT	energy_poa_isotropic needs albedo (ground reflectance (0 to 1)). Example: solve('energy_poa_isotropic', tilt=30, surface

Irradiance on a tilted plane (Klucher sky model), `energy_poa_klucher`

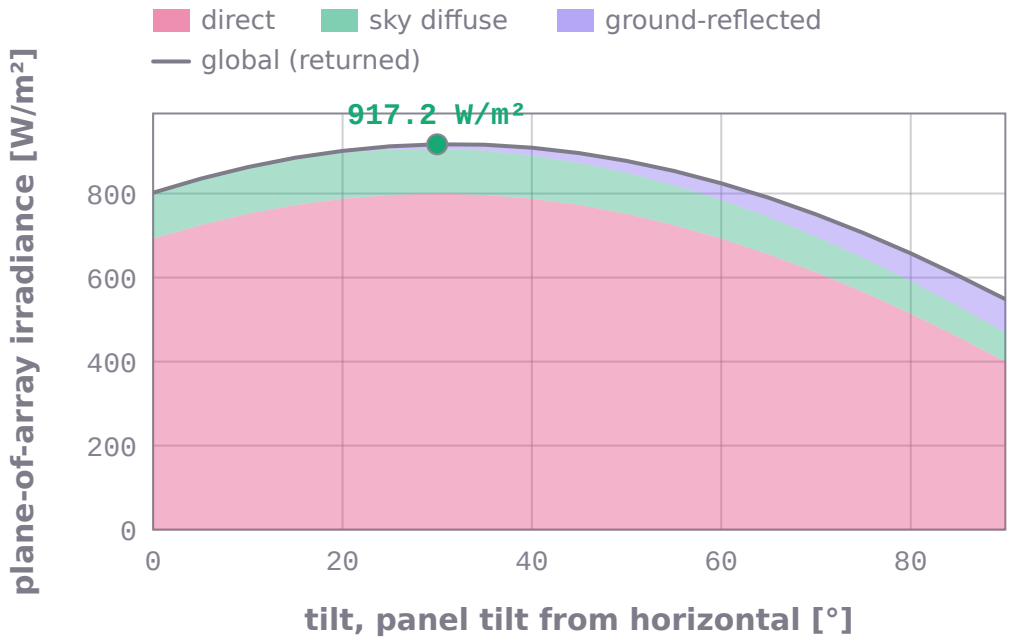
Beam, sky-diffuse (Klucher 1979: isotropic sky brightened near the horizon and around the sun under clear skies) and ground-reflected irradiance on a tilted surface.

$$G_{d,t} = G_d \frac{1+\cos\beta}{2}\left(1 + F\sin^3\frac{\beta}{2}\right)\left(1 + F\cos^2\theta\sin^3\theta_z\right)$$

input	meaning	condition
`tilt`	surface tilt from horizontal in degrees	
`surface_azimuth`	surface azimuth in degrees clockwise from north (south = 180)	
`zenith`	solar zenith angle in degrees	
`azimuth`	solar azimuth in degrees clockwise from north	
`dni`	direct normal irradiance	
`ghi`	global horizontal irradiance	
`dhi`	diffuse horizontal irradiance	
`albedo`	ground reflectance (0 to 1)	

output	meaning
`poa_global`	total irradiance on the plane
`poa_direct`	beam irradiance on the plane
`poa_sky_diffuse`	sky-diffuse irradiance on the plane
`poa_ground_diffuse`	ground-reflected irradiance on the plane
`aoi_deg`	angle of incidence in degrees

- **Method:** sky = DHI (1 + cos beta)/2 (1 + F sin^3(beta/2)) (1 + F cos^2(theta) sin^3(z)), F = 1 - (DHI/GHI)^2
- **Assumes:** sun above the horizon; DHI <= GHI; 0 <= albedo <= 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Irradiance on a tilted panel split into its parts by the package (Irradiance on a tilted plane (Klucher sky model)) for the worked example's sky (DNI 800, GHI 792.8, DHI 100 W/m², Sun at zenith 30°, azimuth 180°, panel facing 180°, albedo 0.2) at tilts from 0 to 90°, stacked: direct, sky diffuse and ground-reflected, the line their returned total. Marked: tilt 30°, direct 800, sky 106.6, ground 10.62, global 917.2 W/m² (angle of incidence 0°).

```
energy_systems_ai999x.solve('energy_poa_klucher', tilt=30, surface_azimuth=180, zenith=30,
azimuth=180, dni=800, ghi=792.820323027551, dhi=100, albedo=0.2)
# -> poa_global = 917.188 W/m^2, poa_direct = 800 W/m^2, poa_sky_diffuse = 106.566 W/m^2,
poa_ground_diffuse = 10.6218 W/m^2, aoi_deg = 0   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Plane of array irradiance (Klucher): tilt = 30, surface_azimuth = 180, zenith = 30, azimuth = 180, dni = 800 W/m^2, ghi = 792.820323027551 W/m^2, dhi = 100 W/m^2, albedo = 0.2	SOLVED	poa_global = 917.188 W/m^2, poa_direct = 800 W/m^2, poa_sky_diffuse = 106.566 W/m^2, poa_ground_diffuse = 10.6218 W/m^2, aoi_deg = 0
Plane of array irradiance (Klucher): tilt = 30, surface_azimuth = 180, zenith = 30, azimuth = 180, dni = 800 W/m^2, ghi = 792.820323027551 W/m^2, dhi = 100 W/m^2, albedo = 1.5	INVALID_INPUT	Irradiance on a tilted plane (Klucher sky model): the albedo must lie between 0 and 1.
Plane of array irradiance (Klucher): tilt = 30, surface_azimuth = 180, zenith = 30, azimuth = 180, dni = 800 W/m^2, ghi = 792.820323027551 W/m^2, dhi = 100 W/m^2	NEEDS_INPUT	energy_poa_klucher needs albedo (ground reflectance (0 to 1)). Example: solve('energy_poa_klucher', tilt=30, surface_azi

Irradiance on a tilted plane (Hay-Davies sky model), `energy_poa_haydavies`

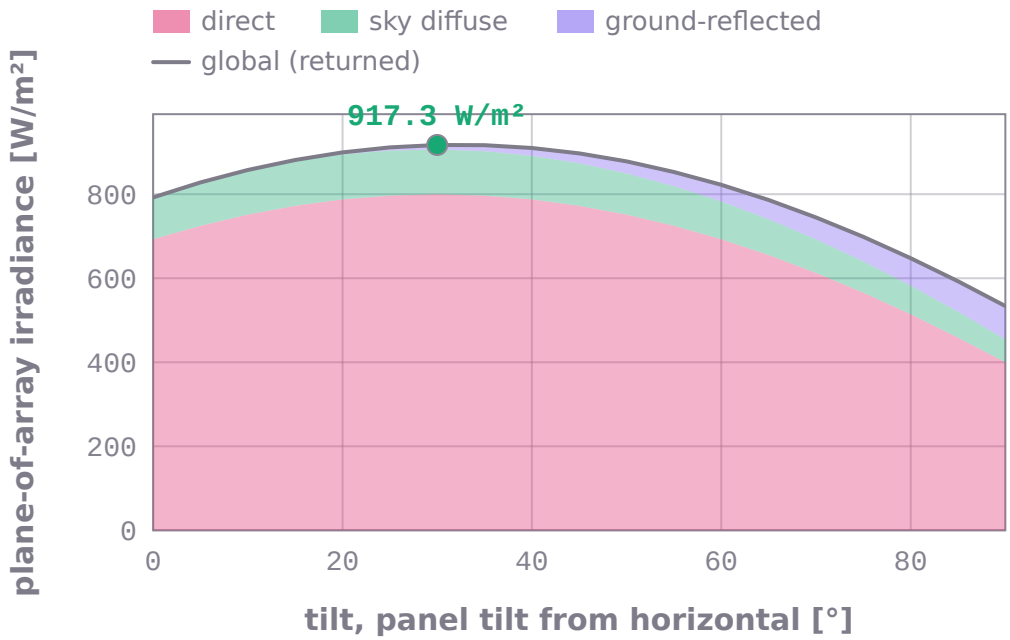
Beam, sky-diffuse (Hay and Davies 1980: isotropic plus circumsolar weighted by the anisotropy index DNI/G_on) and ground-reflected irradiance on a tilted surface; G_on from day n (Spencer, 1366.1 W/m^2).

$$G_{d,t} = G_d \left[A R_b + (1 - A) \frac{1 + \cos\beta}{2} \right],\ A = \frac{G_{bn}}{G_{on}}$$

input	meaning	condition
`tilt`	surface tilt from horizontal in degrees	
`surface_azimuth`	surface azimuth in degrees clockwise from north (south = 180)	
`zenith`	solar zenith angle in degrees	
`azimuth`	solar azimuth in degrees clockwise from north	
`dni`	direct normal irradiance	
`ghi`	global horizontal irradiance	
`dhi`	diffuse horizontal irradiance	
`albedo`	ground reflectance (0 to 1)	
`n`	day of the year (1 = 1 January)	

output	meaning
`poa_global`	total irradiance on the plane
`poa_direct`	beam irradiance on the plane
`poa_sky_diffuse`	sky-diffuse irradiance on the plane
`poa_ground_diffuse`	ground-reflected irradiance on the plane
`aoi_deg`	angle of incidence in degrees

- **Method:** sky = DHI [A $R_b + (1 - A)$(1 + cos beta)/2], A = DNI/G_on, $R_b = \frac{\cos(\theta)}{\cos(z)}$
- **Assumes:** sun above the horizon; DHI <= GHI; 0 <= albedo <= 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Irradiance on a tilted panel split into its parts by the package (Irradiance on a tilted plane (Hay-Davies sky model)) for the worked example's sky (DNI 800, GHI 792.8, DHI 100 W/m², Sun at zenith 30°, azimuth 180°, panel facing 180°, albedo 0.2) at tilts from 0 to 90°, stacked: direct, sky diffuse and ground-reflected, the line their returned total. Marked: tilt 30°, direct 800, sky 106.7, ground 10.62, global 917.3 W/m² (angle of incidence 0°).

```
energy_systems_ai999x.solve('energy_poa_haydavies', tilt=30, surface_azimuth=180,
zenith=30, azimuth=180, dni=800, ghi=792.820323027551, dhi=100, albedo=0.2, n=172)
# -> poa_global = 917.342 W/m^2, poa_direct = 800 W/m^2, poa_sky_diffuse = 106.72 W/m^2,
poa_ground_diffuse = 10.6218 W/m^2, aoi_deg = 0   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Plane of array irradiance (Hay-Davies): tilt = 30, surface_azimuth = 180, zenith = 30, azimuth = 180, dni = 800 W/m^2, ghi = 792.820323027551 W/m^2, dhi = 100 W/m^2, albedo = 0.2, n = 172	SOLVED	poa_global = 917.342 W/m^2, poa_direct = 800 W/m^2, poa_sky_diffuse = 106.72 W/m^2, poa_ground_diffuse = 10.6218 W/m^2, aoi_deg = 0
Plane of array irradiance (Hay-Davies): tilt = 30, surface_azimuth = 180, zenith = 30, azimuth = 180, dni = 800 W/m^2, ghi = 792.820323027551 W/m^2, dhi = 100 W/m^2, albedo = 1.5, n = 172	INVALID_INPUT	Irradiance on a tilted plane (Hay-Davies sky model): the albedo must lie between 0 and 1.
Plane of array irradiance (Hay-Davies): tilt = 30, surface_azimuth = 180, zenith = 30, azimuth = 180, dni = 800 W/m^2, ghi = 792.820323027551 W/m^2, dhi = 100 W/m^2, n = 172	NEEDS_INPUT	energy_poa_haydavies needs albedo (ground reflectance (0 to 1)). Example: solve('energy_poa_haydavies', tilt=30, surface

Irradiance on a tilted plane (Reindl sky model), `energy_poa_reindl`

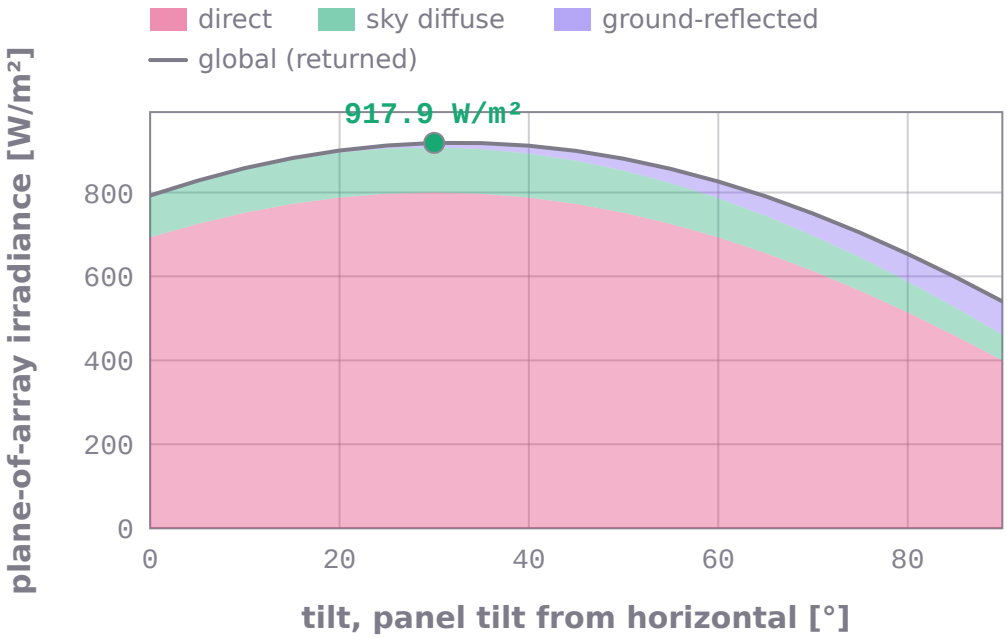
Beam, sky-diffuse (Reindl 1990: Hay-Davies plus horizon brightening sqrt(BHI/GHI) sin^3(beta/2)) and ground-reflected irradiance on a tilted surface; G_on from day n (Spencer).

$$G_{d,t} = G_d \left[(1 - A) \frac{1 + \cos\beta}{2} \left(1 + \sqrt{\frac{G_b}{G}} \sin^3 \frac{\beta}{2} \right) + A R_b \right]$$

input	meaning	condition
`tilt`	surface tilt from horizontal in degrees	
`surface_azimuth`	surface azimuth in degrees clockwise from north (south = 180)	
`zenith`	solar zenith angle in degrees	
`azimuth`	solar azimuth in degrees clockwise from north	
`dni`	direct normal irradiance	
`ghi`	global horizontal irradiance	
`dhi`	diffuse horizontal irradiance	
`albedo`	ground reflectance (0 to 1)	
`n`	day of the year (1 = 1 January)	

output	meaning
`poa_global`	total irradiance on the plane
`poa_direct`	beam irradiance on the plane
`poa_sky_diffuse`	sky-diffuse irradiance on the plane
`poa_ground_diffuse`	ground-reflected irradiance on the plane
`aoi_deg`	angle of incidence in degrees

- **Method:** sky = DHI [$(1 - A)$(1 + cos beta)/2 (1 + sqrt(BHI/GHI) sin^3(beta/2)) + A R_b]
- **Assumes:** sun above the horizon; DHI <= GHI; 0 <= albedo <= 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Irradiance on a tilted panel split into its parts by the package (Irradiance on a tilted plane (Reindl sky model)) for the worked example's sky (DNI 800, GHI 792.8, DHI 100 W/m², Sun at zenith 30°, azimuth 180°, panel facing 180°, albedo 0.2) at tilts from 0 to 90°, stacked: direct, sky diffuse and ground-reflected, the line their returned total. Marked: tilt 30°, direct 800, sky 107.3, ground 10.62, global 917.9 W/m² (angle of incidence 0°).

```
energy_systems_ai999x.solve('energy_poa_reindl', tilt=30, surface_azimuth=180, zenith=30,
azimuth=180, dni=800, ghi=792.820323027551, dhi=100, albedo=0.2, n=172)
# -> poa_global = 917.939 W/m^2, poa_direct = 800 W/m^2, poa_sky_diffuse = 107.317 W/m^2,
poa_ground_diffuse = 10.6218 W/m^2, aoi_deg = 0   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Plane of array irradiance (Reindl): tilt = 30, surface_azimuth = 180, zenith = 30, azimuth = 180, dni = 800 W/m^2, ghi = 792.820323027551 W/m^2, dhi = 100 W/m^2, albedo = 0.2, n = 172	SOLVED	poa_global = 917.939 W/m^2, poa_direct = 800 W/m^2, poa_sky_diffuse = 107.317 W/m^2, poa_ground_diffuse = 10.6218 W/m^2, aoi_deg = 0
Plane of array irradiance (Reindl): tilt = 30, surface_azimuth = 180, zenith = 30, azimuth = 180, dni = 800 W/m^2, ghi = 792.820323027551 W/m^2, dhi = 100 W/m^2, albedo = 1.5, n = 172	INVALID_INPUT	Irradiance on a tilted plane (Reindl sky model): the albedo must lie between 0 and 1.
Plane of array irradiance (Reindl): tilt = 30, surface_azimuth = 180, zenith = 30, azimuth = 180, dni = 800 W/m^2, ghi = 792.820323027551 W/m^2, dhi = 100 W/m^2, n = 172	NEEDS_INPUT	energy_poa_reindl needs albedo (ground reflectance (0 to 1)). Example: solve('energy_poa_reindl', tilt=30, surface_azimu

Diffuse and direct irradiance from global horizontal irradiance (Erbs), `energy_erbs_decomposition`

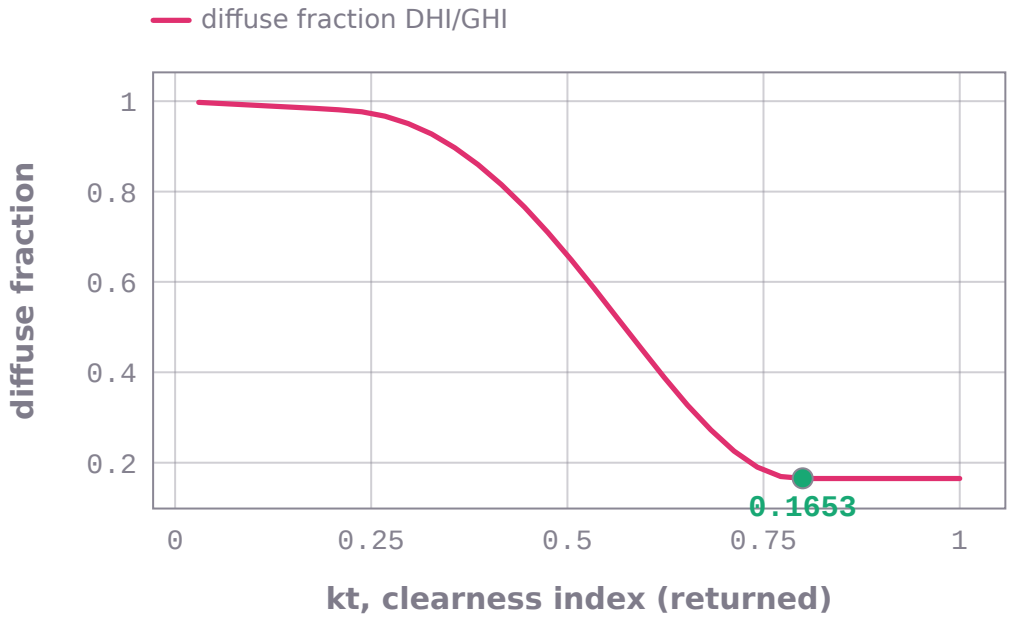
The clearness index, diffuse fraction, diffuse horizontal and direct normal irradiance estimated from a measured global horizontal irradiance by the Erbs (1982) correlation.

$$\frac{G_d}{G} = 0.9511 - 0.1604k_t + 4.388k_t^2 - 16.638k_t^3 + 12.336k_t^4$$

input	meaning	condition
`ghi`	global horizontal irradiance	
`zenith`	solar zenith angle in degrees	
`n`	day of the year (1 = 1 January)	

output	meaning
`kt`	clearness index GHI/(G_on cos z)
`diffuse_fraction`	diffuse fraction DHI/GHI
`dhi`	diffuse horizontal irradiance
`dni`	direct normal irradiance

- **Method:** kt = GHI/(G_on cos z); DHI/GHI = 1 - 0.09 kt $(kt \le 0.22)$, $0.9511 - 0.1604\ kt + 4.388\ kt^2 - 16.638\ kt^3 + 12.336\ kt^4$ (<= 0.8), 0.165; DNI = (GHI - DHI)/cos z
- **Assumes:** 0 <= zenith <= 85; GHI > 0; kt is capped at 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Diffuse fraction against clearness index in the package's Erbs decomposition: global horizontal irradiance stepped from 20 to 1000 W/m² at a solar zenith of 60° on day 172 (the worked example's), both axes the package's returns. Overcast skies (low kt) are nearly all diffuse. Marked: the worked example, GHI 528.6 W/m², kt 0.8, diffuse fraction 0.1653, DHI 87.37 and DNI 882.6 W/m².

```
energy_systems_ai999x.solve('energy_erbs_decomposition', ghi=528.6494370285654, zenith=60,
n=172)
# -> kt = 0.8, diffuse_fraction = 0.16527, dhi = 87.3697 W/m^2, dni = 882.56 W/m^2
(SOLVED)
```

```
energy_systems_ai999x.solve('energy_erbs_decomposition', ghi=141.3981805, zenith=0, n=1)
# -> kt = 0.1, diffuse_fraction = 0.991, dhi = 140.126 W/m^2, dni = 1.27258 W/m^2
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Erbs decomposition: ghi = 141.3981805 W/m^2, zenith = 0, n = 1	SOLVED	kt = 0.1, diffuse_fraction = 0.991, dhi = 140.126 W/m^2, dni = 1.27258 W/m^2
Erbs decomposition: ghi = 600 W/m^2, zenith = 88, n = 172	INVALID_INPUT	Diffuse and direct irradiance from global horizontal irradiance (Erbs): the zenith angle must lie between 0 and 85 degre

Clear-sky global horizontal irradiance (Haurwitz), `energy_clear_sky_haurwitz`

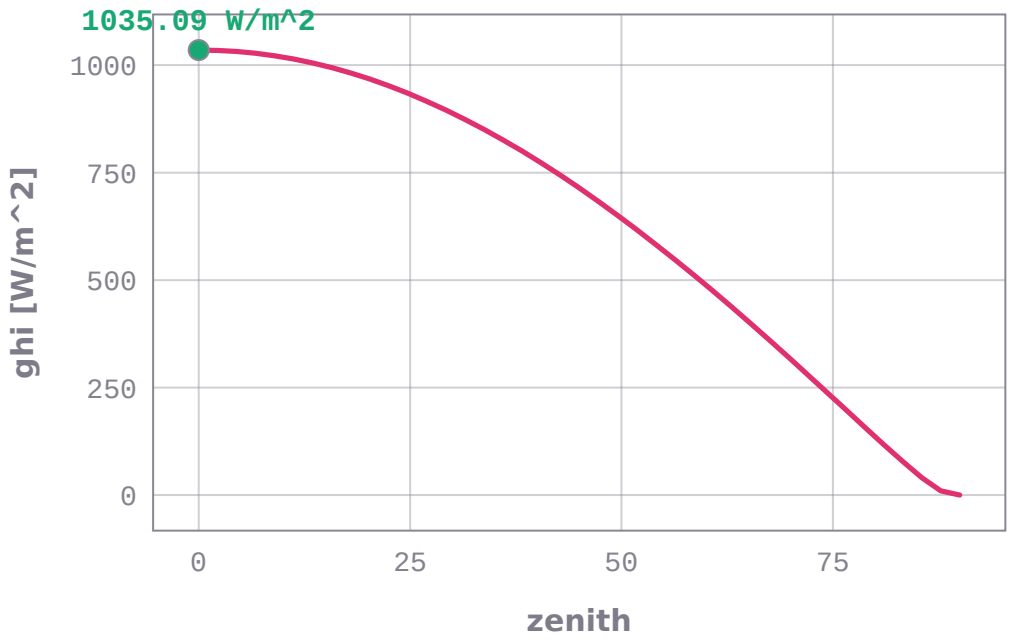
The global horizontal irradiance under a clear sky from the solar zenith angle alone (Haurwitz 1945): 1098 cos z exp(−0.059/cos z).

$$G_{cs} = 1098 \cos z \, e^{-0.059/\cos z}$$

input	meaning	condition
`zenith`	apparent solar zenith angle in degrees	

output	meaning
`ghi`	clear-sky global horizontal irradiance (0 when the sun is below the horizon)

- **Method:** GHI = 1098 cos z exp(−0.059/cos z) for cos $z > 0$, else 0
- **Assumes:** 0 <= zenith <= 180
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



ghi (clear-sky global horizontal irradiance (0 when the sun is below the horizon)) against zenith (apparent solar zenith angle in degrees) from 0 to 90. 41 points computed by energy-systems-ai999x; marked: the worked example, zenith = 0, ghi = 1035.09 W/m^2.

```
energy_systems_ai999x.solve('energy_clear_sky_haurwitz', zenith=0)
# -> ghi = 1035.09 W/m^2   (SOLVED)
```

```
energy_systems_ai999x.solve('energy_clear_sky_haurwitz', zenith=95)
# -> ghi = 0 W/m^2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Clear-sky GHI (Haurwitz): zenith = 0	SOLVED	ghi = 1035.09 W/m^2
Haurwitz clear sky: zenith = 200	INVALID_INPUT	Clear-sky global horizontal irradiance (Haurwitz): the zenith angle must lie between 0 and 180 degrees.

PV cell temperature from NOCT, `energy_cell_temperature_noct`

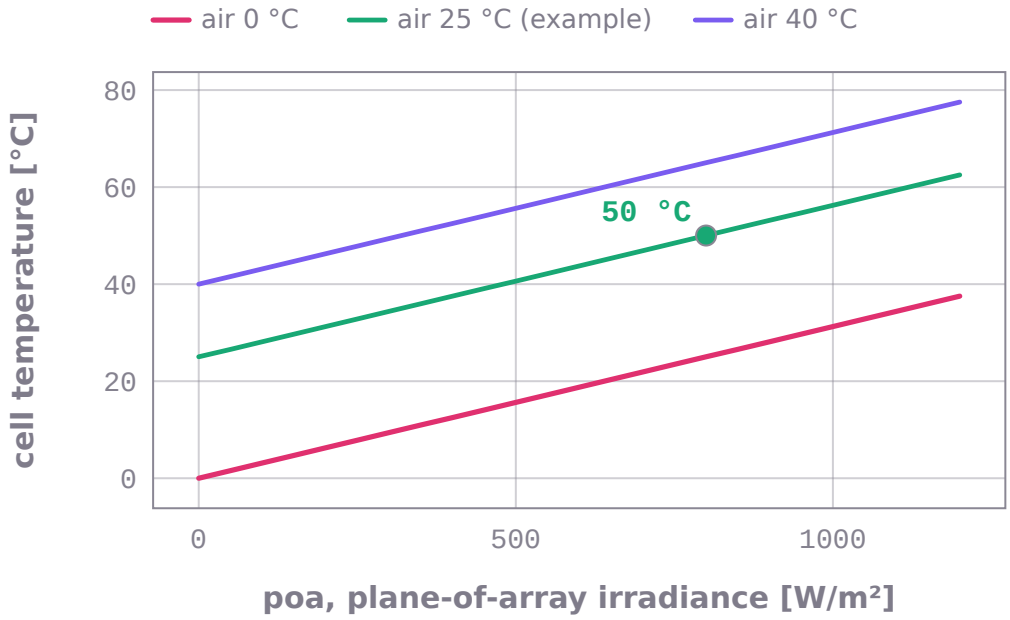
The operating temperature of a PV cell from the plane-of-array irradiance, the air temperature and the module's nominal operating cell temperature (NOCT): T_c = T_a + (NOCT - 20 degC) $G/800.$

$$T_c = T_a + \frac{NOCT - 20^\circ\mathrm{C}}{800\ \mathrm{W/m^2}}\,G$$

input	meaning	condition
`poa`	plane-of-array irradiance	
`T_air`	ambient air temperature	
`noct`	nominal operating cell temperature (at 800 W/m^2, 20 degC, 1 m/s)	

output	meaning
`T_cell`	cell temperature
`T_cell_C`	cell temperature in degC

- **Method:** T_c = T_a + (NOCT - 20)/800 G (Ross model with the NOCT coefficient)
- **Assumes:** irradiance >= 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Cell temperature by the nominal-operating-cell-temperature rule (NOCT 45 °C, 318.15 K, the worked example's) against irradiance for 3 air temperatures, one run per point. Marked: the worked example, 800 W/m² in 25 °C air, 50 °C.

```
energy_systems_ai999x.solve('energy_cell_temperature_noct', poa=800, T_air=298.15,
noct=318.15)
# -> T_cell = 323.15 K, T_cell_C = 50   (SOLVED)
```

```
energy_systems_ai999x.solve('energy_cell_temperature_noct', poa=1000, T_air=293.15,
noct=320.15)
# -> T_cell = 326.9 K, T_cell_C = 53.75   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Cell temperature (NOCT): poa = 800 W/m^2, T_air = 25 degC, noct = 45 degC	SOLVED	T_cell = 323.15 K, T_cell_C = 50
Cell temperature (NOCT): poa = -800 W/m^2, T_air = 25 degC, noct = 45 degC	INVALID_INPUT	PV cell temperature from NOCT: the plane-of-array irradiance must be zero or positive.

PV module temperature (Faiman model), `energy_cell_temperature_faiman`

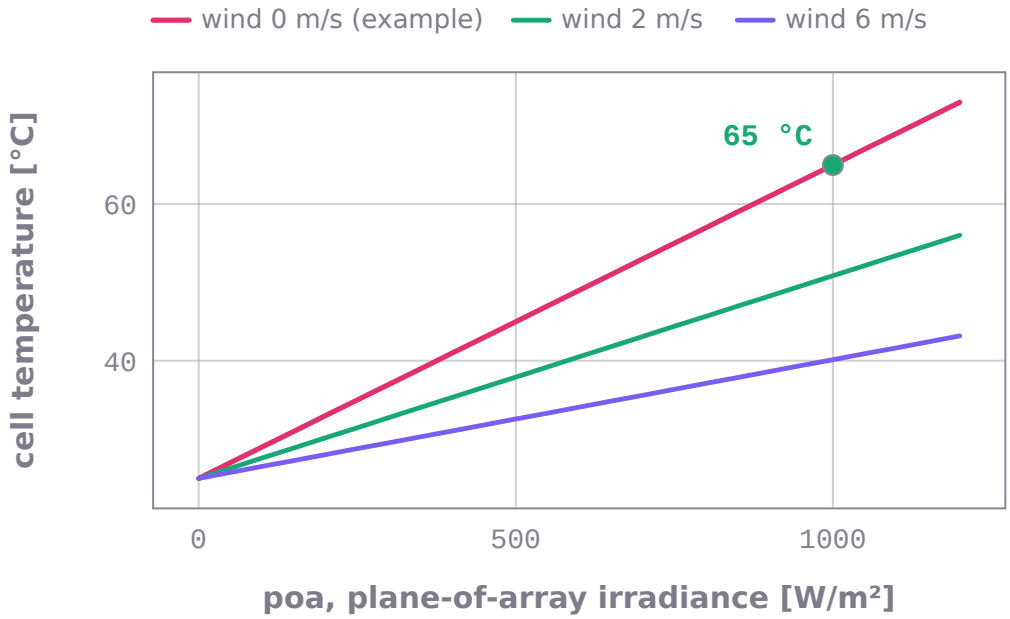
Module (cell) temperature by the Faiman heat-loss model used in IEC 61853: T = T_a + G/(u0 + u1 v).

$$T_m = T_a + \frac{G}{u_0 + u_1 v}$$

input	meaning	condition
`poa`	plane-of-array irradiance	
`T_air`	ambient air temperature	
`wind`	wind speed at module height	
`u0`	constant heat-loss coefficient in W/(m^2 K) (default 25)	
`u1`	wind heat-loss coefficient in W s/(m^3 K) (default 6.84)	

output	meaning
`T_cell`	module temperature
`T_cell_C`	module temperature in degC

- **Method:** T = T_a + G/(u0 + u1 v)
- **Assumes:** $G \geq 0$; $u_0 > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



PV cell temperature against irradiance in air at 25 °C (298.15 K) for 3 wind speeds, one run of the package's Faiman model per point. Wind cools the module. Marked: the worked example, 1000 W/m² with wind 0 m/s, 65 °C.

```
energy_systems_ai999x.solve('energy_cell_temperature_faiman', poa=1000, T_air=298.15,
wind=0)
# -> T_cell = 338.15 K, T_cell_C = 65   (SOLVED)
```

```
energy_systems_ai999x.solve('energy_cell_temperature_faiman', poa=800, T_air=293.15,
wind=1, u0=30, u1=10)
# -> T_cell = 313.15 K, T_cell_C = 40   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Module temperature (Faiman): poa = 1000 W/m^2, T_air = 25 degC, wind = 0 m/s	SOLVED	T_cell = 338.15 K, T_cell_C = 65
Faiman temperature: poa = 800 W/m^2, T_air = 20 degC, wind = 1 m/s, u0 = 30, u1 = 10	SOLVED	T_cell = 313.15 K, T_cell_C = 40
Module temperature (Faiman): poa = 1000 W/m^2, T_air = 25 degC	NEEDS_INPUT	energy_cell_temperature_faiman needs wind (wind speed at module height). Example: solve('energy_cell_temperature_faiman'

PV cell and module temperature (Sandia SAPM model), `energy_cell_temperature_sapm`

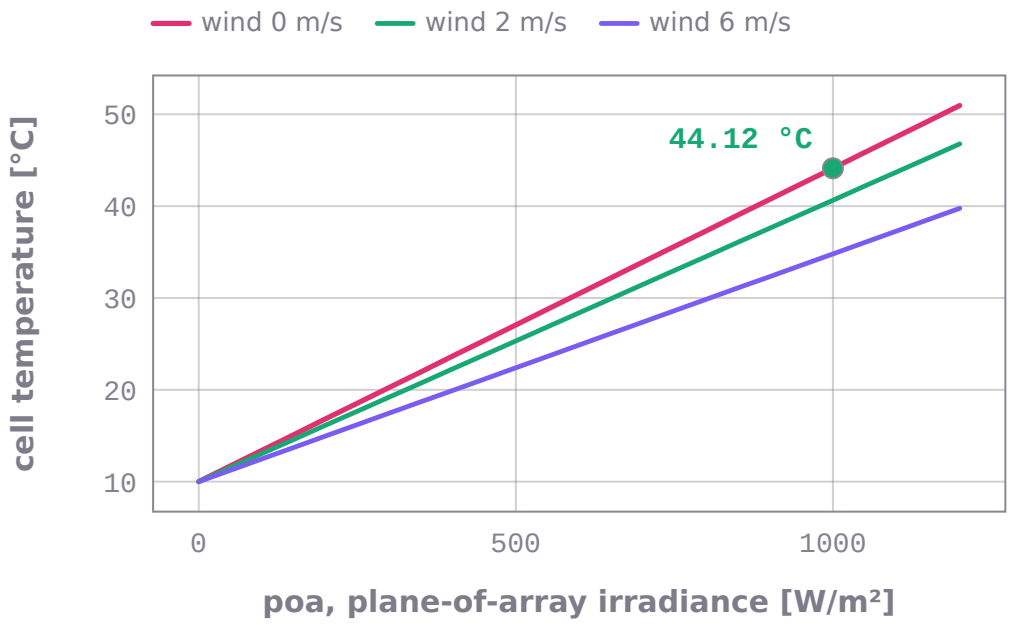
Back-of-module and cell temperature by the Sandia Array Performance Model, $T_m = G$ exp(a + b v) + T_a, $T_c = T_m + (G/1000)$ dT, for the standard module/mounting parameter sets.

$$T_m = G\,e^{a+bv} + T_a, \quad T_c = T_m + \frac{G}{1000}\Delta T$$

input	meaning	condition
`poa`	plane-of-array irradiance	
`T_air`	ambient air temperature	
`wind`	wind speed at 10 m	
`mount`	module construction and mounting (Sandia parameter set)	

output	meaning
`T_module_C`	back-of-module temperature in degC
`T_cell_C`	cell temperature in degC
`T_cell`	cell temperature

- **Method:** $T_m = G$ exp(a + b v) + T_a; $T_c = T_m + G\,dT/1000$ (a, b, dT from the Sandia table)
- **Assumes:** $G \geq 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Cell temperature against irradiance (Sandia model, mounting 'open_rack_glass_glass', air at 10 °C) for three wind speeds, one run per point. Marked: the worked example, 1000 W/m² in still air, cell 44.12 °C, module back 41.12 °C.

```
energy_systems_ai999x.solve('energy_cell_temperature_sapm', poa=1000, T_air=283.15, wind=0,
mount='open_rack_glass_glass')
# -> T_module_C = 41.117, T_cell_C = 44.117, T_cell = 317.267 K   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Cell temperature (SAPM): poa = 1000 W/m^2, T_air = 10 degC, wind = 0 m/s, mount = open_rack_glass_glass	SOLVED	T_module_C = 41.117, T_cell_C = 44.117, T_cell = 317.267 K
Cell temperature (SAPM): poa = 1000 W/m^2, T_air = 10 degC, wind = 0 m/s, mount = on_the_moon	INVALID_INPUT	mount: one of open_rack_glass_glass, close_mount_glass_glass, open_rack_glass_polymer, insulated_back_glass_polymer; got

PV array DC power with temperature derating (PVWatts), `energy_pvwatts_dc`

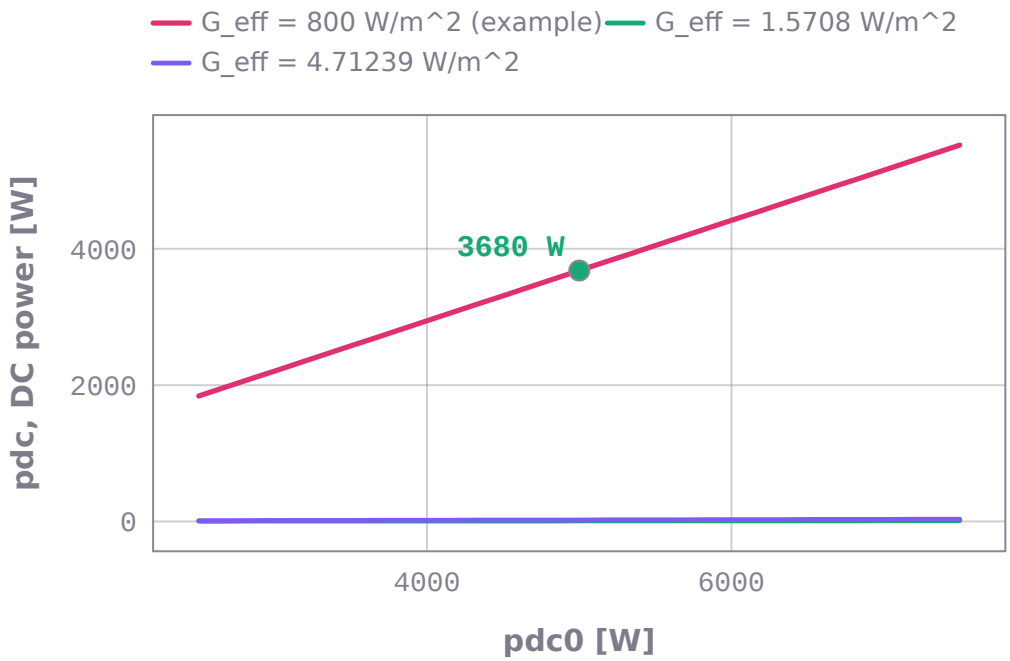
DC power of a PV array from the effective irradiance and cell temperature (PVWatts): $P = (G/1000)$ P_dc0 (1 + gamma (T_c - 25 degC)).

$$P_{dc} = \frac{G}{1000} P_{dc0} \left[1 + \gamma (T_c - 25°\mathrm{C})\right]$$

input	meaning	condition
`G_eff`	effective (transmitted) irradiance on the cells	
`T_cell`	cell temperature	
`pdc0`	DC rating at 1000 W/m^2 and 25 degC	
`gamma`	power temperature coefficient per K (e.g. -0.004, $or -0.4$%)	

output	meaning
`pdc`	DC power
`pdc_kW`	DC power in kW

- **Method:** $P_dc = (G/1000)$ P_dc0 $[1 + \gamma\,(T_c - 25)]$
- **Assumes:** $-0.02 \le \gamma \le 0.01$ per K
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



pdc (DC power) against pdc0 (DC rating at 1000 W/m^2 and 25 degC) from 2500 W to 7500 W. One curve for each value of G_eff (effective (transmitted) irradiance on the cells): 800 W/m^2, 1.5708 W/m^2, 4.71239 W/m^2. Other inputs as in the worked example: T_cell = 318.15 K, gamma = -0.004. 31 points per curve computed by energy-systems-ai999x; marked: the worked example, pdc0 = 5000 W, pdc = 3680 W.

```
energy_systems_ai999x.solve('energy_pvwatts_dc', G_eff=800, T_cell=318.15, pdc0=5000,
gamma=-0.004)
# -> pdc = 3680 W, pdc_kW = 3.68   (SOLVED)
```

```
energy_systems_ai999x.solve('energy_pvwatts_dc', G_eff=1000, T_cell=298.15, pdc0=300,
gamma=-0.0035)
# -> pdc = 300 W, pdc_kW = 0.3   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
PVWatts DC power: G_eff = 800 W/m^2, T_cell = 45 degC, pdc0 = 5 kW, gamma = -0.4%	SOLVED	pdc = 3680 W, pdc_kW = 3.68
PVWatts DC power: G_eff = 800 W/m^2, T_cell = 45 degC, pdc0 = 5 kW, gamma = -0.4	INVALID_INPUT	PV array DC power with temperature derating (PVWatts): the temperature coefficient gamma must lie between -0.02 and 0.01

PV array output at operating temperature, `energy_pv_array_output`

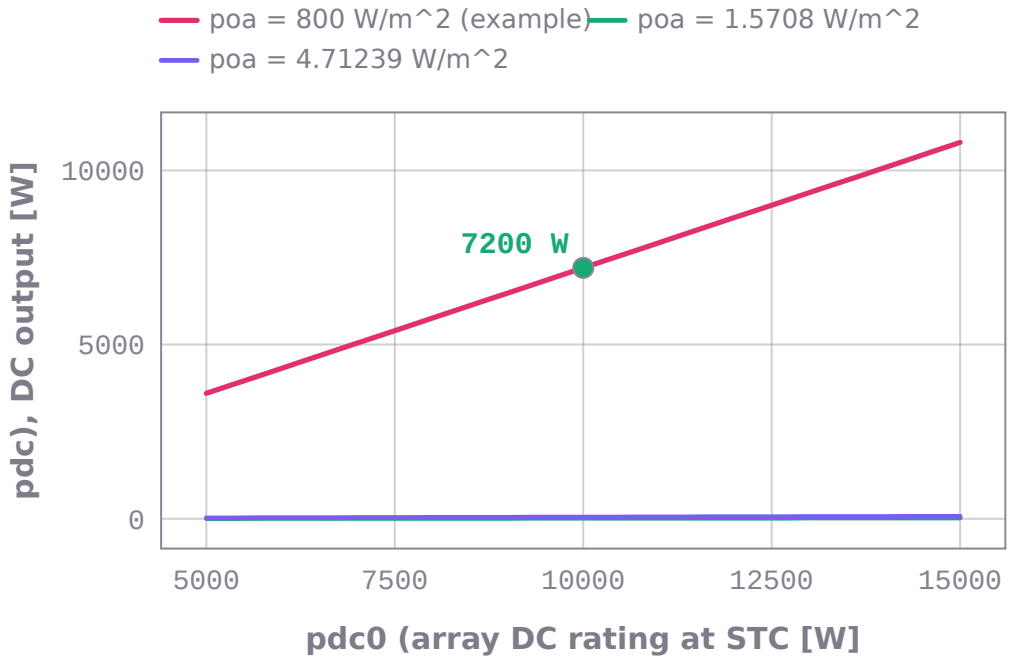
Cell temperature from NOCT and the resulting DC output of an array with a power temperature coefficient, from the plane-of-array irradiance and the air temperature.

$$P = \frac{G}{1000} P_{dc0} \left[1 + \gamma \left(T_a + \frac{NOCT - 20}{800} G - 25 \right) \right]$$

input	meaning	condition
`poa`	plane-of-array irradiance	
`T_air`	ambient air temperature	
`noct`	nominal operating cell temperature	
`pdc0`	array DC rating at STC	
`gamma`	power temperature coefficient per K (e.g. -0.004, $or -0.4\%$)	

output	meaning
`T_cell`	cell temperature
`T_cell_C`	cell temperature in degC
`pdc`	DC output
`derate`	temperature derating factor $1 + \gamma\,(T_c - 25)$

- **Method:** T_c = T_a + (NOCT - 20) $G/800$; $P = (G/1000)$ P_dc0 $[1 + \gamma\,(T_c - 25)]$
- **Assumes:** $G > 0$; $-0.02 \le \gamma \le 0.01$ per K
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



pdc (DC output) against pdc0 (array DC rating at STC) from 5000 W to 15000 W. One curve for each value of poa (plane-of-array irradiance): 800 W/m^2, 1.5708 W/m^2, 4.71239 W/m^2. Other inputs as in the worked example: T_air = 298.15 K, noct = 318.15 K, gamma = -0.004. 31 points per curve computed by energy-systems-ai999x; marked: the worked example, pdc0 = 10000 W, pdc = 7200 W.

```
energy_systems_ai999x.solve('energy_pv_array_output', poa=800, T_air=298.15, noct=318.15,
pdc0=10000, gamma=-0.004)
# -> T_cell = 323.15 K, T_cell_C = 50, pdc = 7200 W, derate = 0.9   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
PV array output: poa = 800 W/m^2, T_air = 25 degC, noct = 45 degC, pdc0 = 10 kW, gamma = -0.4%	SOLVED	T_cell = 323.15 K, T_cell_C = 50, pdc = 7200 W, derate = 0.9
PV array output: poa = 0 W/m^2, T_air = 25 degC, noct = 45 degC, pdc0 = 10 kW, gamma = -0.4%	INVALID_INPUT	PV array output at operating temperature: the plane-of-array irradiance must be a positive number.

Inverter AC output and efficiency (PVWatts), `energy_pvwatts_inverter`

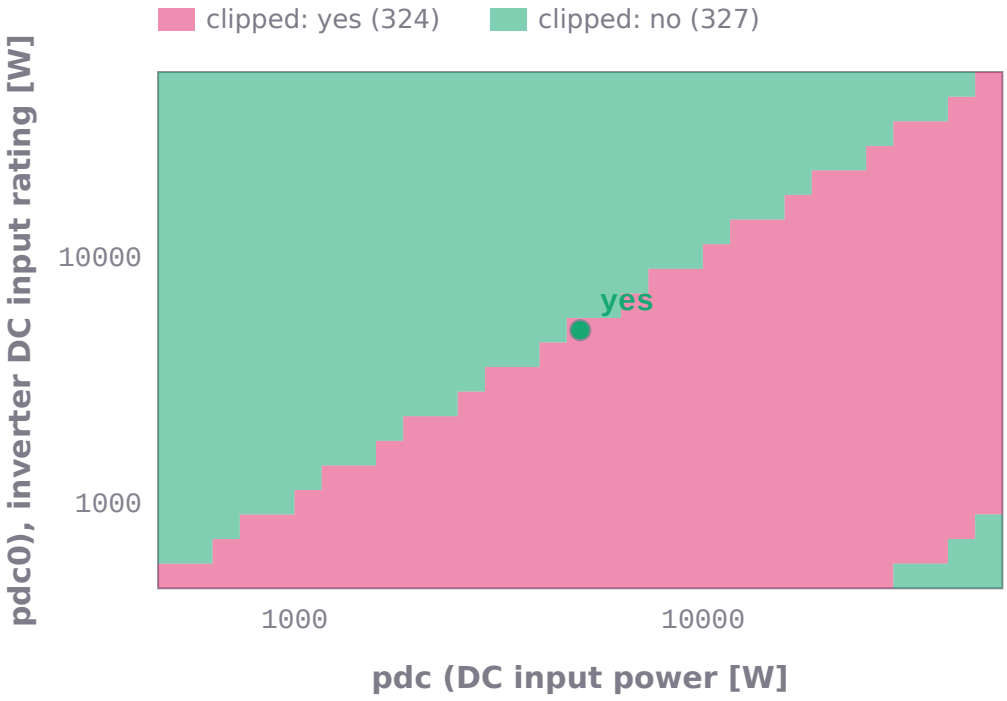
AC output and efficiency of an inverter at a DC input by the PVWatts part-load efficiency curve, with clipping at the AC rating eta_nom P_dc0.

$$\eta = \frac{\eta_{nom}}{\eta_{ref}}\left(-0.0162\zeta - \frac{0.0059}{\zeta} + 0.9858\right),\ \zeta = \frac{P_{dc}}{P_{dc0}}$$

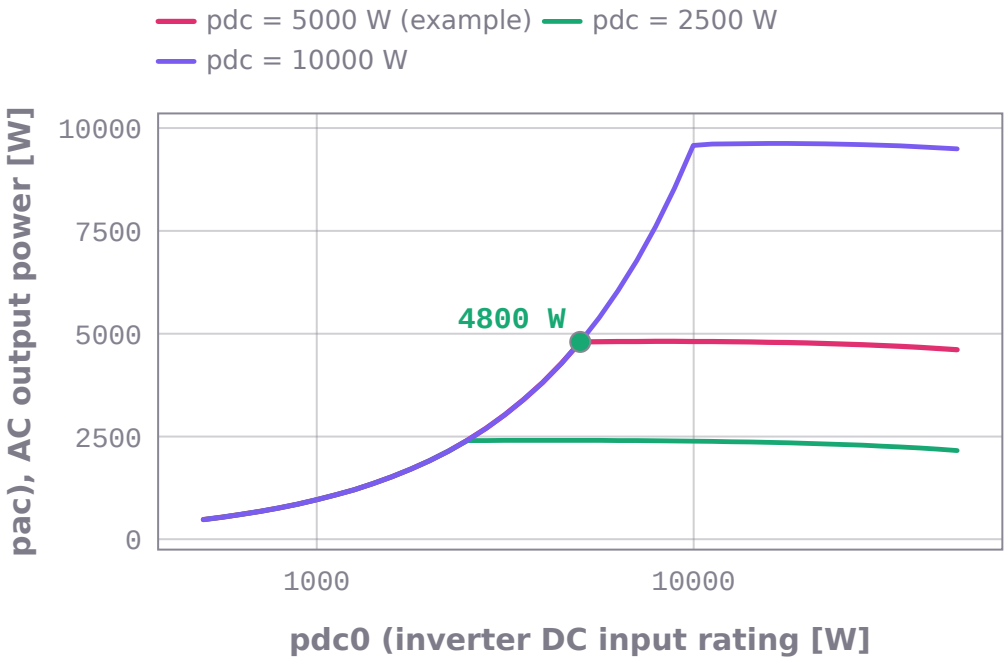
input	meaning	condition
`pdc`	DC input power	
`pdc0`	inverter DC input rating	
`eta_nom`	nominal efficiency (default 0.96)	
`eta_ref`	reference efficiency of the curve (default 0.9637)	

output	meaning
`pac`	AC output power
`efficiency`	conversion efficiency P_ac/P_dc
`clipped`	true when the output is limited at the AC rating

- **Method:** $\zeta = P_dc/P_dc0$; eta = (eta_nom/eta_ref)(-0.0162 zeta - 0.0059/zeta + 0.9858); P_ac = min(eta P_dc, eta_nom P_dc0)
- **Assumes:** $P_dc > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



true when the output is limited at the AC rating (clipped) over pdc (DC input power) from 500 W to 50000 W and pdc0 (inverter DC input rating) from 500 W to 50000 W (logarithmic axes): each of the 31 × 21 cells is one run of energy-systems-ai999x, coloured by the verdict it returned, yes: 324 cells, no: 327 cells. Marked: the worked example, pdc = 5000 W, pdc0 = 5000 W → yes.



pac (AC output power) against pdc0 (inverter DC input rating) from 500 W to 50000 W on a logarithmic axis. One curve for each value of pdc (DC input power): 5000 W, 2500 W, 10000 W. 41 points per curve computed by energy-systems-ai999x; marked: the worked example, pdc0 = 5000 W, pac = 4800 W.

```
energy_systems_ai999x.solve('energy_pvwatts_inverter', pdc=5000, pdc0=5000)
# -> pac = 4800 W, efficiency = 0.96, clipped = True   (SOLVED)
```

```
energy_systems_ai999x.solve('energy_pvwatts_inverter', pdc=6000, pdc0=5000)
# -> pac = 4800 W, efficiency = 0.8, clipped = True   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Inverter output (PVWatts): pdc = 6 kW, pdc0 = 5 kW	SOLVED	pac = 4800 W, efficiency = 0.8, clipped = True
Inverter output (PVWatts): pdc = 0 kW, pdc0 = 5 kW	INVALID_INPUT	Inverter AC output and efficiency (PVWatts): the DC input power must be a positive number.

Incidence angle modifier (ASHRAE), `energy_iam_ashrae`

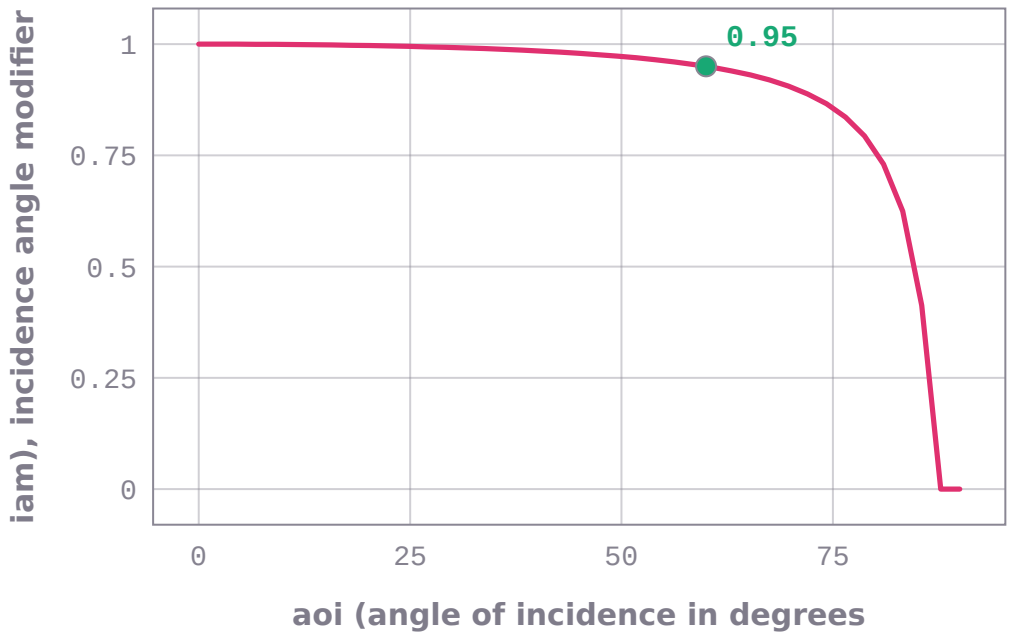
The fraction of beam irradiance transmitted into a PV module at an angle of incidence, by the ASHRAE model IAM = 1 - b0 (1/cos(theta) - 1), floored at 0.

$$IAM = 1 - b_0 \left(\frac{1}{\cos \theta} - 1 \right)$$

input	meaning	condition
`aoi`	angle of incidence in degrees	
`b0`	ASHRAE coefficient (default 0.05)	

output	meaning
`iam`	incidence angle modifier

- **Method:** IAM = max(0, $1 - b_0$ (sec(theta) - 1)); 0 for $\theta \geq 90$
- **Assumes:** 0 <= aoi <= 180
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



iam (incidence angle modifier) against aoi (angle of incidence in degrees) from 0 to 90. 41 points computed by energy-systems-ai999x; marked: the worked example, aoi = 60, iam = 0.95.

```
energy_systems_ai999x.solve('energy_iam_ashrae', aoi=60)
# -> iam = 0.95   (SOLVED)
```

```
energy_systems_ai999x.solve('energy_iam_ashrae', aoi=0)
# -> iam = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Incidence angle modifier (ASHRAE): aoi = 60	SOLVED	iam = 0.95
ASHRAE IAM: aoi = 60, b0 = 2	INVALID_INPUT	Incidence angle modifier (ASHRAE): b0 must lie between 0 and 1.

Incidence angle modifier from Fresnel reflection and absorption, `energy_iam_physical`

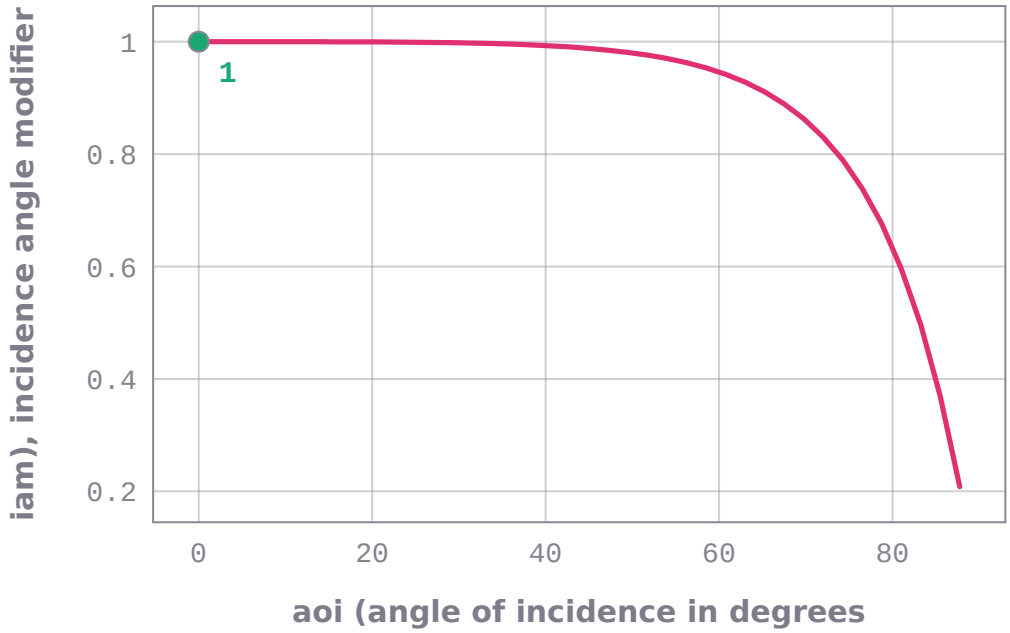
The incidence angle modifier of a glass cover from Fresnel reflection (both polarizations) and absorption in the glass (De Soto / physical model), normalised to normal incidence.

$$IAM = \frac{\frac{1}{2}[(1 - r_s) + (1 - r_p)]e^{-KL/\cos\theta_r}}{(1 - r_0)e^{-KL}}$$

input	meaning	condition
`aoi`	angle of incidence in degrees	
`n`	refractive index of the glass (default 1.526)	
`K`	glazing extinction coefficient in $1/m$ (default 4)	
`L`	glazing thickness (default 2 mm)	

output	meaning
`iam`	incidence angle modifier

- **Method:** Snell's law, Fresnel reflectances r_s, r_p, transmission $\tau = (1 - r)$ exp(-K L/cos theta_r), IAM = (tau_s + tau_p)/2/tau(0)
- **Assumes:** 0 <= aoi < 90; $1 < n < 3$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



iam (incidence angle modifier) against aoi (angle of incidence in degrees) from 0 to 90. 40 points computed by energy-systems-ai999x; marked: the worked example, aoi = 0, iam = 1.

```
energy_systems_ai999x.solve('energy_iam_physical', aoi=0)
# -> iam = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Incidence angle modifier (physical): aoi = 0	SOLVED	iam = 1
Incidence angle modifier (physical): aoi = 95	INVALID_INPUT	Incidence angle modifier from Fresnel reflection and absorption: the angle of incidence must lie between 0 and 90 degree

Combined PV system losses (PVWatts), `energy_pvwatts_losses`

The total system loss of a PV system from its individual loss percentages, combined multiplicatively as in PVWatts: 1 - prod(1 - L_i); unstated losses take the PVWatts defaults (soiling 2%, shading 3%, snow 0%, mismatch 2%, wiring 2%, connections 0.5%, light-induced degradation 1.5%, nameplate 1%, age 0%, availability 3%).

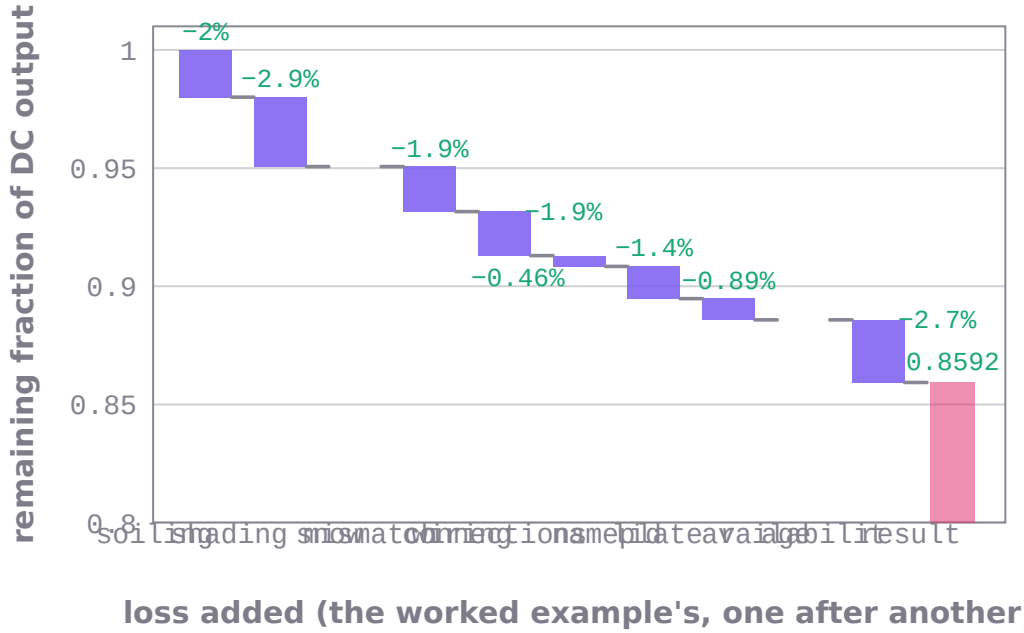
$$L = 1 - \prod_i (1 - L_i)$$

input	meaning	condition
`soiling`	soiling loss as a fraction or a percentage (default 2.0%)	
`shading`	shading loss as a fraction or a percentage (default 3.0%)	
`snow`	snow loss as a fraction or a percentage (default 0.0%)	
`mismatch`	mismatch loss as a fraction or a percentage (default 2.0%)	
`wiring`	wiring loss as a fraction or a percentage (default 2.0%)	
`connections`	connections loss as a fraction or a percentage (default 0.5%)	

input	meaning	condition
`lid`	lid loss as a fraction or a percentage (default 1.5%)	
`nameplate_rating`	nameplate rating loss as a fraction or a percentage (default 1.0%)	
`age`	age loss as a fraction or a percentage (default 0.0%)	
`availability`	availability loss as a fraction or a percentage (default 3.0%)	

output	meaning
`total_loss`	total loss as a fraction
`performance_factor`	product of (1 - loss)
`total_loss_pct`	total loss in percent

- **Method:** L = 1 - prod(1 - L_i)
- **Assumes:** each loss between 0 and 100%
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



System losses of a PV array (PVWatts loss model) for the worked example (every loss at its default; defaults elsewhere): each step adds one loss to those before it and the package returns the remaining performance factor; a step's drop (printed) is the difference of two returned factors. The losses multiply, so the total 14.08 % is less than their sum. Marked: performance factor 0.8592.

```
energy_systems_ai999x.solve('energy_pvwatts_losses', )
# -> total_loss = 0.140757, performance_factor = 0.859243, total_loss_pct = 14.0757
(SOLVED)
```

```
energy_systems_ai999x.solve('energy_pvwatts_losses', soiling=0.0, shading=0.0, snow=0.0,
mismatch=0.0, wiring=0.0, connections=0.0, lid=0.0, nameplate_rating=0.0, age=0.0,
availability=0.0)
# -> total_loss = 0, performance_factor = 1, total_loss_pct = 0   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
PVWatts losses: soiling = 0%, shading = 0%, snow = 0%, mismatch = 0%, wiring = 0%, connections = 0%, lid = 0%, nameplate_rating = 0%, age = 0%, availability = 10%	SOLVED	total_loss = 0.1, performance_factor = 0.9, total_loss_pct = 10
PV system losses: soiling = 120%	INVALID_INPUT	Combined PV system losses (PVWatts): the soiling loss must lie between 0 and 100%.

Power in the wind and the Betz limit, `energy_wind_power`

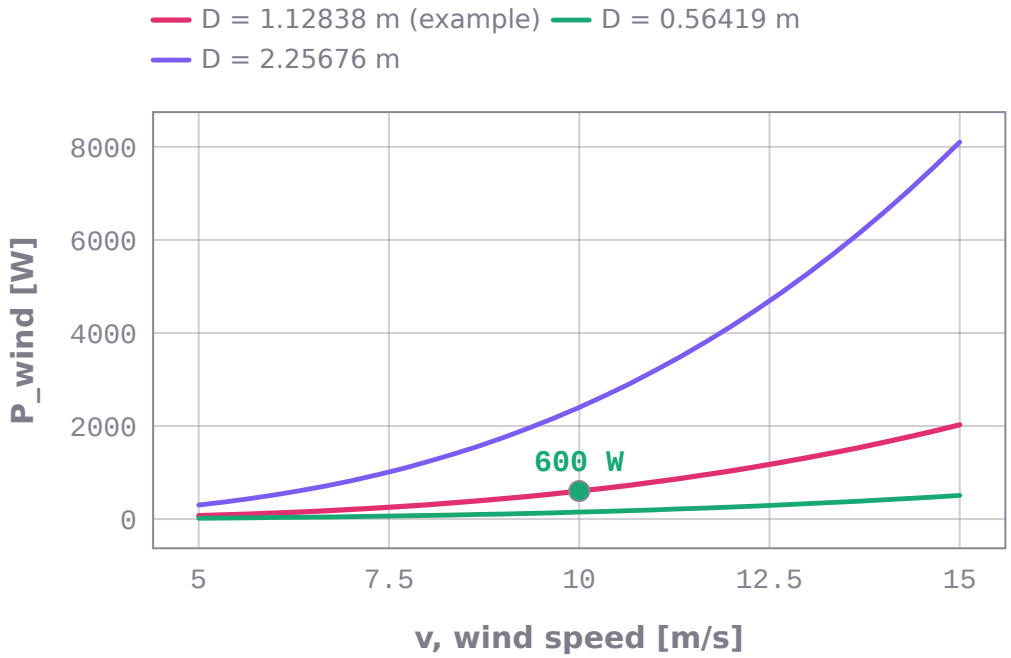
The kinetic power of the wind through a rotor's swept area, the maximum a turbine can extract (Betz limit 16/27), the power density, and the power at a given power coefficient.

$$P = \tfrac{1}{2}\rho A v^3, \quad P_{max} = \tfrac{16}{27} \cdot \tfrac{1}{2}\rho A v^3$$

input	meaning	condition
`rho`	air density	
`D`	rotor diameter	
`v`	wind speed	
`cp`	power coefficient (optional, at most 16/27)	

output	meaning
`P_wind`	power in the wind through the swept area
`P_betz`	maximum extractable power (Betz)
`power_density`	wind power density in W/m^2
`area`	swept area
`P`	power at the given power coefficient

- **Method:** $P = (1/2)$ rho A v^3, $A = pi\ D^2/4$; P_Betz = (16/27) P; P(cp) = cp P
- **Assumes:** rho, D, $v > 0$; $0 < cp \leq 16/27$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



P_wind (power in the wind through the swept area) against v (wind speed) from 5 m/s to 15 m/s. One curve for each value of D (rotor diameter): 1.12838 m, 0.56419 m, 2.25676 m. Other inputs as in the worked example: rho = 1.2 kg/m^3. 31 points per curve computed by energy-systems-ai999x; marked: the worked example, v = 10 m/s, P_wind = 600 W.

```
energy_systems_ai999x.solve('energy_wind_power', rho=1.2, D=1.1283791670955126, v=10)
# -> P_wind = 600 W, P_betz = 355.556 W, power_density = 600, area = 1 m^2   (SOLVED)
```

```
energy_systems_ai999x.solve('energy_wind_power', rho=1.225, D=80, v=10, cp=0.4)
# -> P_wind = 3.07876e+06 W, P_betz = 1.82445e+06 W, power_density = 612.5, area = 5026.55
m^2, P = 1.2315e+06 W   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Power in the wind: rho = 1.225 kg/m^3, D = 80 m, v = 10 m/s, cp = 0.4	SOLVED	P_wind = 3.07876e+06 W, P_betz = 1.82445e+06 W, power_density = 612.5, area = 5026.55 m^2, P = 1.2315e+06 W
Wind power: rho = 1.2 kg/m^3, D = 100 m, v = 12 m/s, cp = 0.7	INVALID_INPUT	Power in the wind and the Betz limit: the power coefficient must lie between 0 and the Betz limit 16/27 = 0.593.
Wind power: rho = 1.2 kg/m^3, D = 100 m	NEEDS_INPUT	energy_wind_power needs v (wind speed). Example: solve('energy_wind_power', rho='1.2 kg/m^3', D='1.12838 m', v='10 m/s')

Actuator-disc (momentum theory) rotor performance, `energy_actuator_disc`

Power and thrust coefficients, disc and far-wake speeds, power and thrust of an ideal rotor (actuator disc) with axial induction factor a; $a = 1/3$ gives the Betz optimum $Cp = 16/27$.

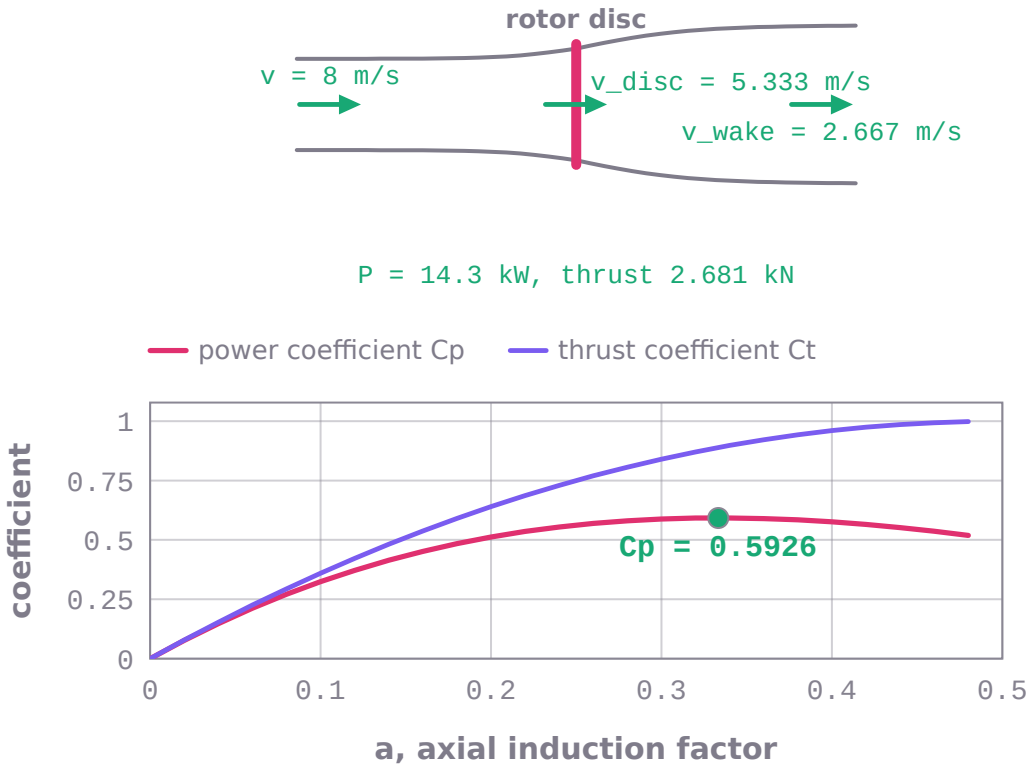
$$C_P = 4a(1-a)^2, \quad C_T = 4a(1-a)$$

input	meaning	condition
a	axial induction factor $(0 \le a < 0.5)$	

input	meaning	condition
`rho`	air density	
`D`	rotor diameter	
`v`	free-stream wind speed	

output	meaning
`Cp`	power coefficient 4a(1-a)^2
`Ct`	thrust coefficient 4a(1-a)
`v_disc`	speed at the disc
`v_wake`	far-wake speed
`P`	power extracted
`T`	thrust on the rotor

- **Method:** $Cp = 4a$(1 - a)^2, $Ct = 4a$(1 - a), v_disc = v(1 - a), v_wake = v(1 - 2a); $P = Cp\,(1/2)$ rho A v^3, $T = Ct\,(1/2)$ rho A v^2
- **Assumes:** $0 \le a < 0.5$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Top: the stream tube through an ideal rotor (diameter 10 m, free wind 8 m/s, air 1.2 kg/m³, induction factor a = 0.3333, the worked example) with the package's speeds at the disc (5.333 m/s) and in the wake (2.667 m/s); the tube widths are a sketch whose end radii follow continuity of those speeds. Bottom: the package's Cp and Ct against a. Marked: Cp = 0.5926, Ct = 0.8889, P = 14.3 kW, thrust 2.681 kN.

```
energy_systems_ai999x.solve('energy_actuator_disc', a=0.3333333333333333, rho=1.2, D=10,
v=8)
# -> Cp = 0.592593, Ct = 0.888889, v_disc = 5.33333 m/s, v_wake = 2.66667 m/s, P = 14297.7
W, T = 2680.83 N   (SOLVED)
```

```
energy_systems_ai999x.solve('energy_actuator_disc', a=0.25, rho=1.2, D=10, v=8)
# -> Cp = 0.5625, Ct = 0.75, v_disc = 6 m/s, v_wake = 4 m/s, P = 13571.7 W, T = 2261.95 N
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Actuator disc: a = 0.25, rho = 1.2 kg/m^3, D = 10 m, v = 8 m/s	SOLVED	Cp = 0.5625, Ct = 0.75, v_disc = 6 m/s, v_wake = 4 m/s, P = 13571.7 W, T = 2261.95 N
Actuator disc: a = 0.6, rho = 1.2 kg/m^3, D = 10 m, v = 8 m/s	INVALID_INPUT	Actuator-disc (momentum theory) rotor performance: the axial induction factor a must lie in [0, 0.5) (momentum theory fa

Wind speed at another height (power law), `energy_wind_shear_power_law`

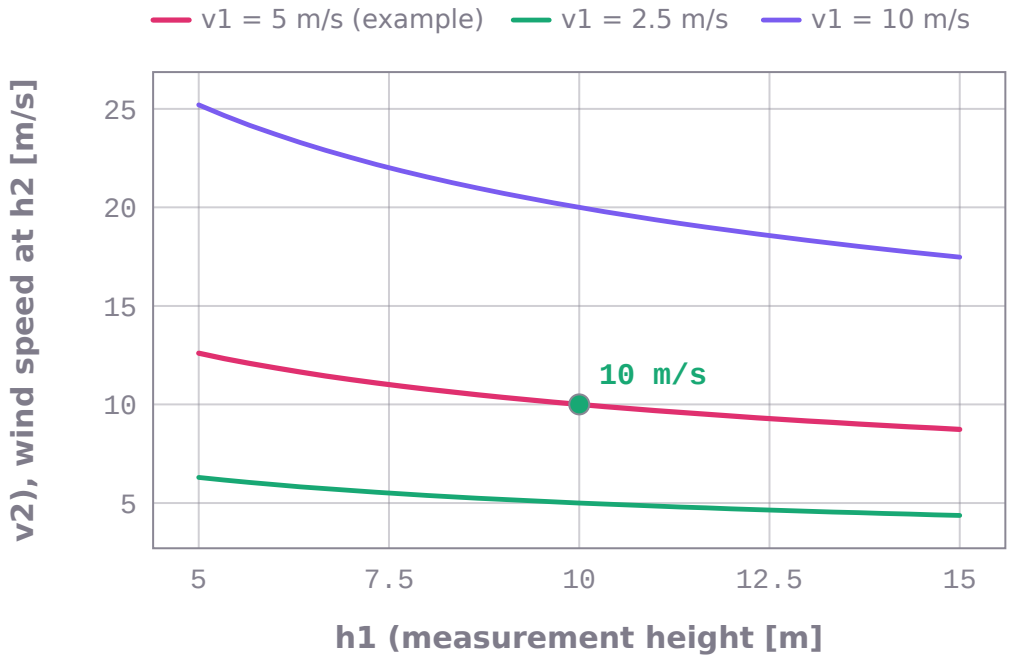
The wind speed at height h2 from a measurement at h1 by the power (Hellman) law $v_2 = v_1\,(h_2/h_1)^{\alpha}$; $\alpha = 1/7$ when not stated (the one-seventh law).

$$v_2 = v_1\left(\frac{h_2}{h_1}\right)^{\alpha}$$

input	meaning	condition
`v1`	measured wind speed	
`h1`	measurement height	
`h2`	target height (e.g. hub height)	
`alpha`	shear exponent (default 1/7)	

output	meaning
`v2`	wind speed at h2

- **Method:** $v_2 = v_1\,(h_2/h_1)^{\alpha}$
- **Assumes:** heights > 0; $0 \le \alpha \le 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



v2 (wind speed at h2) against h1 (measurement height) from 5 m to 15 m. One curve for each value of v1 (measured wind speed): 5 m/s, 2.5 m/s, 10 m/s. Other inputs as in the worked example: h2 = 80 m, alpha = 0.3333333333333333. 31 points per curve computed by energy-systems-ai999x; marked: the worked example, h1 = 10 m, v2 = 10 m/s.

```
energy_systems_ai999x.solve('energy_wind_shear_power_law', v1=5, h1=10, h2=80,
alpha=0.3333333333333333)
# -> v2 = 10 m/s   (SOLVED)
```

```
energy_systems_ai999x.solve('energy_wind_shear_power_law', v1=6, h1=10, h2=1280)
# -> v2 = 12 m/s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Wind shear (power law): v1 = 6 m/s, h1 = 10 m, h2 = 1280 m	SOLVED	v2 = 12 m/s
Wind shear (power law): v1 = 5 m/s, h1 = 10 m, h2 = 80 m, alpha = 0.3333333333333333	SOLVED	v2 = 10 m/s
Wind shear (power law): v1 = 5 m/s, h1 = 0 m, h2 = 80 m	INVALID_INPUT	Wind speed at another height (power law): the wind speed and the heights must be a positive number.

Wind speed at another height (logarithmic profile), `energy_wind_shear_log`

The wind speed at height h2 from a measurement at h1 by the logarithmic wind profile with roughness length z0 and an optional obstacle height d (displacement 0.7 d).

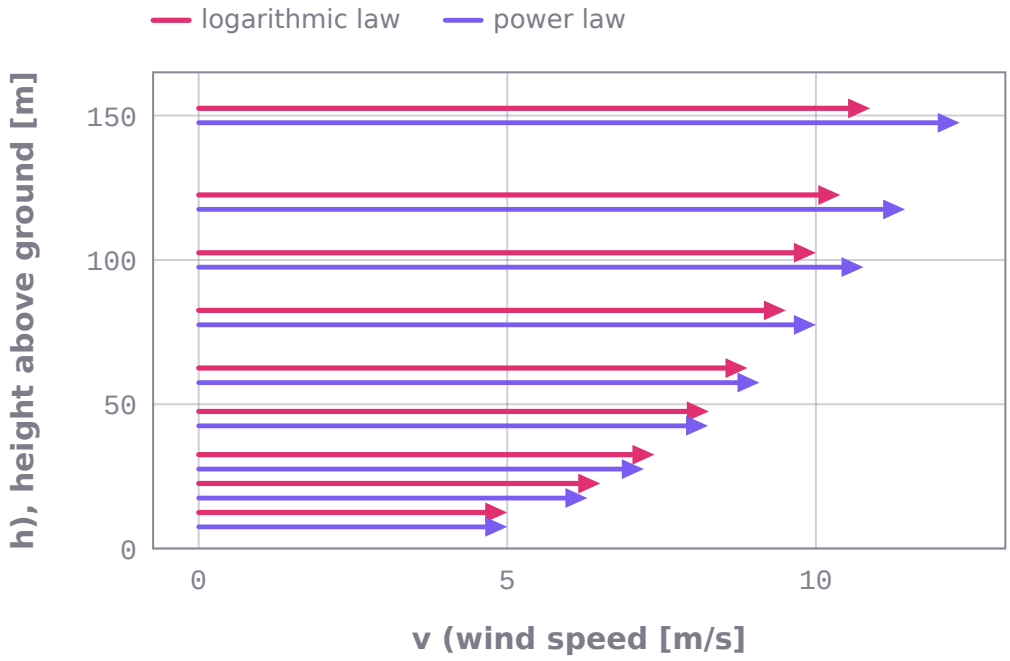
$$v_2 = v_1 \frac{\ln\left((h_2 - 0.7d)/z_0\right)}{\ln\left((h_1 - 0.7d)/z_0\right)}$$

input	meaning	condition
`v1`	measured wind speed	

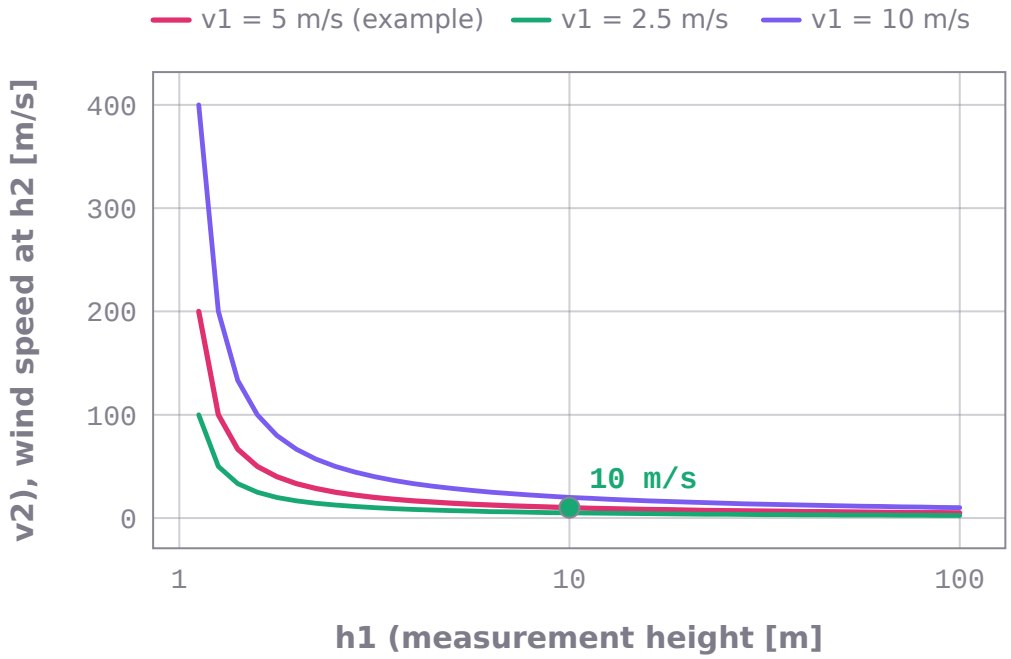
input	meaning	condition
`h1`	measurement height	
`h2`	target height	
`z0`	surface roughness length	
`d`	obstacle height (default 0; displacement height 0.7 d)	

output	meaning
`v2`	wind speed at h2

- **Method:** $v_2 = v_1$ ln((h2 - 0.7 d)/z0)/ln((h1 - 0.7 d)/z0)
- **Assumes:** $h - 0.7\,d > z_0$ at both heights
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Wind speed at nine heights extrapolated from 5 m/s measured at 10 m, drawn as arrows whose length is the speed: logarithmic law (roughness length 1 m) and power law (exponent 0.333333), each computed by the package. At 100 m: 10 and 10.7722 m/s; at 150 m: 10.8805 and 12.3311 m/s.



v2 (wind speed at h2) against h1 (measurement height) from 1 m to 100 m on a logarithmic axis. One curve for each value of v1 (measured wind speed): 5 m/s, 2.5 m/s, 10 m/s. Other inputs as in the worked example: h2 = 100 m, z0 = 1 m. 40 points per curve computed by energy-systems-ai999x; marked: the worked example, h1 = 10 m, v2 = 10 m/s.

```
energy_systems_ai999x.solve('energy_wind_shear_log', v1=5, h1=10, h2=100, z0=1)
# -> v2 = 10 m/s   (SOLVED)
```

```
energy_systems_ai999x.solve('energy_wind_shear_log', v1=4, h1=9.718281828459045,
h2=27.085536923187664, z0=1, d=10)
# -> v2 = 12 m/s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Wind shear (log profile): v1 = 5 m/s, h1 = 10 m, h2 = 100 m, z0 = 1 m	SOLVED	v2 = 10 m/s
Wind shear (log profile): v1 = 5 m/s, h1 = 10 m, h2 = 100 m, z0 = 20 m	INVALID_INPUT	Wind speed at another height (logarithmic profile): both heights must exceed the displacement height 0.7 d plus the roug

Air density at hub height, `energy_hub_air_density`

Temperature, pressure and air density at a turbine's hub height from a pressure and a temperature measured at other heights: temperature gradient -6.5 K/km, pressure gradient -1/8 hPa per m, then the barometric (1.225 kg/m^3 at 288.15 K, 1013.3 hPa) or the ideal-gas (R = 287.058 J/(kg K)) density.

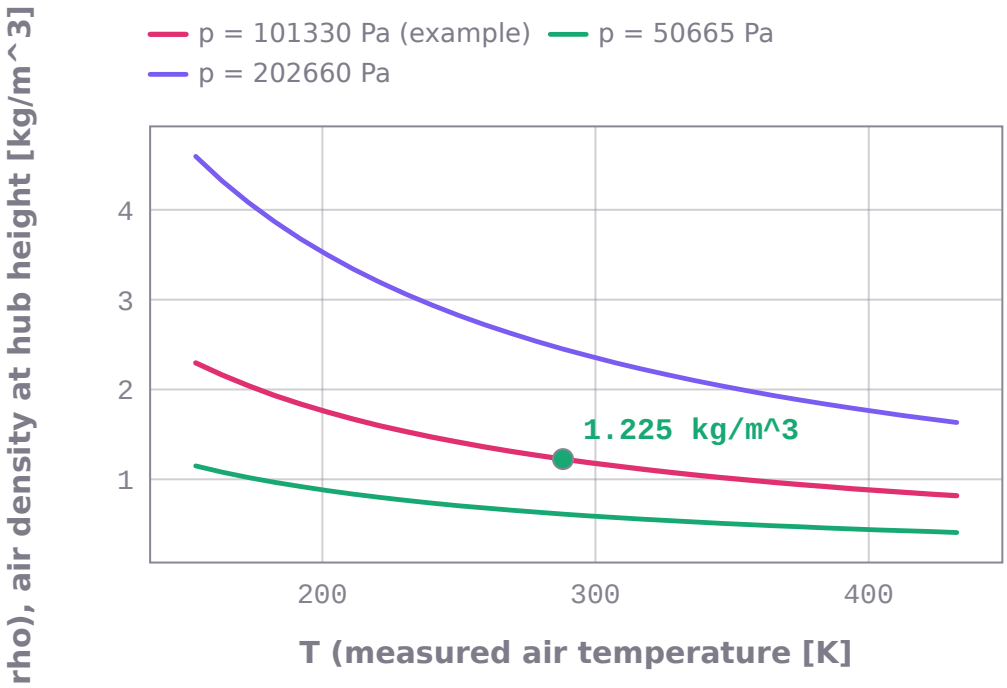
$$\rho_{hub} = 1.225 \, \frac{p_{hub}}{101330 \text{ Pa}} \, \frac{288.15 \text{ K}}{T_{hub}}$$

input	meaning	condition
`p`	measured air pressure	
`h_p`	height of the pressure measurement	

input	meaning	condition
`T`	measured air temperature	
`h_T`	height of the temperature measurement	
`hub`	hub height	
`model`	density model (default barometric)	

output	meaning
`T_hub`	temperature at hub height
`p_hub`	pressure at hub height
`rho`	air density at hub height

- **Method:** T_hub = T - 0.0065 (h_hub - h_T); p_hub = p - 12.5 Pa/m (h_hub - h_p); rho = p_hub/(R T_hub) (ideal gas) or 1.225 (p_hub/101330)(288.15/T_hub) (barometric)
- **Assumes:** pressure and temperature stay positive at hub height
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



rho (air density at hub height) against T (measured air temperature) from 144.075 K to 432.225 K. One curve for each value of p (measured air pressure): 101330 Pa, 50665 Pa, 202660 Pa. Other inputs as in the worked example: h_p = 0 m, h_T = 0 m, hub = 0 m. 30 points per curve computed by energy-systems-ai999x; marked: the worked example, T = 288.15 K, rho = 1.225 kg/m^3.

```
energy_systems_ai999x.solve('energy_hub_air_density', p=101330, h_p=0, T=288.15, h_T=0,
hub=0)
# -> T_hub = 288.15 K, p_hub = 101330 Pa, rho = 1.225 kg/m^3   (SOLVED)
```

```
energy_systems_ai999x.solve('energy_hub_air_density', p=101330, h_p=0, T=288.15, h_T=0,
hub=0, model='ideal_gas')
# -> T_hub = 288.15 K, p_hub = 101330 Pa, rho = 1.22504 kg/m^3   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Air density at hub height: p = 1013.3 hPa, h_p = 0 m, T = 288.15 K, h_T = 0 m, hub = 0 m	SOLVED	T_hub = 288.15 K, p_hub = 101330 Pa, rho = 1.225 kg/m^3
Air density at hub height: p = 1013 hPa, h_p = 0 m, T = 15 degC, h_T = 2 m	NEEDS_INPUT	energy_hub_air_density needs hub (hub height). Example: solve('energy_hub_air_density', p='101330 Pa', h_p='0 m', T='288

Turbine output from its power curve, `energy_power_curve_output`

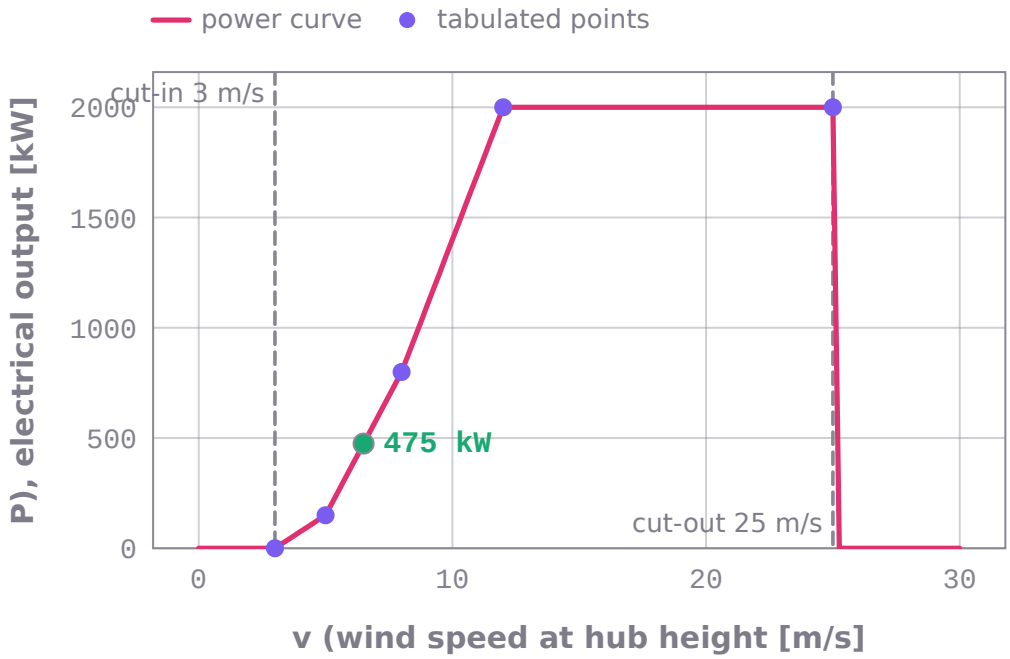
The electrical output of a wind turbine at a wind speed by linear interpolation of its power curve (zero below the first and above the last listed speed).

$$P(v) = P_i + (P_{i+1} - P_i)\frac{v - v_i}{v_{i+1} - v_i}$$

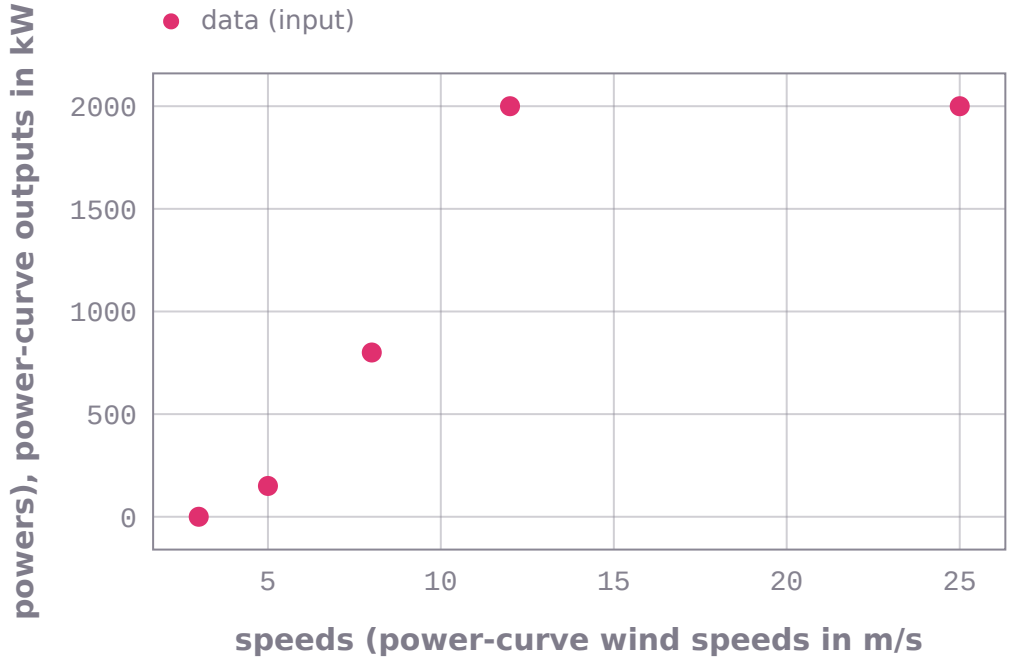
input	meaning	condition
`speeds`	power-curve wind speeds in m/s (increasing)	
`powers`	power-curve outputs in kW	
`v`	wind speed at hub height	

output	meaning
`P_kW`	output in kW
`P`	output

- **Method:** linear interpolation between the curve points; 0 outside the listed speeds
- **Assumes:** speeds strictly increasing; one power per speed
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Wind turbine output against wind speed from the tabulated power curve (speeds 3, 5, 8, 12, 25 m/s, powers 0, 150, 800, 2000, 2000 kW), computed by the package at 121 speeds from 0 to 30 m/s: nothing below cut-in at 3 m/s, rated 2000 kW up to cut-out at 25 m/s, nothing above. Marked: v = 6.5 m/s, P = 475 kW.



The data powers (power-curve outputs in kW) against speeds (power-curve wind speeds in m/s (increasing)) as given in the worked example (5 points). Also returned: P_kW = 475, P = 475000 W. Other inputs: v = 6.5 m/s.



P (output) against v (wind speed at hub height) from 0.65 m/s to 65 m/s on a logarithmic axis. Other inputs as in the worked example: speeds = 3, 5, 8, 12, 25, powers = 0, 150, 800, 2000, 2000. 41 points computed by energy-systems-ai999x; marked: the worked example, v = 6.5 m/s, P = 475000 W.

```
energy_systems_ai999x.solve('energy_power_curve_output', speeds='3, 5, 8, 12, 25',
powers='0, 150, 800, 2000, 2000', v=6.5)
# -> P_kW = 475, P = 475000 W   (SOLVED)
```

```
energy_systems_ai999x.solve('energy_power_curve_output', speeds=[3, 5, 8, 12, 25], powers=
[0, 150, 800, 2000, 2000], v=26)
# -> P_kW = 0, P = 0 W   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Power curve output: speeds = [3, 5, 8, 12, 25], powers = [0, 150, 800, 2000, 2000], v = 6.5 m/s	SOLVED	P_kW = 475, P = 475000 W
Power curve output: speeds = [3, 5, 4, 12, 25], powers = [0, 150, 800, 2000, 2000], v = 6.5 m/s	INVALID_INPUT	Turbine output from its power curve: the wind speeds of the curve must increase strictly.
Power curve output: speeds = [3, 5, 8, 12, 25], powers = [0, 150, 800, 2000, 2000]	NEEDS_INPUT	energy_power_curve_output needs v (wind speed at hub height). Example: solve('energy_power_curve_output', speeds='3, 5,

Turbine output from a density-corrected power curve, `energy_power_curve_density_corrected`

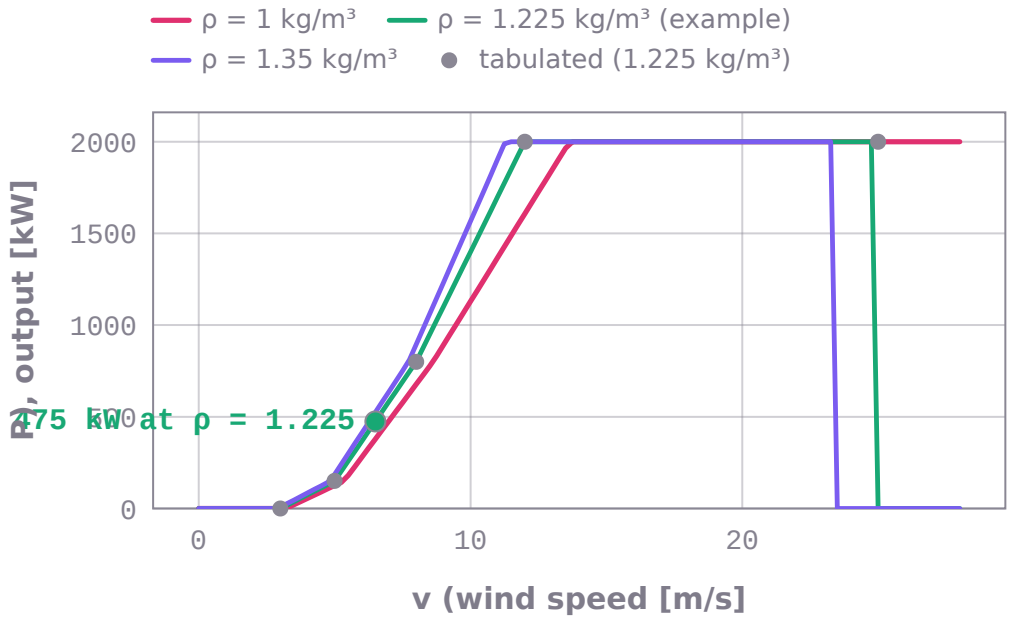
Turbine output at a wind speed and air density, with the power curve's speeds rescaled by $(1.225/\rho)^e$, e rising from $1/3$ below 7.5 m/s to $2/3$ above 12.5 m/s (Svenningsen's correction, as in windpowerlib).

$$v_i' = v_i \left(\frac{1.225}{\rho} \right)^{e(v_i)}, \; e \in [\tfrac{1}{3}, \tfrac{2}{3}]$$

input	meaning	condition
`speeds`	power-curve wind speeds in m/s at 1.225 kg/m^3 (increasing)	
`powers`	power-curve outputs in kW	
`v`	wind speed at hub height	
`rho`	air density at hub height	

output	meaning
`P_kW`	output in kW
`P`	output

- **Method:** v_i' = v_i $(1.225/\rho)^e$(v_i); interpolate P(v) on (v_i', P_i)
- **Assumes:** $0.5 < \rho < 1.6\ kg/m^3$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The worked example's tabulated power curve (speeds [3, 5, 8, 12, 25] m/s, powers [0, 150, 800, 2000, 2000] kW at standard air density) corrected by the package to 3 air densities, each curve a run per speed. Thinner air shifts the curve to higher speeds. Marked: the worked example, 6.5 m/s at ρ = 1.225 kg/m³ → 475 kW.

```
energy_systems_ai999x.solve('energy_power_curve_density_corrected', speeds=[3, 5, 8, 12,
25], powers=[0, 150, 800, 2000, 2000], v=6.5, rho=1.225)
# -> P_kW = 475, P = 475000 W   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Density-corrected power curve: speeds = [3, 5, 8, 12, 25], powers = [0, 150, 800, 2000, 2000], v = 6.5 m/s, rho = 1.225 kg/m^3	SOLVED	P_kW = 475, P = 475000 W
Density-corrected power curve: speeds = [3, 5, 8, 12, 25], powers = [0, 150, 800, 2000, 2000], v = 6.5 m/s, rho = 3 kg/m^3	INVALID_INPUT	Turbine output from a density-corrected power curve: the air density must lie between 0.5 and 1.6 kg/m^3.

Weibull wind statistics and mean power density, `energy_weibull_wind`

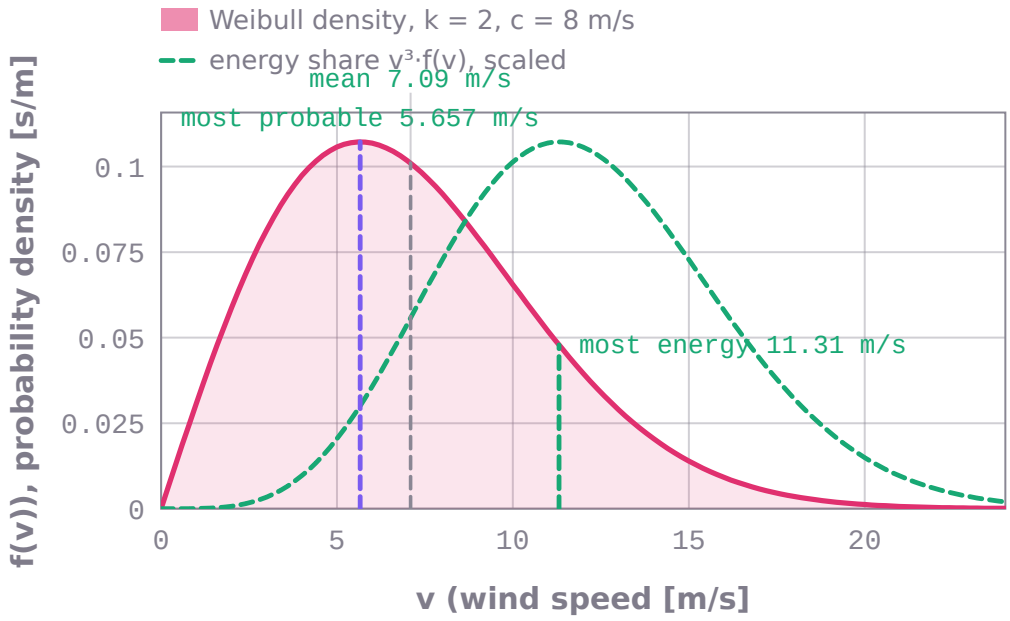
Mean wind speed, mean wind power density, energy pattern factor, most probable speed and the speed carrying most energy for a Weibull wind-speed distribution with shape k and scale c.

$$\bar{v} = c\,\Gamma\left(1+\tfrac{1}{k}\right), \quad \frac{\bar{P}}{A} = \tfrac{1}{2}\rho c^3\,\Gamma\left(1+\tfrac{3}{k}\right)$$

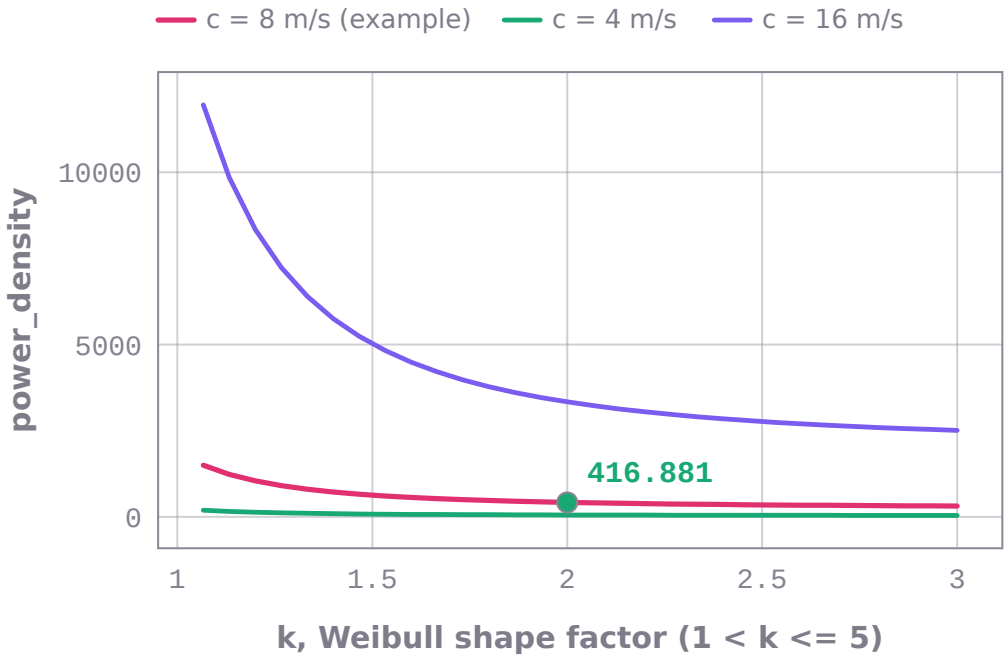
input	meaning	condition
`k`	Weibull shape factor $(1 < k \le 5)$	
`c`	Weibull scale factor	
`rho`	air density	

output	meaning
`v_mean`	mean wind speed c Gamma(1 + 1/k)
`power_density`	mean wind power density in W/m^2
`energy_pattern_factor`	mean of v^3 over the cube of the mean
`v_most_probable`	most probable wind speed
`v_max_energy`	wind speed carrying the most energy

- **Method:** v_mean = c Gamma(1 + 1/k); $P/A = (1/2)$ rho c^3 Gamma(1 + 3/k); $v_m p = c$ ((k - 1)/k)^(1/k); v_maxE = c ((k + 2)/k)^(1/k)
- **Assumes:** $1 < k \le 5$; $c > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Weibull wind-speed distribution with shape k = 2 and scale c = 8 m/s (the worked example). The density curve and the dashed energy-share curve are drawn from the stated k and c by the Weibull formula; the marked speeds are the package's: most probable 5.657 m/s, mean 7.09 m/s and the speed carrying the most energy 11.31 m/s. The package also returns the mean power density 416.9 W/m² (air density 1.225 kg/m³) and energy pattern factor 1.91.



power_density (mean wind power density in W/m^2) against k (Weibull shape factor (1 < k <= 5)) from 1 to 3. One curve for each value of c (Weibull scale factor): 8 m/s, 4 m/s, 16 m/s. Other inputs as in the worked example: rho = 1.225 kg/m^3. 30 points per curve computed by energy-systems-ai999x; marked: the worked example, k = 2, power_density = 416.881.

```
energy_systems_ai999x.solve('energy_weibull_wind', k=2, c=8, rho=1.225)
# -> v_mean = 7.08982 m/s, power_density = 416.881, energy_pattern_factor = 1.90986,
v_most_probable = 5.65685 m/s, v_max_energy = 11.3137 m/s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Weibull wind statistics: k = 2, c = 8 m/s, rho = 1.225 kg/m^3	SOLVED	v_mean = 7.08982 m/s, power_density = 416.881, energy_pattern_factor = 1.90986, v_most_probable = 5.65685 m/s, v_max_energy = 11.3137 m/s
Weibull wind statistics: k = 0.5, c = 8 m/s, rho = 1.225 kg/m^3	INVALID_INPUT	Weibull wind statistics and mean power density: the Weibull shape k must lie in (1, 5].

Annual energy production from a power curve and a Weibull distribution,

`energy_weibull_aep`

Mean output, annual energy production and capacity factor of a wind turbine whose power curve (linear between points, zero outside) operates in winds with a Weibull distribution (shape k, scale c).

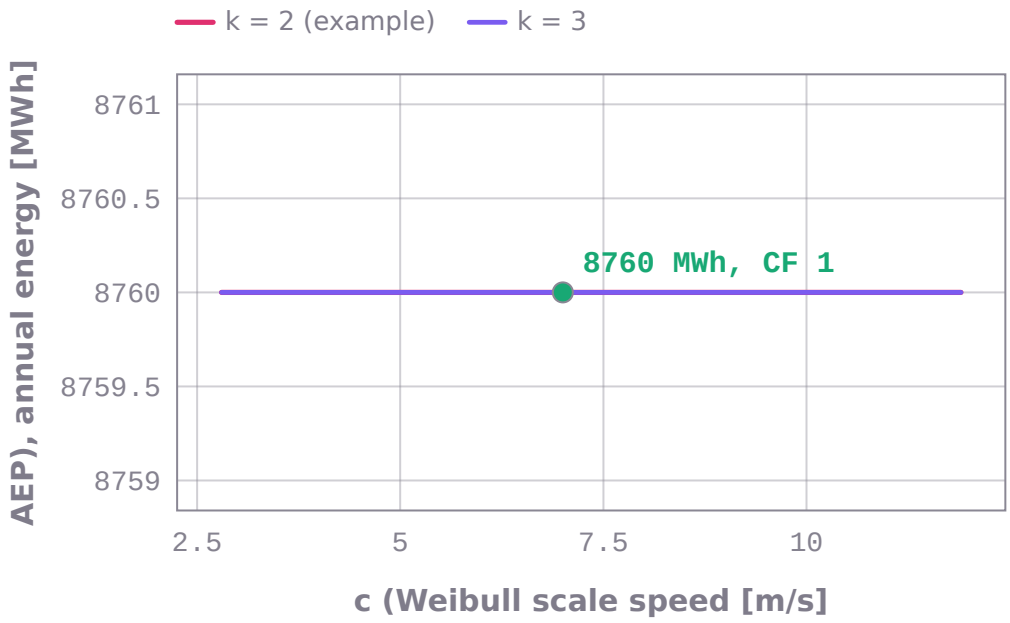
$$\bar{P} = \int_0^\infty P(v)\,\frac{k}{c}\left(\frac{v}{c}\right)^{k-1} e^{-(v/c)^k}\,dv, \quad AEP = 8760\,\mathrm{h}\cdot\bar{P}$$

input	meaning	condition
`speeds`	power-curve wind speeds in m/s (increasing)	
`powers`	power-curve outputs in kW	
`k`	Weibull shape factor $(1 < k \le 5)$	

input	meaning	condition
`c`	Weibull scale factor	
`hours`	hours in the period (default 8760)	

output	meaning
`mean_power_kW`	mean output in kW
`AEP_MWh`	energy over the period in MWh
`capacity_factor`	mean output over the largest output of the curve

- **Method:** P_mean = sum over segments of integral (P_i + s (v - v_i)) f(v) dv, each in closed form: F(b) - F(a) and c Gamma(1 + 1/k) [P($1 + 1/k$, (b/c)^k) - P($1 + 1/k$, (a/c)^k)]
- **Assumes:** $1 < k \leq 5$; speeds strictly increasing
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Annual energy of the worked example's turbine (power curve: speeds [0, 100] m/s, powers [1000, 1000] kW) in Weibull winds of shape k = 2 (and 3) against the scale speed c, each point one run of the package. Marked: the worked example, c = 7 m/s, gives 8760 MWh, mean power 1000 kW and capacity factor 1.

```
energy_systems_ai999x.solve('energy_weibull_aep', speeds=[0, 100], powers=[1000, 1000],
k=2, c=7)
# -> mean_power_kW = 1000, AEP_MWh = 8760, capacity_factor = 1   (SOLVED)
```

```
energy_systems_ai999x.solve('energy_weibull_aep', speeds=[0, 1000], powers=[0, 1000], k=2,
c=7)
# -> mean_power_kW = 6.20359, AEP_MWh = 54.3434, capacity_factor = 0.00620359   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Annual energy production: speeds = [0, 1000], powers = [0, 1000], k = 2, c = 7 m/s	SOLVED	mean_power_kW = 6.20359, AEP_MWh = 54.3434, capacity_factor = 0.00620359

question	outcome	result
Annual energy production: speeds = [3, 5, 8, 12, 25], powers = [0, 150, 800, 2000, 2000], k = 7, c = 7 m/s	INVALID_INPUT	Annual energy production from a power curve and a Weibull distribution: the Weibull shape k must lie in (1, 5].

Balanced three-phase power from line quantities, `energy_three_phase_power`

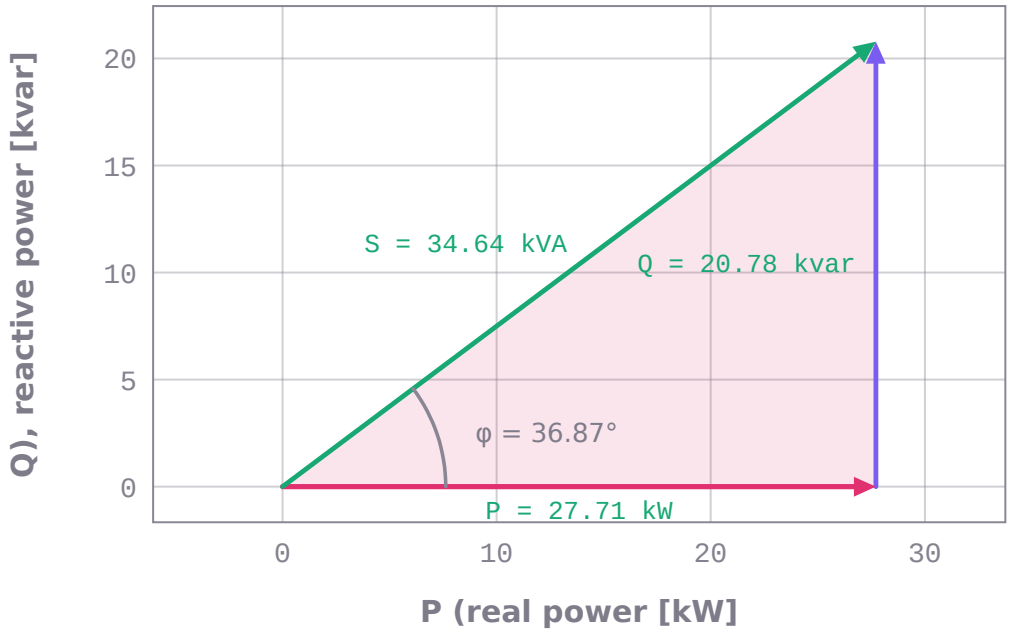
Real, reactive and apparent power of a balanced three-phase load from the line voltage, line current and power factor, with the phase voltage and phase current of a star or delta connection.

$$P = \sqrt{3}\,V_L I_L \cos\phi, \quad Q = \sqrt{3}\,V_L I_L \sin\phi$$

input	meaning	condition
`VL`	line-to-line rms voltage	
`IL`	line current (rms)	
`pf`	power factor $(0 < pf \leq 1)$	
`connection`	connection of the load (star = wye)	
`pf_type`	lagging (inductive, default) or leading (capacitive)	

output	meaning
`P`	real power (three-phase total)
`Q`	reactive power in var (positive lagging)
`S`	apparent power in VA
`V_phase`	voltage across each phase of the load
`I_phase`	current in each phase of the load

- **Method:** $S = sqrt$(3) V_L I_L, $P = S$ pf, $Q = S$ sin(acos pf); star: $V_{ph} = \frac{V_L}{\sqrt{3}}$, $I_ph = I_L$; delta: $V_ph = V_L$, $I_{ph} = \frac{I_L}{\sqrt{3}}$
- **Assumes:** balanced load; $0 < pf \leq 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Power triangle of the worked example: a star-connected load at 400 V line, 50 A, power factor 0.8. The package returns P = 27.71 kW, Q = 20.78 kvar and S = 34.64 kVA (phase voltage 230.9 V, phase current 50 A); φ = atan(Q/P) = 36.87° is derived from the returned P and Q.

```
energy_systems_ai999x.solve('energy_three_phase_power', VL=400, IL=50, pf=0.8,
connection='star')
# -> P = 27712.8 W, Q = 20784.6 var, S = 34641 VA, V_phase = 230.94 V, I_phase = 50 A
(SOLVED)
```

```
energy_systems_ai999x.solve('energy_three_phase_power', VL=400, IL=17.32050807568877,
pf=1.0, connection='delta')
# -> P = 12000 W, Q = 0 var, S = 12000 VA, V_phase = 400 V, I_phase = 10 A   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Three-phase power: VL = 400 V, IL = 50 A, pf = 0.8, connection = star	SOLVED	P = 27712.8 W, Q = 20784.6 var, S = 34641 VA, V_phase = 230.94 V, I_phase = 50 A
Three-phase power: VL = 400 V, IL = 50 A, pf = 1.2, connection = star	INVALID_INPUT	Balanced three-phase power from line quantities: the power factor must lie in (0, 1].
Three-phase power: VL = 400 V, IL = 50 A, pf = 0.8	NEEDS_INPUT	energy_three_phase_power needs connection (connection of the load (star = wye)). Example: solve('energy_three_phase_powe

Per-unit impedance on a new base, `energy_per_unit_change_of_base`

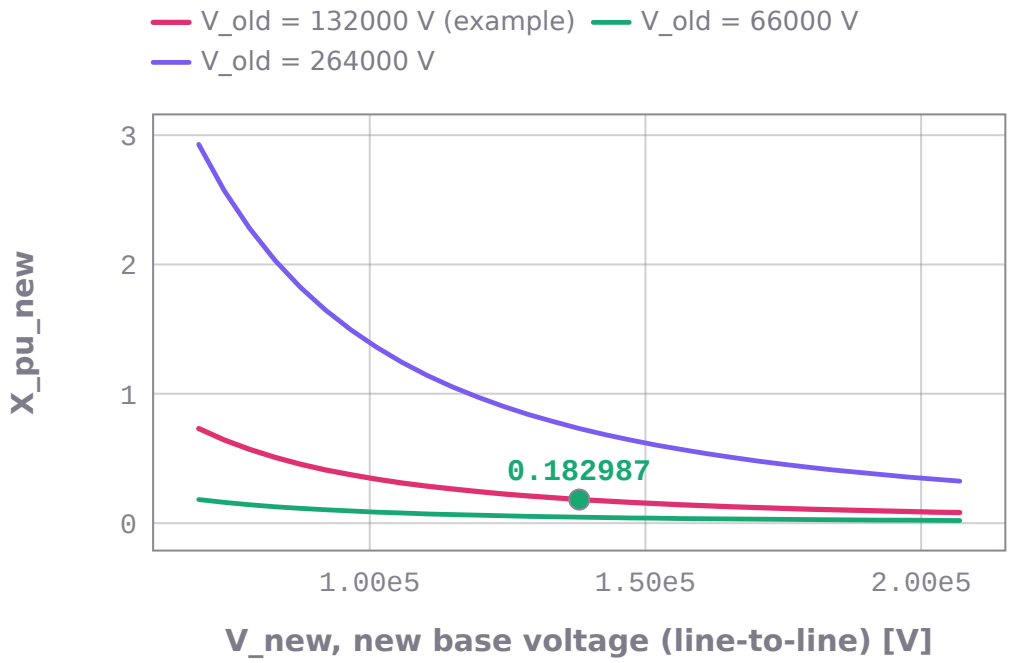
An impedance given in per unit on equipment ratings (S_old, V_old) converted to the per-unit system of the network (S_new, V_new).

$$Z_{pu,new} = Z_{pu,old}\left(\frac{V_{old}}{V_{new}}\right)^2 \frac{S_{new}}{S_{old}}$$

input	meaning	condition
`R_pu`	per-unit resistance on the old base	
`X_pu`	per-unit reactance on the old base	
`S_old`	old base (equipment rating) in VA	
`V_old`	old base voltage (line-to-line)	
`S_new`	new (system) base power in VA	
`V_new`	new base voltage (line-to-line)	

output	meaning
`R_pu_new`	per-unit resistance on the new base
`X_pu_new`	per-unit reactance on the new base
`Z_pu_new`	per-unit impedance magnitude on the new base

- **Method:** Z_new = Z_old (V_old/V_new)^2 (S_new/S_old)
- **Assumes:** $X_p u > 0$; positive bases
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



X_pu_new (per-unit reactance on the new base) against V_new (new base voltage (line-to-line)) from 69000 V to 207000 V. One curve for each value of V_old (old base voltage (line-to-line)): 132000 V, 66000 V, 264000 V. Other inputs as in the worked example: R_pu = 0.0, X_pu = 0.1, S_old = 50000000.0 VA, S_new = 100000000.0 VA. 31 points per curve computed by energy-systems-ai999x; marked: the worked example, V_new = 138000 V, X_pu_new = 0.182987.

```
energy_systems_ai999x.solve('energy_per_unit_change_of_base', R_pu=0.0, X_pu=0.1,
S_old=50000000.0, V_old=132000.0, S_new=100000000.0, V_new=138000.0)
# -> R_pu_new = 0, X_pu_new = 0.182987, Z_pu_new = 0.182987   (SOLVED)
```

```
energy_systems_ai999x.solve('energy_per_unit_change_of_base', R_pu=0.01, X_pu=0.1,
S_old=10000000.0, V_old=11000.0, S_new=100000000.0, V_new=11000.0)
# -> R_pu_new = 0.1, X_pu_new = 1, Z_pu_new = 1.00499   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Per-unit change of base: R_pu = 0.01, X_pu = 0.1, S_old = 10 MVA, V_old = 11 kV, S_new = 100 MVA, V_new = 11 kV	SOLVED	R_pu_new = 0.1, X_pu_new = 1, Z_pu_new = 1.00499
Per-unit change of base: R_pu = 0.01, X_pu = 0.1, S_old = 10 MVA, V_old = 11 kV, S_new = 100 MVA	NEEDS_INPUT	energy_per_unit_change_of_base needs V_new (new base voltage (line-to-line)). Example: solve('energy_per_unit_change_of_

Short transmission line: sending-end voltage, regulation and efficiency, `energy_short_line`

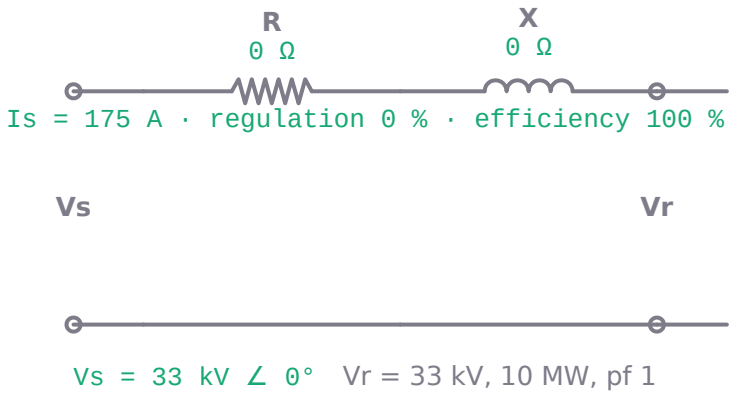
Sending-end voltage and its angle, line current, losses, voltage regulation and efficiency of a short three-phase line (series R + jX, no shunt admittance) feeding a load at a known receiving-end voltage.

$$\mathbf{V}_s = \mathbf{V}_r + \mathbf{I}\,(R + jX), \quad \mathrm{VR} = \frac{|V_s| - |V_r|}{|V_r|}$$

input	meaning	condition
`Vr`	receiving-end line-to-line voltage	
`P`	load real power (three-phase)	
`pf`	load power factor	
`R`	line resistance per phase (total)	
`X`	line reactance per phase (total)	
`pf_type`	lagging (default) or leading load	

output	meaning
`Vs`	sending-end line voltage
`delta_deg`	angle of the sending voltage ahead of the receiving voltage, degrees
`I`	receiving-end line current
`losses`	line losses (three-phase)
`Ps`	sending-end real power
`regulation_pct`	voltage regulation in percent
`efficiency_pct`	transmission efficiency in percent

- **Method:** $V_s = V_r + I$ (R + jX) per phase, I = P/(sqrt(3) V_r pf) at angle -acos(pf); regulation = (V_s - V_r)/V_r; efficiency = P/(P + 3 I^2 R)
- **Assumes:** short line: shunt admittance neglected; balanced three-phase load
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Short-line model: a series impedance 0 + j0 Ω (the worked example's) carrying 10 MW at power factor 1 lagging to a 33 kV receiving end. The package returns the sending voltage 33 kV at 0°, current 175 A, losses 0 kW, regulation 0 % and efficiency 100 %. Marked: every returned value.

```
energy_systems_ai999x.solve('energy_short_line', Vr=33000.0, P=10000000.0, pf=1.0, R=0.0,
X=0.0)
# -> Vs = 33000 V, delta_deg = 0, I = 174.955 A, losses = 0 W, Ps = 1e+07 W, regulation_pct
= 0, efficiency_pct = 100   (SOLVED)
```

```
energy_systems_ai999x.solve('energy_short_line', Vr=1732.0508075688772, P=300000, pf=1.0,
R=1.0, X=0.0)
# -> Vs = 1905.26 V, delta_deg = 0, I = 100 A, losses = 30000 W, Ps = 330000 W,
regulation_pct = 10, efficiency_pct = 90.9091   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Short line: Vr = 33 kV, P = 10 MW, pf = 1, R = 0 ohm, X = 0 ohm	SOLVED	Vs = 33000 V, delta_deg = 0, I = 174.955 A, losses = 0 W, Ps = 1e+07 W, regulation_pct = 0, efficiency_pct = 100
Short line: Vr = 33 kV, P = 10 MW, pf = 0.8, R = 5 ohm, X = 20 ohm, pf_type = sideways	INVALID_INPUT	pf_type: one of lagging, leading; got 'sideways'.
Short line: Vr = 33 kV, P = 10 MW, pf = 0.8, R = 5 ohm	NEEDS_INPUT	energy_short_line needs X (line reactance per phase (total)). Example: solve('energy_short_line', Vr='33000 V', P='1e+07

Medium transmission line (nominal pi), `energy_medium_line`

ABCD constants of the nominal-pi model and the sending-end voltage, current and power, losses, regulation and efficiency of a medium-length three-phase line feeding a load.

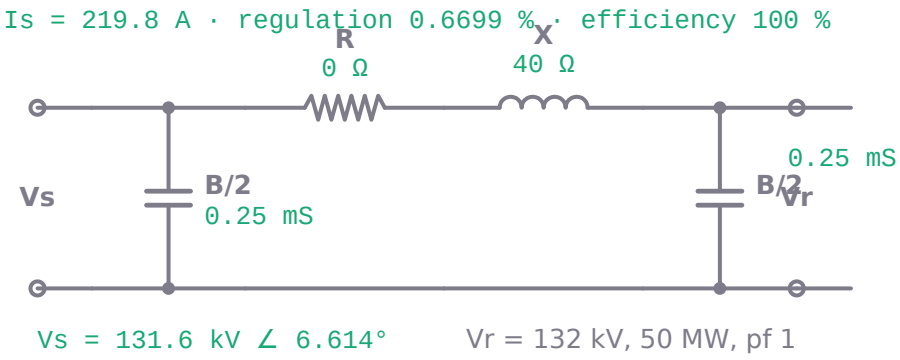
$$\begin{bmatrix} V_s \\ I_s \end{bmatrix} = \begin{bmatrix} 1 + \frac{ZY}{2} & Z \\ Y\left(1 + \frac{ZY}{4}\right) & 1 + \frac{ZY}{2} \end{bmatrix} \begin{bmatrix} V_r \\ I_r \end{bmatrix}$$

input	meaning	condition
`Vr`	receiving-end line-to-line voltage	
`P`	load real power (three-phase)	
`pf`	load power factor	

input	meaning	condition
`R`	line resistance per phase (total)	
`X`	line reactance per phase (total)	
`B`	total shunt susceptance of the line per phase	
`pf_type`	lagging (default) or leading load	

output	meaning
`Vs`	sending-end line voltage
`delta_deg`	angle of the sending voltage ahead of the receiving voltage, degrees
`Is`	sending-end current
`Ps`	sending-end real power
`losses`	line losses (three-phase)
`regulation_pct`	voltage regulation (V_s/|A| - V_r)/V_r in percent
`efficiency_pct`	transmission efficiency in percent
`A_mag`	|A| (= |D|)
`A_angle_deg`	angle of A in degrees
`B_mag`	|B| in ohms
`B_angle_deg`	angle of B in degrees

- **Method:** $A = D = 1 + ZY/2$, $B = Z$, $C = Y$(1 + ZY/4); $V_s = A$ V_r + B I_r, $I_s = C$ V_r + D I_r; regulation = (V_s/|A| - V_r)/V_r
- **Assumes:** nominal-pi model (half the shunt admittance at each end); balanced load
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Nominal-π model of a medium line: series 0 + j40 Ω, half the shunt susceptance (0.5 mS in all) at each end (the worked example's), delivering 50 MW at power factor 1 lagging at 132 kV. The package returns Vs = 131.6 kV at 6.614°, Is = 219.8 A, losses 0 kW, regulation 0.6699 %, efficiency 100 %, and the constants A = 0.99∠0°, B = 40∠90° Ω.

```
energy_systems_ai999x.solve('energy_medium_line', Vr=132000.0, P=50000000.0, pf=1.0, R=0.0,
X=40.0, B=0.0005)
# -> Vs = 131555 V, delta_deg = 6.61355, Is = 219.801 A, Ps = 5e+07 W, losses = 0 W,
regulation_pct = 0.669903, efficiency_pct = 100, A_mag = 0.99, A_angle_deg = 0, B_mag = 40
ohm, B_angle_deg = 90   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Medium line: Vr = 132 kV, P = 50 MW, pf = 1, R = 0 ohm, X = 40 ohm, B = 500 uS	SOLVED	Vs = 131555 V, delta_deg = 6.61355, Is = 219.801 A, Ps = 5e+07 W, losses = 0 W, regulation_pct = 0.669903, efficiency_pct = 100, A_mag = 0.99, A_angle_deg = 0, B_mag = 40 ohm, B_angle_deg = 90
Medium line: Vr = 132 kV, P = 50 MW, pf = 0.8, R = 10 ohm, X = 40 ohm	NEEDS_INPUT	energy_medium_line needs B (total shunt susceptance of the line per phase). Example: solve('energy_medium_line', Vr='132

Long transmission line (exact distributed parameters), `energy_long_line`

Characteristic impedance, propagation constant, exact ABCD constants and the sending-end quantities, regulation and efficiency of a long three-phase line with uniformly distributed R, X and shunt B (the Ferranti rise appears as a negative regulation at light load).

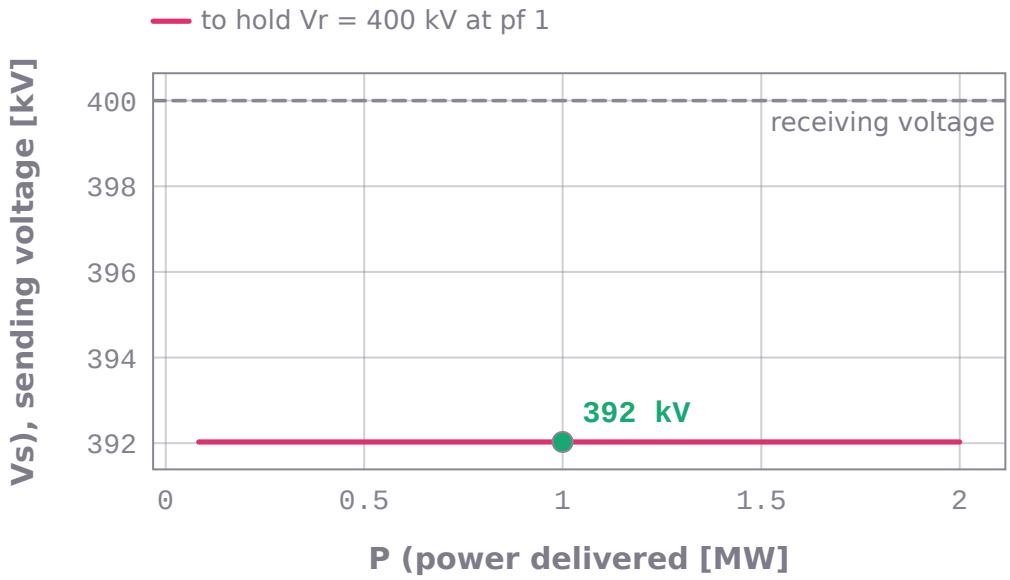
$$A = \cosh \gamma l,\ B = Z_c \sinh \gamma l,\ C = \frac{\sinh \gamma l}{Z_c}, \quad Z_c = \sqrt{\frac{z}{y}},\ \gamma = \sqrt{zy}$$

input	meaning	condition
`Vr`	receiving-end line-to-line voltage	
`P`	load real power (three-phase)	
`pf`	load power factor	
`R`	line resistance per phase (total)	
`X`	line reactance per phase (total)	
`B`	total shunt susceptance of the line per phase	
`pf_type`	lagging (default) or leading load	

output	meaning
`Vs`	sending-end line voltage
`delta_deg`	angle of the sending voltage ahead of the receiving voltage, degrees
`Is`	sending-end current
`Ps`	sending-end real power
`losses`	line losses (three-phase)
`regulation_pct`	voltage regulation (V_s/|A| - V_r)/V_r in percent
`efficiency_pct`	transmission efficiency in percent
`A_mag`	|A| (= |D|)
`A_angle_deg`	angle of A in degrees
`B_mag`	|B| in ohms
`B_angle_deg`	angle of B in degrees
`Zc_mag`	characteristic (surge) impedance magnitude
`Zc_angle_deg`	angle of Zc in degrees
`alpha_l`	attenuation alpha l in nepers

output	meaning
`beta_l`	phase shift beta l in radians

- **Method:** gamma $l = \sqrt{ZY}$, $Zc = \sqrt{\frac{Z}{Y}}$; A = D = cosh(gamma l), $B = Zc$ sinh(gamma l), C = sinh(gamma l)/Zc
- **Assumes:** uniformly distributed parameters; balanced load
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



A long line with the worked example's totals R = 0 Ω, X = 100 Ω, B = 0.4 mS (distributed parameters): the sending voltage the package returns to hold 400 kV at the receiving end, against the power delivered at power factor 1 lagging, light loads need less than the receiving voltage (line charging). Marked: at 1 MW, Vs = 392 kV at 0.0363°, efficiency 100 %; the package's constants A = 0.9801∠0°, B = 99.33∠90° Ω, surge impedance 500 Ω, βl = 0.2 rad.

```
energy_systems_ai999x.solve('energy_long_line', Vr=400000.0, P=1000000.0, pf=1.0, R=0.0,
X=100.0, B=0.0004)
# -> Vs = 392027 V, delta_deg = 0.0362951, Is = 91.7723 A, Ps = 1e+06 W, losses = 0 W,
regulation_pct = 2.00641e-05, efficiency_pct = 100, A_mag = 0.980067, A_angle_deg = 0,
B_mag = 99.3347 ohm, B_angle_deg = 90, Zc_mag = 500 ohm, Zc_angle_deg = 0, alpha_l = 0,
beta_l = 0.2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Long line: Vr = 400 kV, P = 1 MW, pf = 1, R = 0 ohm, X = 100 ohm, B = 400 uS	SOLVED	Vs = 392027 V, delta_deg = 0.0362951, Is = 91.7723 A, Ps = 1e+06 W, losses = 0 W, regulation_pct = 2.00641e-05, efficiency_pct = 100, A_mag = 0.980067, A_angle_deg = 0, B_mag = 99.3347 ohm, B_angle_deg = 90, Zc_mag = 500 ohm, Zc_angle_deg = 0, alpha_l = 0, beta_l = 0.2
Long line: Vr = 400 kV, P = 1 MW, pf = 1, R = 0 ohm, X = 100 ohm, B = 0 uS	INVALID_INPUT	Long transmission line (exact distributed parameters): the shunt susceptance must be a positive number.

Inductance, capacitance and surge impedance of a three-phase overhead line, `energy_line_parameters`

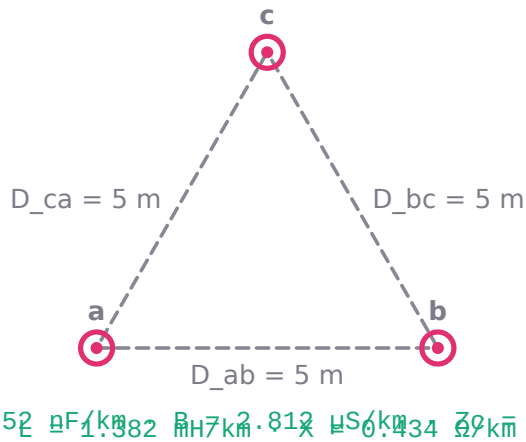
Per-phase inductance, capacitance, reactance, susceptance, surge impedance (and surge-impedance loading at a voltage) of a fully transposed three-phase line from its phase spacings, conductor radius and geometric mean radius, with optional bundled conductors.

$$L = 2\times10^{-7}\ln\frac{GMD}{GMR}\ \mathrm{H/m},\quad C = \frac{2\pi\varepsilon_0}{\ln(GMD/r)}\ \mathrm{F/m}$$

input	meaning	condition
`D_ab`	distance between phases a and b	
`D_bc`	distance between phases b and c	
`D_ca`	distance between phases c and a	
`r`	conductor (outer) radius	
`f`	system frequency	
`gmr`	conductor geometric mean radius (default r $e^{-1/4}$, a solid round conductor)	
`n_bundle`	subconductors per phase (1 to 4, regular polygon)	
`spacing`	distance between neighbouring subconductors of a bundle	
`V`	line voltage for the surge-impedance loading (optional)	

output	meaning
`GMR`	geometric mean radius of a phase (bundle)
`L_mH_per_km`	inductance per phase in mH/km
`C_nF_per_km`	capacitance to neutral in nF/km
`X_ohm_per_km`	series reactance in ohm/km
`B_uS_per_km`	shunt susceptance in uS/km
`Zc`	surge impedance of the lossless line sqrt(L/C)
`SIL_MW`	surge-impedance loading V^2/Zc in MW

- **Method:** GMD = (D_ab D_bc D_ca)^(1/3); $L = 2e-7$ ln(GMD/GMR) H/m; $C = 2$ pi eps0/ln(GMD/r) F/m; bundles: GMR_b = (GMR d^{n-1})^$(1/n)$ (2^(1/8) (GMR d^3)^(1/4) = 1.09 (GMR d^3)^(1/4) for 4)
- **Assumes:** fully transposed line; spacings much larger than the conductors; ground effect neglected
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The worked example's three-phase arrangement drawn to scale (spacings 5, 5 and 5 m; conductor radius 10 mm, GMR 5 mm, 50 Hz, single conductors). Marked: the package's per-phase inductance 1.382 mH/km, reactance 0.434 Ω/km, capacitance 8.952 nF/km, susceptance 2.812 µS/km and surge impedance 392.8 Ω.

```
energy_systems_ai999x.solve('energy_line_parameters', D_ab=5, D_bc=5, D_ca=5, r=0.01, f=50,
gmr=0.005)
# -> GMR = 0.005 m, L_mH_per_km = 1.38155, C_nF_per_km = 8.95189, X_ohm_per_km = 0.434027,
B_uS_per_km = 2.81232, Zc = 392.849 ohm   (SOLVED)
```

```
energy_systems_ai999x.solve('energy_line_parameters', D_ab=6, D_bc=6, D_ca=6, r=0.012,
f=60, gmr=0.01, n_bundle=2, spacing=0.4)
# -> GMR = 0.0632456 m, L_mH_per_km = 0.910498, C_nF_per_km = 12.4699, X_ohm_per_km =
0.34325, B_uS_per_km = 4.70106, Zc = 270.214 ohm   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Line parameters: D_ab = 5 m, D_bc = 5 m, D_ca = 5 m, r = 1 cm, f = 50 Hz, gmr = 5 mm	SOLVED	GMR = 0.005 m, L_mH_per_km = 1.38155, C_nF_per_km = 8.95189, X_ohm_per_km = 0.434027, B_uS_per_km = 2.81232, Zc = 392.849 ohm
Line parameters: D_ab = 5 m, D_bc = 5 m, D_ca = 5 m, r = 1 cm, f = 50 Hz, gmr = 2 cm	INVALID_INPUT	Inductance, capacitance and surge impedance of a three-phase overhead line: the geometric mean radius cannot exceed the

Receiving-end voltage of a line feeding a constant-power load (two-bus load flow), `energy_two_bus_load_flow`

The receiving-end voltage magnitude and angle, current and losses when a three-phase line of impedance R + jX supplies a constant P + jQ load from a fixed sending-end voltage (the exact two-bus load flow).

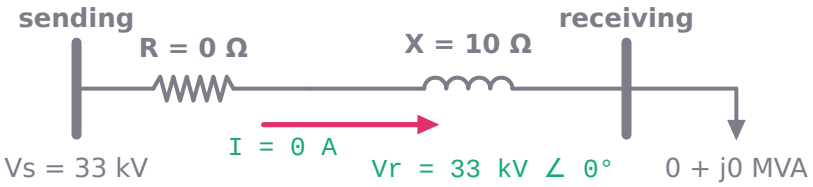
$$V_r^4 + \left[2(PR + QX) - V_s^2\right]V_r^2 + (P^2 + Q^2)(R^2 + X^2) = 0$$

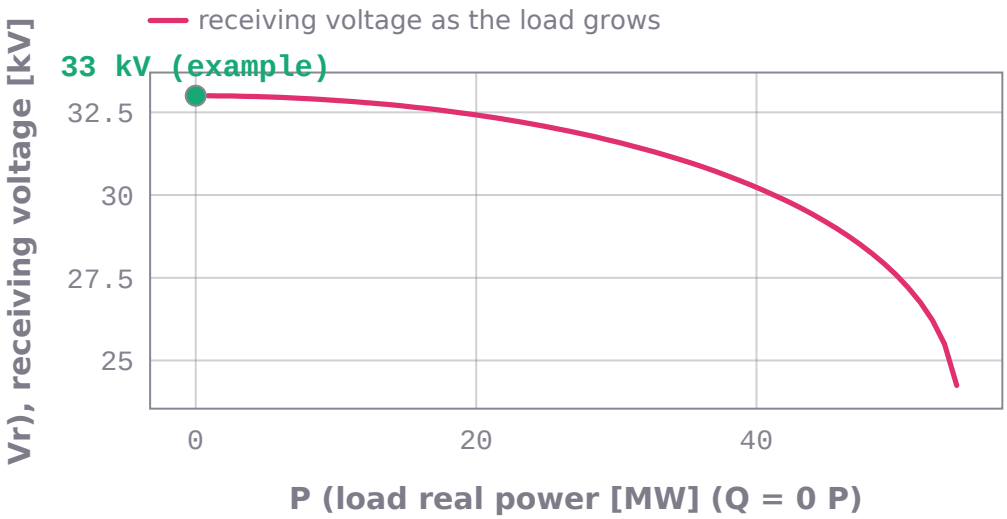
input	meaning	condition
`Vs`	sending-end line-to-line voltage (slack)	
`R`	line resistance per phase	
`X`	line reactance per phase	

input	meaning	condition
`P`	load real power (three-phase)	
`Q`	load reactive power in var (negative for a capacitive load)	

output	meaning
`Vr`	receiving-end line voltage
`delta_deg`	receiving-end voltage angle in degrees (negative: lagging the source)
`I`	line current
`P_loss`	real power loss
`Q_loss`	reactive power absorbed by the line in var
`voltage_drop_pct`	(V_s - V_r)/V_s in percent

- **Method:** V_r^4 + [2(PR + QX) - V_s^2] $V_r^2 + (P^2 + Q^2)(R^2 + X^2) = 0$, larger root; angle from $V_s = V_r + (PR + QX)/V_r + j$(PX - QR)/V_r
- **Assumes:** the load is within the line's transfer limit (a real solution exists); the high-voltage (normal) solution is reported
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)





A 33 kV source feeding a load of 0 + j0 MVA over a line of R = 0 Ω, X = 10 Ω (the worked example). Top: the package returns Vr = 33 kV at δ = 0°, I = 0 A, voltage drop 0 %, losses 0 kW. Bottom: Vr against the load at the same power factor (Q = 0 P), one run per point; points the package refuses (no voltage solution) are left out, so the curve stops near the largest load the line can carry (54.28 MW drawn). Marked: the worked example's load.

```
energy_systems_ai999x.solve('energy_two_bus_load_flow', Vs=33000.0, R=0.0, X=10.0, P=0.0,
Q=0.0)
# -> Vr = 33000 V, delta_deg = 0, I = 0 A, P_loss = 0 W, Q_loss = 0 var, voltage_drop_pct =
0   (SOLVED)
```

```
energy_systems_ai999x.solve('energy_two_bus_load_flow', Vs=2000.0, R=0.0, X=1.0,
P=1000000.0, Q=0.0)
# -> Vr = 1931.85 V, delta_deg = -15, I = 298.858 A, P_loss = 0 W, Q_loss = 267949 var,
voltage_drop_pct = 3.40742   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Two-bus load flow: Vs = 2 kV, R = 0 ohm, X = 1 ohm, P = 1 MW, Q = 0 var	SOLVED	Vr = 1931.85 V, delta_deg = -15, I = 298.858 A, P_loss = 0 W, Q_loss = 267949 var, voltage_drop_pct = 3.40742
Two-bus load flow: Vs = 33 kV, R = 3 ohm, X = 12 ohm, P = 100 MW, Q = 50 Mvar	INVALID_INPUT	Receiving-end voltage of a line feeding a constant-power load (two-bus load flow): the load exceeds what the line can de

DC power flow, `energy_dc_power_flow`

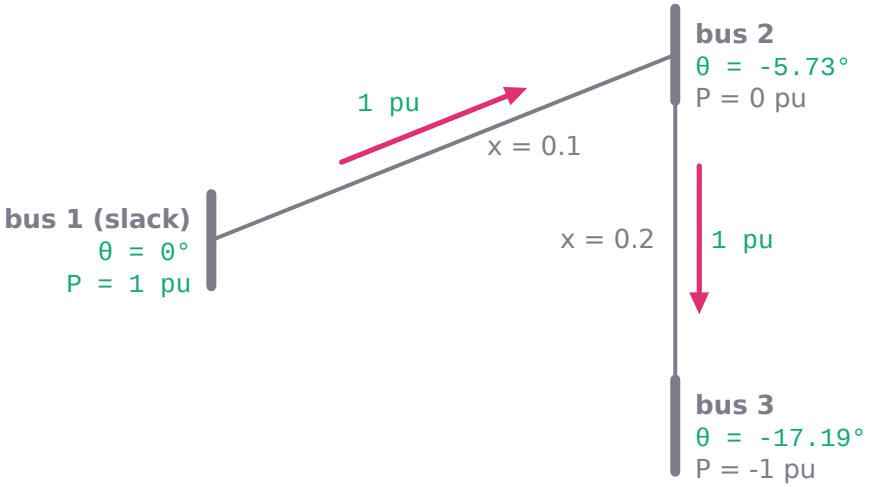
Bus voltage angles and line flows of a network by the DC (linearised) power flow: lossless lines, flat 1 pu voltages, $P_ij = (theta_i - theta_j)/x_ij$; bus 1 is the slack (angle 0) and takes the balance.

$$\mathbf{B}'\boldsymbol{\theta} = \mathbf{P}, \quad P_{ij} = \frac{\theta_i - \theta_j}{x_{ij}}$$

input	meaning	condition
`lines`	lines as 'i-j x' with the series reactance x in per unit (buses numbered 1..n)	
`P`	net injections (generation minus load) of buses 2..n in per unit	

output	meaning
`theta_deg`	bus voltage angles in degrees, buses 1..n
`flows`	line flows from the first to the second named bus, per unit, in the order given
`P_slack`	slack (bus 1) injection in per unit

- **Method:** B' $\theta = P$ (reduced susceptance matrix, slack removed), solved by Gaussian elimination; $P_ij = (theta_i - theta_j)/x_ij$
- **Assumes:** connected network; positive reactances; bus 1 is the slack
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



DC power flow of the worked example's 3-bus network (lines 1-2 0.1, 2-3 0.2: from–to and series reactance in per unit; injections 0, -1 pu at buses 2, 3, bus 1 the slack). The package returns the bus angles θ = 0°, -5.73°, -17.19°, the line flows 1, 1 pu (arrows point the way power flows) and the slack injection 1 pu. Marked: every angle, flow and injection.

```
energy_systems_ai999x.solve('energy_dc_power_flow', lines='1-2 0.1, 2-3 0.2', P='0, -1')
# -> theta_deg = [0.0, -5.72957795131, -17.1887338539], flows = [1.0, 1.0], P_slack = 1
(SOLVED)
```

```
energy_systems_ai999x.solve('energy_dc_power_flow', lines='1-2 0.1, 1-2 0.1', P='-1')
# -> theta_deg = [0.0, -2.86478897565], flows = [0.5, 0.5], P_slack = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
DC power flow: lines = 1-2 0.1; 2-3 0.2, P = [0, -1]	SOLVED	theta_deg = [0.0, -5.72957795131, -17.1887338539], flows = [1.0, 1.0], P_slack = 1
DC power flow: lines = 1-2 0.1; 2-3 0.2, P = [0, -1, 2]	INVALID_INPUT	DC power flow: give the net injections of buses 2 to 3 (2 values); bus 1 is the slack.
DC power flow: lines = 1-2 0.1; 2-3 0.2	NEEDS_INPUT	energy_dc_power_flow needs P (net injections (generation minus load) of buses 2..n in per unit). Example: solve('energy_

AC load flow by Newton-Raphson, `energy_newton_load_flow`

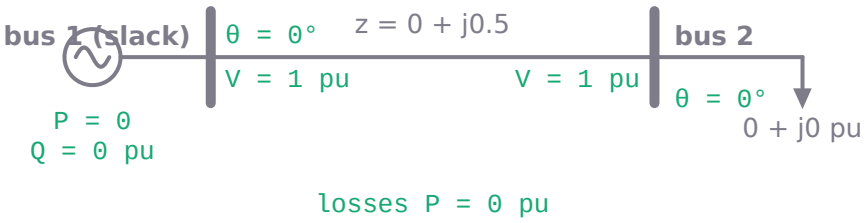
Bus voltage magnitudes and angles, slack-bus power, PV-bus reactive power and losses of a small network by the Newton-Raphson load flow (per unit; pi-model lines; bus 1 is the slack, generator buses hold their voltage; reactive limits are not enforced).

$$\begin{bmatrix}\Delta\mathbf{P}\\ \Delta\mathbf{Q}\end{bmatrix} = \begin{bmatrix}\mathbf{H} & \mathbf{N}\\ \mathbf{M} & \mathbf{L}\end{bmatrix}\begin{bmatrix}\Delta\boldsymbol{\theta}\\ \Delta|\mathbf{V}|\end{bmatrix}$$

input	meaning	condition
`lines`	one row per line: from bus, to bus, r, x, total shunt b (per unit)	
`loads`	one row per load: bus, P, Q (per unit)	
`gens`	optional rows per voltage-controlled (PV) generator: bus, P, V (per unit)	
`V1`	slack-bus voltage magnitude in per unit (default 1.0)	

output	meaning
`V`	voltage magnitudes in per unit, buses 1..n
`angle_deg`	voltage angles in degrees, buses 1..n
`P_slack`	slack real power injection (pu)
`Q_slack`	slack reactive power injection (pu)
`P_loss`	total real power loss (pu)
`Q_gen`	reactive power of each PV generator (pu), in the order given

- **Method:** polar Newton-Raphson on the P and Q mismatches with the analytic Jacobian, solved by Gaussian elimination, to a mismatch below $1e-13$ pu
- **Assumes:** buses numbered 1..n, bus 1 the slack; the load flow converges from a flat start
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



AC load flow (Newton–Raphson) of the worked example's 2-bus network: lines 1 2 0.0 0.5 0.0 (from, to, r, x, total shunt b in per unit), loads 2 0.0 0.0 (bus, P, Q in pu), bus 1 the slack at 1 pu. The package returns the voltages 1∠0°, 1∠0° pu, the slack injection P = 0, Q = 0 pu and the real power loss 0 pu. Marked: every returned value.

```
energy_systems_ai999x.solve('energy_newton_load_flow', lines='1 2 0.0 0.5 0.0', loads='2
0.0 0.0')
# -> V = [1.0, 1.0], angle_deg = [0.0, 0.0], P_slack = 0, Q_slack = 0, P_loss = 0, Q_gen =
[]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Load flow: lines = [1 2 0 0.5 0], loads = [2 0 0]	SOLVED	V = [1.0, 1.0], angle_deg = [0.0, 0.0], P_slack = 0, Q_slack = 0, P_loss = 0, Q_gen = []
Load flow: lines = [1 2 0.02 0.06 0.03], loads = [2 50 20]	INVALID_INPUT	AC load flow by Newton-Raphson: the load flow diverged (the network cannot carry this loading).
Load flow: loads = [2 0.5 0.2]	NEEDS_INPUT	energy_newton_load_flow needs lines (one row per line: from bus, to bus, r, x, total shunt b (per unit)). Example: solve

Economic dispatch with quadratic costs and limits, `energy_economic_dispatch`

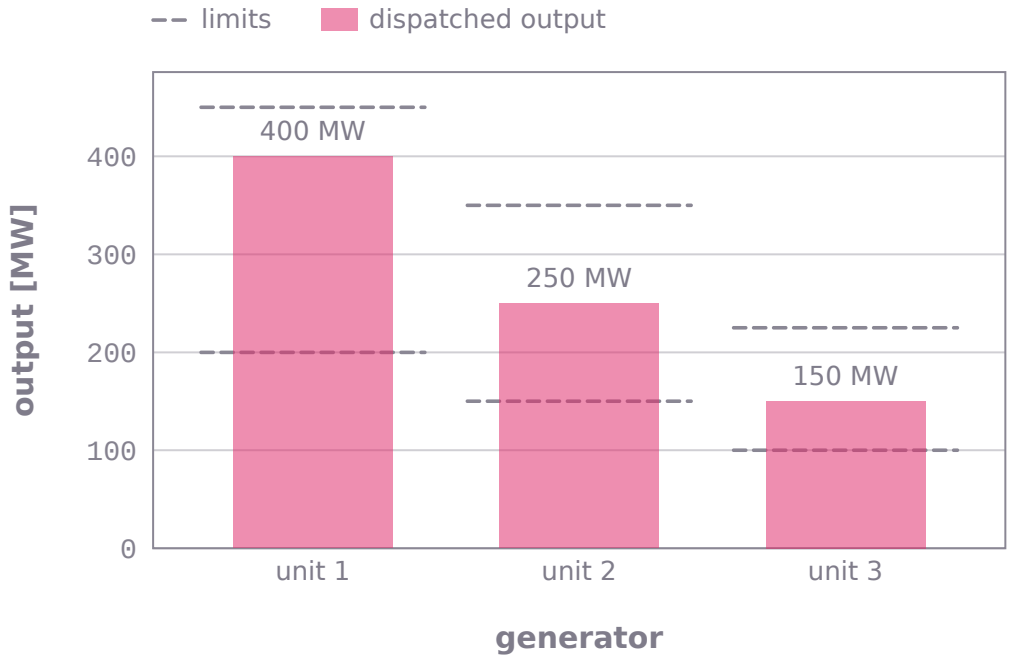
The least-cost outputs of thermal units with costs a + b P + c P^2 ($/h) and output limits that together meet a demand (losses neglected): every unit between its limits runs at the same incremental cost lambda.

$$\frac{dC_i}{dP_i} = b_i + 2c_iP_i = \lambda \ \ (P_{min} < P_i < P_{max}), \quad \sum_i P_i = D$$

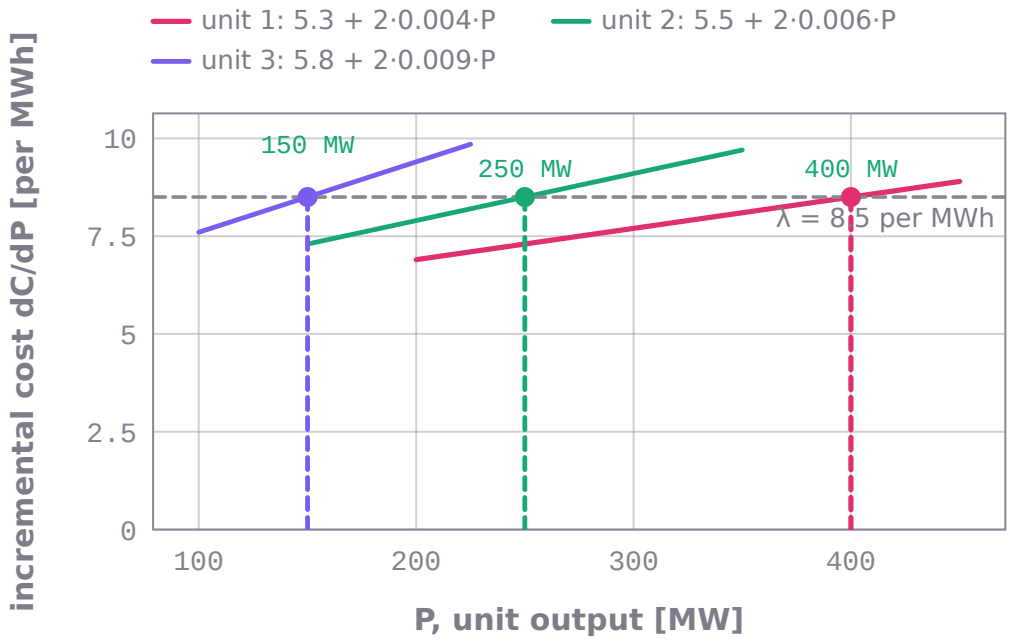
input	meaning	condition
`costs`	one row per unit: a ($/h), b($/MWh), c ($/MW^{2}$ h), Pmin (MW), Pmax (MW)	
`demand`	total demand in MW	

output	meaning
`lambda_`	system incremental cost in $/MWh
`P`	unit outputs in MW, in the order given
`cost`	total cost in $/h

- **Method:** equal incremental cost: P_i(lambda) = clip((lambda - b_i)/(2 c_i), Pmin_i, Pmax_i), lambda by bisection on sum $P_i = D$
- **Assumes:** $c > 0$ for every unit; sum Pmin <= demand <= sum Pmax; at least one unit between its limits
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Economic dispatch of 3 generators with quadratic cost curves (rows a, b, c, Pmin, Pmax: 500 5.3 0.004 200 450; 400 5.5 0.006 150 350; 200 5.8 0.009 100 225) meeting a demand of 800 MW: the package returns outputs 400, 250, 150 MW (bars, dashed lines the limits) at a common incremental cost λ = 8.5 per MWh, total cost 6682.5 per hour.



Economic dispatch of the worked example's three units (cost a + bP + cP², rows a, b, c, Pmin, Pmax: 500 5.3 0.004 200 450; 400 5.5 0.006 150 350; 200 5.8 0.009 100 225) meeting 800 MW. Each line is a unit's incremental cost b + 2cP between its limits, drawn from the stated coefficients; the package's common incremental cost λ = 8.5 per MWh cuts every line at that unit's dispatched output. Marked: outputs 400, 250, 150 MW, total cost 6682 per hour.

```
energy_systems_ai999x.solve('energy_economic_dispatch', costs='500 5.3 0.004 200 450; 400
5.5 0.006 150 350; 200 5.8 0.009 100 225', demand=800)
# -> lambda_ = 8.5, P = [400.0, 250.0, 150.0], cost = 6682.5   (SOLVED)
```

```
energy_systems_ai999x.solve('energy_economic_dispatch', costs=[[0, 10, 0.01, 0, 500], [0,
10, 0.01, 0, 500]], demand=400)
# -> lambda_ = 14, P = [200.0, 200.0], cost = 4800   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Economic dispatch: costs = [500 5.3 0.004 200 450]; [400 5.5 0.006 150 350]; [200 5.8 0.009 100 225], demand = 800 MW	SOLVED	lambda_ = 8.5, P = [400.0, 250.0, 150.0], cost = 6682.5
Economic dispatch: costs = [500 5.3 0.004 200 450]; [400 5.5 0.006 150 350], demand = 900 MW	INVALID_INPUT	Economic dispatch with quadratic costs and limits: the demand must lie between the total minimum and total maximum outpu

Three-phase (symmetrical) fault current by the bus impedance matrix,

`energy_symmetrical_fault`

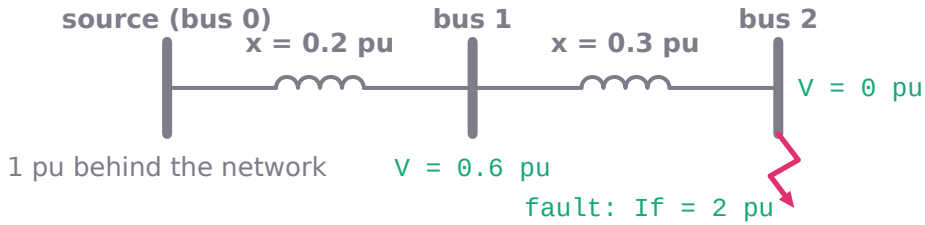
The fault current, Thevenin reactance and the bus voltages during a balanced three-phase fault at a bus of a reactance network (per unit, prefault no-load voltage Vf everywhere; every branch to node 0 is a source).

$$I_f = \frac{V_f}{Z_{ff} + Z_f}, \quad V_i = V_f - Z_{if} I_f$$

input	meaning	condition
`network`	branches as 'i-j x' with reactances in per unit; node 0 is the reference, each branch to 0 a source (generator or infinite bus)	
`fault_bus`	faulted bus	
`Vf`	prefault voltage in per unit (default 1.0)	
`Xf`	fault reactance in per unit (default 0, a bolted fault)	
`S_base`	base power, for the fault current in amperes (optional)	
`V_base`	base line voltage at the faulted bus (optional)	

output	meaning
`If`	fault current in per unit
`X_th`	Thevenin reactance at the faulted bus (Z_ff) in per unit
`V`	bus voltage magnitudes during the fault (pu), buses 1..n
`If_A`	fault current in amperes

- **Method:** column f of Zbus from Ybus $z = e_f$; $I_f = V_f/(Z_ff + jX_f)$; $V_i = V_f - Z_if$ I_f
- **Assumes:** reactance network; no prefault load current (all prefault voltages Vf)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Three-phase fault at bus 2 of the worked example's network (0-1 0.2, 1-2 0.3: from–to and reactance in per unit; bus 0 is the source at 1 pu). The package returns the fault current If = 2 pu, the Thevenin reactance at the fault X_th = 0.5 pu and the bus voltages during the fault V1 = 0.6, V2 = 0 pu. Marked: If and every voltage.

```
energy_systems_ai999x.solve('energy_symmetrical_fault', network='0-1 0.2, 1-2 0.3',
fault_bus=2)
# -> If = 2, X_th = 0.5, V = [0.6, 0.0]   (SOLVED)
```

```
energy_systems_ai999x.solve('energy_symmetrical_fault', network='0-1 0.1', fault_bus=1,
S_base=100000000.0, V_base=100000.0)
# -> If = 10, X_th = 0.1, V = [0.0], If_A = 5773.5 A   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Symmetrical fault: network = 0-1 0.2; 1-2 0.3, fault_bus = 2	SOLVED	If = 2, X_th = 0.5, V = [0.6, 0.0]
Three-phase fault: network = 0-1 0.1, bus = 1, S_base = 100 MVA, V_base = 100 kV	SOLVED	If = 10, X_th = 0.1, V = [0.0], If_A = 5773.5 A
Symmetrical fault: network = 0-1 0.2; 1-2 0.3, fault_bus = 5	INVALID_INPUT	Three-phase (symmetrical) fault current by the bus impedance matrix: the faulted bus must be one of 1..2.

Single line-to-ground fault (symmetrical components), `energy_slg_fault`

Phase and sequence currents and phase voltages for a fault from phase a to ground through Zf: the three sequence networks in series.

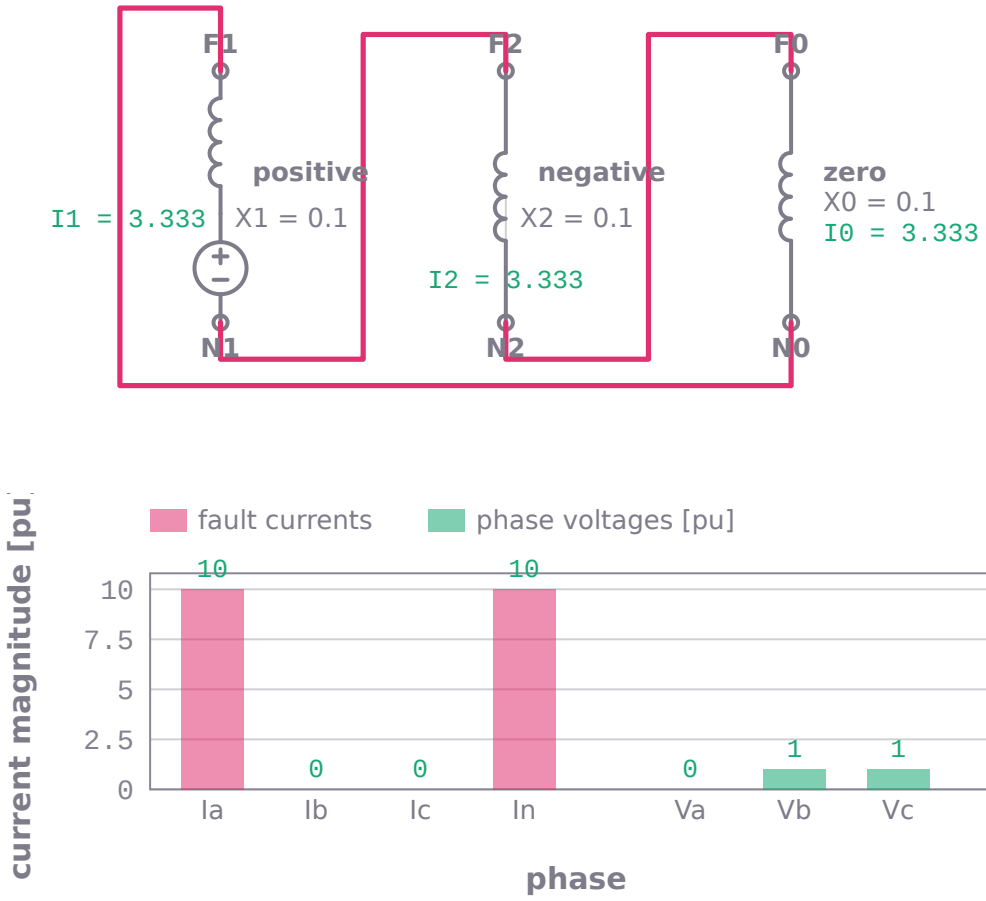
$$I_a = 3I_0 = \frac{3V_f}{Z_1 + Z_2 + Z_0 + 3Z_f}$$

input	meaning	condition
`X1`	positive-sequence Thevenin reactance (pu)	
`X2`	negative-sequence reactance (pu)	
`X0`	zero-sequence reactance (pu)	
`Rf`	fault resistance (pu, default 0)	
`Xf`	fault reactance (pu, default 0)	

input	meaning	condition
`Vf`	prefault voltage (pu, default 1.0)	

output	meaning
`Ia`	|I_a| (pu)
`Ib`	|I_b| (pu)
`Ic`	|I_c| (pu)
`In`	neutral (earth) current |3 I_0| (pu)
`I1`	|I_1| (pu)
`I2`	|I_2| (pu)
`I0`	|I_0| (pu)
`Va`	|V_a| (pu)
`Vb`	|V_b| (pu)
`Vc`	|V_c| (pu)

- **Method:** I1 = I2 = I0 = Vf/(Z1 + Z2 + Z0 + 3 Zf); $\boldsymbol{I_a = 3}$ I_0
- **Assumes:** sequence networks from the Thevenin reactances at the fault; prefault no-load voltage Vf
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



A single line-to-ground fault with the worked example's Thevenin reactances X1 = 0.1, X2 = 0.1, X0 = 0.1 pu (a bolted fault from a 1 pu prefault voltage unless stated). Top: the sequence networks connected in series (the same current flows through all three). Bottom: the package's fault currents Ia = 10, Ib = 0, Ic = 0, In = 10 pu and phase voltages Va = 0, Vb = 1, Vc = 1 pu. Marked: the sequence currents I1 = 3.333, I2 = 3.333, I0 = 3.333 pu.

```
energy_systems_ai999x.solve('energy_slg_fault', X1=0.1, X2=0.1, X0=0.1)
# -> Ia = 10, Ib = 0, Ic = 0, In = 10, I1 = 3.33333, I2 = 3.33333, I0 = 3.33333, Va = 0, Vb
= 1, Vc = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
SLG fault: X1 = 0.1, X2 = 0.1, X0 = 0.1	SOLVED	Ia = 10, Ib = 0, Ic = 0, In = 10, I1 = 3.33333, I2 = 3.33333, I0 = 3.33333, Va = 0, Vb = 1, Vc = 1
SLG fault: X1 = 0.1, X2 = 0.1, X0 = -0.1	INVALID_INPUT	Single line-to-ground fault (symmetrical components): the sequence reactances must be a positive number.
SLG fault: X1 = 0.1, X2 = 0.1	NEEDS_INPUT	energy_slg_fault needs X0 (zero-sequence reactance (pu)). Example: solve('energy_slg_fault', X1=0.1, X2=0.1, X0=0.1).

Line-to-line fault (symmetrical components), `energy_ll_fault`

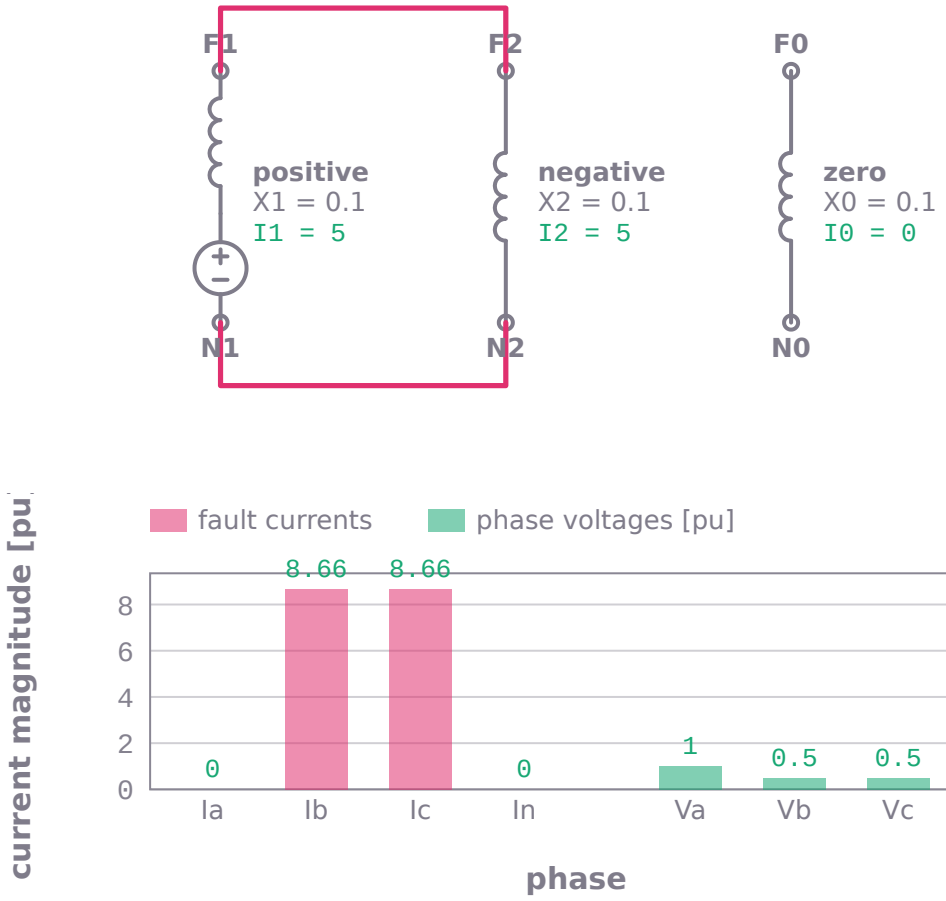
Phase and sequence currents and phase voltages for a fault between phases b and c through Zf: positive and negative networks in parallel opposition.

$$I_b = -I_c = -j\sqrt{3}\,\frac{V_f}{Z_1 + Z_2 + Z_f}$$

input	meaning	condition
`X1`	positive-sequence Thevenin reactance (pu)	
`X2`	negative-sequence reactance (pu)	
`X0`	zero-sequence reactance (pu)	
`Rf`	fault resistance (pu, default 0)	
`Xf`	fault reactance (pu, default 0)	
`Vf`	prefault voltage (pu, default 1.0)	

output	meaning
`Ia`	|I_a| (pu)
`Ib`	|I_b| (pu)
`Ic`	|I_c| (pu)
`In`	neutral (earth) current |3 I_0| (pu)
`I1`	|I_1| (pu)
`I2`	|I_2| (pu)
`I0`	|I_0| (pu)
`Va`	|V_a| (pu)
`Vb`	|V_b| (pu)
`Vc`	|V_c| (pu)

- **Method:** $I_1 = -I_2 = Vf/(Z_1 + Z_2 + Zf)$, $I_0 = 0$; |I_b| = sqrt(3) |I_1|
- **Assumes:** sequence networks from the Thevenin reactances at the fault; prefault no-load voltage Vf
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



A line-to-line fault with the worked example's Thevenin reactances X1 = 0.1, X2 = 0.1, X0 = 0.1 pu (a bolted fault from a 1 pu prefault voltage unless stated). Top: the sequence networks connected in parallel; the zero-sequence network stays open. Bottom: the package's fault currents Ia = 0, Ib = 8.66, Ic = 8.66, In = 0 pu and phase voltages Va = 1, Vb = 0.5, Vc = 0.5 pu. Marked: the sequence currents I1 = 5, I2 = 5, I0 = 0 pu.

```
energy_systems_ai999x.solve('energy_ll_fault', X1=0.1, X2=0.1, X0=0.1)
# -> Ia = 0, Ib = 8.66025, Ic = 8.66025, In = 0, I1 = 5, I2 = 5, I0 = 0, Va = 1, Vb = 0.5,
Vc = 0.5   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
LL fault: X1 = 0.1, X2 = 0.1, X0 = 0.1	SOLVED	Ia = 0, Ib = 8.66025, Ic = 8.66025, In = 0, I1 = 5, I2 = 5, I0 = 0, Va = 1, Vb = 0.5, Vc = 0.5
LL fault: X1 = 0.1, X2 = 0.1, X0 = -0.1	INVALID_INPUT	Line-to-line fault (symmetrical components): the sequence reactances must be a positive number.
LL fault: X1 = 0.1, X2 = 0.1	NEEDS_INPUT	energy_ll_fault needs X0 (zero-sequence reactance (pu)). Example: solve('energy_ll_fault', X1=0.1, X2=0.1, X0=0.1).

Double line-to-ground fault (symmetrical components), `energy_dlg_fault`

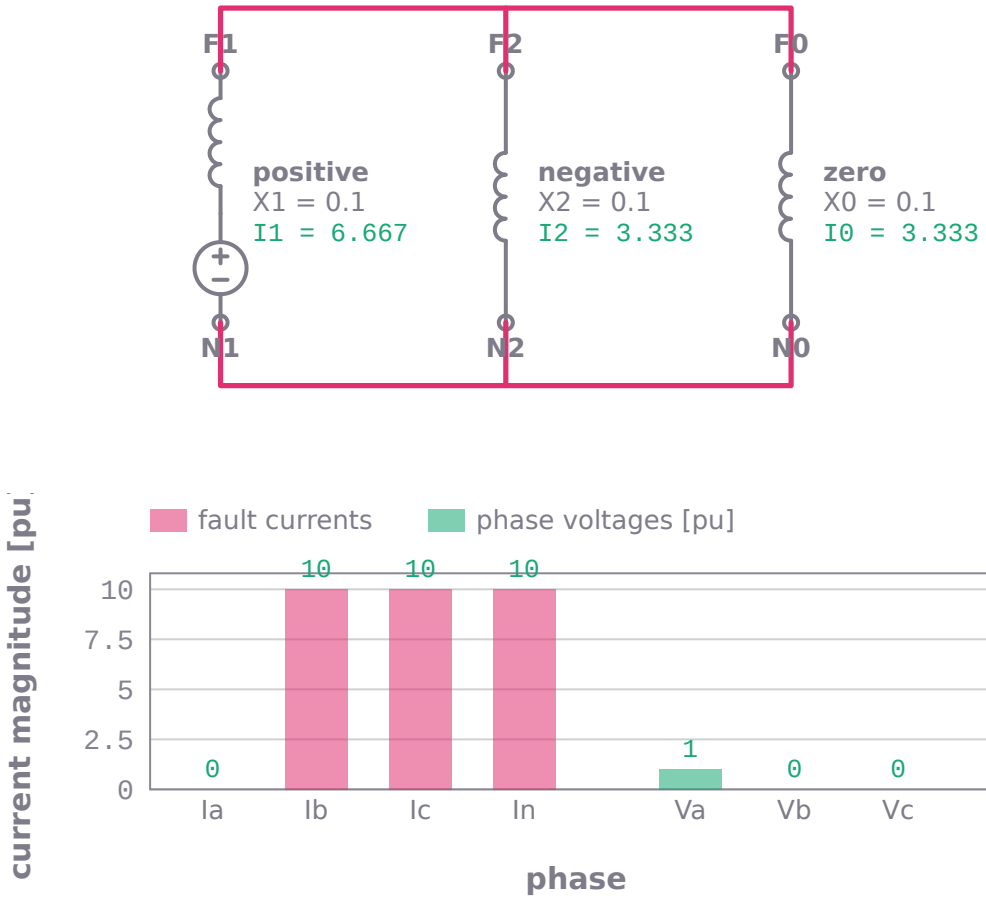
Phase, neutral and sequence currents and phase voltages for a fault from phases b and c to ground through Zf: negative and (zero + 3Zf) networks in parallel.

$$I_1 = \frac{V_f}{Z_1 + \frac{Z_2(Z_0+3Z_f)}{Z_2+Z_0+3Z_f}}$$

input	meaning	condition
`X1`	positive-sequence Thevenin reactance (pu)	
`X2`	negative-sequence reactance (pu)	
`X0`	zero-sequence reactance (pu)	
`Rf`	fault resistance (pu, default 0)	
`Xf`	fault reactance (pu, default 0)	
`Vf`	prefault voltage (pu, default 1.0)	

output	meaning
`Ia`	|I_a| (pu)
`Ib`	|I_b| (pu)
`Ic`	|I_c| (pu)
`In`	neutral (earth) current |3 I_0| (pu)
`I1`	|I_1| (pu)
`I2`	|I_2| (pu)
`I0`	|I_0| (pu)
`Va`	|V_a| (pu)
`Vb`	|V_b| (pu)
`Vc`	|V_c| (pu)

- **Method:** I1 = Vf/(Z1 + Z2 || $(Z_0 + 3Zf)$); $I_2 = -I_1\,(Z_0 + 3Zf)/(Z_2 + Z_0 + 3Zf)$; $I_0 = -I_1\,Z_2/(Z_2 + Z_0 + 3Zf)$
- **Assumes:** sequence networks from the Thevenin reactances at the fault; prefault no-load voltage Vf
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



A double line-to-ground fault with the worked example's Thevenin reactances X1 = 0.1, X2 = 0.1, X0 = 0.1 pu (a bolted fault from a 1 pu prefault voltage unless stated). Top: the sequence networks connected all three in parallel. Bottom: the package's fault currents Ia = 0, Ib = 10, Ic = 10, In = 10 pu and phase voltages Va = 1, Vb = 0, Vc = 0 pu. Marked: the sequence currents I1 = 6.667, I2 = 3.333, I0 = 3.333 pu.

```
energy_systems_ai999x.solve('energy_dlg_fault', X1=0.1, X2=0.1, X0=0.1)
# -> Ia = 0, Ib = 10, Ic = 10, In = 10, I1 = 6.66667, I2 = 3.33333, I0 = 3.33333, Va = 1,
Vb = 0, Vc = 0   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
DLG fault: X1 = 0.1, X2 = 0.1, X0 = 0.1	SOLVED	Ia = 0, Ib = 10, Ic = 10, In = 10, I1 = 6.66667, I2 = 3.33333, I0 = 3.33333, Va = 1, Vb = 0, Vc = 0
DLG fault: X1 = 0.1, X2 = 0.1, X0 = -0.1	INVALID_INPUT	Double line-to-ground fault (symmetrical components): the sequence reactances must be a positive number.
DLG fault: X1 = 0.1, X2 = 0.1	NEEDS_INPUT	energy_dlg_fault needs X0 (zero-sequence reactance (pu)). Example: solve('energy_dlg_fault', X1=0.1, X2=0.1, X0=0.1).

Power transfer and power angle between two voltages, `energy_power_angle`

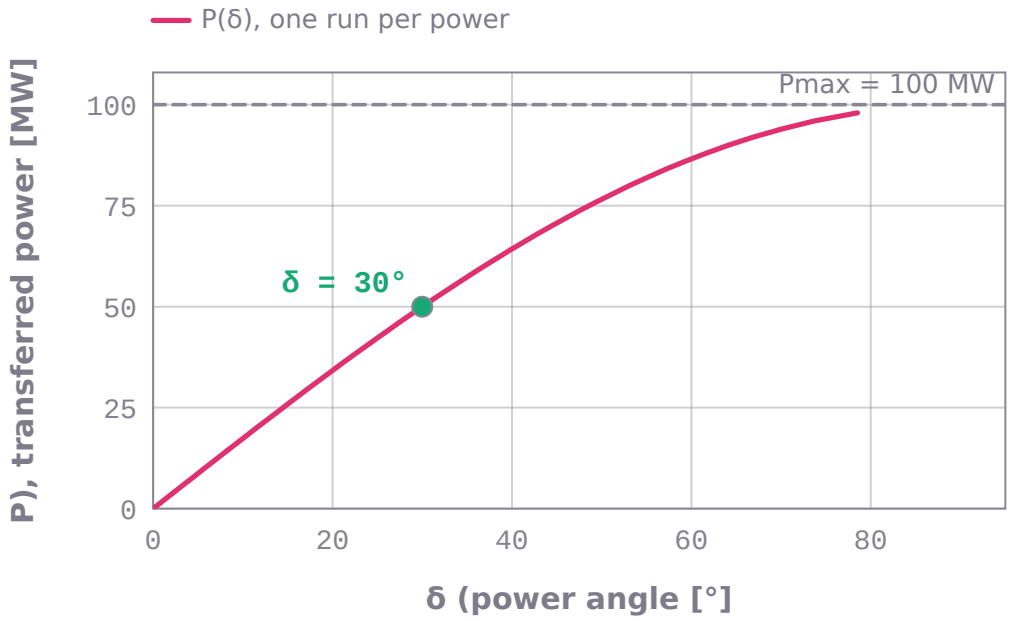
The power angle needed to transfer P over a lossless reactance X between a sending voltage V2 (generator) and a receiving voltage V1 (infinite bus), the steady-state limit V1 V_2/X, the reactive power at both ends and the stability margin.

$$P = \frac{V_1 V_2}{X} \sin\delta, \quad P_{max} = \frac{V_1 V_2}{X}$$

input	meaning	condition
`V1`	receiving (infinite-bus) line voltage	
`V2`	sending (generator) line voltage	
`X`	transfer reactance per phase	
`P`	real power transferred (three-phase)	

output	meaning
`delta_deg`	power angle (V2 ahead of V1) in degrees
`Pmax`	steady-state limit V1 V_2/X
`Q_sent`	reactive power sent from the generator end in var
`Q_received`	reactive power arriving at the infinite bus in var
`margin_pct`	stability margin (Pmax - P)/Pmax in percent

- **Method:** $P = V_1$ V2 sin(delta)/X; Q_sent = (V2^2 - V1 V2 cos delta)/X; Q_received = (V1 V2 cos delta - V1^2)/X
- **Assumes:** lossless transfer reactance; |P| < V1 V_2/X
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Power transferred between two buses (V1 = 100 kV, V2 = 100 kV, X = 100 Ω, the worked example) against the power angle: each point is the package's angle for a given power from 0 to Pmax. Marked: the operating point P = 50 MW at δ = 30°, the limit Pmax = 100 MW (stability margin 50 %), sending reactive power 13.4 Mvar.

```
energy_systems_ai999x.solve('energy_power_angle', V1=100000.0, V2=100000.0, X=100.0,
P=50000000.0)
# -> delta_deg = 30, Pmax = 1e+08 W, Q_sent = 1.33975e+07 var, Q_received = -1.33975e+07
var, margin_pct = 50   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Power angle: V1 = 100 kV, V2 = 100 kV, X = 100 ohm, P = 50 MW	SOLVED	delta_deg = 30, Pmax = 1e+08 W, Q_sent = 1.33975e+07 var, Q_received = -1.33975e+07 var, margin_pct = 50
Power angle: V1 = 100 kV, V2 = 100 kV, X = 100 ohm, P = 150 MW	INVALID_INPUT	Power transfer and power angle between two voltages: the power exceeds the steady-state limit V1 V2/X.

Critical clearing angle and time (equal-area criterion), `energy_critical_clearing`

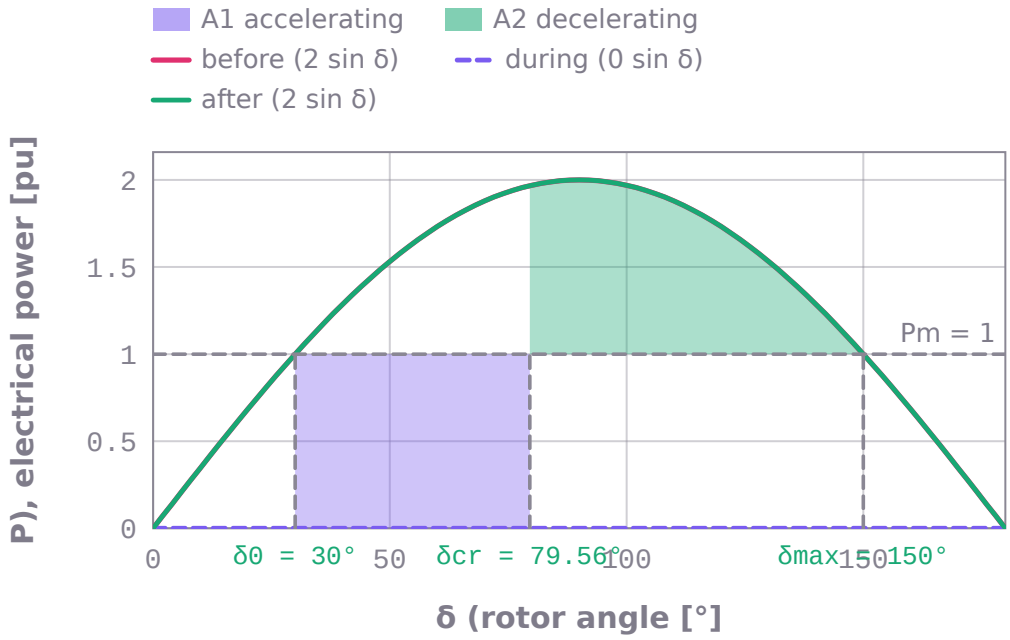
The initial rotor angle, the largest allowable angle and the critical clearing angle of a machine against an infinite bus by the equal-area criterion, for given pre-fault, during-fault and post-fault transfer limits; with H and f, the critical clearing time for a fault that blocks all transfer.

$$\cos\delta_{cr} = \frac{P_m(\delta_{max} - \delta_0) + P_{max,3}\cos\delta_{max} - P_{max,2}\cos\delta_0}{P_{max,3} - P_{max,2}}$$

input	meaning	condition
`Pm`	mechanical input power (pu)	
`Pmax_pre`	pre-fault transfer limit (pu)	
`Pmax_fault`	transfer limit during the fault (pu; 0 for a fault at the terminals)	
`Pmax_post`	post-fault transfer limit (pu)	
`H`	inertia constant H (MJ/MVA, i.e. seconds), for the clearing time	
`f`	system frequency, for the clearing time	

output	meaning
`delta0_deg`	initial rotor angle in degrees
`delta_max_deg`	largest angle before loss of synchronism in degrees
`delta_cr_deg`	critical clearing angle in degrees
`t_cr`	critical clearing time (Pmax_fault = 0)

- **Method:** cos(delta_cr) = [Pm (delta_max - delta0) + Pmax_post cos(delta_max) - Pmax_fault cos(delta0)]/(Pmax_post - Pmax_fault); $t_cr = sqrt$(4 H (delta_cr - delta0)/(omega_s Pm))
- **Assumes:** Pmax_fault < Pm < Pmax_pre, Pmax_post; classical model (constant voltage behind transient reactance, no damping)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Equal-area criterion for the worked example's machine: mechanical power Pm = 1 pu and transfer limits 2 (before), 0 (during) and 2 pu (after the fault). The power curves are Pmax·sin δ drawn from these stated limits; the angles are the package's: initial δ0 = 30°, critical clearing angle δcr = 79.56° and the largest angle δmax = 150°. Clearing at δcr makes the accelerating area A1 equal to the decelerating area A2. Marked: δ0, δcr, δmax.

```
energy_systems_ai999x.solve('energy_critical_clearing', Pm=1.0, Pmax_pre=2.0,
Pmax_fault=0.0, Pmax_post=2.0)
# -> delta0_deg = 30, delta_max_deg = 150, delta_cr_deg = 79.562   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Critical clearing: Pm = 1, Pmax_pre = 2, Pmax_fault = 0, Pmax_post = 2	SOLVED	delta0_deg = 30, delta_max_deg = 150, delta_cr_deg = 79.562
Critical clearing: Pm = 1, Pmax_pre = 2, Pmax_fault = 1.5, Pmax_post = 2	INVALID_INPUT	Critical clearing angle and time (equal-area criterion): the during-fault transfer limit must be below the mechanical po

Frequency deviation and load sharing under governor droop, `energy_droop_sharing`

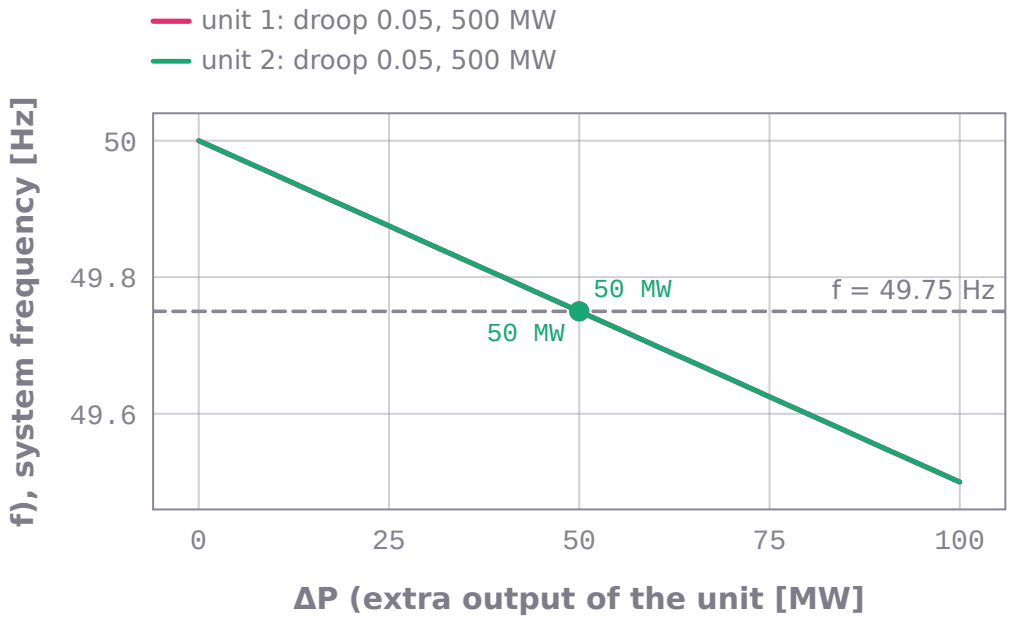
The steady-state frequency drop after a load step and each generator's output change when governors with droop R (per unit on their own rating) share the load, with optional load damping D.

$$\Delta f = -\frac{\Delta P_L}{\sum_i \frac{S_i}{R_i f_0} + D}$$

input	meaning	condition
`R`	droop of each governor in per unit on its own rating (e.g. 0.05)	
`ratings`	rating of each generator in MW	
`dP`	load increase in MW (negative for a load decrease)	
`f0`	nominal frequency	
`D`	load damping in MW/Hz (default 0)	

output	meaning
`df`	frequency change in Hz
`f`	new frequency in Hz
`dP_gen`	output change of each generator in MW
`dP_load_damping`	load relief from damping in MW

- **Method:** beta = sum(S_i/(R_i f0)) + D; $df = -dP/\beta$; $dP_i = -df$ S_i/(R_i f0)
- **Assumes:** primary control only (no secondary/AGC); R, ratings > 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



2 generators on droop control (the worked example's droops [0.05, 0.05] on ratings [500, 500] MW, nominal 50 Hz) taking up a load increase stepped from 0 to 200 MW: each line joins the package's frequency and that unit's share for every step. Marked: the worked example's 100 MW increase settles at 49.75 Hz with shares 50, 50 MW.

```
energy_systems_ai999x.solve('energy_droop_sharing', R=[0.05, 0.05], ratings=[500, 500],
dP=100, f0=50)
# -> df = -0.25, f = 49.75, dP_gen = [50.0, 50.0], dP_load_damping = 0   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Droop sharing: R = [0.05, 0.05], ratings = [500, 500], dP = 100 MW, f0 = 50 Hz	SOLVED	df = -0.25, f = 49.75, dP_gen = [50.0, 50.0], dP_load_damping = 0
Droop sharing: R = [0.05, 0.05], ratings = [500], dP = 100 MW, f0 = 50 Hz	INVALID_INPUT	Frequency deviation and load sharing under governor droop: give one droop and one rating per generator.

Transformer regulation and efficiency from its equivalent circuit,

`energy_transformer_performance`

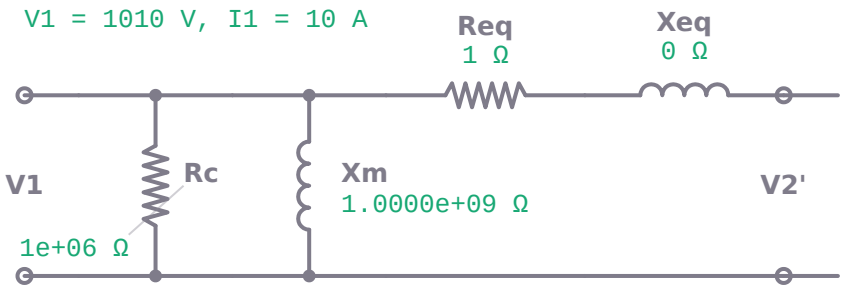
Primary voltage needed, voltage regulation, primary current and input power factor, copper and core losses and efficiency of a single-phase transformer (approximate equivalent circuit referred to the primary, shunt branch at the primary terminals) delivering a fraction of its rating at rated secondary voltage.

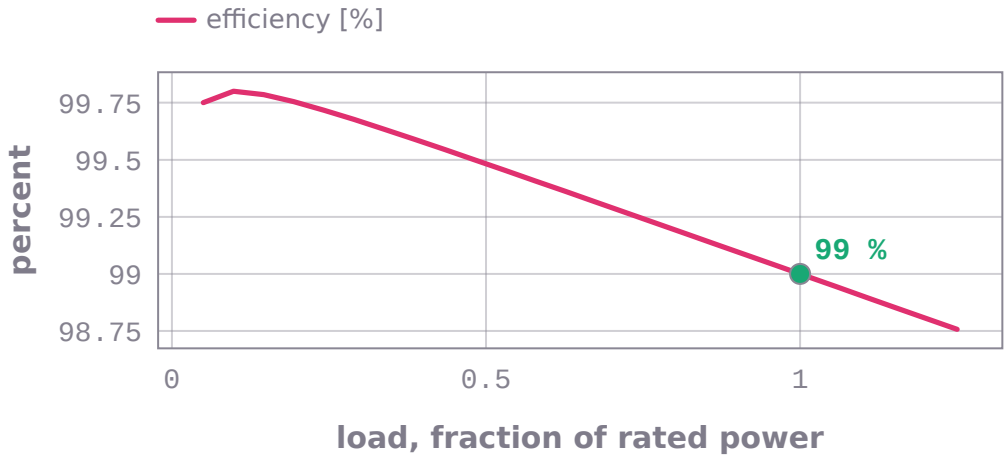
$$\mathbf{V}_1 = a\mathbf{V}_2 + \frac{\mathbf{I}_2}{a}(R_{eq} + jX_{eq}), \quad \eta = \frac{P_{out}}{P_{out} + P_{cu} + P_{core}}$$

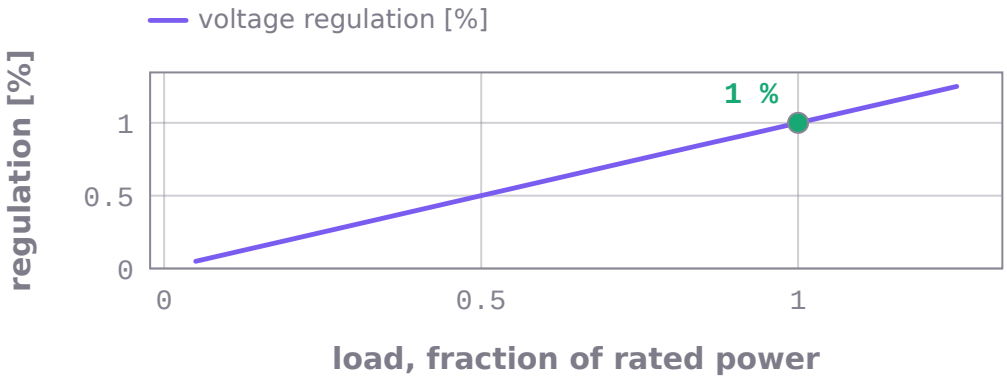
input	meaning	condition
`S`	rating in VA	
`V1`	rated primary voltage	
`V2`	rated secondary voltage	
`Req`	series resistance referred to the primary	
`Xeq`	series leakage reactance referred to the primary	
`Rc`	core-loss resistance (primary side)	
`Xm`	magnetizing reactance (primary side)	
`load`	load as a fraction of the rating (or a percentage)	
`pf`	load power factor	
`pf_type`	lagging (default) or leading load	

output	meaning
`V1_required`	primary voltage for rated secondary voltage at this load
`regulation_pct`	voltage regulation in percent
`I1`	primary current
`pf_in`	input power factor
`P_out`	output power
`P_cu`	copper loss
`P_core`	core loss
`efficiency_pct`	efficiency in percent

- **Method:** $V_1 = a\,V_2 + I_2$' (Req + jXeq), I2' = I_2/a at angle -acos(pf); P_cu = |I2'|^2 Req; P_core = V1^2/Rc; eta = P_out/(P_out + P_cu + P_core)
- **Assumes:** approximate equivalent circuit (shunt branch at the primary terminals); 0 < load <= 2
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)







The worked example's 10 kVA, 1000/100 V transformer (referred to the high-voltage side: Req = 1 Ω, Xeq = 0 Ω, Rc = 1e+06 Ω, Xm = 1.0000e+09 Ω) supplying power factor 1 lagging. Top: the circuit; at 1 of rated load the package returns V1 = 1010 V, I1 = 10 A, copper loss 100 W, core loss 1.02 W. Middle and bottom: efficiency and regulation against load, one run per point. Marked: the worked example's load, 99 % and 1 %.

```
energy_systems_ai999x.solve('energy_transformer_performance', S=10000.0, V1=1000, V2=100,
Req=1.0, Xeq=0.0, Rc=1000000.0, Xm=1000000000.0, load=1.0, pf=1.0)
# -> V1_required = 1010 V, regulation_pct = 1, I1 = 10.001 A, pf_in = 1, P_out = 10000 W,
P_cu = 100 W, P_core = 1.0201 W, efficiency_pct = 98.9999   (SOLVED)
```

```
energy_systems_ai999x.solve('energy_transformer_performance', S=10000.0, V1=1000, V2=100,
Req=1.0, Xeq=0.0, Rc=1000000000.0, Xm=1000000000.0, load=1.0, pf=1.0)
# -> V1_required = 1010 V, regulation_pct = 1, I1 = 10 A, pf_in = 1, P_out = 10000 W, P_cu
= 100 W, P_core = 0.0010201 W, efficiency_pct = 99.0099   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Transformer regulation and efficiency: S = 10 kVA, V1 = 1000 V, V2 = 100 V, Req = 1 ohm, Xeq = 0 ohm, Rc = 1e9 ohm, Xm = 1e9 ohm, load = 100%, pf = 1	SOLVED	V1_required = 1010 V, regulation_pct = 1, I1 = 10 A, pf_in = 1, P_out = 10000 W, P_cu = 100 W, P_core = 0.0010201 W, efficiency_pct = 99.0099
Transformer regulation and efficiency: S = 10 kVA, V1 = 1000 V, V2 = 100 V, Req = 1 ohm, Xeq = 0 ohm, Rc = 1e9 ohm, Xm = 1e9 ohm, load = 300%, pf = 1	INVALID_INPUT	Transformer regulation and efficiency from its equivalent circuit: the load must lie between 0 and 2 times the rating (a

Transformer parameters from open-circuit and short-circuit tests, `energy_transformer_tests`

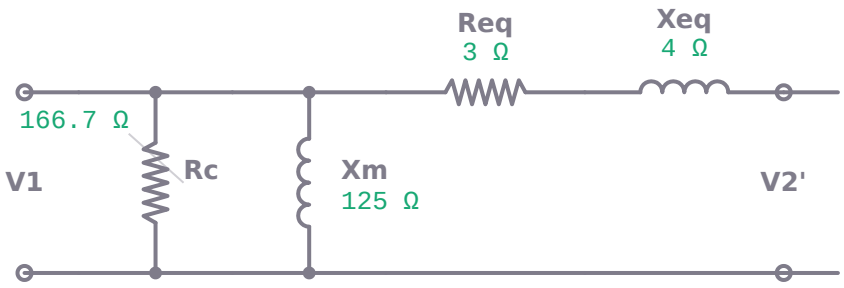
Core-loss resistance and magnetizing reactance from the open-circuit test, and the equivalent series resistance, reactance and impedance from the short-circuit test (each on the side where the test was made).

$$R_c = \frac{V_{oc}^2}{P_{oc}},\ X_m = \frac{1}{\sqrt{Y_{oc}^2 - G_c^2}},\ R_{eq} = \frac{P_{sc}}{I_{sc}^2},\ X_{eq} = \sqrt{Z_{eq}^2 - R_{eq}^2}$$

input	meaning	condition
`Voc`	open-circuit test voltage	
`Ioc`	open-circuit test current	
`Poc`	open-circuit test power	
`Vsc`	short-circuit test voltage	
`Isc`	short-circuit test current	
`Psc`	short-circuit test power	

output	meaning
`Rc`	core-loss resistance (open-circuit side)
`Xm`	magnetizing reactance (open-circuit side)
`Req`	equivalent series resistance (short-circuit side)
`Xeq`	equivalent leakage reactance (short-circuit side)
`Zeq`	equivalent series impedance magnitude
`pf_oc`	open-circuit power factor
`pf_sc`	short-circuit power factor

- **Method:** $Rc = V_oc^2 / P_oc$, $Xm = \frac{1}{\sqrt{\left(\frac{I_{oc}}{V_{oc}}\right)^2 - \left(\frac{P_{oc}}{V_{oc}^2}\right)^2}}$; Req = P_sc/I_sc^2, $Xeq = \sqrt{\left(\frac{V_{sc}}{I_{sc}}\right)^2 - Req^2}$
- **Assumes:** $P < V$ I in both tests
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Approximate equivalent circuit of a transformer from its two tests (the worked example: open circuit 100 V, 1 A, 60 W; short circuit 50 V, 10 A, 300 W), referred to the side the tests were made on. Marked: the package's core-loss resistance Rc = 166.7 Ω and magnetising reactance Xm = 125 Ω (shunt), series Req = 3 Ω and Xeq = 4 Ω (|Zeq| = 5 Ω); power factors 0.6 and 0.6.

```
energy_systems_ai999x.solve('energy_transformer_tests', Voc=100, Ioc=1, Poc=60, Vsc=50,
Isc=10, Psc=300)
# -> Rc = 166.667 ohm, Xm = 125 ohm, Req = 3 ohm, Xeq = 4 ohm, Zeq = 5 ohm, pf_oc = 0.6,
pf_sc = 0.6   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Transformer tests: Voc = 100 V, Ioc = 1 A, Poc = 60 W, Vsc = 50 V, Isc = 10 A, Psc = 300 W	SOLVED	Rc = 166.667 ohm, Xm = 125 ohm, Req = 3 ohm, Xeq = 4 ohm, Zeq = 5 ohm, pf_oc = 0.6, pf_sc = 0.6
Transformer tests: Voc = 100 V, Ioc = 1 A, Poc = 160 W, Vsc = 50 V, Isc = 10 A, Psc = 300 W	INVALID_INPUT	Transformer parameters from open-circuit and short-circuit tests: the open-circuit power must be below V I (power factor

Load for maximum transformer efficiency, `energy_transformer_max_efficiency`

The load (fraction of rating and apparent power) at which a transformer's efficiency is greatest (copper loss equal to core loss), that maximum efficiency and the full-load efficiency at a power factor.

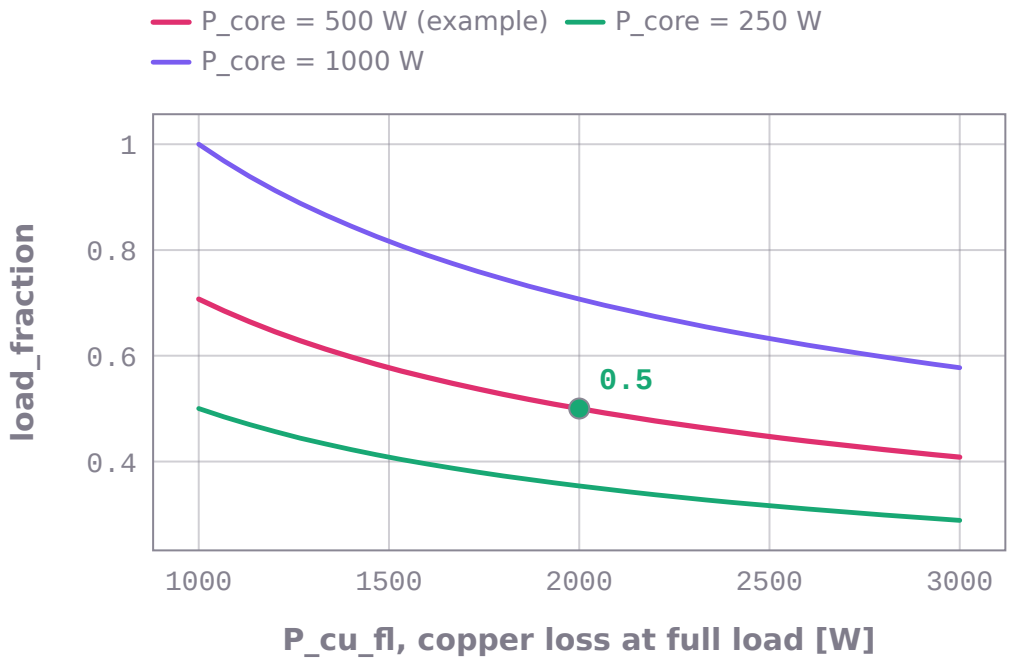
$$x_{\eta,max} = \sqrt{\frac{P_{core}}{P_{cu,fl}}}, \quad \eta_{max} = \frac{xS\cos\phi}{xS\cos\phi + 2P_{core}}$$

input	meaning	condition
`S`	rating in VA	
`P_core`	core (no-load) loss	
`P_cu_fl`	copper loss at full load	
`pf`	load power factor	

output	meaning
`load_fraction`	load fraction for maximum efficiency sqrt(P_core/P_cu)
`S_at_max`	apparent power at maximum efficiency in VA
`efficiency_max_pct`	maximum efficiency in percent

output	meaning
`efficiency_full_load_pct`	full-load efficiency in percent

- **Method:** $x = sqrt$(P_core/P_cu,fl); eta_max = x S pf/(x S $pf + 2$ P_core)
- **Assumes:** constant core loss, copper loss proportional to the load squared
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



load_fraction (load fraction for maximum efficiency sqrt(P_core/P_cu)) against P_cu_fl (copper loss at full load) from 1000 W to 3000 W. One curve for each value of P_core (core (no-load) loss): 500 W, 250 W, 1000 W. Other inputs as in the worked example: S = 100000.0 VA, pf = 1.0. 31 points per curve computed by energy-systems-ai999x; marked: the worked example, P_cu_fl = 2000 W, load_fraction = 0.5.

```
energy_systems_ai999x.solve('energy_transformer_max_efficiency', S=100000.0, P_core=500,
P_cu_fl=2000, pf=1.0)
# -> load_fraction = 0.5, S_at_max = 50000 VA, efficiency_max_pct = 98.0392,
efficiency_full_load_pct = 97.561   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Maximum transformer efficiency: S = 100 kVA, P_core = 500 W, P_cu_fl = 2000 W, pf = 1	SOLVED	load_fraction = 0.5, S_at_max = 50000 VA, efficiency_max_pct = 98.0392, efficiency_full_load_pct = 97.561
Maximum transformer efficiency: S = 100 kVA, P_core = 0 W, P_cu_fl = 2000 W, pf = 1	INVALID_INPUT	Load for maximum transformer efficiency: the rating and the losses must be a positive number.

Transformer under load from nameplate data, `energy_transformer_loading`

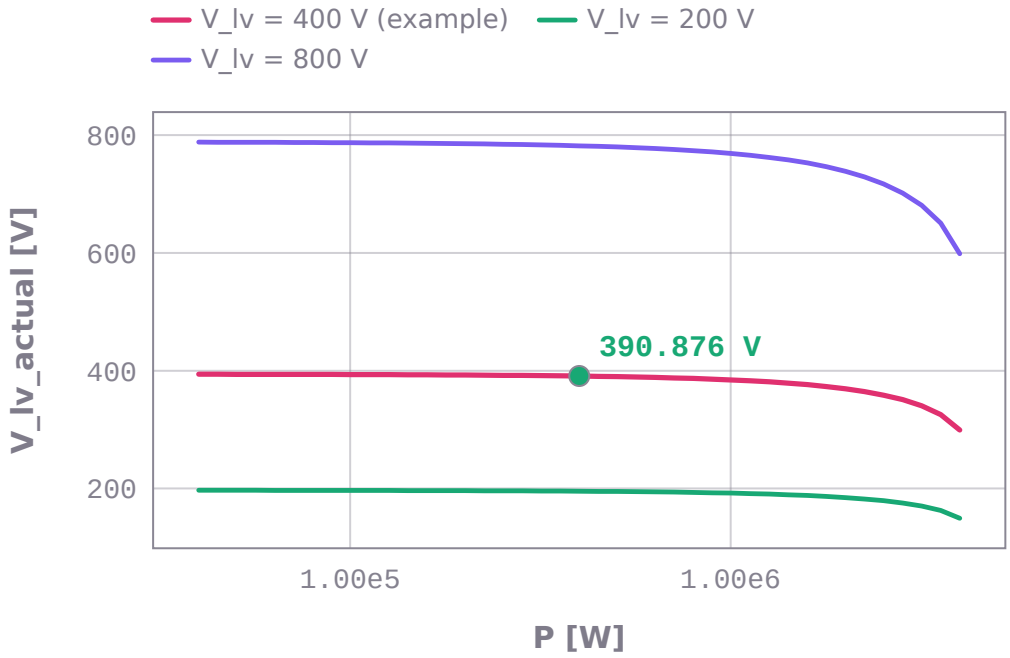
Secondary voltage, losses, efficiency and loading of a transformer fed at rated primary voltage and supplying a P + jQ load, from its nameplate: rating, rated voltages, short-circuit voltage vk and its resistive part vkr, no-load loss and no-load current (T equivalent circuit).

$$z_k = v_{kr} + j\sqrt{v_k^2 - v_{kr}^2}, \quad y_m = \frac{P_0}{S_n} - j\sqrt{i_0^2 - \left(\frac{P_0}{S_n}\right)^2}$$

input	meaning	condition
`sn`	rating in VA	
`V_hv`	rated (and applied) high-voltage side voltage	
`V_lv`	rated low-voltage side voltage	
`vk`	short-circuit voltage as a fraction or percentage of rated voltage	
`vkr`	resistive part of the short-circuit voltage (fraction or %)	
`P0`	no-load (iron) loss	
`i0`	no-load current as a fraction or percentage of rated current	
`P`	load real power on the low-voltage side	
`Q`	load reactive power in var	

output	meaning
`V_lv_actual`	low-voltage side voltage under load
`V_lv_pu`	low-voltage side voltage in per unit
`delta_deg`	phase shift of the secondary voltage in degrees (no vector-group shift)
`P_loss`	total real loss
`Q_loss`	reactive power absorbed in var
`efficiency_pct`	efficiency in percent
`loading_pct`	loading: the larger winding current over its rated current, percent

- **Method:** per unit on the rating: z = vkr + j sqrt(vk^2 - vkr^2), $y_m = P_0/S - j$ sqrt(i0^2 - (P0/S)^2); the T circuit solved for the load-bus voltage by Newton iteration
- **Assumes:** T equivalent circuit (series impedance halved on each side of the magnetizing branch); rated voltage applied, nominal tap
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



V_lv_actual (low-voltage side voltage under load) against P (load real power on the low-voltage side) from 40000 W to 4e+06 W on a logarithmic axis. One curve for each value of V_lv (rated low-voltage side voltage): 400 V, 200 V, 800 V. Other inputs as in the worked example: sn = 630000.0 VA, V_hv = 20000.0 V, vk = 0.06, vkr = 0.012, P0 = 1500 W, i0 = 0.003, Q = 150000.0 var. 41 points per curve computed by energy-systems-ai999x; marked: the worked example, P = 400000 W, V_lv_actual = 390.876 V.

```
energy_systems_ai999x.solve('energy_transformer_loading', sn=630000.0, V_hv=20000.0,
V_lv=400, vk=0.06, vkr=0.012, P0=1500, i0=0.003, P=400000.0, Q=150000.0)
# -> V_lv_actual = 390.876 V, V_lv_pu = 0.977189, delta_deg = -2.02483, P_loss = 5120.79 W,
Q_loss = 19030.9 var, efficiency_pct = 98.736, loading_pct = 69.6777   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Transformer under load: sn = 630 kVA, V_hv = 20 kV, V_lv = 400 V, vk = 6%, vkr = 1.2%, P0 = 1.5 kW, i0 = 0.3%, P = 400 kW, Q = 150 kvar	SOLVED	V_lv_actual = 390.876 V, V_lv_pu = 0.977189, delta_deg = -2.02483, P_loss = 5120.79 W, Q_loss = 19030.9 var, efficiency_pct = 98.736, loading_pct = 69.6777
Transformer under load: sn = 630 kVA, V_hv = 20 kV, V_lv = 400 V, vk = 6%, vkr = 7%, P0 = 1.5 kW, i0 = 0.3%, P = 400 kW, Q = 150 kvar	INVALID_INPUT	Transformer under load from nameplate data: need 0 < vkr < vk (short-circuit voltages as fractions or percentages, vk be

Load sharing between two transformers in parallel, `energy_parallel_transformers`

How two transformers with equal voltage ratios but different ratings and short-circuit impedances share a load: the apparent, real and reactive power each delivers and its loading on its own rating.

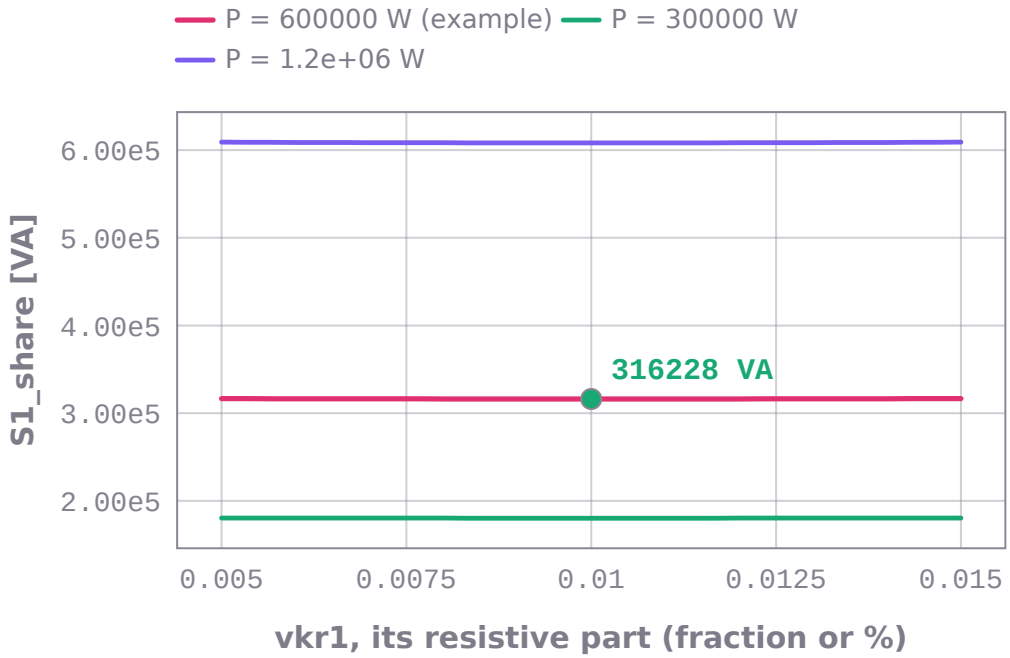
$$\mathbf{S}_1 = \mathbf{S}_L\left(\frac{\mathbf{Z}_2}{\mathbf{Z}_1+\mathbf{Z}_2}\right)^*, \quad \mathbf{S}_2 = \mathbf{S}_L\left(\frac{\mathbf{Z}_1}{\mathbf{Z}_1+\mathbf{Z}_2}\right)^*$$

input	meaning	condition
`S1`	rating of transformer 1 (VA)	

input	meaning	condition
`vk1`	short-circuit voltage of transformer 1 (fraction or %)	
`vkr1`	its resistive part (fraction or %)	
`S2`	rating of transformer 2 (VA)	
`vk2`	short-circuit voltage of transformer 2 (fraction or %)	
`vkr2`	its resistive part (fraction or %)	
`V_lv`	common secondary rated voltage	
`P`	load real power	
`Q`	load reactive power in var	

output	meaning
`S1_share`	apparent power delivered by transformer 1 in VA
`S2_share`	apparent power delivered by transformer 2 in VA
`P1`	real power of transformer 1
`Q1`	reactive power of transformer 1 in var
`P2`	real power of transformer 2
`Q2`	reactive power of transformer 2 in var
`share1_pct_of_rating`	transformer 1 output over its rating, percent
`share2_pct_of_rating`	transformer 2 output over its rating, percent

- **Method:** impedances in ohms on the common secondary: Z_i = (vkr_i + j sqrt(vk_i^2 - vkr_i^2)) V^2/S_i ; $S_1 = S_L$ conj($Z_2/(Z_1 + Z_2)$), $S_2 = S_L$ conj($Z_1/(Z_1 + Z_2)$)
- **Assumes:** equal voltage ratios (no circulating current); magnetizing branches neglected
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



S1_share (apparent power delivered by transformer 1 in VA) against vkr1 (its resistive part (fraction or %)) from 0.005 to 0.015. One curve for each value of P (load real power): 600000 W, 300000 W, 1.2e+06 W. Other inputs as in the worked example: S1 = 500000.0 VA, vk1 = 0.05, S2 = 500000.0 VA, vk2 = 0.05, vkr2 = 0.01, V_lv = 400 V, Q = 200000.0 var. 31 points per curve computed by energy-systems-ai999x; marked: the worked example, vkr1 = 0.01, S1_share = 316228 VA.

```
energy_systems_ai999x.solve('energy_parallel_transformers', S1=500000.0, vk1=0.05,
vkr1=0.01, S2=500000.0, vk2=0.05, vkr2=0.01, V_lv=400, P=600000.0, Q=200000.0)
# -> S1_share = 316228 VA, S2_share = 316228 VA, P1 = 300000 W, Q1 = 100000 var, P2 =
300000 W, Q2 = 100000 var, share1_pct_of_rating = 63.2456, share2_pct_of_rating = 63.2456
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Parallel transformers: S1 = 500 kVA, vk1 = 5%, vkr1 = 1%, S2 = 500 kVA, vk2 = 5%, vkr2 = 1%, V_lv = 400 V, P = 600 kW, Q = 200 kvar	SOLVED	S1_share = 316228 VA, S2_share = 316228 VA, P1 = 300000 W, Q1 = 100000 var, P2 = 300000 W, Q2 = 100000 var, share1_pct_of_rating = 63.2456, share2_pct_of_rating = 63.2456
Parallel transformers: S1 = 500 kVA, vk1 = 5%, vkr1 = 6%, S2 = 500 kVA, vk2 = 5%, vkr2 = 1%, V_lv = 400 V, P = 600 kW, Q = 200 kvar	INVALID_INPUT	Load sharing between two transformers in parallel: need 0 <= vkr < vk below 50% for each transformer.

Induction motor performance from its equivalent circuit, `energy_induction_motor`

Line and rotor currents, power factor, air-gap and converted power, induced torque, output and efficiency of a three-phase induction motor at a slip, from its per-phase equivalent circuit.

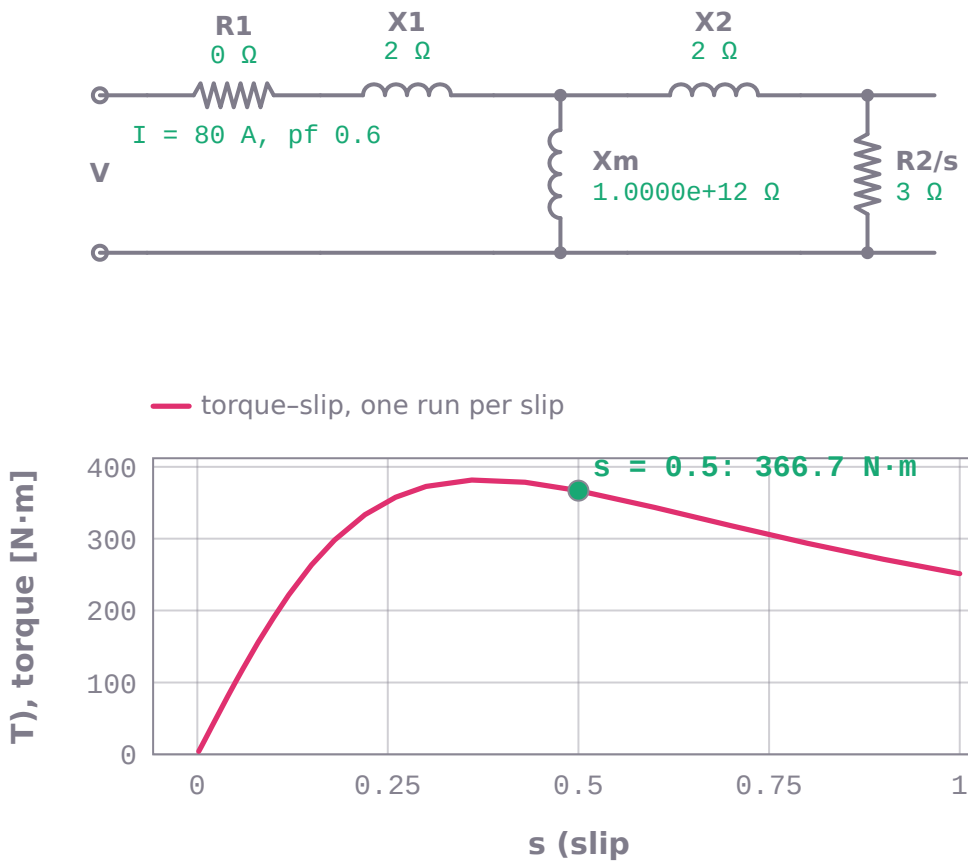
$$T_{ind} = \frac{3I_2^2 R_2/s}{\omega_s}, \quad n_s = \frac{120f}{p}$$

input	meaning	condition
VL	line voltage	

input	meaning	condition
`f`	supply frequency	
`poles`	number of poles (even)	
`R1`	stator resistance per phase	
`X1`	stator leakage reactance per phase	
`R2`	rotor resistance per phase referred to the stator	
`X2`	rotor leakage reactance referred to the stator	
`Xm`	magnetizing reactance per phase	
`slip`	slip $(0 < s \leq 1)$	
`connection`	stator connection	
`P_rot`	rotational (friction, windage and core) loss, default 0	

output	meaning
`I_line`	line current
`I2`	rotor current referred to the stator
`pf`	power factor
`P_in`	input power
`P_ag`	air-gap power
`P_conv`	converted (developed) power
`T`	induced torque
`P_out`	output power
`efficiency_pct`	efficiency in percent

- **Method:** Z = R1 + jX1 + jXm || $(R_2/s + jX2)$; $P_ag = 3\ I_2^2\ R_2/s$; P_conv = (1 - s) P_ag; $T = P_ag/omega_s$; $n_s = 120$ f/p
- **Assumes:** steady state, balanced supply; 0 < slip <= 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Top: the per-phase equivalent circuit of the worked example's motor (692.82 V, 50 Hz, 4 poles, star, R1 = 0 Ω, X1 = 2 Ω, R2 = 1.5 Ω, X2 = 2 Ω, Xm = 1.0000e+12 Ω per phase) at slip 0.5, the rotor resistance appearing as R2/s = 3 Ω. Bottom: the package's torque against slip from standstill (s = 1) to near synchronous speed. Marked: at s = 0.5 the package returns T = 366.7 N·m, line current 80 A, power factor 0.6, air-gap power 57.6 kW, output 28.8 kW, efficiency 50 %.

```
energy_systems_ai999x.solve('energy_induction_motor', VL=692.8203230275509, f=50, poles=4,
R1=0.0, X1=2.0, R2=1.5, X2=2.0, Xm=1000000000000.0, slip=0.5, connection='star')
# -> I_line = 80 A, I2 = 80 A, pf = 0.6, P_in = 57600 W, P_ag = 57600 W, P_conv = 28800 W,
T = 366.693 N*m, P_out = 28800 W, efficiency_pct = 50   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Induction motor: VL = 460 V, f = 60 Hz, poles = 4, R1 = 0.641 ohm, X1 = 1.106 ohm, R2 = 0.332 ohm, X2 = 0.464 ohm, Xm = 26.3 ohm, slip = 2.2%, connection = star	SOLVED	I_line = 18.8919 A, I2 = 16.171 A, pf = 0.832122, P_in = 12525.1 W, P_ag = 11838.8 W, P_conv = 11578.4 W, T = 62.8068 N*m, P_out = 11578.4 W, efficiency_pct = 92.4409
Induction motor: VL = 460 V, f = 60 Hz, poles = 5, R1 = 0.641 ohm, X1 = 1.106 ohm, R2 = 0.332 ohm, X2 = 0.464 ohm, Xm = 26.3 ohm, slip = 2.2%, connection = star	INVALID_INPUT	Induction motor performance from its equivalent circuit: the number of poles must be an even whole number (2, 4, 6, ...)

Induction motor breakdown (maximum) torque and starting torque,

`energy_induction_max_torque`

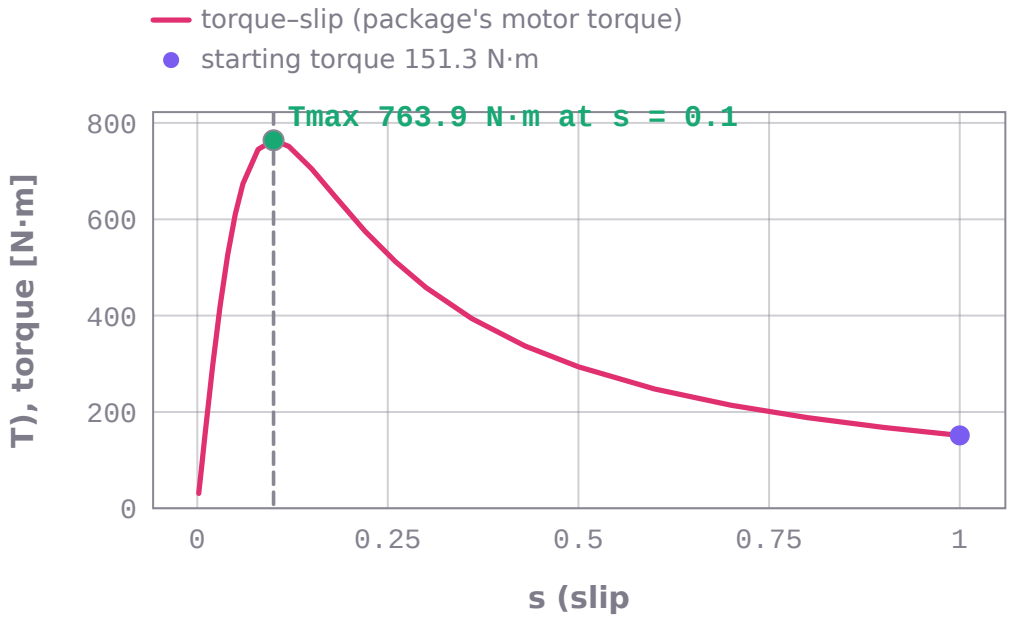
The slip at maximum torque, the maximum (breakdown) torque, the starting torque and current, and the Thevenin equivalent of the stator and magnetizing branch, for a three-phase induction motor.

$$s_{max} = \frac{R_2}{\sqrt{R_{th}^2 + (X_{th} + X_2)^2}}, \quad T_{max} = \frac{3V_{th}^2}{2\omega_s \left[R_{th} + \sqrt{R_{th}^2 + (X_{th} + X_2)^2}\right]}$$

input	meaning	condition
`VL`	line voltage	
`f`	supply frequency	
`poles`	number of poles (even)	
`R1`	stator resistance per phase	
`X1`	stator leakage reactance per phase	
`R2`	rotor resistance per phase referred to the stator	
`X2`	rotor leakage reactance referred to the stator	
`Xm`	magnetizing reactance per phase	
`connection`	stator connection	

output	meaning
`s_maxT`	slip at maximum torque
`T_max`	maximum (pull-out) torque
`T_start`	starting torque $(s = 1)$
`I_start`	starting line current
`V_th`	Thevenin voltage per phase
`R_th`	Thevenin resistance
`X_th`	Thevenin reactance

- **Method:** Thevenin: $V_th = V$ jXm/(R1 + j(X1 + Xm)), $Z_th = Z_1$ || jXm; $s_{max} = \frac{R_2}{\sqrt{R_{th}^2+(X_{th}+X_2)^2}}$; T_max = 3 V_th^2/(2 omega_s (R_th + sqrt(R_th^2 + (X_th + X2)^2)))
- **Assumes:** constant parameters (no deep-bar or saturation effects)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Torque–slip curve of the worked example's induction motor (692.82 V, 50 Hz, 4 poles, star, R1 = 0 Ω, X1 = 1 Ω, R2 = 0.2 Ω, X2 = 1 Ω, Xm = 1.0000e+12 Ω per phase): each point is the package's torque at one slip. Marked: the package's breakdown torque T_max = 763.9 N·m at slip 0.1 and starting torque 151.3 N·m (starting current 199 A, Thevenin voltage 400 V).

```
energy_systems_ai999x.solve('energy_induction_max_torque', VL=692.8203230275509, f=50,
poles=4, R1=0.0, X1=1.0, R2=0.2, X2=1.0, Xm=1000000000000.0, connection='star')
# -> s_maxT = 0.1, T_max = 763.944 N*m, T_start = 151.276 N*m, I_start = 199.007 A, V_th =
400 V, R_th = 0 ohm, X_th = 1 ohm   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Induction motor maximum torque: VL = 460 V, f = 60 Hz, poles = 4, R1 = 0.641 ohm, X1 = 1.106 ohm, R2 = 0.332 ohm, X2 = 0.464 ohm, Xm = 26.3 ohm	NEEDS_INPUT	energy_induction_max_torque needs connection (stator connection). Example: solve('energy_induction_max_torque', VL='692.
Induction motor breakdown torque: VL = 460 V, f = 60 Hz, poles = 4, R1 = 0 ohm, X1 = 1 ohm, R2 = 0.2 ohm, X2 = 1 ohm, Xm = 1e12 ohm, connection = star	SOLVED	s_maxT = 0.1, T_max = 280.643 N*m, T_start = 55.5729 N*m, I_start = 132.132 A, V_th = 265.581 V, R_th = 0 ohm, X_th = 1 ohm

Synchronous generator internal EMF, power angle and regulation,

`energy_synchronous_generator`

The internal (excitation) EMF, power angle and voltage regulation of a star-connected cylindrical-rotor synchronous generator delivering a current at a power factor, from its synchronous reactance and armature resistance.

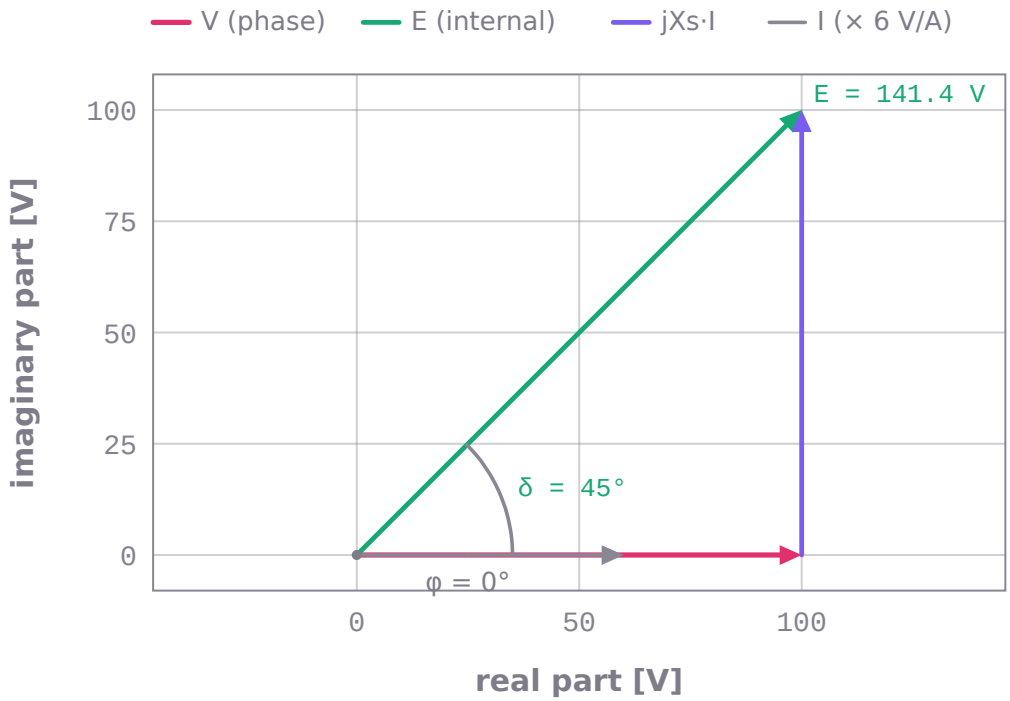
$$\mathbf{E}_A = \mathbf{V}_\phi + (R_A + jX_S)\mathbf{I}_A$$

input	meaning	condition
`VL`	terminal line voltage	
`IL`	armature (line) current	

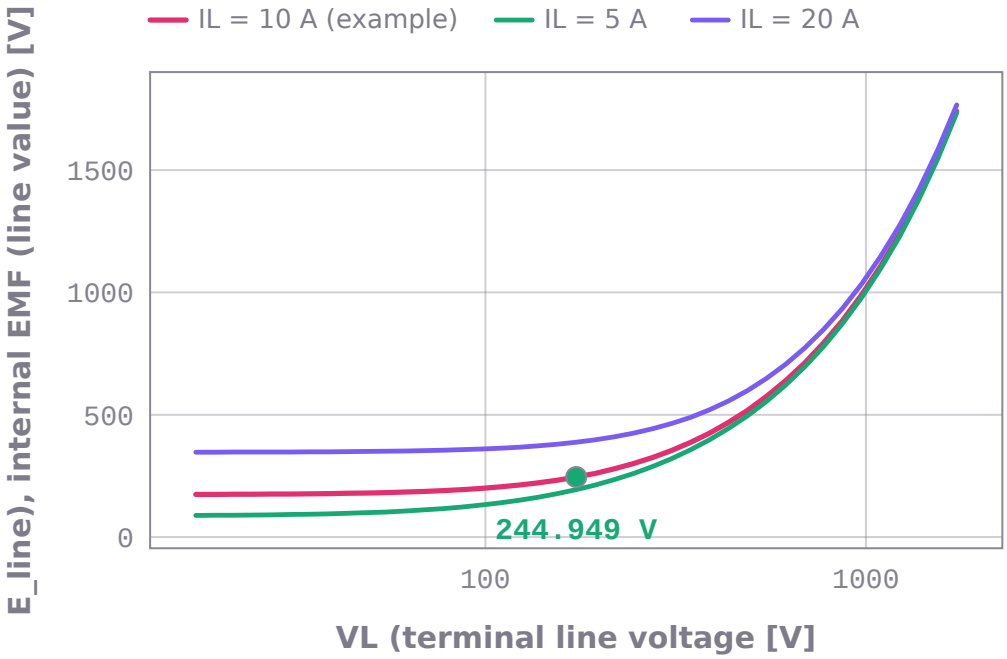
input	meaning	condition
`pf`	power factor	
`Xs`	synchronous reactance per phase	
`Ra`	armature resistance per phase (default 0)	
`pf_type`	lagging (default) or leading	

output	meaning
`E_line`	internal EMF (line value)
`E_phase`	internal EMF per phase
`delta_deg`	power (torque) angle in degrees
`regulation_pct`	voltage regulation (E - V)/V in percent
`P`	real power output
`Q`	reactive power output in var

- **Method:** E = V + (Ra + jXs) I per phase, I at angle -acos(pf)
- **Assumes:** cylindrical rotor (no saliency), star connection, unsaturated
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Phasor diagram of the worked example's synchronous generator per phase: terminal 173.205 V line (V = 100 V phase, the reference), 10 A at power factor 1 lagging, synchronous reactance Xs = 10 Ω, armature resistance neglected (default). The package returns E = 141.4 V per phase (244.9 V line) at load angle δ = 45°, regulation 41.42 %, P = 3 kW, Q = 0 kvar; the third side joins the tip of V to the tip of E (jXs·I). Current drawn at 6 V per A. Marked: E, δ, φ.



E_line (internal EMF (line value)) against VL (terminal line voltage) from 17.3205 V to 1732.05 V on a logarithmic axis. One curve for each value of IL (armature (line) current): 10 A, 5 A, 20 A. Other inputs as in the worked example: pf = 1.0, Xs = 10.0 ohm. 41 points per curve computed by energy-systems-ai999x; marked: the worked example, VL = 173.205 V, E_line = 244.949 V.

```
energy_systems_ai999x.solve('energy_synchronous_generator', VL=173.20508075688772, IL=10,
pf=1.0, Xs=10.0)
# -> E_line = 244.949 V, E_phase = 141.421 V, delta_deg = 45, regulation_pct = 41.4214, P =
3000 W, Q = 0 var   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Synchronous generator: VL = 173.20508075688772 V, IL = 10 A, pf = 1, Xs = 10 ohm	SOLVED	E_line = 244.949 V, E_phase = 141.421 V, delta_deg = 45, regulation_pct = 41.4214, P = 3000 W, Q = 0 var
Synchronous generator: VL = 480 V, IL = 60 A, pf = 0.8, Xs = 0 ohm	INVALID_INPUT	Synchronous generator internal EMF, power angle and regulation: the terminal voltage, the current and the synchronous re

Peukert exponent and run time at a new discharge current, `energy_peukert`

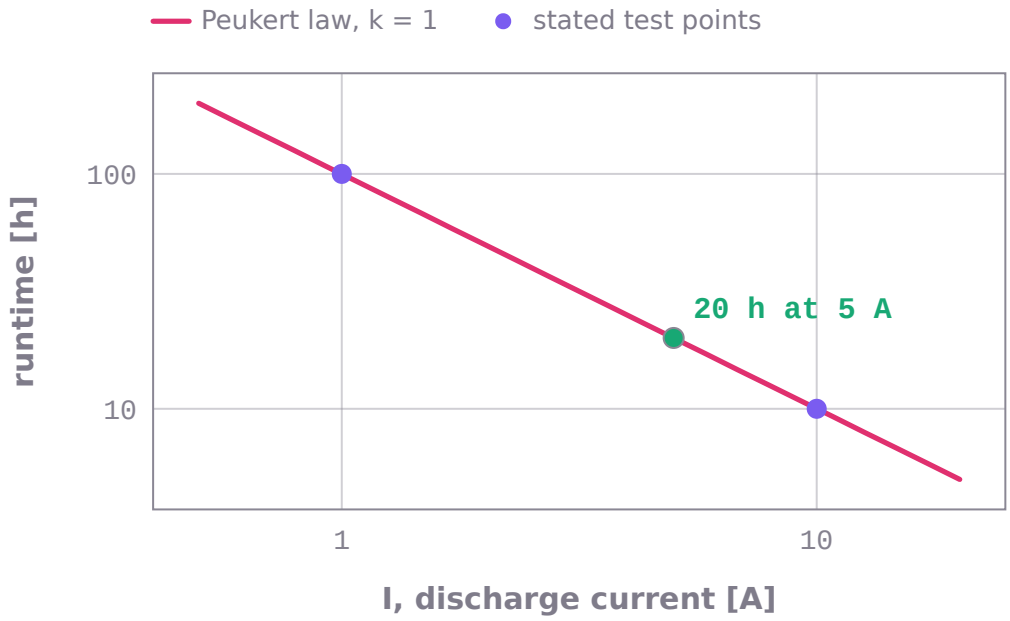
Peukert's law $t = C_p / I^k$ fitted (least squares on the logarithms) to two or more constant-current discharge tests, then the run time and delivered capacity at another current.

$$t = \frac{C_p}{I^k} \;\Rightarrow\; \ln t = \ln C_p - k \ln I$$

input	meaning	condition
`currents`	test discharge currents in A	
`times`	run time of each test in hours	
`I`	new discharge current	

output	meaning
`k`	Peukert exponent
`C_p`	Peukert capacity C_p in A^k h
`runtime_h`	run time at I in hours
`capacity_Ah`	capacity delivered at I in Ah

- **Method:** ln $t = ln$ C_p - k ln I fitted by least squares; t(I) = C_p I^{-k}
- **Assumes:** at least two tests at different currents; constant temperature and cut-off voltage
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Battery runtime against discharge current on logarithmic axes from the worked example's two tests (100 h at 1 A, 10 h at 10 A): the package fits Peukert's exponent k = 1 and capacity C_p = 100, and each point of the line is its runtime at one current. Marked: at 5 A, 20 h, 100 Ah delivered.

```
energy_systems_ai999x.solve('energy_peukert', currents=[1, 10], times=[100, 10], I=5)
# -> k = 1, C_p = 100, runtime_h = 20, capacity_Ah = 100   (SOLVED)
```

```
energy_systems_ai999x.solve('energy_peukert', currents='2, 8', times='50, 10', I=4)
# -> k = 1.16096, C_p = 111.803, runtime_h = 22.3607, capacity_Ah = 89.4427   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Peukert: currents = [1, 10], times = [100, 10], I = 5 A	SOLVED	k = 1, C_p = 100, runtime_h = 20, capacity_Ah = 100
Peukert: currents = [5, 5], times = [20, 4], I = 10 A	INVALID_INPUT	Peukert exponent and run time at a new discharge current: the test currents must be different.

Battery pack series-parallel sizing, `energy_battery_pack_sizing`

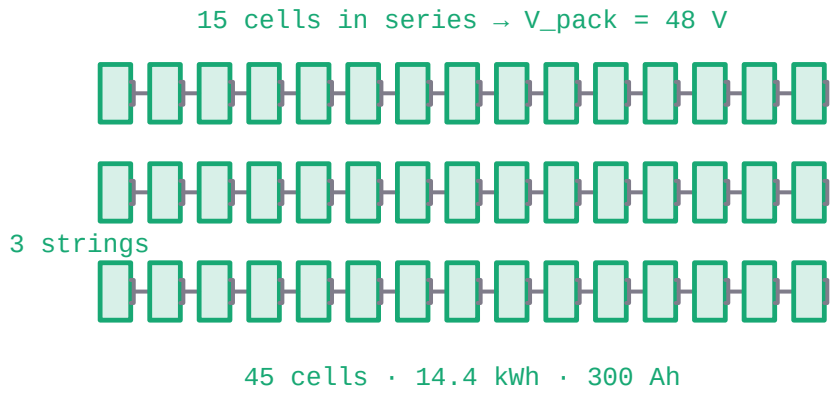
The number of cells in series to reach a pack voltage and of strings in parallel to reach a pack energy, with the resulting pack voltage, energy and capacity (fewest cells; nominal values).

$$N_s = \left\lceil \frac{V_{pack}}{V_{cell}} \right\rceil, \quad N_p = \left\lceil \frac{E_{pack}}{N_s V_{cell} Q_{cell}} \right\rceil$$

input	meaning	condition
`V_target`	required nominal pack voltage	
`E_target`	required pack energy (Wh or kWh)	
`V_cell`	nominal cell voltage	
`Q_cell`	cell capacity (Ah accepted)	

output	meaning
`N_series`	cells in series
`N_parallel`	strings in parallel
`N_cells`	total cells
`V_pack`	nominal pack voltage
`E_pack_kWh`	pack energy in kWh
`Q_pack_Ah`	pack capacity in Ah

- **Method:** N_s = ceil(V/V_cell); N_p = ceil(E/(N_s V_cell Q_cell))
- **Assumes:** nominal voltages and capacities; a target met within $1e-9$ counts as met
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Battery pack sized by the package for the worked example: target 48 V and 10 kWh from cells of 3.2 V and 100 Ah. Each box is one cell; a row is one series string. Marked: 15 in series, 3 in parallel, 45 cells, pack 48 V, 14.4 kWh, 300 Ah.

```
energy_systems_ai999x.solve('energy_battery_pack_sizing', V_target=48, E_target=36000000.0,
V_cell=3.2, Q_cell=360000)
# -> N_series = 15, N_parallel = 3, N_cells = 45, V_pack = 48 V, E_pack_kWh = 14.4,
Q_pack_Ah = 300   (SOLVED)
```

```
energy_systems_ai999x.solve('energy_battery_pack_sizing', V_target=400,
E_target=216000000.0, V_cell=3.7, Q_cell=18000)
# -> N_series = 109, N_parallel = 30, N_cells = 3270, V_pack = 403.3 V, E_pack_kWh =
60.495, Q_pack_Ah = 150   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Battery pack sizing: V_target = 48 V, E_target = 10 kWh, V_cell = 3.2 V, Q_cell = 100 Ah	SOLVED	N_series = 15, N_parallel = 3, N_cells = 45, V_pack = 48 V, E_pack_kWh = 14.4, Q_pack_Ah = 300
Battery pack sizing: V_target = 48 V, E_target = 10 kWh, V_cell = 0 V, Q_cell = 100 Ah	INVALID_INPUT	Battery pack series-parallel sizing: the target voltage and energy and the cell voltage and capacity must be a positive

Battery terminal voltage under constant current (one-RC equivalent circuit), `energy_battery_rc_discharge`

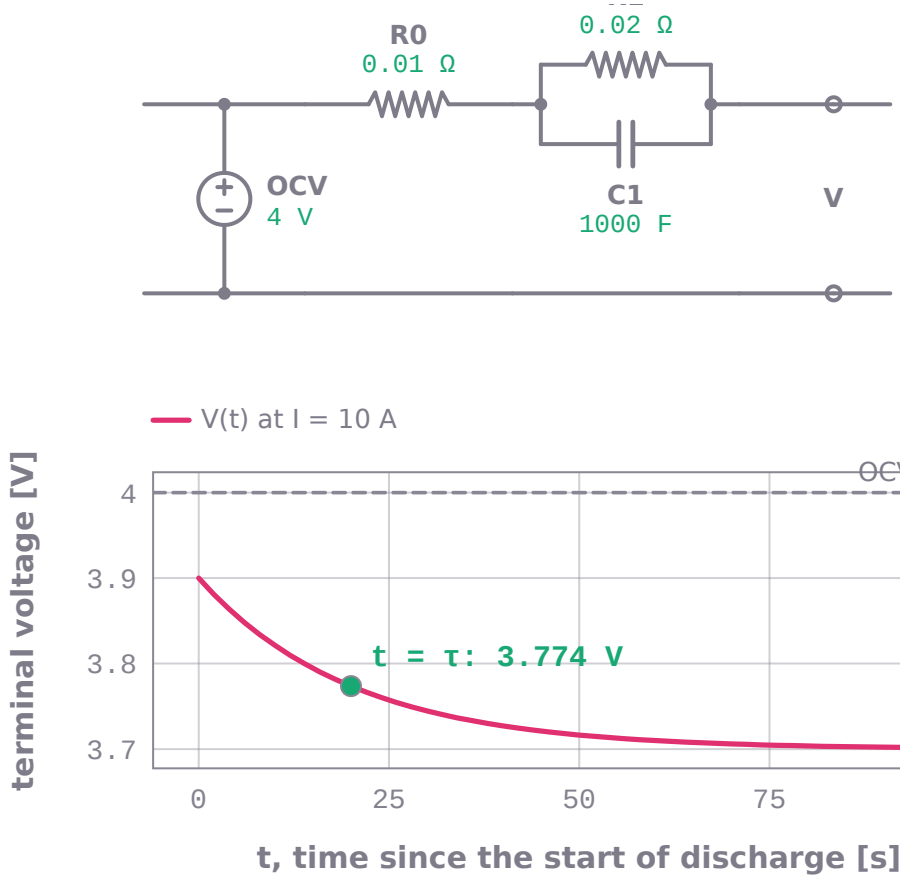
Terminal voltage, polarization (RC) voltage, energy delivered and heat generated after a constant-current discharge of duration t, for a cell modelled as an open-circuit voltage with a series resistance R0 and one R1 || C1 branch (constant OCV).

$$V_t(t) = V_{oc} - IR_0 - IR_1\left(1 - e^{-t/R_1C_1}\right)$$

input	meaning	condition
`OCV`	open-circuit voltage (held constant)	
`R0`	series (ohmic) resistance	
`R1`	polarization resistance	
`C1`	polarization capacitance	
`I`	discharge current	
`t`	time since the current step	

output	meaning
`V_terminal`	terminal voltage
`V_rc`	voltage across the RC branch
`E_delivered`	energy delivered to the load
`E_Wh`	energy delivered in Wh
`heat`	heat generated in R0 and R1
`tau`	time constant R1 C1

- **Method:** $V_1 = I$ R1 (1 - e^(-t/tau)), V_t = OCV - I $R_0 - V_1$; energy and heat integrated in closed form
- **Assumes:** constant open-circuit voltage over the interval (short pulse or flat OCV curve); RC branch initially relaxed
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Top: the one-RC battery model with the worked example's values (OCV = 4 V, R0 = 0.01 Ω, R1 = 0.02 Ω, C1 = 1000 F). Bottom: terminal voltage during a constant 10 A discharge, one run per time from 0 to 5τ; the instant drop is I·R0 and the slower one charges R1–C1 with τ = 20 s (returned). Marked: at t = τ the package returns 3.774 V, 0.2126 Wh delivered and 26.72 J of heat.

```
energy_systems_ai999x.solve('energy_battery_rc_discharge', OCV=4.0, R0=0.01, R1=0.02,
C1=1000, I=10, t=0)
# -> V_terminal = 3.9 V, V_rc = 0 V, E_delivered = 0 J, E_Wh = 0, heat = 0 J, tau = 20 s
(SOLVED)
```

```
energy_systems_ai999x.solve('energy_battery_rc_discharge', OCV=4.0, R0=0.01, R1=0.02,
C1=1000, I=10, t=13.862943611198906)
# -> V_terminal = 3.8 V, V_rc = 0.1 V, E_delivered = 532.929 J, E_Wh = 0.148036, heat =
16.5888 J, tau = 20 s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Battery RC model: OCV = 4 V, R0 = 10 mohm, R1 = 20 mohm, C1 = 1000 F, I = 10 A, t = 0 s	SOLVED	V_terminal = 3.9 V, V_rc = 0 V, E_delivered = 0 J, E_Wh = 0, heat = 0 J, tau = 20 s
Battery RC model: OCV = 4 V, R0 = 10 mohm, R1 = 20 mohm, C1 = 1000 F, I = 500 A, t = 60 s	INVALID_INPUT	Battery terminal voltage under constant current (one-RC equivalent circuit): the terminal voltage would fall to zero: th

Hydrogen fuel-cell stack: consumption, power and efficiency, `energy_fuel_cell_stack`

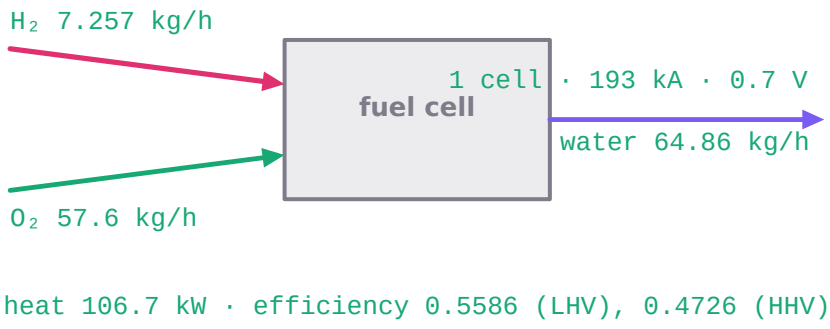
Hydrogen and oxygen consumption, water production (Faraday's law), efficiency on the lower and higher heating value and heat released of a hydrogen fuel-cell stack of N cells at a current and cell voltage.

$$\dot{n}_{H_2} = \frac{NI}{2F}, \quad \eta_{LHV} = \frac{NIV_{cell}}{\dot{n}_{H_2}\,\Delta h_{LHV}}$$

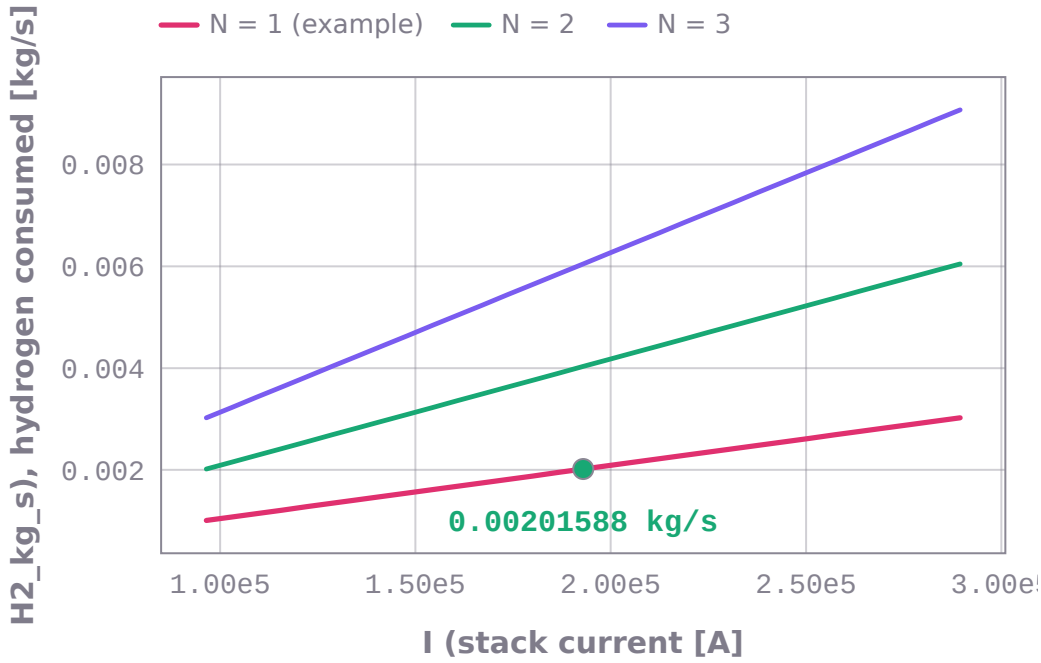
input	meaning	condition
`I`	stack current	
`N`	number of cells in series	
`V_cell`	average cell voltage	
`utilization`	fuel utilization (fraction or %, default 1)	

output	meaning
`H2_kg_s`	hydrogen consumed
`H2_g_h`	hydrogen consumed in g/h
`H2_fed_kg_s`	hydrogen fed (consumed/utilization)
`O2_kg_s`	oxygen consumed
`H2O_kg_s`	water produced
`efficiency_LHV`	efficiency on the lower heating value
`efficiency_HHV`	efficiency on the higher heating value
`heat_LHV`	heat released (water vapour product)

- **Method:** $n_H2 = N\ I/(2F)$; $n_O2 = n_H2/2$; $n_H2O = n_H2$; $P = N$ I V; eta = P/(n_H2 LHV)
- **Assumes:** hydrogen/oxygen cell, 2 electrons per H2; heating values at 25 degC
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Mass and energy balance of a hydrogen fuel cell as the package returns it for the worked example (1 cell at 1.93e+05 A and 0.7 V, full fuel utilisation by default): hydrogen 7.257 kg/h and oxygen 57.6 kg/h in, water 64.86 kg/h out. Marked: heat released 106.7 kW and efficiencies 0.5586 (lower) and 0.4726 (higher heating value).



H2_kg_s (hydrogen consumed) against I (stack current) from 96485.3 A to 289456 A. One curve for each value of N (number of cells in series): 1, 2, 3. Other inputs as in the worked example: V_cell = 0.7 V. 31 points per curve computed by energy-systems-ai999x; marked: the worked example, I = 192971 A, H2_kg_s = 0.00201588 kg/s.

```
energy_systems_ai999x.solve('energy_fuel_cell_stack', I=192970.66424, N=1, V_cell=0.7)
# -> H2_kg_s = 0.00201588 kg/s, H2_g_h = 7257.17, H2_fed_kg_s = 0.00201588 kg/s, O2_kg_s =
0.0159994 kg/s, H2O_kg_s = 0.0180153 kg/s, efficiency_LHV = 0.558581, efficiency_HHV =
0.472593, heat_LHV = 106747 W   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Fuel-cell stack: I = 192970.66424 A, N = 1, V_cell = 0.7 V	SOLVED	H2_kg_s = 0.00201588 kg/s, H2_g_h = 7257.17, H2_fed_kg_s = 0.00201588 kg/s, O2_kg_s = 0.0159994 kg/s, H2O_kg_s = 0.0180153 kg/s, efficiency_LHV = 0.558581, efficiency_HHV = 0.472593, heat_LHV = 106747 W
Fuel-cell stack: I = 200 A, N = 100, V_cell = 1.4 V	INVALID_INPUT	Hydrogen fuel-cell stack: consumption, power and efficiency: a hydrogen fuel cell's voltage must stay below the thermone

Water electrolyser: hydrogen production and energy per kilogram, `energy_electrolyser`

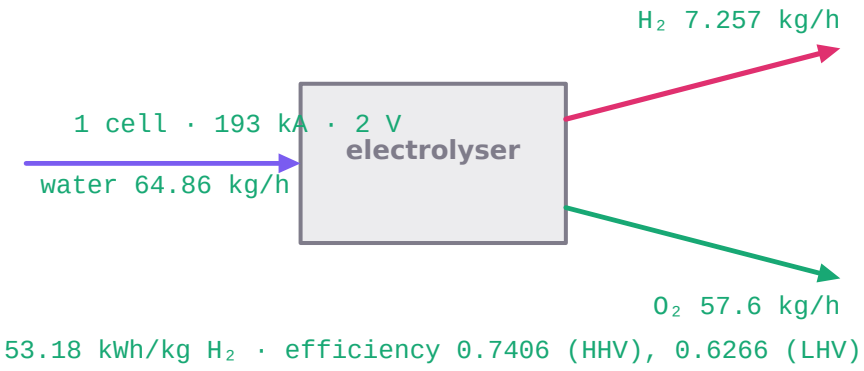
Hydrogen and oxygen production, water consumption, specific energy (kWh per kg H2) and efficiency on the higher and lower heating value of a water electrolyser of N cells.

$$\frac{E}{m_{H_2}} = \frac{2FV_{cell}}{\eta_F M_{H_2}}$$

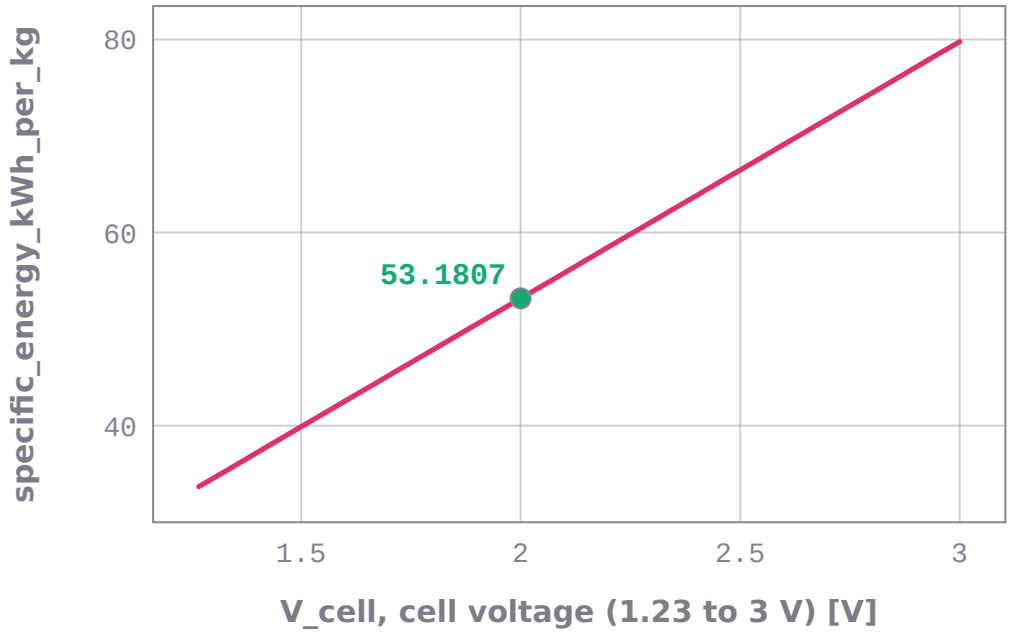
input	meaning	condition
`I`	stack current	
`N`	number of cells in series	
`V_cell`	cell voltage (1.23 to 3 V)	
`faradaic`	Faradaic (current) efficiency, fraction or % (default 1)	

output	meaning
`H2_kg_s`	hydrogen produced
`H2_kg_h`	hydrogen produced in kg/h
`O2_kg_s`	oxygen produced
`H2O_kg_s`	water consumed
`specific_energy_kWh_per_kg`	electricity per kg of hydrogen in kWh/kg
`efficiency_HHV`	efficiency on the higher heating value
`efficiency_LHV`	efficiency on the lower heating value

- **Method:** $n_H2 = eta_F$ N $I/(2F)$; specific energy = N I V/(n_H2 M_H2); eta_HHV = n_H2 HHV/P
- **Assumes:** water electrolysis, 2 electrons per H2; heating values at 25 degC
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Mass balance of a water electrolyser as the package returns it for the worked example (1 cell at 1.93e+05 A (one faraday per second), and 2 V, full faradaic efficiency by default): water in 64.86 kg/h, hydrogen 7.257 kg/h and oxygen 57.6 kg/h out (kg/s × 3600). Marked: specific energy 53.18 kWh per kg of hydrogen and efficiencies 0.7406 (higher) and 0.6266 (lower heating value).



specific_energy_kWh_per_kg (electricity per kg of hydrogen in kWh/kg) against V_cell (cell voltage (1.23 to 3 V)) from 1 V to 3 V. Other inputs as in the worked example: I = 192970.66424 A, N = 1. 27 points computed by energy-systems-ai999x; marked: the worked example, V_cell = 2 V, specific_energy_kWh_per_kg = 53.1807.

```
energy_systems_ai999x.solve('energy_electrolyser', I=192970.66424, N=1, V_cell=2.0)
# -> H2_kg_s = 0.00201588 kg/s, H2_kg_h = 7.25717, O2_kg_s = 0.0159994 kg/s, H2O_kg_s =
0.0180153 kg/s, specific_energy_kWh_per_kg = 53.1807, efficiency_HHV = 0.740594,
efficiency_LHV = 0.626587   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Electrolyser: I = 192970.66424 A, N = 1, V_cell = 2 V	SOLVED	H2_kg_s = 0.00201588 kg/s, H2_kg_h = 7.25717, O2_kg_s = 0.0159994 kg/s, H2O_kg_s = 0.0180153 kg/s, specific_energy_kWh_per_kg = 53.1807, efficiency_HHV = 0.740594, efficiency_LHV = 0.626587
Electrolyser: I = 1000 A, N = 100, V_cell = 1 V	INVALID_INPUT	Water electrolyser: hydrogen production and energy per kilogram: an electrolysis cell operates between the reversible vo

Hydrogen stored in a tank at pressure (real gas), `energy_hydrogen_tank`

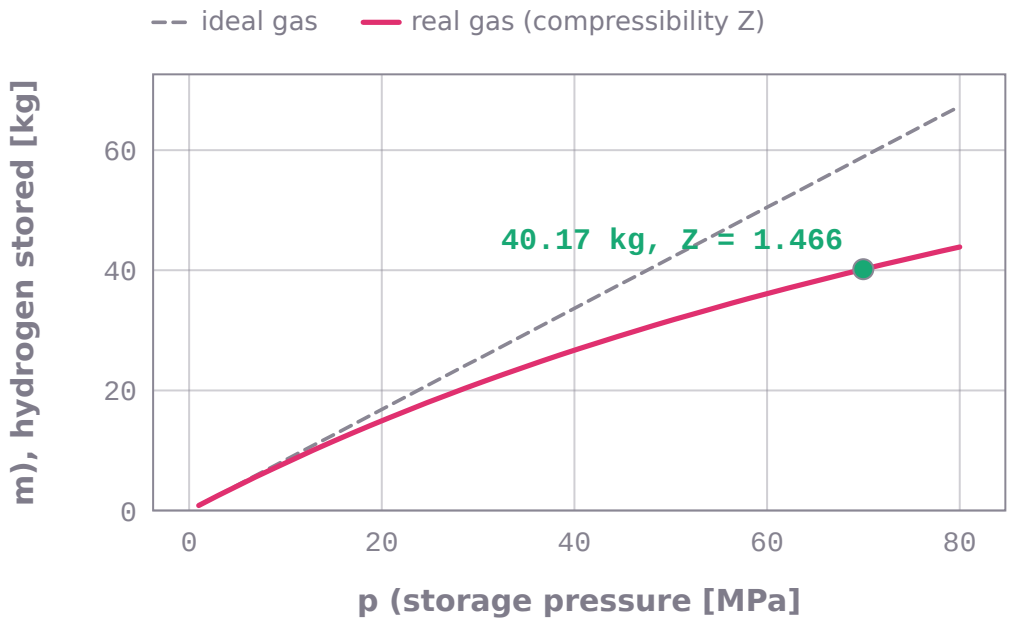
Density, compressibility factor, stored mass and energy (lower heating value) of compressed hydrogen in a tank at a pressure and temperature, by the Leachman (2009) equation of state for normal hydrogen, compared with the ideal-gas estimate.

$$p = \rho R T \left(1 + \delta \frac{\partial \alpha^r}{\partial \delta}\right), \quad Z = \frac{p}{\rho R T}$$

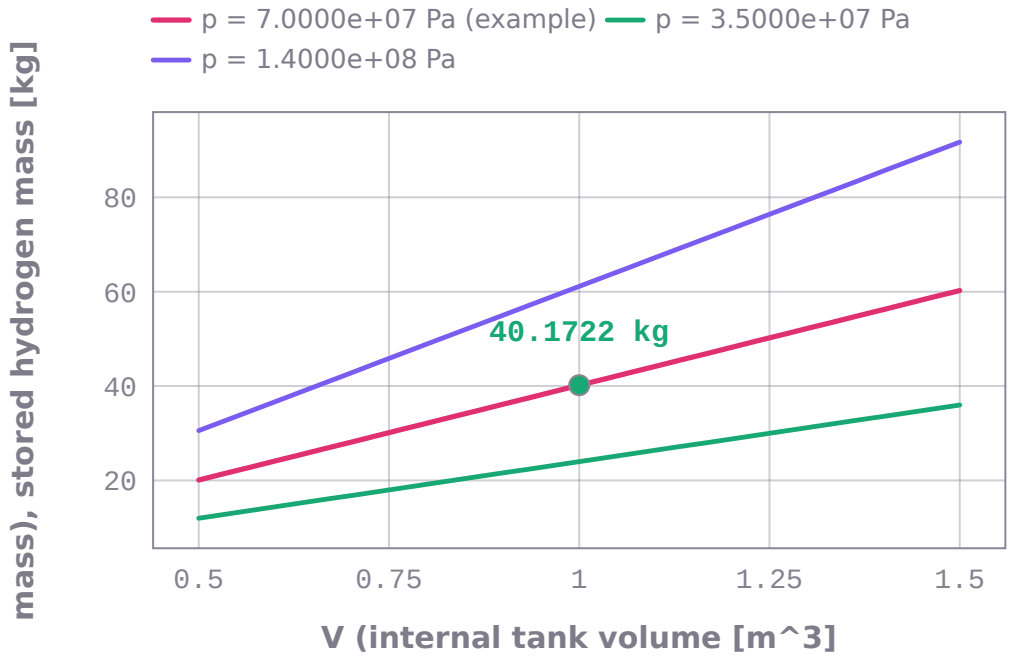
input	meaning	condition
`V`	internal tank volume	
`p`	gas pressure (absolute)	
`T`	gas temperature	

output	meaning
`rho`	hydrogen density (real gas)
`Z`	compressibility factor
`mass`	stored hydrogen mass
`rho_ideal`	ideal-gas density p M/(R T)
`mass_ideal`	ideal-gas mass
`ideal_over_real`	ideal-gas over real density (= Z)
`energy_LHV_kWh`	stored chemical energy (LHV) in kWh

- **Method:** Leachman 2009 Helmholtz equation: $p = \rho$ R T (1 + delta d(alpha_r)/d(delta)), solved for rho; Z = p/(rho R T)
- **Assumes:** 1 kPa <= p <= 1500 bar; 200 $K \leq T \leq 700$ K; normal hydrogen, uniform state
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Hydrogen held in a 1 m³ tank at 288.15 K against pressure, each point a run of the package: the real-gas mass (with its compressibility factor) and the ideal-gas mass. Marked: the worked example at 70 MPa, 40.17 kg (ideal 58.9 kg, Z = 1.466), 1339 kWh of lower heating value.



mass (stored hydrogen mass) against V (internal tank volume) from 0.5 m^3 to 1.5 m^3. One curve for each value of p (gas pressure (absolute)): 7.0000e+07 Pa, 3.5000e+07 Pa, 1.4000e+08 Pa. Other inputs as in the worked example: T = 288.15 K. 31 points per curve computed by energy-systems-ai999x; marked: the worked example, V = 1 m^3, mass = 40.1722 kg.

```
energy_systems_ai999x.solve('energy_hydrogen_tank', V=1.0, p=70000000.0, T=288.15)
# -> rho = 40.1722 kg/m^3, Z = 1.46617, mass = 40.1722 kg, rho_ideal = 58.8992 kg/m^3,
mass_ideal = 58.8992 kg, ideal_over_real = 1.46617, energy_LHV_kWh = 1338.63   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Hydrogen tank: V = 1 m^3, p = 700 bar, T = 288.15 K	SOLVED	rho = 40.1722 kg/m^3, Z = 1.46617, mass = 40.1722 kg, rho_ideal = 58.8992 kg/m^3, mass_ideal = 58.8992 kg, ideal_over_real = 1.46617, energy_LHV_kWh = 1338.63
Hydrogen tank: V = 1 m^3, p = 700 bar, T = 50 K	INVALID_INPUT	Hydrogen stored in a tank at pressure (real gas): the temperature must lie between 200 K and 700 K (gaseous hydrogen).

Minimum (isothermal) work to compress hydrogen, `energy_hydrogen_isothermal_compression`

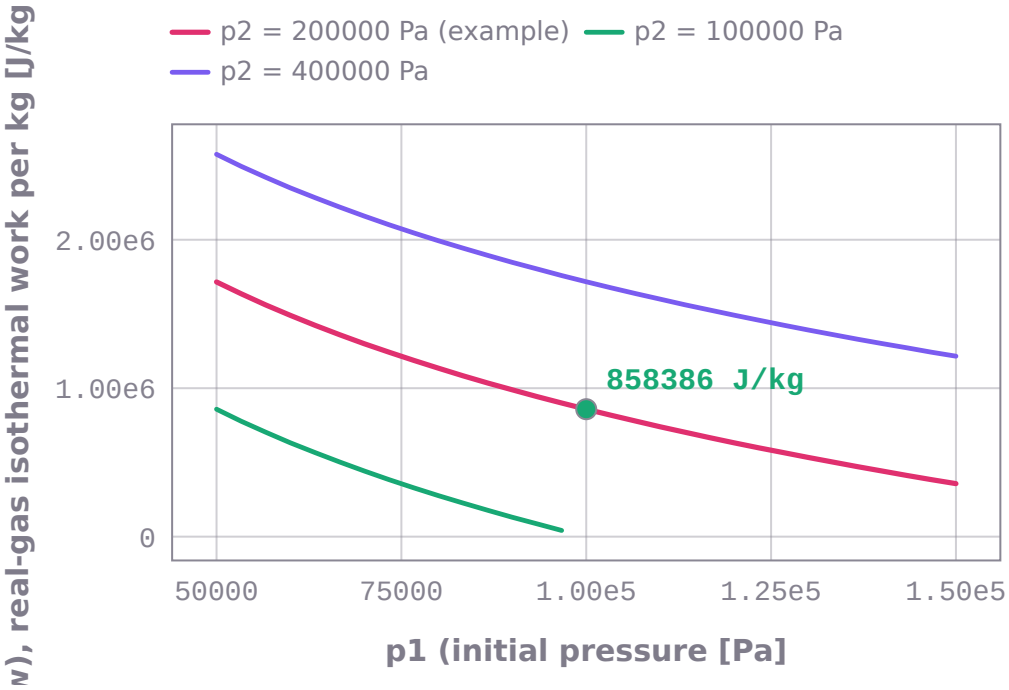
The reversible isothermal work per kilogram to compress hydrogen from p1 to p2 at temperature T (the integral of v dp, equal to the Gibbs-energy rise) for the real gas (Leachman equation of state) and the ideal gas.

$$w = \int_{p_1}^{p_2} v\,dp = g(p_2,T) - g(p_1,T), \quad w_{ideal} = \frac{RT}{M}\ln\frac{p_2}{p_1}$$

input	meaning	condition
`p1`	initial pressure	
`p2`	final pressure	
`T`	compression temperature	

output	meaning
`w`	real-gas isothermal work per kg
`w_kWh_per_kg`	the same in kWh/kg
`w_ideal`	ideal-gas work R T ln(p2/p1)/M
`real_over_ideal`	real over ideal work

- **Method:** $w = g$(p2, T) - g(p1, T), $g/(RT) = ln$(delta) + alpha_r + delta d(alpha_r)/d(delta) + (terms in T only)
- **Assumes:** reversible and isothermal; 200 $K \leq T \leq 700$ K, pressures up to 1500 bar
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



w (real-gas isothermal work per kg) against p1 (initial pressure) from 50000 Pa to 150000 Pa. One curve for each value of p2 (final pressure): 200000 Pa, 100000 Pa, 400000 Pa. Other inputs as in the worked example: T = 300 K. 31 points per curve computed by energy-systems-ai999x; marked: the worked example, p1 = 100000 Pa, w = 858386 J/kg.

```
energy_systems_ai999x.solve('energy_hydrogen_isothermal_compression', p1=100000.0,
p2=200000.0, T=300)
# -> w = 858386 J/kg, w_kWh_per_kg = 0.238441, w_ideal = 857663 J/kg, real_over_ideal =
1.00084   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Hydrogen isothermal compression: p1 = 1 bar, p2 = 2 bar, T = 300 K	SOLVED	w = 858386 J/kg, w_kWh_per_kg = 0.238441, w_ideal = 857663 J/kg, real_over_ideal = 1.00084
Hydrogen isothermal compression: p1 = 700 bar, p2 = 20 bar, T = 300 K	INVALID_INPUT	Minimum (isothermal) work to compress hydrogen: the final pressure must exceed the initial pressure.

Isentropic (adiabatic) compression of hydrogen, `energy_hydrogen_isentropic_compression`

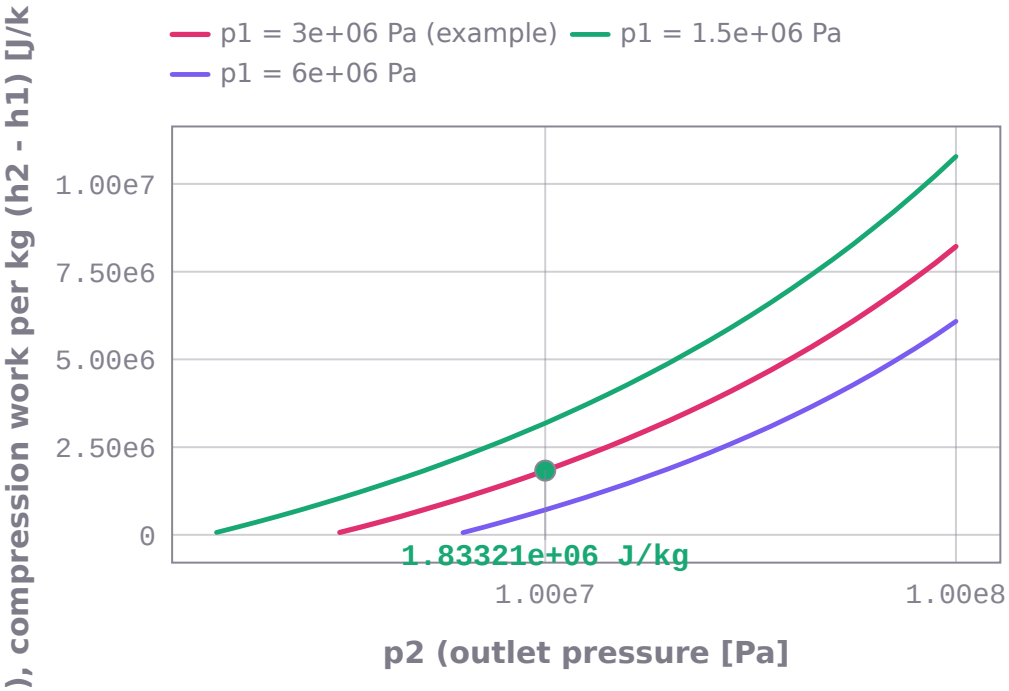
Outlet temperature and work per kilogram of a reversible adiabatic compression of hydrogen from (p1, T1) to p2, from the full Leachman equation of state (real gas, variable heat capacity).

$$s(p_2, T_2) = s(p_1, T_1), \quad w = h_2 - h_1$$

input	meaning	condition
`p1`	inlet pressure	
`T1`	inlet temperature	
`p2`	outlet pressure	

output	meaning
`T2`	outlet temperature
`w`	compression work per kg $(h_2 - h_1)$
`w_kWh_per_kg`	work in kWh/kg

- **Method:** s(p2, T2) = s(p1, T1) solved for T2 by bisection; $w = h$(p2, T2) - h(p1, T1); s and h from the Helmholtz energy (ideal + residual parts)
- **Assumes:** reversible adiabatic, single stage; outlet temperature below 1000 K
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



w (compression work per kg (h2 - h1)) against p2 (outlet pressure) from 1e+06 Pa to 1.0000e+08 Pa on a logarithmic axis. One curve for each value of p1 (inlet pressure): 3e+06 Pa, 1.5e+06 Pa, 6e+06 Pa. Other inputs as in the worked example: T1 = 300.0 K. 31 points per curve computed by energy-systems-ai999x; marked: the worked example, p2 = 1.0000e+07 Pa, w = 1.83321e+06 J/kg.

```
energy_systems_ai999x.solve('energy_hydrogen_isentropic_compression', p1=3000000.0,
T1=300.0, p2=10000000.0)
# -> T2 = 423.294 K, w = 1.83321e+06 J/kg, w_kWh_per_kg = 0.509226   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Hydrogen isentropic compression: p1 = 30 bar, T1 = 300 K, p2 = 100 bar	SOLVED	T2 = 423.294 K, w = 1.83321e+06 J/kg, w_kWh_per_kg = 0.509226
Hydrogen isentropic compression: p1 = 30 bar, T1 = 300 K, p2 = 10 bar	INVALID_INPUT	Isentropic (adiabatic) compression of hydrogen: the final pressure must exceed the initial pressure.

PV single-diode model: current at a voltage, `energy_single_diode_current`

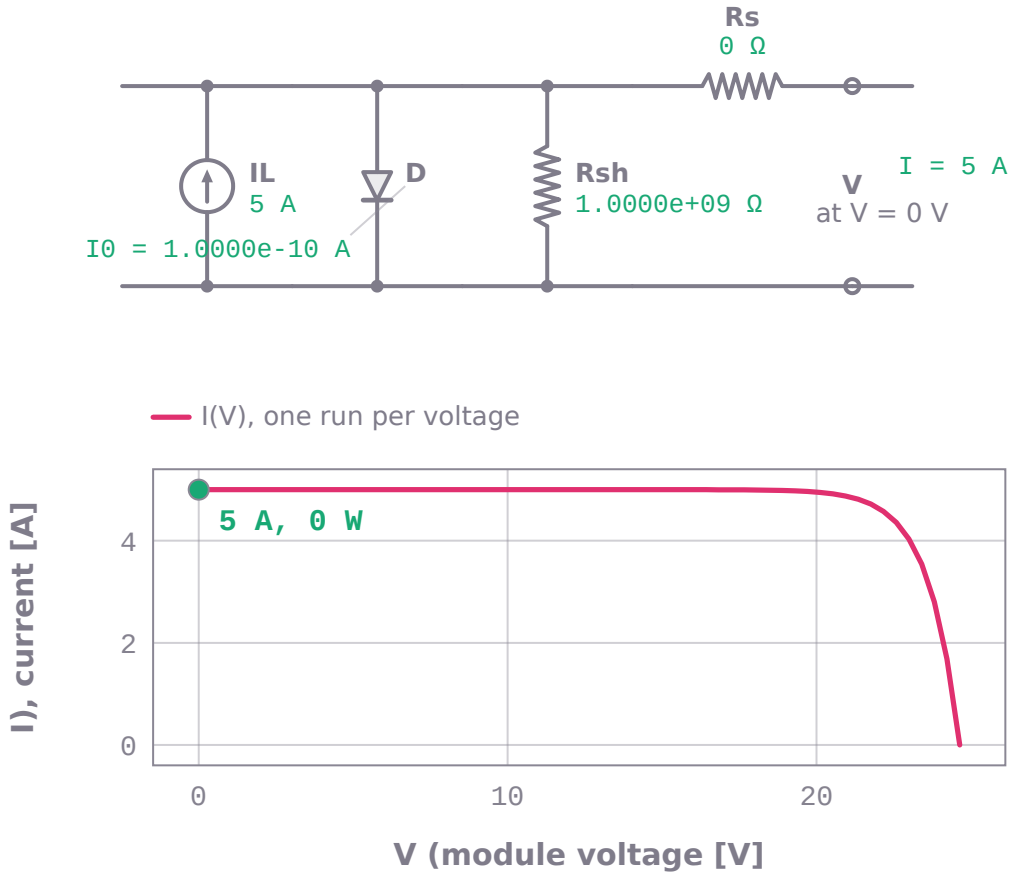
The current (and power) of a PV cell or module at a terminal voltage from the five-parameter single-diode equation $I = IL - I_0$ [exp((V + I Rs)/a) - 1] - (V + I Rs)/Rsh, solved implicitly.

$$I = I_L - I_0 \left[e^{(V+IR_s)/a} - 1 \right] - \frac{V + IR_s}{R_{sh}}$$

input	meaning	condition
`V`	terminal voltage	
`IL`	photocurrent (light-generated current)	
`I0`	diode saturation current	
`Rs`	series resistance	
`Rsh`	shunt resistance	
`nNsVth`	modified ideality factor n Ns k T/q	

output	meaning
`I`	terminal current
`P`	power V I

- **Method:** the implicit single-diode equation solved for I by bisection to full precision
- **Assumes:** IL, I0, Rsh, $a > 0$; $Rs \geq 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Top: the single-diode equivalent circuit (photocurrent source, diode, shunt and series resistance) with the worked example's values (photocurrent 5 A, saturation current 1.0000e-10 A, Rs = 0 Ω, Rsh = 1.0000e+09 Ω, nNsVth = 1 V). Bottom: the package's current at 61 voltages from 0 to Voc = 24.64 V. Marked: at V = 0 V the package returns I = 5 A and P = 0 W.

```
energy_systems_ai999x.solve('energy_single_diode_current', V=0.0, IL=5.0, I0=1e-10, Rs=0.0,
Rsh=1000000000.0, nNsVth=1.0)
# -> I = 5 A, P = 0 W   (SOLVED)
```

```
energy_systems_ai999x.solve('energy_single_diode_current', V=1.0, IL=5.0, I0=1e-10, Rs=0.0,
Rsh=100.0, nNsVth=1.0)
# -> I = 4.99 A, P = 4.99 W   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Single-diode current: V = 1 V, IL = 5 A, I0 = 1e-10 A, Rs = 0 ohm, Rsh = 100 ohm, nNsVth = 1 V	SOLVED	I = 4.99 A, P = 4.99 W
Single-diode current: V = 1 V, IL = 5 A, I0 = 1e-10 A, Rs = 0 ohm, Rsh = 0 ohm, nNsVth = 1 V	INVALID_INPUT	PV single-diode model: current at a voltage: the photocurrent, saturation current, shunt resistance and nNsVth must be a

PV single-diode model: voltage at a current, `energy_single_diode_voltage`

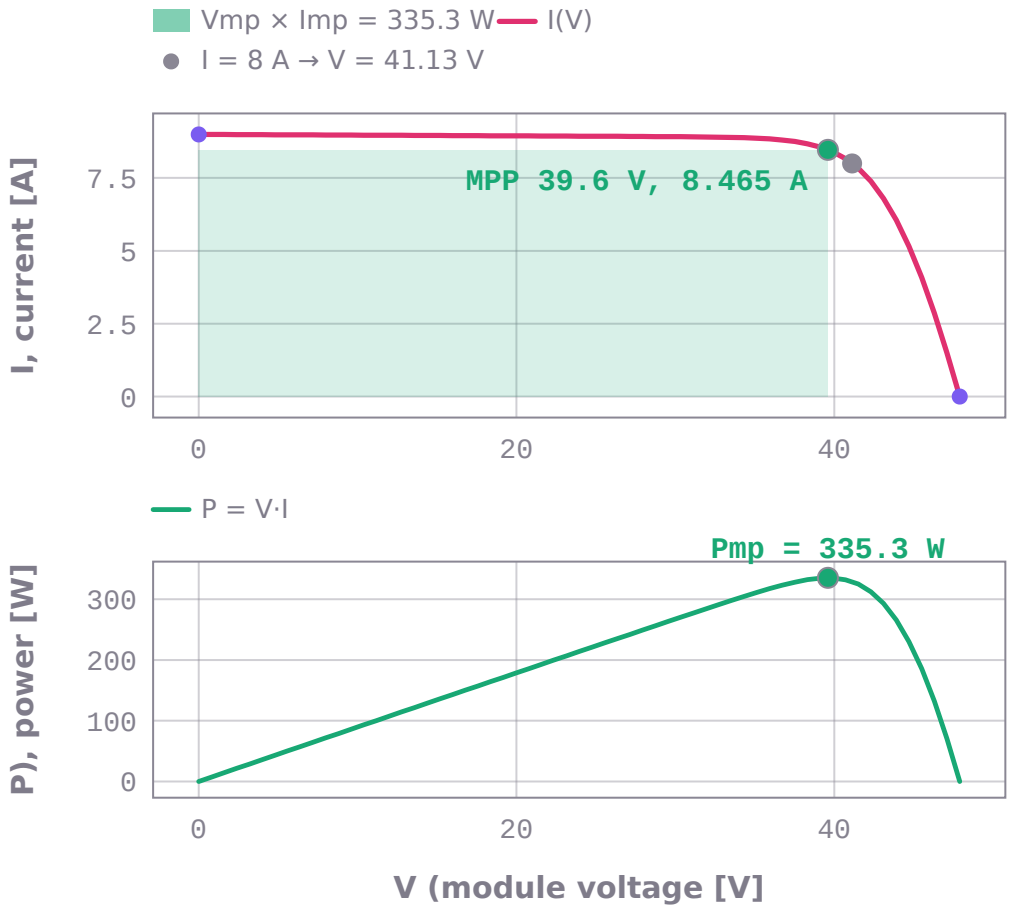
The terminal voltage (and power) of a PV cell or module carrying a current, from the five-parameter single-diode equation solved implicitly for V; $I = 0$ gives the open-circuit voltage.

$$I = I_L - I_0 \left[e^{(V+IR_s)/a} - 1 \right] - \frac{V + IR_s}{R_{sh}}$$

input	meaning	condition
`I`	terminal current	
`IL`	photocurrent (light-generated current)	
`I0`	diode saturation current	
`Rs`	series resistance	
`Rsh`	shunt resistance	
`nNsVth`	modified ideality factor n Ns k T/q	

output	meaning
`V`	terminal voltage
`P`	power V I

- **Method:** the implicit single-diode equation solved for V by bisection to full precision
- **Assumes:** IL, I0, Rsh, $a > 0$; $Rs \geq 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The worked example: at a current of 8 A the package returns V = 41.13 V and P = 329 W (single-diode parameters photocurrent 9 A, saturation current 1.0000e-10 A, Rs = 0.3 Ω, Rsh = 400 Ω, nNsVth = 1.9 V). The I–V (top) and P–V (bottom) curves are the package's currents at 61 voltages. Marked: the asked point and the maximum power point Pmp = 335.3 W at 39.6 V.

```
energy_systems_ai999x.solve('energy_single_diode_voltage', I=8.0, IL=9.0, I0=1e-10, Rs=0.3,
Rsh=400.0, nNsVth=1.9)
# -> V = 41.1302 V, P = 329.042 W   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Single-diode voltage: I = 0 A, IL = 5 A, I0 = 1e-10 A, Rs = 0 ohm, Rsh = 1e15 ohm, nNsVth = 1 V	SOLVED	V = 24.6353 V, P = 0 W
Single-diode voltage: I = 0 A, IL = 5 A, I0 = 0 A, Rs = 0 ohm, Rsh = 100 ohm, nNsVth = 1 V	INVALID_INPUT	PV single-diode model: voltage at a current: the photocurrent, saturation current, shunt resistance and nNsVth must be a

PV single-diode model: short-circuit, open-circuit and maximum power point, `energy_single_diode_mpp`

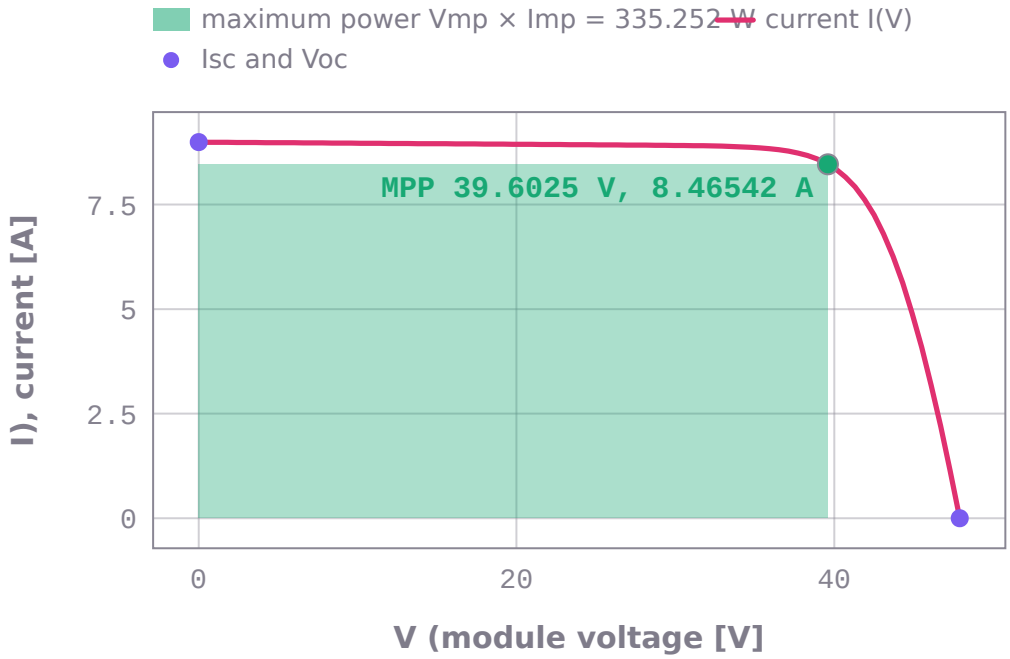
Short-circuit current, open-circuit voltage, the maximum power point (current, voltage, power) and the fill factor of a PV cell or module described by the five-parameter single-diode model.

$$\frac{dP}{dV} = 0, \quad FF = \frac{I_{mp} V_{mp}}{I_{sc} V_{oc}}$$

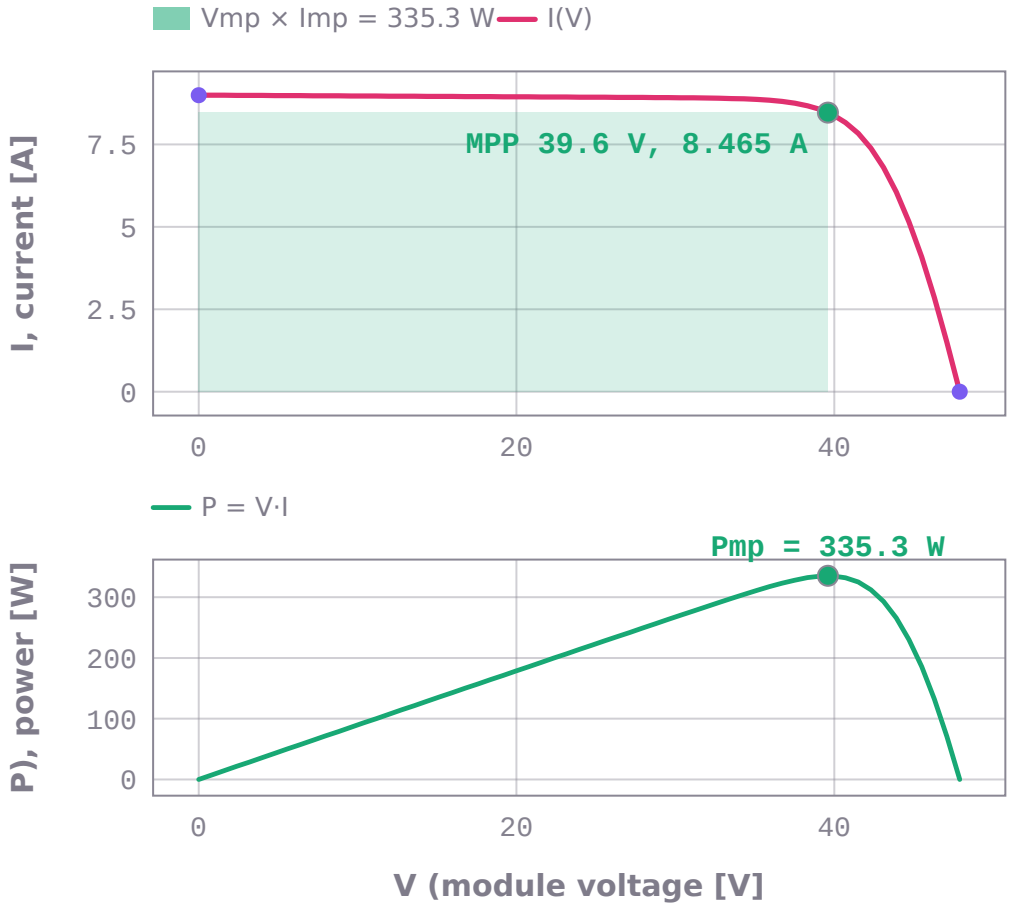
input	meaning	condition
`IL`	photocurrent (light-generated current)	
`I0`	diode saturation current	
`Rs`	series resistance	
`Rsh`	shunt resistance	
`nNsVth`	modified ideality factor n Ns k T/q	

output	meaning
`Isc`	short-circuit current
`Voc`	open-circuit voltage
`Imp`	current at maximum power
`Vmp`	voltage at maximum power
`Pmp`	maximum power
`FF`	fill factor Pmp/(Isc Voc)

- **Method:** I(0) and V(I = 0) from the implicit equation; maximum power where $dP/dV_d = 0$ along the curve parametrized by the diode voltage (bisection)
- **Assumes:** IL, I0, Rsh, $a > 0$; $Rs \geq 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Current–voltage curve of a PV module (single-diode model: photocurrent 9 A, saturation current 1.0000e-10 A, Rs = 0.3 Ω, Rsh = 400 Ω), computed by the package at 81 voltages. The shaded rectangle is the maximum power the package finds, P = 335.252 W at V = 39.6025 V, I = 8.46542 A; it fills 0.778276 of Isc × Voc (8.99326 A × 47.8984 V), the fill factor.



Current–voltage (top) and power–voltage (bottom) curves of a PV module, single-diode model with the worked example's parameters (photocurrent 9 A, saturation current 1.0000e-10 A, Rs = 0.3 Ω, Rsh = 400 Ω, nNsVth = 1.9 V); each point is the package's current at one of 61 voltages, the power is V·I of those points. Marked: the package's maximum power point Vmp = 39.6 V, Imp = 8.465 A, Pmp = 335.3 W, Isc = 8.993 A, Voc = 47.9 V, fill factor 0.7783.

```
energy_systems_ai999x.solve('energy_single_diode_mpp', IL=9.0, I0=1e-10, Rs=0.3, Rsh=400.0,
nNsVth=1.9)
# -> Isc = 8.99326 A, Voc = 47.8984 V, Imp = 8.46542 A, Vmp = 39.6025 V, Pmp = 335.252 W,
FF = 0.778276   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
PV MPP: IL = 5 A, I0 = 1e-10 A, Rs = 0 ohm, Rsh = 1e15 ohm, nNsVth = 1 V	SOLVED	Isc = 5 A, Voc = 24.6353 V, Imp = 4.77798 A, Vmp = 21.5208 V, Pmp = 102.826 W, FF = 0.834788
PV MPP: IL = 5 A, I0 = 1e-10 A, Rs = 0 ohm, nNsVth = 1 V	NEEDS_INPUT	energy_single_diode_mpp needs Rsh (shunt resistance). Example: solve('energy_single_diode_mpp', IL='9 A', I0='1e-10 A',

PV module single-diode parameters at operating conditions (De Soto),

`energy_desoto_parameters`

The five single-diode parameters (photocurrent, saturation current, series and shunt resistance, modified ideality factor) of a PV module at an irradiance and cell temperature from its reference-condition values, by the De Soto (2006) model.

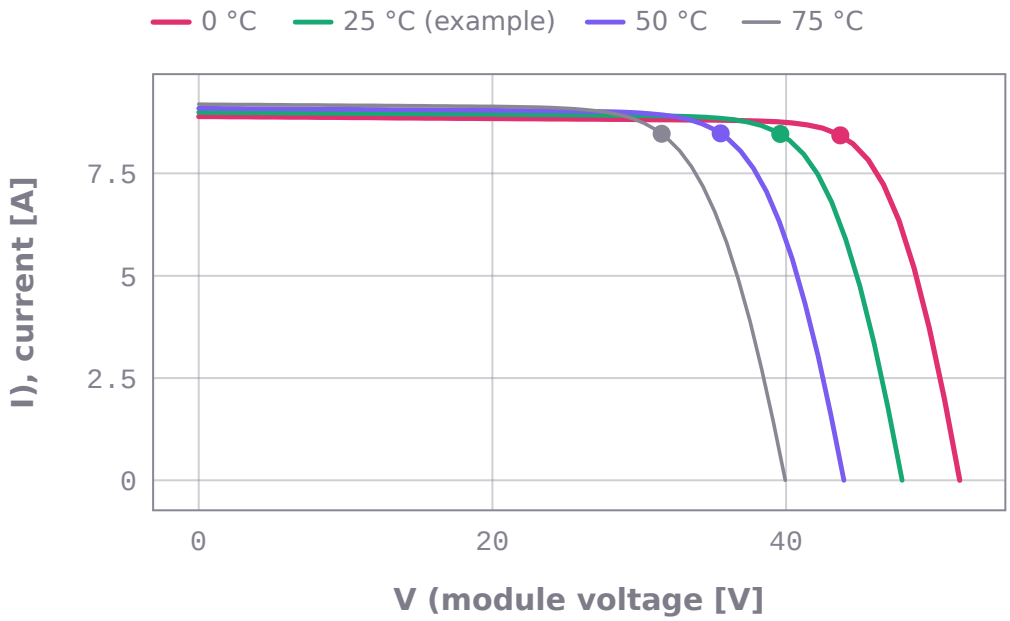
$$I_0 = I_{0,ref}\left(\frac{T}{T_{ref}}\right)^3 \exp\left(\frac{E_{g,ref}}{kT_{ref}} - \frac{E_g}{kT}\right)$$

input	meaning	condition
`G`	effective irradiance	
`T_cell`	cell temperature	
`alpha_sc`	short-circuit current temperature coefficient in A/K	
`a_ref`	modified ideality factor at reference conditions	
`IL_ref`	photocurrent at reference conditions	
`I0_ref`	saturation current at reference conditions	
`Rsh_ref`	shunt resistance at reference conditions	
`Rs`	series resistance	
`EgRef`	band gap at 25 degC in eV (default 1.121, silicon)	
`dEgdT`	band-gap temperature coefficient in $1/K$ (default -0.0002677)	

output	meaning
`IL`	photocurrent
`I0`	saturation current
`Rs`	series resistance
`Rsh`	shunt resistance
`nNsVth`	modified ideality factor

- **Method:** a = a_ref T/T_ref; $IL = (G/1000)$(IL_ref + alpha_sc (T - T_ref)); I0 = I0_ref (T/T_ref)^3 exp(Eg_ref/(k T_ref) - Eg/(k T)), Eg = Eg_ref (1 + dEg/dT (T - T_ref)); Rsh = Rsh_ref $1000/G$

- **Assumes:** reference conditions 1000 W/m^2 and 25 degC
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Module current–voltage curves at 1000 W/m² and four cell temperatures around the worked example's 25 °C: for each, the package's De Soto translation gives the single-diode parameters (the worked example's reference module: photocurrent 9 A, saturation current 1.0000e-10 A, a_ref = 1.9 V, Rsh_ref = 400 Ω, Rs = 0.3 Ω, short-circuit coefficient 0.004 A/K) and its single-diode current draws the curve. Higher temperature lowers the open-circuit voltage. Dots: maximum power points, 0 °C: 368.5 W, 25 °C (example): 335.3 W, 50 °C: 301.4 W, 75 °C: 267 W. At the worked example the package returns IL = 9 A, I0 = 1.0000e-10 A, nNsVth = 1.9 V.

```
energy_systems_ai999x.solve('energy_desoto_parameters', G=1000, T_cell=298.15,
alpha_sc=0.004, a_ref=1.9, IL_ref=9.0, I0_ref=1e-10, Rsh_ref=400.0, Rs=0.3)
# -> IL = 9 A, I0 = 1e-10 A, Rs = 0.3 ohm, Rsh = 400 ohm, nNsVth = 1.9 V   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
De Soto parameters: G = 1000 W/m^2, T_cell = 25 degC, alpha_sc = 0.004, a_ref = 1.9 V, IL_ref = 9 A, I0_ref = 1e-10 A, Rsh_ref = 400 ohm, Rs = 0.3 ohm	SOLVED	IL = 9 A, I0 = 1e-10 A, Rs = 0.3 ohm, Rsh = 400 ohm, nNsVth = 1.9 V
De Soto parameters: G = 0 W/m^2, T_cell = 25 degC, alpha_sc = 0.004, a_ref = 1.9 V, IL_ref = 9 A, I0_ref = 1e-10 A, Rsh_ref = 400 ohm, Rs = 0.3 ohm	INVALID_INPUT	PV module single-diode parameters at operating conditions (De Soto): the effective irradiance must be a positive number.

PV module maximum power at an irradiance and temperature,

`energy_pv_module_at_conditions`

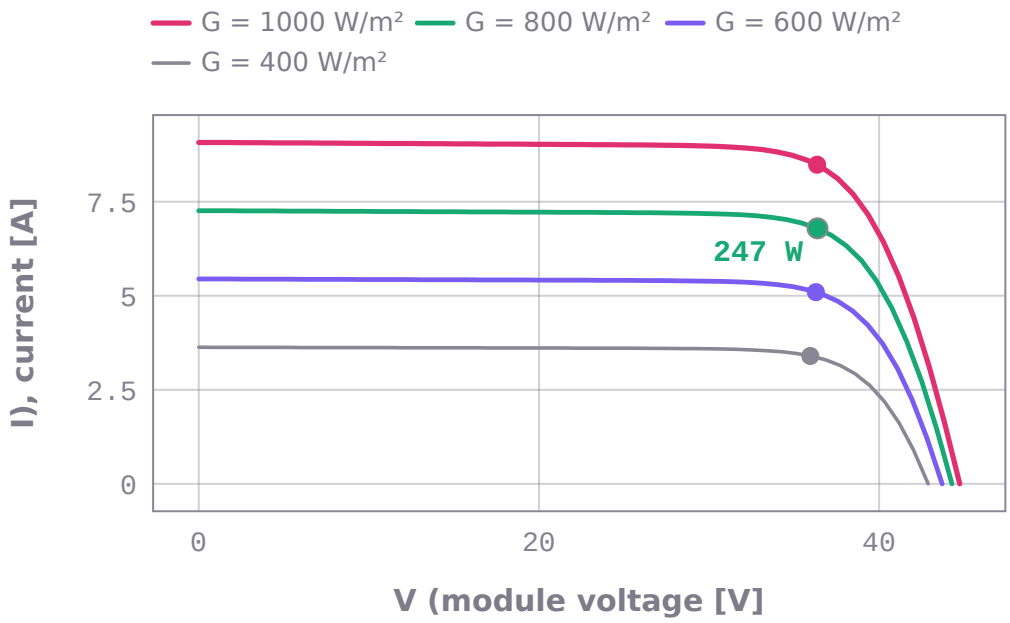
Short-circuit current, open-circuit voltage, maximum-power current, voltage and power, and fill factor of a PV module at an irradiance and cell temperature: the De Soto translation of its reference parameters, then the single-diode maximum power point.

$$P_{mp} = \max_{V} V\, I(V;\, I_L(G,T), I_0(T), R_s, R_{sh}(G), a(T))$$

input	meaning	condition
`G`	effective irradiance	
`T_cell`	cell temperature	
`alpha_sc`	short-circuit current temperature coefficient in A/K	
`a_ref`	modified ideality factor at reference conditions	
`IL_ref`	photocurrent at reference conditions	
`I0_ref`	saturation current at reference conditions	
`Rsh_ref`	shunt resistance at reference conditions	
`Rs`	series resistance	
`EgRef`	band gap at 25 degC in eV (default 1.121, silicon)	
`dEgdT`	band-gap temperature coefficient in $1/K$ (default -0.0002677)	

output	meaning
`Isc`	short-circuit current
`Voc`	open-circuit voltage
`Imp`	current at maximum power
`Vmp`	voltage at maximum power
`Pmp`	maximum power
`FF`	fill factor

- **Method:** De Soto parameter translation, then I_sc, V_oc and the maximum power point of the single-diode equation
- **Assumes:** reference conditions 1000 W/m^2 and 25 degC
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Module current–voltage curves at four irradiances and a cell temperature of 318.15 K (the worked example's), from the package's De Soto parameters (reference photocurrent 9 A, saturation current 1e-10 A, a_ref = 1.9 V, Rsh_ref = 400 Ω, Rs = 0.3 Ω, short-circuit temperature coefficient 0.004 A/K) and its single-diode current, one run per voltage. Dots: the maximum power point of each curve, G = 1000 W/m²: 308.2 W, G = 800 W/m²: 247 W, G = 600 W/m²: 184.9 W, G = 400 W/m²: 122.2 W. Marked: the worked example (G = 800 W/m²) as the package returns it, Pmp = 247 W at 36.37 V, 6.79 A, fill factor 0.7683.

```
energy_systems_ai999x.solve('energy_pv_module_at_conditions', G=800, T_cell=318.15,
alpha_sc=0.004, a_ref=1.9, IL_ref=9.0, I0_ref=1e-10, Rsh_ref=400.0, Rs=0.3)
# -> Isc = 7.25964 A, Voc = 44.2796 V, Imp = 6.79029 A, Vmp = 36.3735 V, Pmp = 246.987 W,
FF = 0.768342   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
PV module at conditions: G = 1000 W/m^2, T_cell = 25 degC, alpha_sc = 0.004, a_ref = 1 V, IL_ref = 5 A, I0_ref = 1e-10 A, Rsh_ref = 1e15 ohm, Rs = 0 ohm	SOLVED	Isc = 5 A, Voc = 24.6353 V, Imp = 4.77798 A, Vmp = 21.5208 V, Pmp = 102.826 W, FF = 0.834788
PV module at conditions: G = 1000 W/m^2, T_cell = 500 K, alpha_sc = 0.004, a_ref = 1 V, IL_ref = 5 A, I0_ref = 1e-10 A, Rsh_ref = 1e15 ohm, Rs = 0 ohm	INVALID_INPUT	PV module maximum power at an irradiance and temperature: the cell temperature must lie between 200 K and 400 K.

Irradiance on a tilted plane (Perez sky model), `energy_poa_perez`

Beam, sky-diffuse and ground-reflected irradiance on a tilted surface with the Perez (1990) anisotropic sky (circumsolar and horizon brightening coefficients by sky-clearness bin, all-sites composite set), the air mass by Kasten-Young and G_on from day n (Spencer).

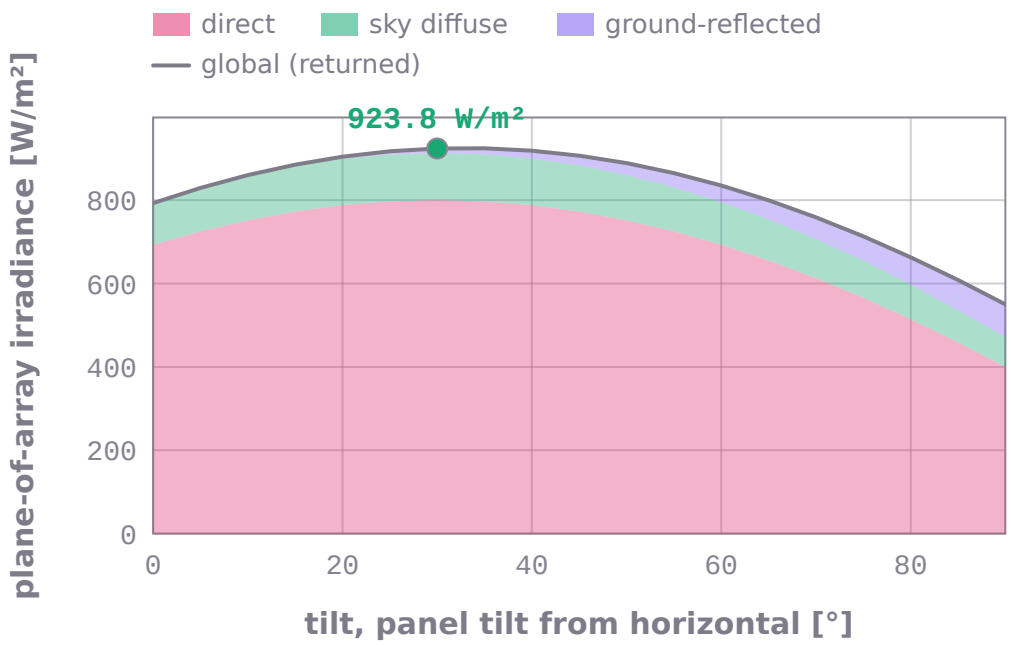
$$G_{d,t} = G_d \left[(1 - F_1) \frac{1 + \cos\beta}{2} + F_1 \frac{a}{b} + F_2 \sin\beta \right]$$

input	meaning	condition
`tilt`	surface tilt from horizontal in degrees	
`surface_azimuth`	surface azimuth in degrees clockwise from north (south = 180)	

input	meaning	condition
`zenith`	solar zenith angle in degrees	
`azimuth`	solar azimuth in degrees clockwise from north	
`dni`	direct normal irradiance	
`ghi`	global horizontal irradiance	
`dhi`	diffuse horizontal irradiance	
`albedo`	ground reflectance (0 to 1)	
`n`	day of the year (1 = 1 January)	

output	meaning
`poa_global`	total irradiance on the plane
`poa_direct`	beam irradiance on the plane
`poa_sky_diffuse`	sky-diffuse irradiance on the plane
`poa_ground_diffuse`	ground-reflected irradiance on the plane
`aoi_deg`	angle of incidence in degrees

- **Method:** eps = ((DHI + DNI)/DHI + 1.041 z^3)/(1 + 1.041 z^3), Delta = DHI AM/G_on; F1, F2 from the bin's coefficients; sky = DHI [(1 − F_1)(1 + cos beta)/2 + F1 a/b + F_2 sin beta]
- **Assumes:** 0 <= zenith < 87; DHI > 0; DHI <= GHI
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Irradiance on a tilted panel split into its parts by the package (Irradiance on a tilted plane (Perez sky model)) for the worked example's sky (DNI 800, GHI 792.8, DHI 100 W/m², Sun at zenith 30°, azimuth 180°, panel facing 180°, albedo 0.2) at tilts from 0 to 90°, stacked: direct, sky diffuse and ground-reflected, the line their returned total. Marked: tilt 30°, direct 800, sky 113.2, ground 10.62, global 923.8 W/m² (angle of incidence 0°).

```
energy_systems_ai999x.solve('energy_poa_perez', tilt=30, surface_azimuth=180, zenith=30,
azimuth=180, dni=800, ghi=792.820323027551, dhi=100, albedo=0.2, n=172)
# -> poa_global = 923.778 W/m^2, poa_direct = 800 W/m^2, poa_sky_diffuse = 113.156 W/m^2,
poa_ground_diffuse = 10.6218 W/m^2, aoi_deg = 0   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Plane of array irradiance (Perez): tilt = 30, surface_azimuth = 180, zenith = 30, azimuth = 180, dni = 800 W/m^2, ghi = 792.820323027551 W/m^2, dhi = 100 W/m^2, albedo = 0.2, n = 172	SOLVED	poa_global = 923.778 W/m^2, poa_direct = 800 W/m^2, poa_sky_diffuse = 113.156 W/m^2, poa_ground_diffuse = 10.6218 W/m^2, aoi_deg = 0
Plane of array irradiance (Perez): tilt = 30, surface_azimuth = 180, zenith = 30, azimuth = 180, dni = 800 W/m^2, ghi = 792.820323027551 W/m^2, dhi = 0 W/m^2, albedo = 0.2, n = 172	INVALID_INPUT	Irradiance on a tilted plane (Perez sky model): the Perez model needs a positive diffuse irradiance.

Clear-sky irradiance (Ineichen-Perez with Linke turbidity), `energy_clear_sky_ineichen`

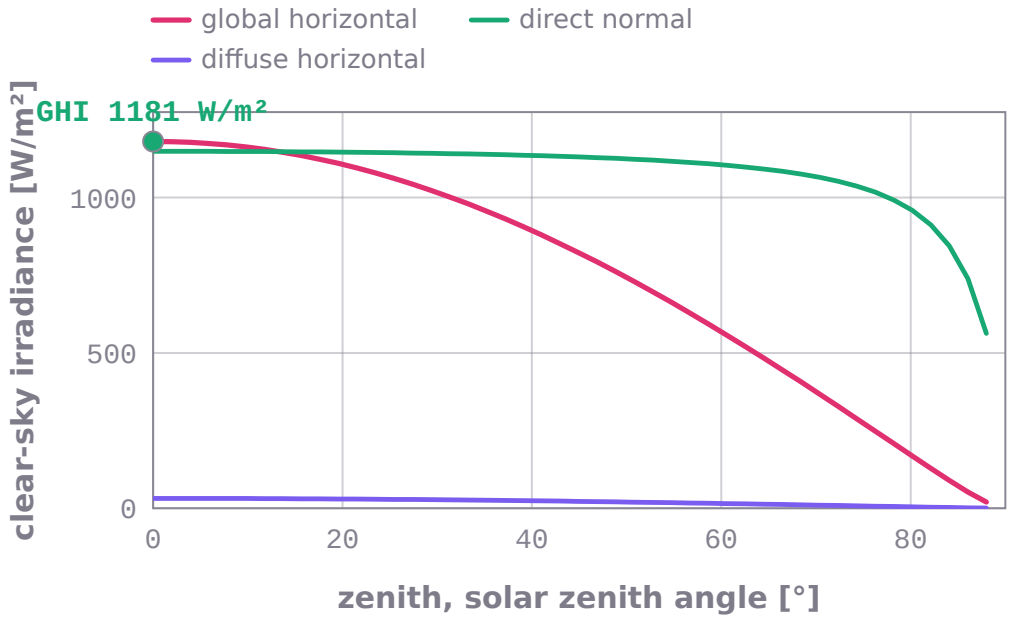
Clear-sky global horizontal, direct normal and diffuse horizontal irradiance from the apparent solar zenith, the Linke turbidity and the site altitude (Ineichen and Perez 2002), with the pressure-corrected air mass and G_on from day n.

$$G_{cs} = c_{g1} G_{on} \cos z \, e^{-c_{g2} AM (f_{h1} + f_{h2}(T_L - 1))}$$

input	meaning	condition
`zenith`	apparent solar zenith angle in degrees	
`linke_turbidity`	Linke turbidity factor (1 to 10; about 2 to 5 for clear skies)	
`altitude`	site altitude above sea level	
`n`	day of the year (1 = 1 January)	

output	meaning
`ghi`	clear-sky global horizontal irradiance
`dni`	clear-sky direct normal irradiance
`dhi`	clear-sky diffuse horizontal irradiance
`AM_absolute`	absolute air mass (Kasten-Young, standard pressure at the altitude)

- **Method:** GHI = cg1 G_on cos z exp(-cg2 AM (fh1 + fh2 $(TL - 1)$)); DNI = min(b G_on exp(-0.09 AM $(TL - 1)$), GHI (1 - (0.1 - 0.2 e^-TL)/(0.1 + 0.882/fh1))/cos z); $fh1 = e^{-h/8000}$, $fh2 = e^{-h/1250}$
- **Assumes:** sun above the horizon; $1 \leq TL \leq 10$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Clear-sky irradiance against the Sun's zenith angle for the worked example's sky (Linke turbidity 1, altitude 0 m, day 1), each point a run of the package: global horizontal, direct normal and diffuse horizontal. Marked: at zenith 0° the package returns GHI 1181, DNI 1149 and DHI 31.77 W/m² (absolute air mass 0.9997).

```
energy_systems_ai999x.solve('energy_clear_sky_ineichen', zenith=0, linke_turbidity=1,
altitude=0, n=1)
# -> ghi = 1180.76 W/m^2, dni = 1148.99 W/m^2, dhi = 31.7724 W/m^2, AM_absolute = 0.999712
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Clear-sky irradiance (Ineichen): zenith = 30, linke_turbidity = 3, altitude = 500 m, n = 172	SOLVED	ghi = 885.003 W/m^2, dni = 910.142 W/m^2, dhi = 96.7974 W/m^2, AM_absolute = 1.08721
Clear-sky irradiance (Ineichen): zenith = 95, linke_turbidity = 3, altitude = 500 m, n = 172	INVALID_INPUT	Clear-sky irradiance (Ineichen-Perez with Linke turbidity): the sun must be above the horizon (apparent zenith below 90
Clear-sky irradiance (Ineichen): zenith = 30, linke_turbidity = 3, n = 172	NEEDS_INPUT	energy_clear_sky_ineichen needs altitude (site altitude above sea level). Example: solve('energy_clear_sky_ineichen', ze

Direct normal irradiance from global horizontal irradiance (DISC model), `energy_disc_dni`

The direct normal irradiance estimated from global horizontal irradiance by Maxwell's DISC model (clearness index and pressure-corrected air mass), with the clearness index and the air mass used.

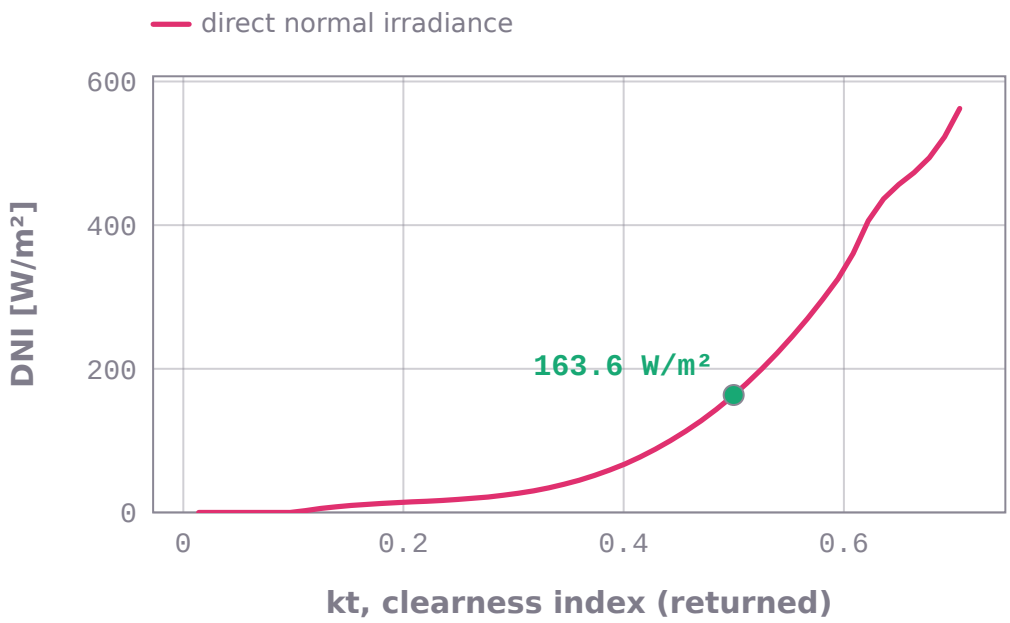
$$G_{bn} = I_0 \left[K_{nc}(AM) - \left(a + b\, e^{c\, AM} \right) \right]$$

input	meaning	condition
`ghi`	global horizontal irradiance	
`zenith`	solar zenith angle in degrees	
`n`	day of the year (1 = 1 January)	

input	meaning	condition
`pressure`	site pressure (default 101325 Pa)	

output	meaning
`dni`	direct normal irradiance (0 when the model gives a negative value)
`kt`	clearness index (solar constant 1370 W/m^2)
`airmass`	absolute air mass (Kasten 1966, capped at 12)

- **Method:** kt = GHI/(I0 cos z); Kn = Knc(AM) - (a(kt) + b(kt) exp(c(kt) AM)); DNI = Kn I0
- **Assumes:** 0 <= zenith <= 85; GHI > 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Direct normal irradiance estimated by the package (DISC) from global horizontal irradiance stepped from 20 to 1000 W/m², Sun at zenith 0° on day 1 (the worked example's), plotted against the clearness index it returns. Marked: the worked example, GHI 709.009 W/m², gives kt 0.5 and DNI 163.6 W/m² (air mass 0.9995).

```
energy_systems_ai999x.solve('energy_disc_dni', ghi=709.0092500000001, zenith=0, n=1)
# -> dni = 163.645 W/m^2, kt = 0.5, airmass = 0.999494   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
DISC model: ghi = 709.0092500000001 W/m^2, zenith = 0, n = 1	SOLVED	dni = 163.645 W/m^2, kt = 0.5, airmass = 0.999494
DISC model: ghi = 600 W/m^2, zenith = 89, n = 172	INVALID_INPUT	Direct normal irradiance from global horizontal irradiance (DISC model): the zenith angle must lie between 0 and 85 degr

Base impedance and per-unit impedance, `energy_per_unit_impedance`

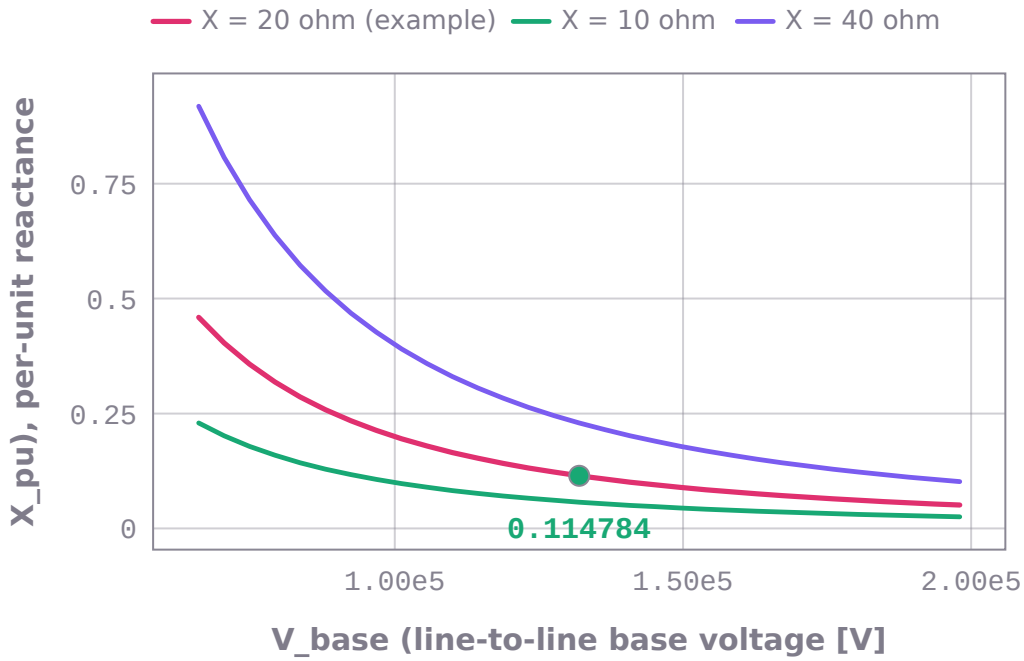
The base impedance Z_base = V_base^2/S_base and base current of a three-phase per-unit system, and an impedance in ohms expressed in per unit.

$$Z_{base} = \frac{V_{base}^2}{S_{base}}, \quad Z_{pu} = \frac{Z}{Z_{base}}$$

input	meaning	condition
`R`	resistance per phase (ohm)	
`X`	reactance per phase (ohm)	
`S_base`	three-phase base power (VA)	
`V_base`	line-to-line base voltage	

output	meaning
`Z_base`	base impedance V_base^2/S_base
`I_base`	base current S_base/(sqrt(3) V_base)
`R_pu`	per-unit resistance
`X_pu`	per-unit reactance
`Z_pu`	per-unit impedance magnitude

- **Method:** Z_base = V_base^2/S_base, I_base = S_base/(sqrt(3) V_base), Z_pu = Z/Z_base
- **Assumes:** R, $X \geq 0$, not both 0; positive bases; three-phase base, line-to-line voltage
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



X_pu (per-unit reactance) against V_base (line-to-line base voltage) from 66000 V to 198000 V. One curve for each value of X (reactance per phase (ohm)): 20 ohm, 10 ohm, 40 ohm. Other inputs as in the worked example: R = 5.0 ohm, S_base = 100000000.0 VA. 31 points per curve computed by energy-systems-ai999x; marked: the worked example, V_base = 132000 V, X_pu = 0.114784.

```
energy_systems_ai999x.solve('energy_per_unit_impedance', R=5.0, X=20.0, S_base=100000000.0,
V_base=132000.0)
# -> Z_base = 174.24 ohm, I_base = 437.387 A, R_pu = 0.0286961, X_pu = 0.114784, Z_pu =
0.118317   (SOLVED)
```

```
energy_systems_ai999x.solve('energy_per_unit_impedance', R=0.0, X=10.0, S_base=1000000.0,
V_base=10000.0)
# -> Z_base = 100 ohm, I_base = 57.735 A, R_pu = 0, X_pu = 0.1, Z_pu = 0.1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Per-unit impedance: R = 5 ohm, X = 20 ohm, S_base = 100 MVA, V_base = 132 kV	SOLVED	Z_base = 174.24 ohm, I_base = 437.387 A, R_pu = 0.0286961, X_pu = 0.114784, Z_pu = 0.118317
Per-unit impedance: R = 5 ohm, X = 20 ohm, S_base = 0 MVA, V_base = 132 kV	INVALID_INPUT	Base impedance and per-unit impedance: the base power and base voltage must be a positive number.
Per-unit impedance: R = 5 ohm, X = 20 ohm, S_base = 100 MVA	NEEDS_INPUT	energy_per_unit_impedance needs V_base (line-to-line base voltage). Example: solve('energy_per_unit_impedance', R='5 ohm

Geometric mean distance of a three-phase line, `energy_geometric_mean_distance`

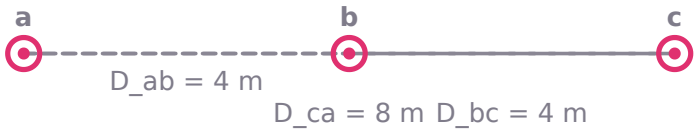
The geometric mean distance GMD = (D_ab D_bc D_ca)^(1/3) between the phases of a fully transposed three-phase line.

$$GMD = \sqrt[3]{D_{ab} D_{bc} D_{ca}}$$

input	meaning	condition
`D_ab`	distance between phases a and b	
`D_bc`	distance between phases b and c	
`D_ca`	distance between phases c and a	

output	meaning
`GMD`	geometric mean distance
`ratio_to_mean`	GMD divided by the arithmetic mean spacing

- **Method:** GMD = (D_ab D_bc D_ca)^(1/3)
- **Assumes:** positive spacings that form a triangle (or a straight line); fully transposed line
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)




```
GMD = 5.04 m (0.9449 × the mean spacing)
```

Phase conductors of a three-phase line drawn to scale from the worked example's spacings D_ab = 4 m, D_bc = 4 m, D_ca = 8 m (a flat arrangement when one spacing is the sum of the other two). Marked: the package's geometric mean distance GMD = 5.04 m, 0.9449 of the arithmetic mean spacing.

```
energy_systems_ai999x.solve('energy_geometric_mean_distance', D_ab=4.0, D_bc=4.0, D_ca=8.0)
# -> GMD = 5.03968 m, ratio_to_mean = 0.944941   (SOLVED)
```

```
energy_systems_ai999x.solve('energy_geometric_mean_distance', D_ab=2.0, D_bc=2.0, D_ca=2.0)
# -> GMD = 2 m, ratio_to_mean = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Geometric mean distance: D_ab = 4 m, D_bc = 4 m, D_ca = 8 m	SOLVED	GMD = 5.03968 m, ratio_to_mean = 0.944941
Geometric mean distance: D_ab = 1 m, D_bc = 1 m, D_ca = 8 m	INVALID_INPUT	Geometric mean distance of a three-phase line: the three spacings must form a triangle (or a line).
Geometric mean distance: D_ab = 4 m, D_bc = 4 m	NEEDS_INPUT	energy_geometric_mean_distance needs D_ca (distance between phases c and a). Example: solve('energy_geometric_mean_dista

Synchronous speed, rotor speed and slip of an AC machine, `energy_synchronous_speed`

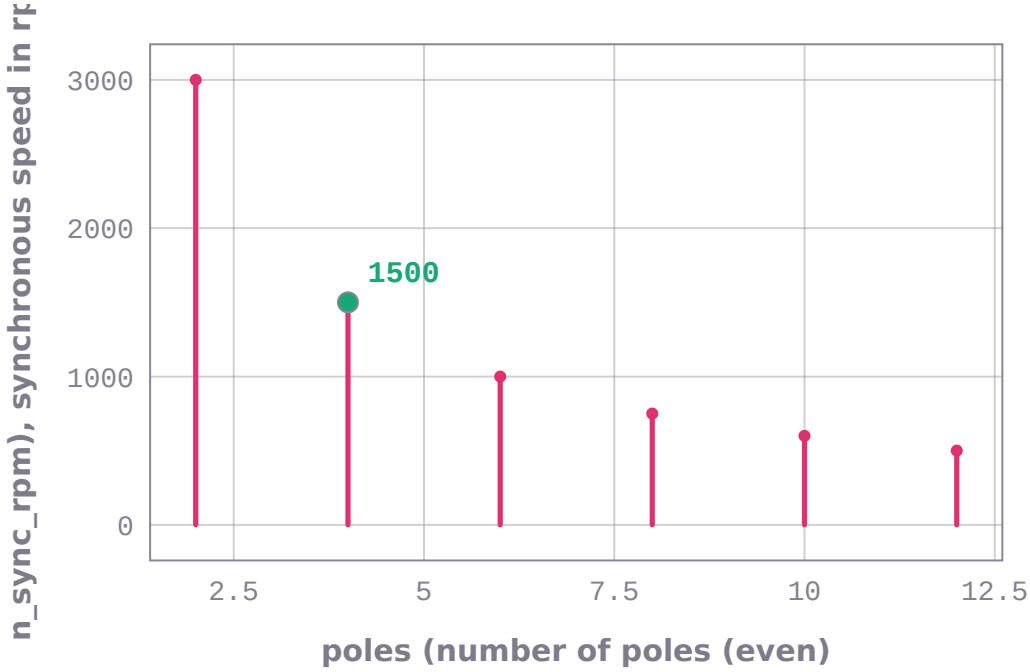
The synchronous speed $n_s = 120$ f/P of an AC machine and, with the slip, the rotor speed $n = n_s\,(1 - s)$ (or, with the rotor speed, the slip) and the rotor (slip) frequency.

$$n_s = \frac{120 f}{P}, \quad n = n_s(1 - s)$$

input	meaning	condition
`f`	supply frequency	
`poles`	number of poles (even)	
`s`	slip (fraction or %, optional)	
`n_r`	rotor speed (rpm, optional)	

output	meaning
`n_sync_rpm`	synchronous speed in rpm
`omega_sync`	synchronous speed in rad/s
`n_rotor_rpm`	rotor speed in rpm
`slip`	slip (n_s - n)/n_s
`slip_frequency`	rotor current frequency s f

- **Method:** $n_s = 120$ f/P; $n = n_s\,(1 - s)$; $s = (n_s - n)/n_s$; $f_r = s$ f
- **Assumes:** $f > 0$; even number of poles; at most one of slip and rotor speed
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



n_sync_rpm (synchronous speed in rpm) against poles (number of poles (even)) from 1 to 12. Other inputs as in the worked example: f = 50.0 Hz. 6 points computed by energy-systems-ai999x; marked: the worked example, poles = 4, n_sync_rpm = 1500.

```
energy_systems_ai999x.solve('energy_synchronous_speed', f=50.0, poles=4)
# -> n_sync_rpm = 1500, omega_sync = 157.08 rad/s   (SOLVED)
```

```
energy_systems_ai999x.solve('energy_synchronous_speed', f=60.0, poles=4, s=0.03)
# -> n_sync_rpm = 1800, omega_sync = 188.496 rad/s, n_rotor_rpm = 1746, slip = 0.03,
slip_frequency = 1.8 Hz   (SOLVED)
```

```
energy_systems_ai999x.solve('energy_synchronous_speed', f=60.0, poles=6,
n_r=120.42771838760873)
# -> n_sync_rpm = 1200, omega_sync = 125.664 rad/s, n_rotor_rpm = 1150, slip = 0.0416667,
slip_frequency = 2.5 Hz   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Synchronous speed: f = 50 Hz, poles = 4	SOLVED	n_sync_rpm = 1500, omega_sync = 157.08 rad/s
Synchronous speed: f = 60 Hz, poles = 4, s = 3%	SOLVED	n_sync_rpm = 1800, omega_sync = 188.496 rad/s, n_rotor_rpm = 1746, slip = 0.03, slip_frequency = 1.8 Hz
Synchronous speed: f = 60 Hz, poles = 6, n_r = 1150 rpm	SOLVED	n_sync_rpm = 1200, omega_sync = 125.664 rad/s, n_rotor_rpm = 1150, slip = 0.0416667, slip_frequency = 2.5 Hz
Synchronous speed: f = 50 Hz, poles = 5	INVALID_INPUT	Synchronous speed, rotor speed and slip of an AC machine: the number of poles must be an even whole number from 2 to 200
Synchronous speed: f = 50 Hz	NEEDS_INPUT	energy_synchronous_speed needs poles (number of poles (even)). Example: solve('energy_synchronous_speed', f='50 Hz', pol

Electrical power of a fuel-cell or electrolyser stack, `energy_stack_electrical_power`

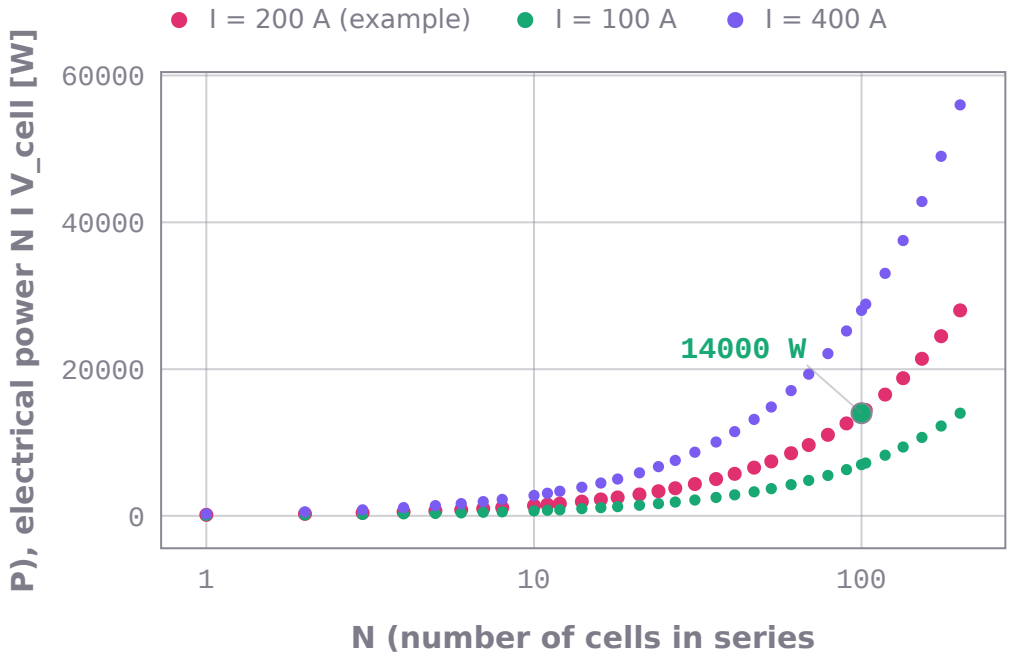
The electrical power $P = N$ I V_cell delivered by a fuel-cell stack (or drawn by an electrolyser stack) of N cells in series at a current and an average cell voltage, and the stack voltage.

$$P = N\,I\,V_{cell}$$

input	meaning	condition
`I`	stack current	
`N`	number of cells in series	
`V_cell`	average cell voltage	

output	meaning
`P`	electrical power N I V_cell
`P_kW`	electrical power in kW
`V_stack`	stack voltage N V_cell

- **Method:** $P = N$ I V_cell, V_stack = N V_cell
- **Assumes:** cells in series carrying the same current; 0 < V_cell <= 5 V
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



P (electrical power N I V_cell) against N (number of cells in series) from 1 to 200 on a logarithmic axis. One curve for each value of I (stack current): 200 A, 100 A, 400 A. Other inputs as in the worked example: V_cell = 0.7 V. 33 points per curve computed by energy-systems-ai999x; marked: the worked example, N = 100, P = 14000 W.

```
energy_systems_ai999x.solve('energy_stack_electrical_power', I=200.0, N=100, V_cell=0.7)
# -> P = 14000 W, P_kW = 14, V_stack = 70 V   (SOLVED)
```

```
energy_systems_ai999x.solve('energy_stack_electrical_power', I=1000.0, N=100, V_cell=1.9)
# -> P = 190000 W, P_kW = 190, V_stack = 190 V   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Stack electrical power: I = 200 A, N = 100, V_cell = 0.7 V	SOLVED	P = 14000 W, P_kW = 14, V_stack = 70 V
Stack electrical power: I = 200 A, N = 0, V_cell = 0.7 V	INVALID_INPUT	N: must be at least 1, got 0.
Stack electrical power: I = 200 A, N = 100	NEEDS_INPUT	energy_stack_electrical_power needs V_cell (average cell voltage). Example: solve('energy_stack_electrical_power', I='20

State of charge by coulomb counting, `energy_coulomb_counting`

The state of charge of a battery after a sequence of constant currents over equal time steps (positive on discharge), by coulomb counting SOC = SOC0 - integral(I dt)/Q, with its lowest and highest value and the charge delivered.

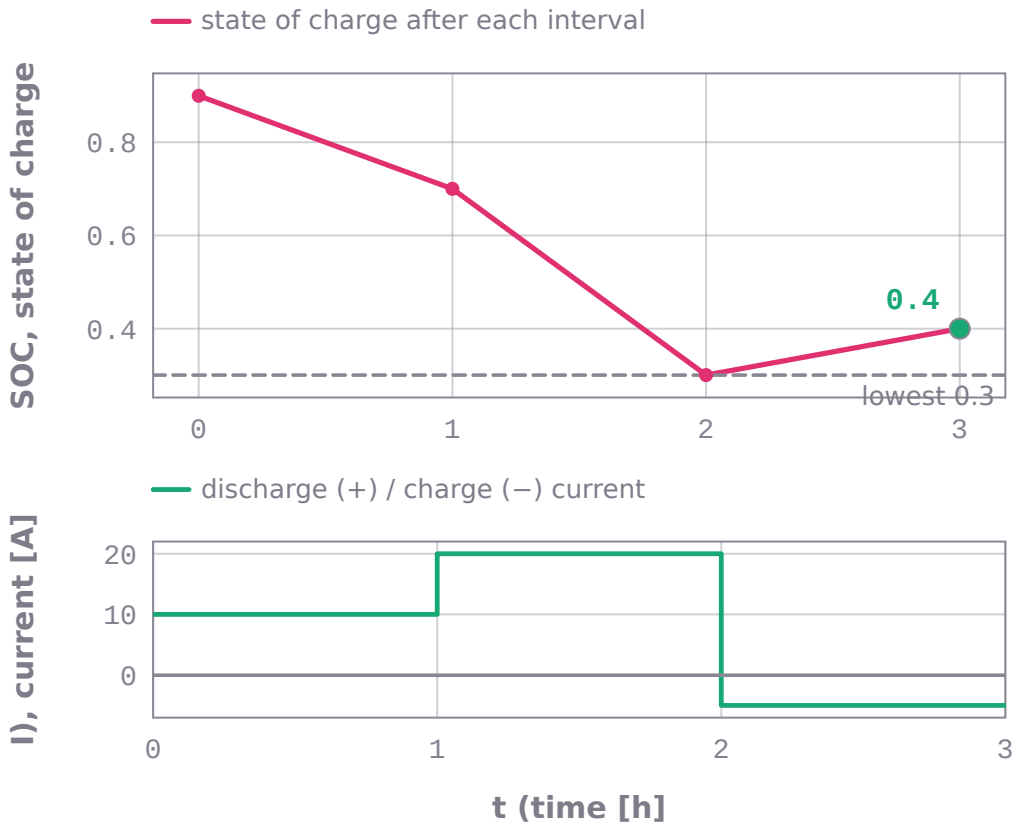
$$SOC(t) = SOC_0 - \frac{1}{Q}\int_0^t I\,dt$$

input	meaning	condition
`SOC0`	initial state of charge (fraction or %)	

input	meaning	condition
`Q`	capacity (Ah, mAh, C)	
`currents`	current in each time step in A (positive on discharge)	
`dt`	length of each time step	

output	meaning
`SOC`	final state of charge
`SOC_min`	lowest state of charge
`SOC_max`	highest state of charge
`charge_out`	net charge delivered
`charge_out_Ah`	net charge delivered in Ah

- **Method:** SOC_k = SOC0 - (sum_{$j \leq k$} I_j dt)/Q
- **Assumes:** 0 <= SOC0 <= 1; Q, $dt > 0$; the state of charge stays within 0 to 100%; no coulombic losses
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



A battery of 50 Ah starting at SOC 0.9 through the worked example's profile of 3 intervals of 3600 s at currents ['10', '20', '-5'] A (bottom). Top: the package's SOC after each interval (each point one run on the profile up to that interval; constant current between points). Marked: final SOC 0.4, lowest 0.3, net charge out 25 Ah.

```
energy_systems_ai999x.solve('energy_coulomb_counting', SOC0=0.9, Q=180000.0, currents=[10,
20, -5], dt=3600.0)
# -> SOC = 0.4, SOC_min = 0.3, SOC_max = 0.9, charge_out = 90000 C, charge_out_Ah = 25
(SOLVED)
```

```
energy_systems_ai999x.solve('energy_coulomb_counting', SOC0=1.0, Q=7200.0, currents='1, 1', dt=1800.0)
# -> SOC = 0.5, SOC_min = 0.5, SOC_max = 1, charge_out = 3600 C, charge_out_Ah = 1 (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Coulomb counting: SOC0 = 90%, Q = 50 Ah, currents = [10, 20, -5], dt = 1 h	SOLVED	SOC = 0.4, SOC_min = 0.3, SOC_max = 0.9, charge_out = 90000 C, charge_out_Ah = 25
Coulomb counting: SOC0 = 90%, Q = 50 Ah, currents = [100, 20], dt = 1 h	INVALID_INPUT	State of charge by coulomb counting: the state of charge leaves the range 0 to 100% (more charge than the capacity).
Coulomb counting: SOC0 = 90%, Q = 50 Ah, currents = [10, 20, -5]	NEEDS_INPUT	energy_coulomb_counting needs dt (length of each time step). Example: solve('energy_coulomb_counting', SOC0=0.9, Q='1800

Hydroelectric power from flow and head, and the energy over a period, `energy_hydro_power`

The electrical output $P = \eta$ rho g Q H of a hydroelectric plant from the flow rate, the net head and the overall efficiency, the hydraulic power, and the energy over a period (default a year of 8760 h) at a capacity factor (default 1).

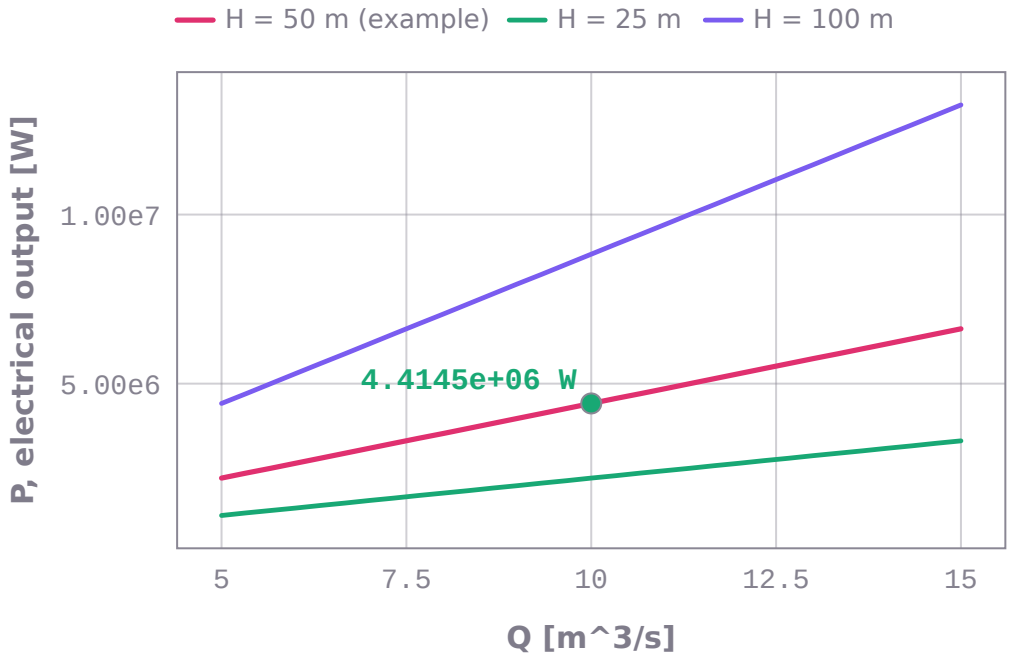
$$P = \eta \rho g Q H, \quad E = P \cdot CF \cdot t$$

input	meaning	condition
`Q`	volume flow rate through the turbines	
`H`	net head	
`eta`	overall efficiency (turbine x generator; fraction or %)	
`rho`	water density (default 1000 kg/m^3)	
`g`	gravitational acceleration (default 9.81 m/s^2)	
`CF`	capacity factor for the energy (fraction or %, default 1)	
`hours`	length of the period in hours (default 8760)	

output	meaning
`P`	electrical output
`P_kW`	electrical output in kW
`P_MW`	electrical output in MW
`P_hydraulic`	hydraulic power rho g Q H
`E`	energy over the period
`E_MWh`	energy over the period in MWh

- **Method:** $P = \eta$ rho g Q H; $E = P$ CF hours
- **Assumes:** Q, $H > 0$; $0 < \eta \leq 1$; $0 < CF \leq 1$; 0 < hours <= 8784; net head (losses in the penstock already deducted)

- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



P (electrical output) against Q (volume flow rate through the turbines) from 5 m^3/s to 15 m^3/s. One curve for each value of H (net head): 50 m, 25 m, 100 m. Other inputs as in the worked example: eta = 0.9. 31 points per curve computed by energy-systems-ai999x; marked: the worked example, Q = 10 m^3/s, P = 4.4145e+06 W.

```
energy_systems_ai999x.solve('energy_hydro_power', Q=10.0, H=50.0, eta=0.9)
# -> P = 4.4145e+06 W, P_kW = 4414.5, P_MW = 4.4145, P_hydraulic = 4.905e+06 W, E =
1.39216e+14 J, E_MWh = 38671   (SOLVED)
```

```
energy_systems_ai999x.solve('energy_hydro_power', Q=2.0, H=100.0, eta=0.85, CF=0.5)
# -> P = 1.6677e+06 W, P_kW = 1667.7, P_MW = 1.6677, P_hydraulic = 1.962e+06 W, E =
2.62963e+13 J, E_MWh = 7304.53   (SOLVED)
```

```
energy_systems_ai999x.solve('energy_hydro_power', Q=1.0, H=10.0, eta=1.0, g=9.80665,
hours=1.0)
# -> P = 98066.5 W, P_kW = 98.0665, P_MW = 0.0980665, P_hydraulic = 98066.5 W, E =
3.53039e+08 J, E_MWh = 0.0980665   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Hydro power: Q = 10 m^3/s, H = 50 m, eta = 90%	SOLVED	P = 4.4145e+06 W, P_kW = 4414.5, P_MW = 4.4145, P_hydraulic = 4.905e+06 W, E = 1.39216e+14 J, E_MWh = 38671
Hydro power: Q = 2 m^3/s, H = 100 m, eta = 85%, CF = 50%	SOLVED	P = 1.6677e+06 W, P_kW = 1667.7, P_MW = 1.6677, P_hydraulic = 1.962e+06 W, E = 2.62963e+13 J, E_MWh = 7304.53
Hydro power: Q = 10 m^3/s, H = 50 m, eta = 120%	INVALID_INPUT	Hydroelectric power from flow and head, and the energy over a period: the efficiency must lie in (0, 1] (e.g. 0.9 or 90%
Hydro power: Q = 10 m^3/s, eta = 90%	NEEDS_INPUT	energy_hydro_power needs H (net head). Example: solve('energy_hydro_power', Q='10 m^3/s', H='50 m', eta=0.9).

Capacity factor and energy of a generating plant, `energy_capacity_factor`

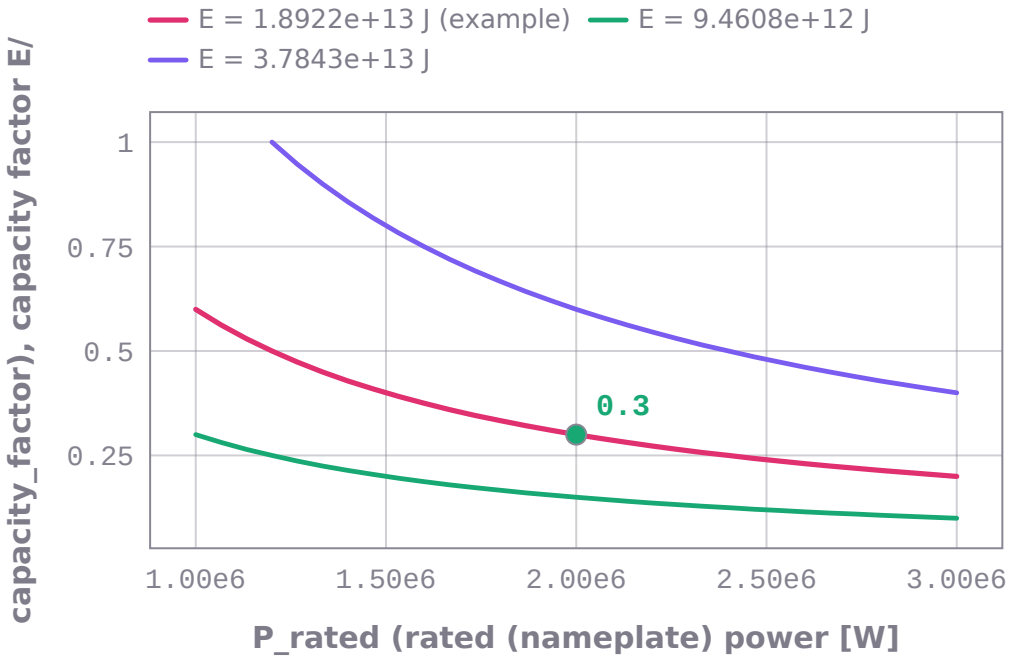
The capacity factor of a plant of any type from its rated power and the energy produced over a period (default a year of 8760 h), or the energy from a capacity factor, with the equivalent full-load hours and the average output.

$$CF = \frac{E}{P_{rated}\, t}$$

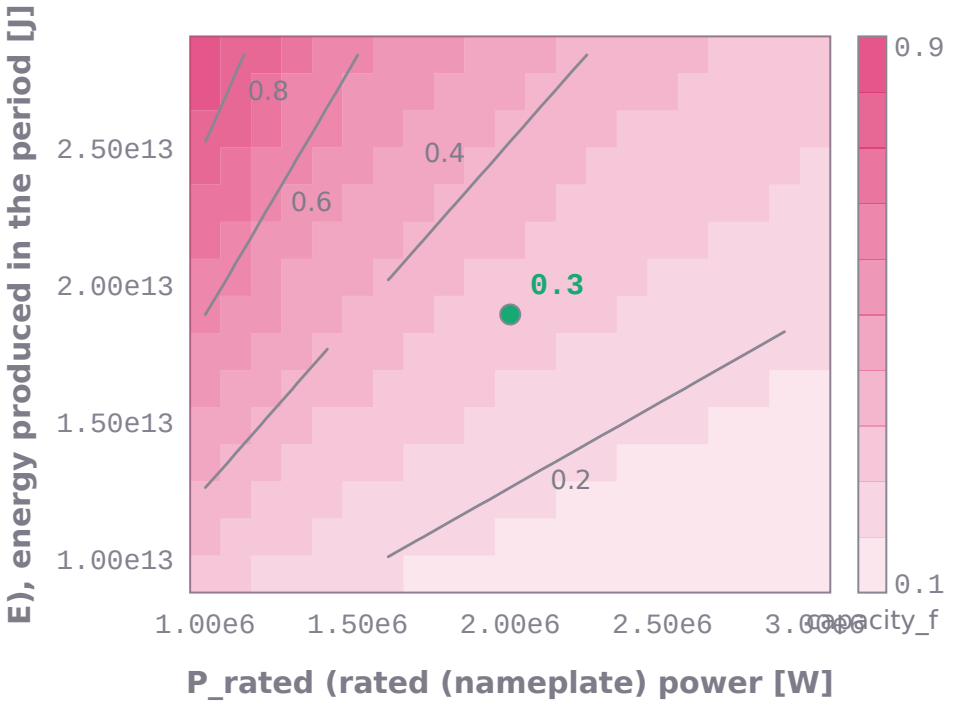
input	meaning	condition
`P_rated`	rated (nameplate) power	
`E`	energy produced in the period (give E or CF)	
`CF`	capacity factor (fraction or %; give E or CF)	
`hours`	length of the period in hours (default 8760)	

output	meaning
`capacity_factor`	capacity factor E/(P_rated t)
`capacity_factor_pct`	capacity factor in percent
`E`	energy produced in the period
`E_MWh`	energy produced in MWh
`full_load_hours`	equivalent full-load hours CF t
`average_power`	average output CF P_rated

- **Method:** CF = E/(P_rated t); $E = CF$ P_rated t; full-load hours = CF t
- **Assumes:** P_rated > 0; exactly one of E and CF; $0 < CF \leq 1$; 0 < hours <= 8784
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



capacity_factor (capacity factor E/(P_rated t)) against P_rated (rated (nameplate) power) from 1e+06 W to 3e+06 W. One curve for each value of E (energy produced in the period (give E or CF)): 1.8922e+13 J, 9.4608e+12 J, 3.7843e+13 J. 31 points per curve computed by energy-systems-ai999x; marked: the worked example, P_rated = 2e+06 W, capacity_factor = 0.3.



capacity_factor (capacity factor E/(P_rated t)) over P_rated (rated (nameplate) power) from 1e+06 W to 3e+06 W and E (energy produced in the period (give E or CF)) from 9.4608e+12 J to 2.8382e+13 J: each of the 21 × 15 cells is one run of energy-systems-ai999x, shaded by its value (light 0.1 to dark 0.9); iso-lines at capacity_factor = 0.2, 0.4, 0.6, 0.8. Marked: the worked example, P_rated = 2e+06 W, E = 1.8922e+13 J, capacity_factor = 0.3.

```
energy_systems_ai999x.solve('energy_capacity_factor', P_rated=2000000.0,
E=18921600000000.0)
# -> capacity_factor = 0.3, capacity_factor_pct = 30, E = 1.89216e+13 J, E_MWh = 5256,
full_load_hours = 2628, average_power = 600000 W   (SOLVED)
```

```
energy_systems_ai999x.solve('energy_capacity_factor', P_rated=500000000.0, CF=0.9)
# -> capacity_factor = 0.9, capacity_factor_pct = 90, E = 1.41912e+16 J, E_MWh = 3.942e+06,
full_load_hours = 7884, average_power = 4.5e+08 W   (SOLVED)
```

```
energy_systems_ai999x.solve('energy_capacity_factor', P_rated=1000000.0, E=21600000000.0,
hours=24.0)
# -> capacity_factor = 0.25, capacity_factor_pct = 25, E = 2.16e+10 J, E_MWh = 6,
full_load_hours = 6, average_power = 250000 W   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Capacity factor: P_rated = 2 MW, E = 5256 MWh	SOLVED	capacity_factor = 0.3, capacity_factor_pct = 30, E = 1.89216e+13 J, E_MWh = 5256, full_load_hours = 2628, average_power = 600000 W
Capacity factor: P_rated = 500 MW, CF = 90%	SOLVED	capacity_factor = 0.9, capacity_factor_pct = 90, E = 1.41912e+16 J, E_MWh = 3.942e+06, full_load_hours = 7884, average_power = 4.5e+08 W
Capacity factor: P_rated = 2 MW, E = 20000 MWh	INVALID_INPUT	Capacity factor and energy of a generating plant: the energy produced cannot exceed rated power x period (capacity facto
Capacity factor: P_rated = 2 MW, E = 5256 MWh, CF = 30%	INVALID_INPUT	Capacity factor and energy of a generating plant: give exactly one of the energy produced E and the capacity factor CF.

Induction motor power flow from slip and losses, `energy_induction_power_flow`

The power flow of a three-phase induction motor from its input power and losses: synchronous and rotor speed, slip, rotor frequency (and rotor emf), air-gap power, rotor copper loss s P_ag, converted power, output, induced and shaft torque, and efficiency (no equivalent-circuit parameters needed).

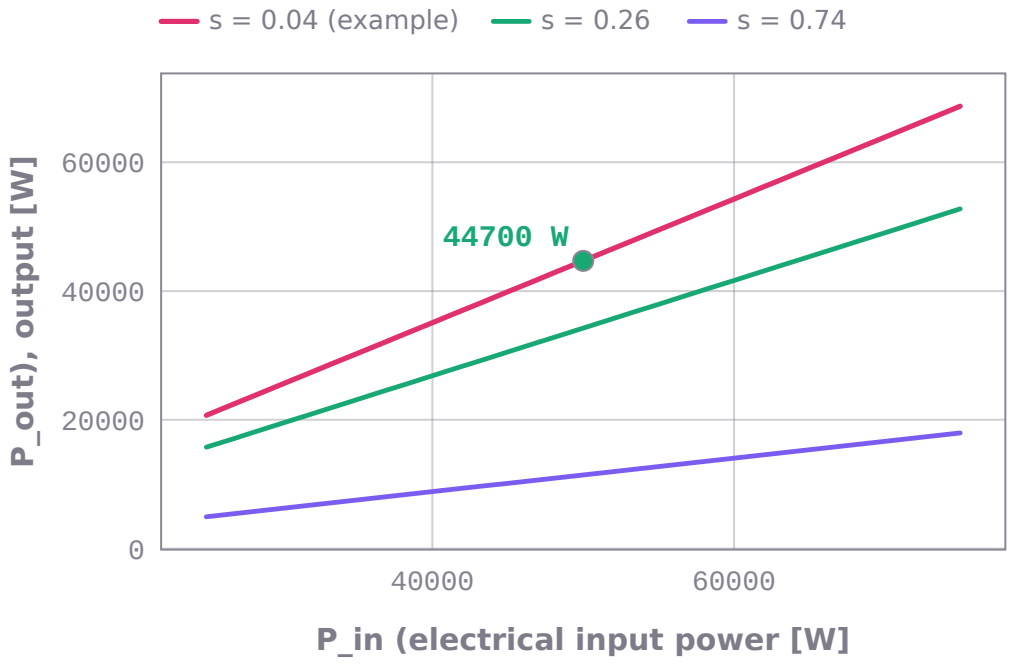
$$P_{ag} = P_{in} - P_{scl} - P_{core}, \quad P_{rcl} = s\,P_{ag}, \quad P_{conv} = (1-s)P_{ag}, \quad \eta = \frac{P_{conv} - P_{fw}}{P_{in}}$$

input	meaning	condition
`f`	supply frequency	
`poles`	number of poles (even)	
`P_in`	electrical input power	
`P_scl`	stator copper loss	
`s`	slip (fraction or %; give s or n_r)	
`n_r`	rotor speed (rpm; give s or n_r)	
`P_core`	core (iron) loss, default 0	
`P_fw`	friction and windage loss, default 0	
`E2`	rotor emf per phase at standstill (optional)	

output	meaning
`n_sync_rpm`	synchronous speed 120 f/p, rpm
`n_rotor_rpm`	rotor speed, rpm

output	meaning
`slip`	slip
`f_rotor`	rotor current frequency s f
`P_ag`	air-gap power P_in - stator losses
`P_rcl`	rotor copper loss s P_ag
`P_conv`	converted (mechanical developed) power $(1 - s)$ P_ag
`P_out`	output (shaft) power P_conv - friction and windage
`T_ind`	induced torque P_ag/omega_s
`T_shaft`	shaft (output) torque P_out/omega_m
`efficiency_pct`	efficiency P_out/P_in in percent
`E_rotor`	rotor emf per phase at the running slip s E2 (with E2)

- **Method:** P_ag = P_in - P_scl - P_core; P_rcl = s P_ag; P_conv = (1 - s) P_ag; P_out = P_conv - P_fw; $T = P_ag/omega_s$; $f_r = s$ f
- **Assumes:** even number of poles; exactly one of s and n_r; $0 < s < 1$ (motoring); stator losses < P_in; P_out > 0; the core loss is taken with the stator losses
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



P_out (output (shaft) power P_conv - friction and windage) against P_in (electrical input power) from 25000 W to 75000 W. One curve for each value of s (slip (fraction or %; give s or n_r)): 0.04, 0.26, 0.74. Other inputs as in the worked example: f = 50.0 Hz, poles = 4, P_scl = 1500.0 W, P_core = 1000.0 W, P_fw = 900.0 W. 31 points per curve computed by energy-systems-ai999x; marked: the worked example, P_in = 50000 W, P_out = 44700 W.

```
energy_systems_ai999x.solve('energy_induction_power_flow', f=50.0, poles=4, s=0.04,
P_in=50000.0, P_scl=1500.0, P_core=1000.0, P_fw=900.0)
# -> n_sync_rpm = 1500, n_rotor_rpm = 1440, slip = 0.04, f_rotor = 2 Hz, P_ag = 47500 W,
P_rcl = 1900 W, P_conv = 45600 W, P_out = 44700 W, T_ind = 302.394 N*m, T_shaft = 296.426
N*m, efficiency_pct = 89.4   (SOLVED)
```

```
energy_systems_ai999x.solve('energy_induction_power_flow', f=60.0, poles=6,
n_r=119.38052083641215, P_in=20000.0, P_scl=800.0, E2=200.0)
# -> n_sync_rpm = 1200, n_rotor_rpm = 1140, slip = 0.05, f_rotor = 3 Hz, P_ag = 19200 W,
P_rcl = 960 W, P_conv = 18240 W, P_out = 18240 W, T_ind = 152.789 N*m, T_shaft = 152.789
N*m, efficiency_pct = 91.2, E_rotor = 10 V   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Induction motor power flow: f = 50 Hz, poles = 4, s = 4%, P_in = 50 kW, P_scl = 1.5 kW, P_core = 1 kW, P_fw = 900 W	SOLVED	n_sync_rpm = 1500, n_rotor_rpm = 1440, slip = 0.04, f_rotor = 2 Hz, P_ag = 47500 W, P_rcl = 1900 W, P_conv = 45600 W, P_out = 44700 W, T_ind = 302.394 N*m, T_shaft = 296.426 N*m, efficiency_pct = 89.4
Induction motor power flow: f = 60 Hz, poles = 6, n_r = 1140 rpm, P_in = 20 kW, P_scl = 800 W, E2 = 200 V	SOLVED	n_sync_rpm = 1200, n_rotor_rpm = 1140, slip = 0.05, f_rotor = 3 Hz, P_ag = 19200 W, P_rcl = 960 W, P_conv = 18240 W, P_out = 18240 W, T_ind = 152.789 N*m, T_shaft = 152.789 N*m, efficiency_pct = 91.2, E_rotor = 10 V
Induction motor power flow: f = 50 Hz, poles = 4, s = 4%, n_r = 1440 rpm, P_in = 50 kW, P_scl = 1.5 kW	INVALID_INPUT	Induction motor power flow from slip and losses: give exactly one of the slip s and the rotor speed n_r.
Induction motor power flow: f = 50 Hz, poles = 4, n_r = 1600 rpm, P_in = 50 kW, P_scl = 1.5 kW	INVALID_INPUT	Induction motor power flow from slip and losses: the slip must lie strictly between 0 and 1 (motoring below synchronous
Induction motor power flow: f = 50 Hz, poles = 4, s = 4%, P_in = 50 kW	NEEDS_INPUT	energy_induction_power_flow needs P_scl (stator copper loss). Example: solve('energy_induction_power_flow', f='50 Hz', p

Electrode kinetics: Butler-Volmer current, overpotential and Tafel slopes,

`energy_butler_volmer`

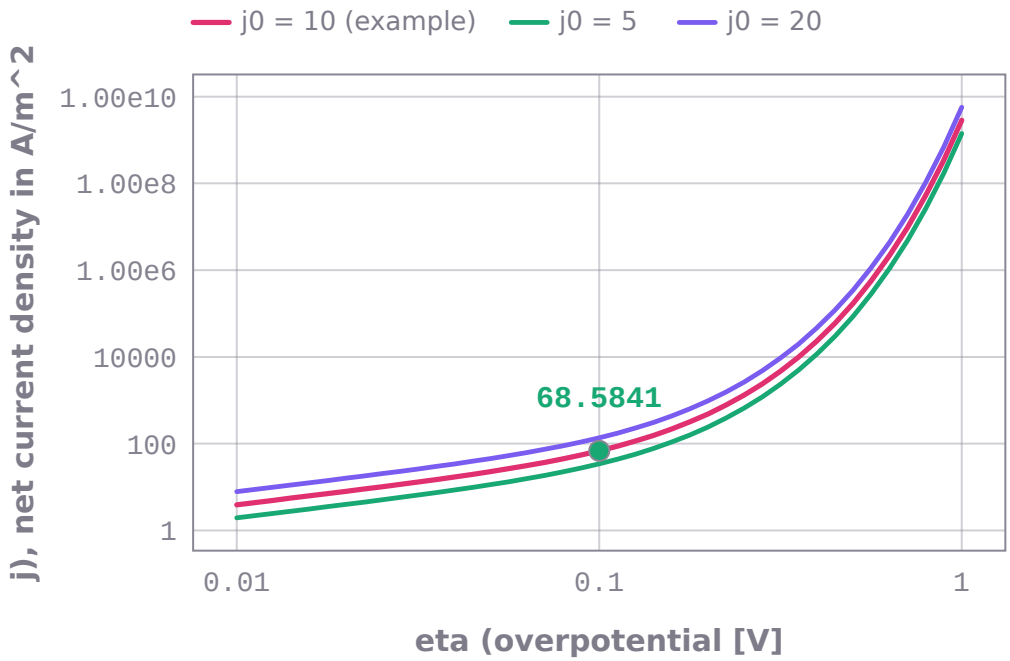
The current density of an electrode reaction at an overpotential by the Butler-Volmer equation (or the overpotential that drives a current density), with the anodic and cathodic partial currents, the Tafel slopes, the Tafel-line overpotential and the charge-transfer resistance.

$$j = j_0 \left[e^{\alpha_a F\eta/RT} - e^{-\alpha_c F\eta/RT} \right], \quad b_a = \frac{2.303\,RT}{\alpha_a F}, \quad \eta \approx b_a \log_{10} \frac{j}{j_0}$$

input	meaning	condition
`j0`	exchange current density in A/m^2 (questions: A/m^2, A/cm^2, mA/cm^2, uA/cm^2)	
`alpha_a`	anodic transfer coefficient (electrons included)	
`alpha_c`	cathodic transfer coefficient (electrons included)	
`T`	electrode temperature	
`eta`	overpotential (anodic positive; give eta or j)	
`j`	current density in A/m^2, anodic positive (give j or eta)	

output	meaning
`j`	net current density in A/m^2 (anodic positive)
`j_mA_cm2`	net current density in mA/cm^2
`eta`	overpotential
`eta_mV`	overpotential in mV
`j_anodic`	anodic partial current density in A/m^2
`j_cathodic`	cathodic partial current density in A/m^2 (negative)
`b_a`	anodic Tafel slope in V per decade
`b_c`	cathodic Tafel slope in V per decade
`b_a_mV`	anodic Tafel slope in mV per decade
`b_c_mV`	cathodic Tafel slope in mV per decade
`R_ct`	charge-transfer resistance at equilibrium in ohm m^2
`R_ct_ohm_cm2`	charge-transfer resistance in ohm cm^2
`eta_tafel`	overpotential of the Tafel line b log10(|j|/j0) (when |j| > j0)

- **Method:** $j = j_0$ [exp(alpha_a F eta/RT) - exp(-alpha_c F eta/RT)] (evaluated with expm1); eta from j by bracketed regula falsi; $b = ln10$ RT/(alpha F); $R_ct = RT$/((alpha_a + alpha_c) F j0)
- **Assumes:** $0 < j_0 \leq 1e6\ A/m^2$; $0 < alpha_a, alpha_c \leq 4$; 200 $K \leq T \leq 1500$ K; exactly one of eta and j; |eta| <= 2 V; no mass-transport (concentration) limitation
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



j (net current density in A/m^2 (anodic positive)) against eta (overpotential (anodic positive; give eta or j)) from 0.01 V to 1 V on a logarithmic axis, logarithmic output axis. One curve for each value of j0 (exchange current density in A/m^2 (questions: A/m^2, A/cm^2, mA/cm^2, uA/cm^2)): 10, 5, 20. Other inputs as in the worked example: alpha_a = 0.5, alpha_c = 0.5, T = 298.15 K. 41 points per curve computed by energy-systems-ai999x; marked: the worked example, eta = 0.1 V, j = 68.5841.

```
energy_systems_ai999x.solve('energy_butler_volmer', j0=10.0, alpha_a=0.5, alpha_c=0.5,
T=298.15, eta=0.1)
# -> j = 68.5841, j_mA_cm2 = 6.85841, eta = 0.1 V, eta_mV = 100, j_anodic = 70.0124,
j_cathodic = -1.42832, b_a = 0.118319, b_c = 0.118319, b_a_mV = 118.319, b_c_mV = 118.319,
R_ct = 0.00256926, R_ct_ohm_cm2 = 25.6926, eta_tafel = 0.0989409 V   (SOLVED)
```

```
energy_systems_ai999x.solve('energy_butler_volmer', j0=10.0, alpha_a=0.5, alpha_c=0.5,
T=298.15, j=1000.0)
# -> j = 1000, j_mA_cm2 = 100, eta = 0.236643 V, eta_mV = 236.643, j_anodic = 1000.1,
j_cathodic = -0.09999, b_a = 0.118319, b_c = 0.118319, b_a_mV = 118.319, b_c_mV = 118.319,
R_ct = 0.00256926, R_ct_ohm_cm2 = 25.6926, eta_tafel = 0.236637 V   (SOLVED)
```

```
energy_systems_ai999x.solve('energy_butler_volmer', j0=0.5, alpha_a=0.3, alpha_c=0.7,
T=353.15, eta=-0.2)
# -> j = -49.6926, j_mA_cm2 = -4.96926, eta = -0.2 V, eta_mV = -200, j_anodic = 0.0696168,
j_cathodic = -49.7622, b_a = 0.233575, b_c = 0.100104, b_a_mV = 233.575, b_c_mV = 100.104,
R_ct = 0.0608642, R_ct_ohm_cm2 = 608.642, eta_tafel = -0.199939 V   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Butler-Volmer: j0 = 1 mA/cm^2, alpha_a = 0.5, alpha_c = 0.5, T = 298.15 K, eta = 100 mV	SOLVED	j = 68.5841, j_mA_cm2 = 6.85841, eta = 0.1 V, eta_mV = 100, j_anodic = 70.0124, j_cathodic = -1.42832, b_a = 0.118319, b_c = 0.118319, b_a_mV = 118.319, b_c_mV = 118.319, R_ct = 0.00256926, R_ct_ohm_cm2 = 25.6926, eta_tafel = 0.0989409 V
Tafel overpotential: j0 = 10 A/m^2, alpha_a = 0.5, alpha_c = 0.5, T = 298.15 K, j = 100 mA/cm^2	SOLVED	j = 1000, j_mA_cm2 = 100, eta = 0.236643 V, eta_mV = 236.643, j_anodic = 1000.1, j_cathodic = -0.09999, b_a = 0.118319, b_c = 0.118319, b_a_mV = 118.319, b_c_mV = 118.319, R_ct = 0.00256926, R_ct_ohm_cm2 = 25.6926, eta_tafel = 0.236637 V
Butler-Volmer: j0 = 0.05 mA/cm^2, alpha_a = 0.3, alpha_c = 0.7, T = 80 degC, eta = -200 mV	SOLVED	j = -49.6926, j_mA_cm2 = -4.96926, eta = -0.2 V, eta_mV = -200, j_anodic = 0.0696168, j_cathodic = -49.7622, b_a = 0.233575, b_c = 0.100104, b_a_mV = 233.575, b_c_mV = 100.104, R_ct = 0.0608642, R_ct_ohm_cm2 = 608.642, eta_tafel = -0.199939 V
Butler-Volmer: j0 = 10 A/m^2, alpha_a = 0.5, alpha_c = 0.5, T = 298.15 K, eta = 0.1 V, j = 100 A/m^2	INVALID_INPUT	Electrode kinetics: Butler-Volmer current, overpotential and Tafel slopes: give exactly one of the overpotential eta and
Butler-Volmer: j0 = 10, alpha_a = 0.5, alpha_c = 0.5, T = 298.15 K, eta = 0.1 V	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: energy_systems_ai999x.templates()
Butler-Volmer: alpha_a = 0.5, alpha_c = 0.5, T = 298.15 K, eta = 0.1 V	NEEDS_INPUT	energy_butler_volmer needs j0 (exchange current density in A/m^2 (questions: A/m^2, A/cm^2, mA/cm^2, uA/cm^2)). Example: